STUDY MANUAL · DERIVED FROM SOURCE

vla-finetune

Every design decision, from the absolute basics to the end — defended from the code, not the docs.

Repository	AdityaKishore0698/VLA-finetune
Package	model_training · CLI `mtrain`
Modules	19 (00 Orientation – 18 Data Provenance & Demo)
Method	source read file by file; docs treated as claims; engine (rde) not available — engine-side behaviour marked inferred
Compiled	2026-11-01

Contents

Module 0 — Orientation

0.1 Prerequisites — the absolute basics
0.2 What the system does, in one paragraph
0.3 The framing sentence
0.4 How the manual is organized (dependency order)
0.5 The stack
0.6 The numbers to know cold
0.7 The two-minute pitch
0.8 Honest gaps to own before an interviewer finds them
The shape to carry into Module 1
Drill — answer from memory, files closed

Module 1 — Foundations: the domain, the boundary, and the package

1.1 Prerequisites — the absolute basics
1.2 What this repository is, measured
1.3 The boundary: what `rde` owns and what this repo owns
1.4 `pyproject.toml`, line by line
1.5 `.gitignore`: what the repo refuses to version
1.6 The package surface: the `__init__.py` files
1.7 The two-environment design (from the README, partly verified)
1.8 Honest gaps to own before an interviewer finds them
The shape to carry into Module 2
Drill — answer from memory, files closed

Module 2 — Configuration as the unit of reproducibility

2.1 Prerequisites — the absolute basics
2.2 The loader: merge first, validate once
2.3 `DatasetRef` and the "identity, not a path" decision
2.4 `DatasetIntervention`: semantics enforced, mostly
2.5 `SplitConfig`
2.6 `TrainingConfig`, `EvaluationConfig`, `ExperimentConfig`
2.7 The shipped configs, read as experiments
2.8 Honest gaps to own before an interviewer finds them
The shape to carry into Module 3
Drill — answer from memory, files closed

Module 3 — The run directory and the cross-run index

3.1 Prerequisites — the absolute basics
3.2 `RunDir` and `make_run_dir`: the no-overwrite rule
3.3 Provenance capture: git and environment
3.4 `JsonlIndex` and `ExperimentIndex`: the cross-run index
3.5 Honest gaps to own before an interviewer finds them
The shape to carry into Module 4
Drill — answer from memory, files closed

Module 4 — Data interventions as lineage

4.1 Prerequisites — the absolute basics
4.2 The version pipeline this module drives
4.3 `compute_length_bucket`
4.4 `resolve_episode_subset`
4.5 `build_intervention_dataset`, and the test set that moves
4.6 Honest gaps to own before an interviewer finds them
The shape to carry into Module 5
Drill — answer from memory, files closed

Module 5 — Pre-training guards: split integrity and the dataset profile

5.1 Prerequisites — the absolute basics
5.2 `verify_split_integrity`
5.3 `build_dataset_profile`, part 1: reuse, don't recompute
5.4 `build_dataset_profile`, part 2: coverage
5.5 Honest gaps to own before an interviewer finds them
The shape to carry into Module 6
Drill — answer from memory, files closed

Module 6 — The training loop around `fit`

6.1 Prerequisites — the absolute basics
6.2 Shape of `Trainer.train`
6.3 Setup: model construction and resume
6.4 The hook: `on_step`
6.5 Calling `fit`, the crash path, and the final checkpoint
6.6 Honest gaps to own before an interviewer finds them
The shape to carry into Module 7
Drill — answer from memory, files closed

Module 7 — Offline evaluation: baselines, tails, time and slices

7.1 Prerequisites — the absolute basics
7.2 `evaluate_baselines`: same call, different inputs
7.3 `error_distribution` and `episode_metrics`
7.4 `temporal_error`: error by phase of the episode
7.5 `stratified_evaluate`: slices that re-run the model
7.6 What isn't here: uncertainty
7.7 Honest gaps to own before an interviewer finds them
The shape to carry into Module 8
Drill — answer from memory, files closed

Module 8 — The feedback artifact: closing the loop

8.1 Prerequisites — the absolute basics
8.2 The code
8.3 The methodological problem: the loop runs on test
8.4 Docs vs code in `FAILURE_ANALYSIS.md`
8.5 Honest gaps to own before an interviewer finds them
The shape to carry into Module 9
Drill — answer from memory, files closed

Module 9 — Model artifact, registry and deployment loader

9.1 Prerequisites — the absolute basics
9.2 `build_model_artifact`
9.3 `ModelRegistry`
9.4 `DeployedModel`: what it checks, and what it doesn't
9.5 Honest gaps to own before an interviewer finds them
The shape to carry into Module 10
Drill — answer from memory, files closed

Module 10 — Reports, figures and cross-run comparison

10.1 Prerequisites — the absolute basics
10.2 `render_experiment_report`
10.3 `render_model_card`
10.4 `comparison.py`
10.5 `figures.py`
10.6 `scripts/plot_results.py`: the README figure
10.7 Honest gaps to own before an interviewer finds them
The shape to carry into Module 11
Drill — answer from memory, files closed

Module 11 — Orchestration: `ExperimentRunner` and the data-fraction sweep

11.1 Prerequisites — the absolute basics
11.2 `ExperimentRunner.__init__` and module constants
11.3 `run`, in order, with what can fail where
11.4 Why a Transaction Script, and when it stops being right
11.5 `run_data_fraction_sweep`
11.6 Honest gaps to own before an interviewer finds them
The shape to carry into Module 12
Drill — answer from memory, files closed

Module 12 — The CLI surface: `mtrain` and the script wrappers

12.1 Prerequisites — the absolute basics
12.2 Structure: lazy imports and thin views
12.3 `train`
12.4 `evaluate` and `test`: the divergent re-implementation
12.5 `rollout`: honesty as a command
12.6 `export`: "paper-ready" with gaps
12.7 `build-intervention`: resolving the design rule
12.8 The script wrappers
12.9 Honest gaps to own before an interviewer finds them
The shape to carry into Module 13
Drill — answer from memory, files closed

Module 13 — The test suite, and the constraint → decision map

13.1 Prerequisites — the absolute basics
13.2 Inventory, verified
13.3 `conftest.py`: the real-engine fixture graph
13.4 What the tests prove well
13.5 What the tests miss, mapped to the gaps they would have caught
13.6 The capstone: constraint → decision map
13.7 Honest gaps to own before an interviewer finds them
The shape to carry into the cross-cutting modules
Drill — answer from memory, files closed

Module 14 — Low-Level Design

14.1 Prerequisites — the absolute basics
14.2 Layering and the dependency rule
14.3 Catalogue of patterns actually present
14.4 SOLID, and where each principle is strained
14.5 Contract map: every boundary, and whether it's enforced
14.6 Sequence diagrams
14.7 State machine of a run (implicit, reconstructed from which files exist)
14.8 Error taxonomy
14.9 Lifetime, sharing and thread safety
14.10 Idempotency table
14.11 Patterns deliberately absent, and the trigger to add each
14.12 Honest gaps to own before an interviewer finds them
The shape to carry into the High-Level Design module
Drill — answer from memory, files closed

Module 15 — High-Level Design & Scaling

15.1 Prerequisites — the absolute basics
15.2 Deployment topology, as it actually is
15.3 The lifecycle of one run, with a latency budget
15.4 Capacity arithmetic: one laptop
15.5 Bottleneck analysis, in the order things break
15.6 The scaling ladder
15.7 Caching strategy
15.8 Multi-tenancy
15.9 Observability: what to measure, quality first
15.10 Cost model
15.11 What survives unchanged: the good bones
15.12 Honest gaps to own before an interviewer finds them
The shape to carry into the Fixes & Roadmap module
Drill — answer from memory, files closed

Module 16 — Fixes & Roadmap

16.1 Prerequisites — the absolute basics
16.2 P0: fix before reporting anything new
16.3 P1: wrong output, lost work, or false documentation
16.4 P2: hygiene, robustness, cost, scale
16.5 The five-fix shortlist: about one day
16.6 Feature roadmap
16.7 Anti-roadmap: tempting changes that would make it worse
16.8 Honest gaps to own before an interviewer finds them
The shape to carry into the Interview Kit
Drill — answer from memory, files closed

Module 17 — Interview Kit

17.1 Prerequisites — the absolute basics
17.2 Pitches
17.3 Question bank
17.4 Behavioural framings (STAR + Reflection)
17.5 Phrases that hurt you, and what to say instead
17.6 Questions to ask them
17.7 The ten sentences that carry the project
17.8 Quick-reference card: every number
17.9 Glossary
17.10 Honest gaps to own before an interviewer finds them
The shape to carry into the Data Provenance & Demo module
Drill — answer from memory, files closed

Module 18 — Data Provenance & the Demo

18.1 Prerequisites — the absolute basics
18.2 Where the data comes from and how it gets in
18.3 The relationship to the sibling project
18.4 What's on disk versus in the repo
18.5 The rehearsed demo script (about 8 minutes)
18.6 Recovery table: what to say when things go wrong
18.7 Honest gaps to own before an interviewer finds them
The shape to carry into Module 0
Drill — answer from memory, files closed

Module 0 — Orientation

Files covered: the whole repository at a glance. This module was written last, after Modules 1–18, so every statement here is a summary of something verified in a later module, with a pointer to it.

Where we are: This is the front door. It assumes nothing. By the end you should know what the system does, the one sentence that frames it, the twenty words you need before Module 1, the stack, the numbers to know cold, and a two-minute pitch you can give today.

The one-line thesis: `vla-finetune` is a thin, provenance-obsessed experiment layer that fine-tunes a robot vision-language-action model on versioned data and reports results honestly. Its most important result is a tie with a nearest-neighbour lookup, and its most important flaw is that its own interventions aren't yet scored on a shared test set.

0.1 Prerequisites — the absolute basics

The minimum vocabulary to start Module 1. Each term is defined fully in the module shown.

Term	One-line meaning	Full treatment
Episode	one recorded demonstration of a robot task, a sequence of frames	M1
Frame	one timestep: camera images, robot state (joint angles), commanded action	M1
Action	the vector sent to the robot: 6 numbers (SO-100), 14 (ALOHA)	M1
Policy	a function from observation to action; the trained model	M1
VLA	vision-language-action model: images plus an instruction plus state in, actions out	M1
SmolVLA	the small VLA used here (`lerobot/smolvla_base`); predicts action *chunks* with *flow matching*, so its outputs are random from call to call	M1
Fine-tuning	continuing to train a pretrained model on new data	M1
Behaviour cloning	training a policy to copy demonstrated actions (supervised regression)	M1
Offline evaluation	comparing predicted vs recorded actions on held-out data, with the robot never moving. **The only evaluation here**	M1, M7
Closed-loop rollout	letting the policy drive a robot or simulator and measuring task success. **Not available; recorded as "unavailable"**	M1, M12
Train / val / test split	disjoint sets of *episodes*: train to learn, val to pick a checkpoint, test to report once	M1, M5
Baseline	a trivial model to beat: *mean-action* (predict the average) and *nearest-neighbour* (copy the action of the closest training robot state)	M7
MSE (normalized)	mean squared action error after per-dimension standardization; mean-action ≈ 1.0	M1, M7
Dataset version	an immutable, named snapshot in the data engine: `raw/v1`, `clean/v1`, `training/v6`	M1, M4
Lineage	the recorded chain of operations that produced a version	M4, M18
Data intervention	a deliberate change to the training data (reduce, imbalance, rebalance) to test what data contributes	M2, M4
The engine (`rde`)	the sibling repository *Episodic Data Engine*: owns data, versions, the evaluator, failure detection and the SmolVLA training step	M1, M18
Run directory	`runs/<date>/<name>/seed_<n>/`, holding everything one experiment produced	M3
Checkpoint	saved model weights plus pre- and post-processors at a training step	M6, M9
"Unavailable" marker	an explicit record that something wasn't measured, and why; never a fabricated value	M5, M10

0.2 What the system does, in one paragraph

Given a YAML experiment file, `mtrain train` asks the data engine to **build a new, lineage-tracked training dataset version** (optionally with an intervention such as "half the episodes" or "upweight short episodes"). It **re-checks the train/val/test split**, **profiles** the data, and **fine-tunes SmolVLA**, validating only on the val split and keeping the best verified checkpoint. It then **scores that checkpoint and two baselines on the test split through the same evaluator**, decomposes the error by tail, episode, phase of the episode and slice, runs the engine's **failure detector** to produce a recommended next data intervention for a human, and writes a **report, model card, per-frame outcomes and index rows**. Everything it can't measure is written down as unavailable (M11).

0.3 The framing sentence

> **"The model layer never owns the data."** Every dataset this package trains on is an immutable, catalog-resolved version produced by the engine, and even a data intervention is executed as a lineage-tracked transform, never a local copy (README; verified in M4).

And the honest counterweight to carry with it:

> **"The data layer owns the model."** The SmolVLA adapter and its gradient step (`fit`) live in the engine; this repository owns the *procedure* around them (M1 §1.3).

0.4 How the manual is organized (dependency order)

#	Module	You'll be able to explain
1	Foundations	the domain, the engine boundary, packaging and two venvs
2	Configuration	the YAML merge, validators, three seeds, why typos vanish
3	Run directory and index	write-once runs, provenance capture, JSONL fragility
4	Data interventions	versions as interventions; **why each intervention gets a different test set**
5	Split integrity and profile	what the leak check can and can't catch; the coverage metric's flaw
6	Training loop	the callback around `fit`; **32-frame validation and the 13× val–test gap**
7	Offline evaluation	baselines through one evaluator; **the nearest-neighbour tie**; slices re-infer
8	Feedback artifact	the flywheel's plumbing; **why it always says "act" and runs on test**
9	Deployment	the artifact; a health check that doesn't cross-check; a replay harness
10	Reports and figures	"not run" discipline; dropped numbers; hard-coded figure markers
11	Orchestration	the runner; **`--seed` doesn't seed**; late validation; sweep flaws
12	CLI	lazy imports; **`mtrain test` overwrites published numbers**
13	Tests and the capstone	real-engine tests, missing invariants; **constraint → decision map**
14	Low-Level Design	layering, patterns, contracts, state machine, errors, idempotency
15	High-Level Design and scaling	latency budget; **under one epoch**; bottleneck order; the ladder
16	Fixes and roadmap	P0/P1/P2, the one-day shortlist, the anti-roadmap
17	Interview kit	pitches, 47 questions, stories, phrase swaps, numbers card, glossary
18	Data provenance and demo	HF → engine → run; the 8-minute demo; recovery table

Reading paths.

- **Short on time:** read this module, then M17, M16 §16.5, and the summary page of M4, M6, M7, M8 and M11.
- **Before a systems interview:** M14, M15, M13 §13.6.
- **Before an ML or research interview:** M4, M6, M7, M8, M15 §15.4.

0.5 The stack

Layer	Technology	Version / note	Where
Language	Python	≥ 3.10 declared; 3.11 used	`pyproject.toml`, README
Packaging	hatchling, uv (sibling path source)	no lockfile	M1
Config	YAML (PyYAML `safe_load`) + Pydantic v2	`extra` not forbidden	M2
CLI	Typer + rich	11 commands, lazy imports	M12
Data engine	`episodic-data-engine` (`rde`), editable sibling	unpinned	M1, M18
Engine storage	catalog + Parquet + video; DuckDB queries	engine-side, partly inferred	M5, M18
Model	SmolVLA via `lerobot`	lerobot 0.4.4; base `lerobot/smolvla_base`	M1, M18
DL framework	PyTorch with CUDA	2.10 (CUDA build); driver 592.82, CUDA 13.1	M15
Numerics / plots	NumPy, matplotlib (Agg)	—	M7, M10
Tracking	run directories + 4 JSONL files	git-ignored	M3
Tests	pytest, real engine fixtures	56 tests, 52 torch-free	M13
Hardware	RTX 4060 Laptop, 8 GB VRAM; ~10 GB VM RAM	single GPU, single process	M15
Data	`lerobot/svla_so100_pickplace`, `lerobot/aloha_static_coffee`	licences not checked	M18

0.6 The numbers to know cold

Data: SO-100: 50 raw → 48 after QC (dropped 9, 31) → 38/5/5, 1,908 test frames, 6-dimensional action, 2 cameras at 30 fps, one task, 326–475 frames per episode. ALOHA: 40 → 32/4/4, 4,400 test frames, 14-dimensional action, 4 cameras at 50 fps, test episodes 1,100 frames each.

Results (test MSE, normalized):

	SmolVLA	Nearest-neighbour	Mean-action	Ratio to mean
SO-100, 1,000 steps	0.0290	0.0239	0.908	3.2%
SO-100, 2,000 steps	**0.0239**	**0.0239**	0.908	2.6%
ALOHA, 1,000 steps	**0.2157**	**0.1062**	1.010	21.4%

Training: batch 4, lr 1e-4 (3e-5 when resumed), warmup 50 (20 when resumed), checkpoint and validation every 100 steps, validation on **32 frames**. 324 / 385 / 576 s. Peak GPU 2.2–3.0 GB of 8.

The derived numbers that change the story:

- val minimum **0.0018** vs test **0.0239**, about 13× (M6)
- ≈ **0.26 epoch** (SO-100) and ≈ **0.11 epoch** (ALOHA) per 1,000 steps (M15)
- re-scoring noise **up to 5%** vs the **18%** continued-training gain (M7)
- evaluation ≈ **45–57%** of run time, about half of it redundant (M15)
- failure rule flags **191 / 1,908** (10%), max episode **13.8%** vs the 0.5 cut (M8)

Code: about 2,700 lines in `src/`; 4 layers; 11 CLI commands; 56 tests; about 200 logged gaps → 7 P0 / 26 P1 / 28 doc corrections / about 60 P2 (M16).

0.7 The two-minute pitch

> "This is the experiment and evaluation layer for fine-tuning SmolVLA, a small vision-language-action robot policy, on real LeRobot datasets: SO-100 pick-and-place and bimanual ALOHA coffee.
>
> The design rests on one rule: the model layer never owns the data. Every data change, like halving the training set or dropping a regime, is executed as a lineage-tracked version in a separate data engine, so any checkpoint traces back to the exact raw episodes it learned from, including which ones QC removed.
>
> One command runs an experiment end to end. It builds the dataset version, re-checks the episode-level splits, fine-tunes with checkpoint selection on validation only, and scores the result against a mean-action and a nearest-neighbour baseline through the same evaluator. It breaks error down by tail, episode, phase and slice, produces a feedback artifact recommending the next data change, and writes a report where anything it can't measure, like closed-loop success, is recorded as unavailable rather than invented.
>
> The results are honest rather than flattering. On SO-100, fine-tuning reached 2.6% of the mean-action error, and continued training gave an 18% gain on an identical test split, but it exactly tied a lookup over robot state. On ALOHA, retrieval was twice as good. Since that run saw about a ninth of an epoch, I read that as undertrained before I read it as retrieval being better.
>
> Auditing the harness itself, the main thing I'd fix is that interventions re-split the data, so each one is scored on a different test set. Freezing one test split, propagating real seeds and adding bootstrap intervals come before any new infrastructure."

(About 2 minutes at a normal pace. Adjust "I" and "we" to your actual role. See the M17 authorship note.)

0.8 Honest gaps to own before an interviewer finds them

The five that matter most, each owned by a later module:

1. **Interventions are scored on different test sets** (filter-then-split). → M4, P0-1
2. **`--seed` doesn't seed; replicates are fake.** → M11, P0-2
3. **Checkpoint selection uses 32 validation frames** (val about 13× below test). → M6, P0-7
4. **The feedback loop runs on test, and can't say "no action".** → M8, P0-5
5. **`mtrain test` overwrites published results.** → M12, P0-6

And the scope facts: offline only, single seed, 5 and 4 test episodes, no CI, the engine unpinned and not in this repo, run artifacts not in git (M1, M18).

The shape to carry into Module 1

```
YAML → mtrain train → [ engine: version + lineage ] → train (val-only) → test vs 2 baselines
                                                                                  │
        report · model card · per-frame outcomes · feedback ◀─────────────────────┘   (rollout: "unavailable")

principle: the model layer never owns the data      counterweight: the data layer owns the model
result:    SmolVLA = state-NN on SO-100; NN 2× better on ALOHA (≈0.11 epoch → undertrained?)
first fix: one frozen test split for every intervention
```

Drill — answer from memory, files closed

1. In one sentence, what does this system do, and what's its organizing principle?
2. What's the difference between offline evaluation and closed-loop rollout, and which does this project do?
3. *Why didn't you just* report the 2.6%-of-mean-action number as the headline?
4. *What breaks in production* if someone treats the README's intervention comparisons as findings today?
5. *Interviewer framing:* "Give me the two-minute version."

Appendix A — Model answers to the drill

A1. "It's an experiment, evaluation and feedback layer that fine-tunes SmolVLA on versioned robot datasets and reports results against baselines. Its principle is that the model layer never owns the data: every dataset, including every intervention, is an immutable, lineage-tracked version in a separate data engine."

Why that answer lands: it gives the what and the principle in under fifteen seconds, with the one word an interviewer will remember: lineage.

A2. "Offline evaluation compares predicted actions with recorded demonstrations on held-out episodes. The robot never moves, so errors can't compound. Closed-loop rollout lets the policy drive a robot or simulator and measures task success. This project only does offline evaluation, and it records rollout as unavailable rather than estimating it, because low offline error doesn't guarantee success once small mistakes push the robot off the demonstrated states."

Why that answer lands: it's precise about the mechanism (compounding error) and shows the project's honesty is a design choice.

A3. "Because the mean-action baseline is the weakest possible comparison. Any model that learned anything beats it. The informative comparison is the nearest-neighbour lookup over robot state, which SmolVLA ties on SO-100 and loses to on ALOHA. Leading with 2.6% would be technically true and misleading, and the whole point of scoring both baselines through the same evaluator is to report the one that matters."

Why that answer lands: it shows you understand baseline selection, and that you'd choose the less flattering framing deliberately.

A4. "They'd be comparing numbers measured on different test sets. Each intervention filters episodes and the engine then re-splits the remainder, so a 'reduced' run is scored on two or three test episodes that aren't even a subset of the baseline's five. Any difference could be which episodes landed in test. Combined with a single seed and a stochastic model, the comparison has no statistical footing. The fix is to freeze one test split and apply interventions to the train split only. Until then, they're pipeline validation, not findings."

Why that answer lands: it names the mechanism, quantifies it and gives the fix, which is the core of the project's methodological story in thirty seconds.

A5. Deliver §0.7.

Why that answer lands: it's bounded, it has a principle, a result, an honest interpretation and a next step, and it invites follow-up in exactly the areas you've prepared.

Module 0 Summary — the one page to revise

What this module is. The entry point: what the system does, its framing sentence and counterweight, twenty terms, the reading map, the stack, the numbers, and the two-minute pitch.

Five claims to make cold

1. The model layer never owns the data; every intervention is a lineage-tracked engine version.
2. The data layer owns the model; `fit` is in the engine, and this repo owns the procedure around it.
3. Offline only: SmolVLA ties state-NN on SO-100 (0.0239) and loses on ALOHA (0.216 vs 0.106).
4. ALOHA saw about 0.11 epoch, so it's undertrained before it's out-retrieved; continued training gained 18% on an identical split.
5. First fix: one frozen test split for all interventions; then real seeds, frozen val subset, val-driven feedback, non-destructive re-scoring.

Decisions (the headline four)

Chose	Over	Because	Cost
Interventions as engine versions	local copies / ID lists	traceable to raw episodes	each intervention gets its own test split
Two baselines through one evaluator	model-only metrics	honest comparison	the result is a tie
"Unavailable" over estimates	proxy success numbers	never fabricate	incomplete-looking reports
Laptop, files, one process	infrastructure	one or two researchers, short runs	concurrency and durability gaps

Numbers and names: §0.6, the results table and the five derived numbers.

Sentences worth memorizing

- "The model layer never owns the data, and the data layer owns the model."
- "Offline, a state lookup tied my VLA; the harness was built to tell me that."
- "My first fix is one frozen test split, because right now each intervention is graded on a different exam."

Module 1 — Foundations: the domain, the boundary, and the package

Files covered: `pyproject.toml`, `.gitignore`, `src/model_training/config/__init__.py`, `src/model_training/data/__init__.py`, `src/model_training/deployment/__init__.py`, `src/model_training/evaluation/__init__.py`, `src/model_training/experiments/__init__.py`, `src/model_training/feedback/__init__.py`, `src/model_training/reports/__init__.py`, `src/model_training/tracking/__init__.py`, `src/model_training/training/__init__.py`

Where we are: This is the first module, so there is nothing before it. Its job is to give you the vocabulary for the rest of the manual and to show exactly where this repository stops and its sibling, the Episodic Data Engine (`rde`), starts. That boundary is the most important structural fact about the project, and getting it wrong in an interview means claiming work this repository doesn't contain.

The one-line thesis: `vla-finetune` is a thin, import-light orchestration layer of about 2,700 lines. It owns *experimental procedure* (config, interventions, validation cadence, selection, reporting, provenance) and delegates every *mechanism* (data storage, transforms, the training step, scoring, failure detection) to a separately installed engine.

1.1 Prerequisites — the absolute basics

Robot learning vocabulary

- **Demonstration / episode.** One recorded attempt at a task, usually a human teleoperating a robot. It's a time-ordered sequence of **frames**. "Episode" is the unit of data in this codebase. Splits, interventions, failure reviews and weights are all per-episode.
- **Frame.** One timestep in an episode: the camera image(s), the robot's **state** (joint positions, sometimes efforts) and the **action** commanded at that moment. Code keys look like `state.joint`, `action.joint` and `rgb.top` (these appear in `tests/smoke/test_smolvla_smoke.py`).
- **Action.** A vector of numbers sent to the robot. For SO-100 it has 6 dimensions (5 joints plus the gripper). For bimanual ALOHA it has 14 (two 7-dimensional arms). The configs record this: the `aloha_coffee_baseline.yaml` header says "14-DoF action across two arms, four cameras".
- **DoF (degrees of freedom).** The number of independently controllable joints.
- **Embodiment.** The physical robot body: its kinematics, cameras and action space. "Cross-embodiment" means the same pipeline or model works across different bodies. The config tag `embodiment: so100` or `aloha` records it.
- **Behaviour cloning (imitation learning).** Training a policy to reproduce the demonstrated action given the observation. It's supervised regression from observation to action. Everything here is behaviour cloning.
- **Policy.** A function from observation to action. The trained model is a policy.
- **VLA (Vision-Language-Action model).** A policy whose inputs are camera images plus a natural-language instruction ("Pick up the cube and place it in the box." is literally a task string in `tests/conftest.py`) plus robot state, and whose output is actions. It's usually built by attaching an action-generating head to a pretrained vision-language model.
- **SmolVLA.** The specific VLA used here, loaded from the Hugging Face checkpoint `lerobot/smolvla_base` (`configs/base.yaml`). Per its public release, which this repository doesn't contain and I can't verify from

here, it's a small VLA from the LeRobot project. Its action expert generates *chunks* of future actions with **flow matching**. The repository's own evidence that actions are stochastic is the README's §2.3 note that re-scoring the same frames moves MSE by up to 5%. The evidence for chunking is the smoke-test comment about an "action-queue bug" where a cached action chunk was reused across batches.

- **Flow matching.** A generative-modelling technique: the network learns a vector field that transports random noise into a sample of the target distribution (here, an action chunk). What matters for this project: **prediction is stochastic**. Score the same input twice and you get different actions. That fact matters later for checkpoint selection (Module 6) and for how to read small metric differences (Module 7).
- **LeRobot.** Hugging Face's robot-learning library and dataset format. The source datasets (`lerobot/svla_so100_pickplace`, `lerobot/aloha_static_coffee`) are LeRobot datasets, and the test fixture fakes one with `rde.testing.make_lerobot_dataset`.
- **Fine-tuning.** Continuing to train a pretrained model on new task data. "Fine-tuning SmolVLA" means starting from `smolvla_base` weights and taking gradient steps on so100 or aloha episodes.

Evaluation vocabulary

- **Offline (open-loop) evaluation.** Feed recorded observations, compare predicted actions with the recorded human actions, and compute error. The robot never acts, so errors can't compound. **This is the only kind of evaluation the project runs.**
- **Closed-loop rollout.** Let the policy actually control a robot or simulator and measure task success. Errors compound because the policy's own actions change what it sees next. The code *refuses* to fake this: `mtrain rollout` writes `"status": "unavailable"`.
- **Why the distinction matters:** a low offline MSE doesn't imply a high success rate. A policy can match demonstrations on average and still fail at the one moment that matters, such as the grasp. Every claim the project makes is offline, and the code is built to keep the two from being confused.
- **MSE in normalized units.** Actions are normalized per dimension (subtract the mean, divide by the standard deviation; the test contract fixtures carry `normalization: {mean, std}`). An MSE of 1.0 means about one standard deviation of error. That's why the mean-action baseline sits near 1.0 (README Table 3: 0.908 and 1.010). Predicting the mean leaves roughly the variance.
- **Baseline.** A trivial model the real model must beat. Here there are two: `mean_action` (always predict the training mean) and `nearest_neighbor` (return the action of the most similar training frame, i.e. retrieval). Both are implemented in `rde`, not here.
- **Train/val/test split.** The **validation** split is used during training to pick a checkpoint. The **test** split is touched once, at the end, to report. Splits are **episode-level**: every frame of an episode goes to the same split. If frames from one episode landed in both train and test, near-identical neighbouring frames would leak and inflate the scores.
- **Leakage.** Any path by which test information influences training or selection.
- **Checkpoint.** A saved snapshot of model weights at a given step.
- **Checkpoint selection.** Choosing which snapshot to report, by a validation metric and never by test.

Data-engineering vocabulary

These terms are defined by the sibling engine. They're inferred from call sites here, not read from the engine's source.

- **Dataset version.** An immutable, named snapshot such as `raw/v1`, `clean/v1` or `training/v6`. The prefix is a **stage** (`rde.schemas.lineage.VersionStage`, with at least `CLEAN` and `TRAINING`, used in `interventions.py`).
  - `raw/vN` is ingested source data.

    - `clean/vN` is the output of transforms (normalization, episode filtering).
    - `training/vN` is a built training set *with split assignments*.

- **Catalog.** The engine's metadata store: which versions exist and which episodes they contain (`catalog.get_manifest`, `catalog.get_episodes`). It's located by `settings.paths.catalog_db`, and the name suggests a database file. I haven't verified the engine.
- **Lineage.** The recorded chain of operations that produced a version (`ingest → transform → build_training_dataset`). The README shows real output of `rde version lineage`.
- **Training contract.** A dict describing a version's modalities: `observation` (each key with a `plane`, `shape` and optional `normalization`) and `action` (`key`, `shape`, `normalization`). The **plane** says where a modality physically lives. `inline` means numeric columns stored with the frame (state). `external` means stored outside the table (images or video). This repo branches on `plane == "inline"` and `plane == "external"` in `profile.py`, `deployment/loader.py` and `reports/model_card.py`.
- **DatasetView.** The read handle on a version. It has `episode_ids(split=...)`, `episodes`, `contract`, `iter_samples`, `get_episode_frames`, `splits()` and `close()`, and it's a context manager (`with engine.get_dataset(...) as view` in `profile.py`).
- **Adapter.** Converts a list of samples into model input tensors (`VisionLanguageAdapter`, `StateAdapter`). An adapter is the "preprocessing" stage.

Python packaging vocabulary

- **`pyproject.toml`.** The standard (PEP 621) file declaring a package's name, dependencies and build system.
- **Build backend.** The tool that turns source into an installable wheel. Here that's `hatchling`.
- **Distribution name vs import name.** You `pip install vla-finetune` but `import model_training`. The two names are independent.
- **Console script.** `[project.scripts] mtrain = "model_training.cli:main"` makes installation create an `mtrain` executable that calls `main()`.
- **Extras / optional dependencies.** `pip install pkg[dev]` adds a named group.
- **Editable install.** `-e`: the installed package points at your source tree, so edits take effect without reinstalling.
- **`src/` layout.** Code lives under `src/` so tests can't accidentally import the uninstalled working copy.
- **Namespace package.** A directory with **no** `__init__.py` that Python still imports (PEP 420), merging every same-named directory on `sys.path`. This matters below.
- **`__all__`.** The list of names a package declares public, and what `from pkg import *` exports.
- **Import-light discipline.** Not importing heavy libraries (torch) at module import time, so cheap commands and tests run without them. It's done with **lazy imports**, meaning imports inside functions.

1.2 What this repository is, measured

Thing	Measured value
Source lines (`src/`)	about 2,700 across 28 files
Largest modules	`runner.py` 314, `cli.py` 306, `profile.py` 303, `trainer.py` 256, `interventions.py` 212
Test functions	49 in `tests/unit`, 3 in `tests/integration`, 4 in `tests/smoke`. The README's "52 unit + integration tests" (line 282) **matches** 49 + 3
CI	**none**: no `.github/`, no other CI config
Lockfile	**none**: no `uv.lock` and no `requirements*.txt`
Engine present in this checkout	**no**: `import rde` fails, and `../episodic-data-engine` doesn't exist here

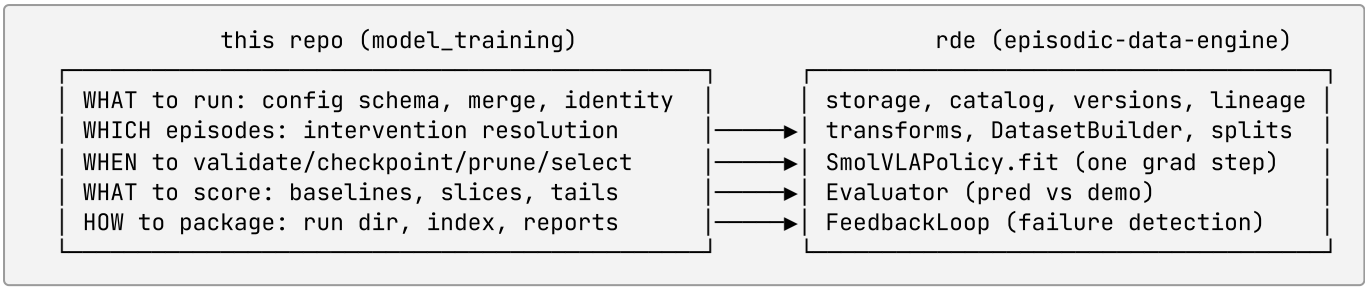
That last row shapes the whole manual. **I couldn't run any test or command while writing it.** Every statement about what `rde` does is either inferred from a call site here (and marked as such) or quoted from this repo's docs (and treated as a claim).

1.3 The boundary: what `rde` owns and what this repo owns

Here is every `rde` symbol imported by `src/`, grouped by who imports it (from grep):

`rde` symbol	Imported by (this repo)	What it does for us
`catalog.Catalog`, `config.Settings/get_settings`	runner, interventions, profile, feedback, cli	locate and read versions
`engine.DataEngine`, `engine.DatasetView`, `engine.Sample`	runner, profile, split_integrity, trainer, offline, loader, cli	open a version for reading
`transformations.TransformPipeline`, `EpisodeSubsetFilter`	interventions	**create** a `clean/vN` by dropping episodes
`dataset_builder.DatasetBuilder`, `build_training_contract`	interventions, profile	**create** a `training/vN` with splits and weights
`versioning.next_version`, `schemas.lineage.VersionStage`	interventions	allocate the next version name
`exploration.queries.DatasetExplorer`	profile, cli	stats, histograms, raw SQL over frames
`model_adapters.models.SmolVLAPolicy`	trainer, runner, deployment artifact and loader, cli	**the model, including `fit()`: the training step itself**
`model_adapters.VisionLanguageAdapter`, `MODEL_REGISTRY`, `build_model`	trainer, runner, offline, loader, cli	preprocessing and baselines
`evaluation.Evaluator`, `EvaluationResult`	trainer, offline, feedback	scoring predictions against demonstrations
`feedback.FeedbackLoop`	feedback	failure detection and episode nomination

So the split is:



Decision: put the model adapter, including `SmolVLAPolicy.fit`, inside the *data* engine.

- *What:* the class that wraps SmolVLA, builds batches and runs forward/backward lives in `rde.model_adapters.models`, not here. `docs/ARCHITECTURE.md` line 35 says so outright: `SmolVLAPolicy` was "Extended in place: `fit()` now performs real gradient steps; `save_checkpoint()` and `from_checkpoint()` were added."
- *Why (inferred):* the adapter has to read the engine's training contract to build feature shapes and normalization statistics. The smoke test calls rde's private `_policy_features_from_contract` and `_dataset_stats_from_contract`. Keeping the model next to the contract keeps those two in lockstep.
- *Alternative:* the model lives in this repo, and the engine exposes only data, a contract and an iterator.
- *Cost:* **the dependency direction is inverted relative to the README's slogan.** The README says "the model layer never owns the data". That's true, but the *data* layer owns the model. The most technically demanding code (batch construction, optimizer, loss, the action-queue fix) isn't in this repository. In an interview, say plainly: "the training step lives in the engine; this repo owns the run around it." Don't claim the optimizer loop as code in this repo.
- *When the answer flips:* once a second model family arrives (a diffusion policy, ACT), its adapter would also have to go into the data engine. At that point a separate `model_adapters` package depended on by both repos is the right shape.

1.4 `pyproject.toml`, line by line

```
[project]
name = "vla-finetune"
version = "0.1.0"
description = "Experiment, training, evaluation, deployment and feedback layer on top of episodic-data-engine"
license = "MIT"
authors = [{ name = "Kushagra Srivastava" }]
requires-python = ">=3.10"
```

- `requires-python = ">=3.10"`: the code uses `X | None` union syntax in annotations at runtime-evaluated positions (for example, Pydantic field types like `float | None` in `schema.py`). Pydantic evaluates those, so Python 3.10 is a real floor and not an arbitrary one. The README installs with 3.11 (`uv venv --python 3.11`), so 3.10 itself is **declared but never shown to have been tested**.
- `authors` and `[project.urls] Repository = "https://github.com/kushagrathisside/VLAFinetuningExperiments"` name a person and a repository other than the checkout this manual was written from. If you present this project as yours, have a one-sentence account of the relationship (fork, collaboration, renamed), because an interviewer who opens the repo will see it. (README line 298 credits the SO-100 runs to `AdityaKishore0698` and the ALOHA run to `chytra3`. See gap 11.)

```
dependencies = [
    "episodic-data-engine",
    "pydantic>=2",
    "pyyaml>=6",
    "typer>=0.9",
    "rich>=13",
    "psutil>=5",
    "matplotlib>=3.8",
    "numpy>=1.24",
]
```

- **Floors only, no ceilings, no lockfile.** `pydantic>=2` admits any future major version. The code uses v2-only API (`model_validator`, `model_validate`, `model_dump(mode="json")`), so the floor is load-bearing, but nothing stops a breaking v3.
  - *Alternative:* a committed `uv.lock`.
  - *Cost:* two installs months apart can resolve different versions. The project compensates *after the fact* by recording versions into `environment.json` (Module 3). That records drift but doesn't prevent it.
- `episodic-data-engine` with **no version constraint at all.** The repository is coupled to the engine's *private* functions (the smoke test imports `_policy_features_from_contract`), yet it declares no compatible engine version. The only real pin is the git commit recorded per run in `git.json` (Module 3).
- `psutil` is a **hard** dependency, yet `trainer.py` and `run_dir.py` both wrap `import psutil` in `try/except ImportError` marked `# pragma: no cover`. A declared dependency can't be missing in a correct install, so those branches are dead defensive code. They're harmless, but don't call them "graceful degradation".
- `matplotlib` is a hard runtime dependency of a library, used only by `reports/figures.py`. That's acceptable because every run renders figures. It does make the "light" environment heavier than it needs to be.

```
[project.optional-dependencies]
# The heavy VLA stack (torch, lerobot) is pulled in transitively through
# episodic-data-engine's own `vla` extra -- installed once, in .venv-vla, per
# the engine's own import-light discipline. Nothing here re-declares torch.
dev = ["pytest>=8", "pytest-cov>=5"]
```

Decision: torch and lerobot are not declared here at all.

- *What:* there's no `vla` extra on this package. You get torch only by installing `episodic-data-engine[vla]`.
- *Why:* single ownership. The engine's `SmolVLAPolicy` is the only code that needs torch, so the engine declares it, and two packages can't pin conflicting torch versions.
- *Alternative:* `vla-finetune[vla] = ["episodic-data-engine[vla]"]`, a pass-through extra that would make `pip install vla-finetune[vla]` work in one step.
- *Cost:* `pyproject.toml` **alone gives no way to get a working training environment.** You have to know to install the engine's extra, which only the README tells you (line 177). And `trainer.py` and `run_dir.py` do `import torch` directly, so this package *does* use torch without declaring it. For `trainer.py` the comment covers it ("safe: policy.load() above already required it to succeed"). It's still an undeclared direct dependency.

`pytest-cov` is declared, but `addopts = "-q"` doesn't enable coverage, and there's no coverage config. Nothing in the repo runs it, so it's an unused dependency.

```
[project.scripts]
mtrain = "model_training.cli:main"
```

The docstring in `cli.py` explains the name: "`mtrain`, not a bare `model`, to avoid colliding with the engine's own `rde model ...` sub-app". That's a real rationale, and it's verifiable because `cli.py` exists and is the target.

```
[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

[tool.hatch.build.targets.wheel]
packages = ["src/model_training"]

[tool.hatch.metadata]
allow-direct-references = true
```

- `packages = ["src/model_training"]` tells hatch where the import package is. That's needed because the import name differs from the distribution name.
- `allow-direct-references = true` permits dependencies written as direct URLs (`pkg @ file:///...` or `git+https://...`). **No dependency in this file is a direct reference.** `episodic-data-engine` is a bare name. The setting is vestigial, probably left over from an earlier `episodic-data-engine @ file:../...` form. Harmless, but it's a loose thread.

```
[tool.uv.sources]
episodic-data-engine = { path = "../episodic-data-engine", editable = true }
```

Decision: resolve the engine from a sibling directory via `uv`.

- *What:* when installed with `uv`, the bare dependency name resolves to the local checkout `../episodic-data-engine`, editable.
- *Why:* the two repos are developed together, so engine edits (like the in-place `SmolVLAPolicy` extensions) take effect immediately.
- *Alternative:* publish the engine to PyPI or a private index and pin a version, or vendor it as a git submodule.
- *Cost:* `[tool.uv.sources]` is **a uv-only table**. Plain `pip install -e .` ignores it and tries to fetch `episodic-data-engine` from PyPI. I couldn't verify whether a package of that name exists there. If it doesn't, the install fails; if a different package squats the name, it installs the wrong code. The install also needs a specific *directory layout* (`../episodic-data-engine` next to this repo), which is why `runner.py` hard-codes `parents[4] / "episodic-data-engine"` (Module 11). The same layout assumption appears in three places, with nothing enforcing it.
- *When the answer flips:* once anyone other than the author installs it, or once CI exists. CI would need to check out both repos side by side.

```
[tool.pytest.ini_options]
testpaths = ["tests"]
addopts = "-q"
```

`testpaths = ["tests"]` matters because there's a file called `scripts/test.py`. Without `testpaths`, pytest's default discovery pattern (`test_*.py` or `*_test.py`) wouldn't match `test.py` anyway, so this is belt-and-braces, not a fix for a real problem.

1.5 `.gitignore`: what the repo refuses to version

```
runs
experiments/*.jsonl
paper_ready/
...
.venv/

# Internal code-review audit -- kept locally as a private record, not published.
docs/CODE_REVIEW.md
```

- `runs` (no trailing slash, no leading slash) ignores **any file or directory named `runs` at any depth**. That's intended for `<repo>/runs/`, where `ExperimentRunner` writes by default.
- **Consequence: the README's results can't be reproduced from the repository alone.** Every run directory, checkpoint and metric file is git-ignored. `docs/figures/results.svg` is committed, but the `runs/...` inputs to `scripts/plot_results.py` aren't. The numbers in README Table 2 are claims backed by files that aren't in the repo. That's normal for ML (checkpoints are gigabytes), but say it before you're asked: "the run artifacts are local; the figure script regenerates from them."
- `experiments/*.jsonl` ignores the cross-run index (Module 3). The pattern is unanchored, so it would also match `src/model_training/experiments/*.jsonl`. No such files exist, so it's harmless.
- `.venv/` is listed, but the README creates both venvs at `../.venv` and `../.venv-vla`, *outside* the repo. The entry is defensive, so the missing `.venv-vla/` entry isn't a real gap. (I raised it as a possible gap in my first survey; on reading the README's install table it isn't one.)
- `docs/CODE_REVIEW.md` is deliberately excluded: there's an internal audit the public repository doesn't contain. Several code comments read like responses to it. For example, `trainer.py`'s crash handler says "A crash here ... previously left the run directory in an undetectable half-finished state", and `registry.py`'s `get` docstring says "returning the first match returned the oldest". Those are fixed bugs narrated in comments. In an interview this is useful: "I ran an adversarial review and fixed what it found; the comments record the before-state."

1.6 The package surface: the `__init__.py` files

There is no `src/model_training/__init__.py`

```
$ ls src/model_training/__init__.py
ls: cannot access 'src/model_training/__init__.py': No such file or directory
```

Every *subpackage* has an `__init__.py`, but the top-level `model_training` doesn't. So `model_training` is an **implicit namespace package** (PEP 420).

- *Probably why:* it was never needed, and nothing reads `model_training.__version__`. `collect_environment` gets the version from installed metadata via `importlib.metadata.version("vla-finetune")` instead.
- *Alternative:* an `__init__.py`, which costs one empty file.
- *Cost:* a namespace package **merges** every directory named `model_training` on `sys.path`. Any other installed project with a top-level `model_training` directory, which is a generic name, would silently interleave with this one, and a `model_training/cli.py` from the wrong tree could be imported. It's also a less common setup, which some tooling (mypy without `namespace_packages`, older coverage configs) handles worse. This looks accidental rather than designed; nothing in the code or docs mentions it.

The nine subpackage `__init__.py` files are re-export façades

Each one re-exports a curated public API with an explicit `__all__`, for example:

```
# src/model_training/deployment/__init__.py
from .artifact import ModelArtifact, build_model_artifact
from .loader import DeployedModel, DeploymentHealthError, LatencyBreakdown
from .registry import ModelRegistry

__all__ = ["DeployedModel", "DeploymentHealthError", "LatencyBreakdown",
           "ModelArtifact", "ModelRegistry", "build_model_artifact"]
```

- **Pattern:** *Façade* at the package level. Callers write `from model_training.deployment import DeployedModel` and don't depend on the file layout.
- **Why:** module files can be reorganized without breaking importers.
- **Cost 1: being exported is not being used.** The façades make names look first-class. Grep shows several exported names that **nothing in `src/` or `scripts/` calls**:

Exported name	Used by `src/` / `scripts/` ?	Used by tests?
`deployment.ModelRegistry`	**no** (the runner writes models via `ExperimentIndex.record_model`, not the registry)	yes, `test_deployment.py`
`deployment.DeployedModel`	**no** (only named in a model-card string)	yes
`experiments.run_data_fraction_sweep`	**no** (no CLI command calls it)	**no** (verified in Module 11: no test references it)
`reports.build_model_dataset_matrix`	**no**	no

These are covered properly in Modules 9, 10 and 11. For now the point is that **the public API is wider than the wired system.**

- **Cost 2: eager imports defeat laziness.** Importing a subpackage executes every module it re-exports. `from model_training.experiments import ExperimentRunner` imports `runner.py`, which at module top imports `..deployment.artifact`, which at module top does `from rde.model_adapters.models import SmolVLAPolicy`. So *any* import of the runner imports the engine's SmolVLA module. The runner's careful lazy `from ..training.trainer import Trainer` inside `if config.training.model == "smolvla":` **saves nothing on the rde side**, because `deployment/artifact.py` has already pulled in the same `models` module. It works in the light venv only if the engine's `models.py` itself imports torch lazily. The trainer comment ("`import torch # safe: policy.load() above already required it`") implies it does, and the README says `test_experiment_runner.py` passes without torch. **I couldn't verify this without the engine.** If the engine ever adds a top-level `import torch` to `models.py`, the whole light test suite breaks, and nothing in this repo would show why.

The CLI is the one place laziness is done right

`cli.py` imports only `typer`, `yaml` and `rich` at the top. Every `rde` and `model_training` import sits inside a command function, so `mtrain --help` and `mtrain rollout` don't load the engine at all. That's the import-light discipline applied where it pays off most: a CLI should start fast and fail only for the command you actually ran. Module 12 covers it.

1.7 The two-environment design (from the README, partly verified)

The README (lines 176–177) and `docs/ARCHITECTURE.md` (lines 143–144) describe:

Venv	Contents	What runs there
`../.venv` (light)	engine core plus this package, no torch	config, profiling, interventions, tracking, reports, CLI, unit and integration tests, baselines
`../.venv-vla` (full)	`episodic-data-engine[vla]` (torch, lerobot)	real fine-tuning, checkpoint evaluation, the smoke test

What the code confirms: the smoke test begins `torch = pytest.importorskip("torch")` and `pytest.importorskip("lerobot")`, so it skips cleanly in the light venv. The integration test docstring says it "Uses the baseline models (no torch needed)". The runner test uses `"training": {"model": "mean_action"}` precisely so that "the whole runner ... without requiring torch/lerobot" runs.

Decision: two environments rather than one.

- *Why:* torch plus lerobot is gigabytes, and CUDA builds are platform-specific (`docs/IMPLEMENTATION_PLAN.md` line 254 records reinstalling torch with CUDA). Fifty-two tests that need no GPU shouldn't pay for that.
- *Alternative:* one environment with torch always installed, plus test markers.
- *Cost:* two environments drift independently, and **nothing tests that the light environment stays light**. No test asserts `"torch" not in sys.modules` after importing the runner. The design is a convention, not an enforced invariant.

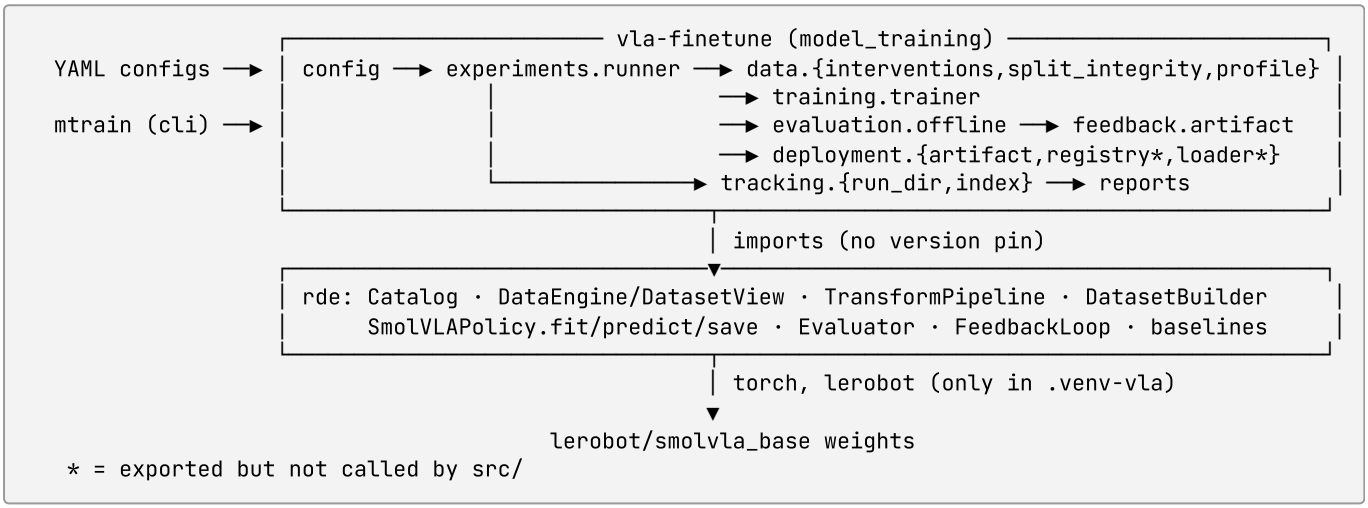
1.8 Honest gaps to own before an interviewer finds them

1. **The training step isn't in this repository.** `SmolVLAPolicy.fit`, batch construction, the optimizer and the loss all live in `rde.model_adapters.models`. *Consequence:* claiming "I wrote the fine-tuning loop" is false for this repo. What this repo owns is the run around it (Module 6).
2. **Nothing can be run from this checkout.** The engine isn't present, `[tool.uv.sources]` needs a sibling directory, and plain `pip` ignores that table. *Consequence:* a reviewer can't reproduce anything without reconstructing your directory layout.
3. **No lockfile, floors-only versions, and an unpinned engine.** *Consequence:* reproducibility depends on after-the-fact `environment.json` and `git.json` capture (Module 3), which records drift but doesn't prevent it.
4. **Coupling to the engine's private API.** `tests/smoke/test_smolvla_smoke.py` imports `_policy_features_from_contract` and `_dataset_stats_from_contract`. *Consequence:* an engine refactor that's legitimate from the engine's side breaks this repo's tests, and the missing version constraint means nothing warns you.
5. **No CI.** *Consequence:* the "52 tests pass" claim is a statement about the author's machine on some date. It isn't continuously enforced.
6. **The top-level package has no `__init__.py`** (implicit namespace package under a generic name). *Consequence:* there's a silent import-merging risk with any other `model_training` on the path.
7. **Light-environment laziness is unenforced** and partly illusory. The runner's lazy trainer import is undone by `deployment/artifact.py`'s eager import. *Consequence:* a top-level torch import added in the engine would break the light suite with no local cause.
8. **Unused declarations.** `allow-direct-references = true` (no direct references), `pytest-cov` (never invoked), `psutil` `ImportError` fallbacks (it's a hard dependency). *Consequence:* small, but each one is a "why is this here?" you should answer as "vestigial", not invent a reason for.
9. **The public API is wider than the wired system** (`ModelRegistry`, `DeployedModel`, `run_data_fraction_sweep`, `build_model_dataset_matrix`). *Consequence:* the package surface overstates

what the pipeline actually exercises.

10. **Undeclared direct torch dependency.** `trainer.py` and `run_dir.py` import torch, but this package never declares it. It's deliberate, delegated to the engine's extra, and the README explains it, but the metadata alone is incomplete.
11. **The `pyproject` author and repository URL point elsewhere.** *Consequence:* you need a one-line provenance story ready. *Correction added during Module 3:* README line 298 states the SO-100 fine-tuning runs were performed by `AdityaKishore0698` and the ALOHA run by `chytra3`. The accurate framing is therefore a collaborative project: one author owns the codebase, you ran the SO-100 experiments and someone else ran ALOHA. Claim exactly that, and be ready for "which code did you write?".

The shape to carry into Module 2



Drill — answer from memory, files closed

1. What's the difference between offline evaluation and closed-loop rollout, and which one does this project do? Why can a model with low offline MSE still fail on the robot?
2. *Why didn't you just* put torch in this package's dependencies, or make one environment for everything?
3. Where does the gradient step actually happen, and what exactly does this repository own about training?
4. *What breaks in production* if a teammate runs `pip install -e .` in a fresh clone?
5. *Interviewer framing:* "Walk me through the dependency structure of this project. Is there anything about it you'd change?"

Appendix A — Model answers to the drill

A1. Offline vs closed-loop. "Offline evaluation feeds the model recorded observations from held-out demonstrations and compares its predicted action with what the human actually did, so it's regression error, in normalized action units here. Closed-loop rollout lets the policy drive a robot or simulator and measures whether the task succeeds. This project only does offline evaluation. There's no simulator or robot attached, and the rollout command writes 'unavailable' instead of inventing a number. Low offline MSE can still fail in closed loop for two reasons. First, compounding error: a small deviation puts the robot into states the demonstrations never visited, and the policy has never seen how to recover. Second, averaging: MSE is dominated by the many easy frames, and a policy can be accurate on average and wrong on the few frames that decide success, like the moment of grasp. That's why I report tails (p99 and max) and per-phase error rather than just the mean, and why I never call the tolerance metric a success rate."

Why that answer lands: it names both mechanisms (distribution shift and averaging) instead of saying "sim-to-real is hard", and it ties them to concrete design choices in the codebase. It also shows you know the limits of your own number before the interviewer has to point them out.

A2. Why not declare torch here or use one environment? "Torch and lerobot are only needed by the SmolVLA adapter, which lives in the engine, so the engine owns that dependency through its `vla` extra. One owner, and no chance of two packages pinning conflicting torch builds. The two environments exist because about 52 of the tests and most of the commands (config, profiling, interventions, reports, baselines) don't need a GPU stack, and a CUDA torch install is several gigabytes and platform-specific. The cost is that it's a convention, not an invariant. Nothing asserts the light environment stays light, and in fact the runner transitively imports the engine's model module, so this relies on the engine importing torch lazily. I'd add a test that imports the runner and asserts torch isn't in `sys.modules`. I'd flip to one environment the moment CI runs on a GPU image anyway."

Why that answer lands: it gives the reason, then names the cost without being asked, then gives the concrete enforcement fix and the condition under which you'd abandon the design. That's the "no, until X" shape.

A3. Where the gradient step happens. "The step itself (batch construction, forward, loss, backward, optimizer, warmup) is `SmolVLAPolicy.fit` in the data engine's `model_adapters` package. I extended that class there because it has to build feature shapes and normalization stats from the engine's training contract. This repository owns the run around the step. It passes an `on_step` callback that does periodic validation on the val split only, periodic checkpointing with verification, pruning that never deletes the best checkpoint, a forced final checkpoint, a crash marker, peak memory tracking, and freeing the GPU before the evaluation model loads. So the procedure is here and the mechanics are there."

Why that answer lands: it's precise about ownership and doesn't overclaim, which interviewers test for directly. And it turns "I didn't write the loop here" into a clean separation of mechanism and procedure, which is a real design idea.

A4. What breaks with `pip install -e .`. "The engine dependency is declared by bare name, and the only thing pointing it at the local checkout is `[tool.uv.sources]`, which only uv reads. Plain pip would go to PyPI for `episodic-data-engine`. That either fails, or worse, installs whatever has that name. Even with uv it needs the engine checked out as a sibling directory, and the runner separately assumes that layout to find the engine's git commit. So a fresh clone isn't installable on its own. The fix is either publishing the engine with a version and pinning a compatible range, or a submodule plus a lockfile, and CI that does exactly the fresh-clone install so this is tested and not just documented."

Why that answer lands: it shows you know the difference between pip and uv semantics, spots the dependency-confusion risk, and ends on a fix that's verified by CI rather than by documentation.

A5. "Walk me through the dependency structure. Anything you'd change?" "Two layers. The engine owns data: versions, lineage, transforms, the builder that assigns splits, the evaluator, failure detection, and also the SmolVLA adapter including `fit`. This package owns experimental procedure: a validated config, resolving an intervention into a lineage-tracked dataset version, the training cadence and checkpoint selection, stratified evaluation against baselines, and packaging everything into a run directory and an index. All rde imports in the CLI are lazy, so commands start fast. Three things I'd change. First, the model adapter living in the data engine inverts the dependency. It's fine with one model, but with a second model family I'd pull adapters into their own package. Second, the engine is unpinned and the smoke test imports two of its private functions, so I'd pin a version range and promote those functions to public. Third, I'd add a lockfile and CI that checks out both repos, because right now 'the tests pass' isn't continuously true."

Why that answer lands: it describes the structure in terms of responsibility, not files, then volunteers three ranked, concrete changes, each tied to a trigger. Being critical of your own architecture is what separates a designer from someone who only implemented it.

Module 1 Summary — the one page to revise

What this module is. The vocabulary of VLA fine-tuning and offline evaluation, and the shape of the codebase: a thin procedural layer (`model_training`, about 2,700 lines) over a separately installed data engine (`rde`) that also owns the model and its training step.

Five claims to make cold

1. All evaluation is offline (predicted vs demonstrated action, normalized units). Rollout is explicitly reported as unavailable.
2. Splits are episode-level, to prevent near-duplicate-frame leakage.
3. `SmolVLAPolicy.fit` lives in the engine. This repo owns cadence, selection, checkpointing and packaging.
4. Torch is owned by the engine's `vla` extra. A light venv runs 52 tests without it, by convention and not by enforcement.
5. The repo isn't installable alone: there's a sibling-directory `uv` source, no lockfile, no CI, and an unpinned engine.

Decisions

Chose	Over	Because	Cost
Model adapter and `fit` inside the engine	adapter in this repo	must share the training contract's shapes and normalization	data layer owns the model; hardest code isn't here
torch via `episodic-data-engine[vla]`	a `vla` extra here	single owner of the heavy stack	`pyproject` alone can't build a training env; undeclared direct torch import
Two venvs	one env	52 tests don't need CUDA	unenforced "lightness"
`[tool.uv.sources]` sibling path	published, pinned engine	co-development with editable engine	pip-incompatible; layout assumption repeated in code
Re-export façades with `__all__`	deep imports	stable public API	eager imports defeat laziness; API wider than what's wired
`mtrain` script name	`model`	avoids `rde model` collision	none

Volunteer unprompted: no CI; no lockfile; private-API coupling; run artifacts are git-ignored so README numbers aren't reproducible from the repo; no top-level `__init__.py`; authorship and URL point elsewhere.

Numbers and names: about 2,700 lines in `src`; 49 unit, 3 integration and 4 smoke tests (README's 52 matches); Python ≥3.10 declared, 3.11 used; `lerobot/smolvla_base`; datasets `so100_pickplace` (6-DoF, 2 cameras, 30 fps) and `aloha_coffee` (14-DoF, 4 cameras, 50 fps); console script `mtrain`; build backend `hatchling`.

Sentences worth memorizing

- "The engine owns the mechanics of one gradient step; this repo owns the procedure around it."
- "Every number I report is offline. The code refuses to produce a rollout number it can't measure."
- "Light-environment import discipline is a convention I'd turn into a test."
- "The run artifacts are deliberately not versioned; the figure script regenerates the README figure from them."

Module 2 — Configuration as the unit of reproducibility

Files covered: `src/model_training/config/schema.py`, `src/model_training/config/loader.py`, `configs/base.yaml`, `configs/smolvla/so100_pickplace.yaml`, `configs/experiments/baseline.yaml`, `configs/experiments/baseline_extended.yaml`, `configs/experiments/reduced.yaml`, `configs/experiments/imbalanced.yaml`, `configs/experiments/rebalanced.yaml`, `configs/experiments/aloha_coffee_baseline.yaml`

Where we are: Module 1 established that this repository owns *procedure* and the engine owns *mechanism*. Procedure starts here. An `ExperimentConfig` is the object every later stage reads, and its fully resolved form is what gets written next to every run. If the config can't be trusted to mean what it says, nothing downstream can be either. That makes this module the right place to test the project's central claim: "every experimental parameter that matters lives here."

The one-line thesis: Configuration is a two-layer YAML deep-merge validated by Pydantic. The validators do enforce *semantic* rules: an imbalanced or rebalanced experiment can't run without a declared target. But the schema accepts unknown keys silently, validates several fields that nothing consumes, and has one imbalanced path that silently becomes a full-data run. So the "single object a run is reproducible from" is reproducible, but it isn't always truthful.

2.1 Prerequisites — the absolute basics

- **YAML.** A human-readable data format. Mappings (`key: value`), lists (`[a, b]` or `- a`), scalars. Two features appear here:
  - `>` (a *folded block scalar*): the following indented lines are joined with spaces into one string. Used for `hypothesis:` and `objective:`.
  - `null`: Python `None`.
- **`yaml.safe_load` vs `yaml.load`.** `safe_load` builds only plain types (dicts, lists, strings, numbers). `load` with the full loader can construct arbitrary Python objects from tags like `!!python/object`. That's a code-execution vector if a config file is untrusted. This code uses `safe_load` and `safe_dump` everywhere.
- **Deep merge.** Combining two nested dicts so that nested dicts are merged key by key, and anything else (scalars, lists) in the override *replaces* the base value.
- **Pydantic (v2).** A library that turns a class with type annotations into a validator. `Model.model_validate(dict)` checks types, applies defaults, coerces where allowed, and raises `ValidationError` on failure. Key terms:
  - `Field(default=..., ge=0.0, le=1.0)`: a default plus numeric bounds (greater-or-equal, less-or-equal).
  - `Field(default_factory=lambda: [...])`: builds a fresh mutable default for each instance, avoiding Python's shared-mutable-default bug.
  - `Literal["a", "b"]`: the value must be exactly one of those.
  - `@model_validator(mode="after")`: a method that runs *after* field validation, on the constructed object. It can check cross-field rules and, as here, mutate fields.

- **Lax mode** (the default): sensible coercions are allowed, such as `"3"` → `3` for an `int` field or dict key. An `int` is **not** coerced to `str` in v2.
- `extra` **behaviour:** by default (`extra="ignore"`) unknown keys are **silently dropped**. `extra="forbid"` makes them errors. Nothing in this schema sets it.
- `model_dump(mode="json")`: export to plain JSON-compatible types (dict keys become strings, and so on).

- **Configuration hierarchy / layered config.** Shared defaults in one file, per-experiment overrides in another. This is the pattern Hydra or OmegaConf implement in fuller form.
- **Resolved config.** The final config after merging and after defaults are applied. Persisting the resolved form means a run doesn't depend on code defaults that might change later.
- **Content-addressed ID vs time-stamped ID.** A content-addressed ID is a hash of the content, so the same content gets the same ID. A time-stamped ID mixes in a clock value, so it's unique per creation.

2.2 The loader: merge first, validate once

```
def _deep_merge(base: dict[str, Any], override: dict[str, Any]) -> dict[str, Any]:
    out = dict(base)
    for k, v in override.items():
        if k in out and isinstance(out[k], dict) and isinstance(v, dict):
            out[k] = _deep_merge(out[k], v)
        else:
            out[k] = v
    return out
```

- **What:** a recursive merge. Dicts merge key by key; everything else is replaced.
- **Why** `dict(base)`**:** a shallow copy, so the caller's base dict isn't mutated at the top level. Nested dicts that the override never touches are *shared* between `out` and `base`, which is safe only because nothing mutates them afterwards.
- **Lists replace, not append.** That's the right semantics for this project. `aloha_coffee_baseline.yaml` sets `slices: [per_length_bucket]` to *replace* base's `[per_task, per_length_bucket]`, and the comment explains why (one task, so `per_task` would duplicate `overall`). Appending would make it impossible to remove a slice.
- **A key can't be deleted.** An override can set `null`, but it can't remove a key. That's fine here: every schema field has a default, so `null` and absent differ only for fields whose default isn't `None`.
- **Alternative:** Hydra or OmegaConf, which give interpolation, command-line overrides (`training.lr=3e-5`), and structured-config validation.
- **Cost of the hand-rolled version:** no CLI overrides, so changing a learning rate means editing or copying a YAML file. That's arguably a *feature* for an experiment log, since every run has a file.
- **When the answer flips:** once sweeps over hyperparameters (not just data fractions) are routine.

```
def load_yaml(path: str | Path) -> dict[str, Any]:
    path = Path(path)
    if not path.exists():
        raise FileNotFoundError(f"no config file at {path}")
    with path.open() as f:
        data = yaml.safe_load(f) or {}
    if not isinstance(data, dict):
        raise ValueError(f"{path} must contain a YAML mapping at the top level")
    return data
```

- `or {}` handles an empty file: `safe_load("")` returns `None`. Without it, the `isinstance` check would raise a misleading "must contain a mapping".
- The `isinstance(data, dict)` check catches a file whose top level is a list or a bare string. Without it, `_deep_merge` would fail later with an `AttributeError` that doesn't name the file. It's a good defensive branch with a specific error.

```
def load_experiment_config(path, base_path=None) -> ExperimentConfig:
    path = Path(path)
    data = load_yaml(path)

    if base_path is None:
        candidate = path.parent.parent / "base.yaml"
        base_path = candidate if candidate.exists() else None

    if base_path is not None:
        base_data = load_yaml(base_path)
        data = _deep_merge(base_data, data)

    data["source_config_path"] = str(path)
    return ExperimentConfig.model_validate(data)
```

Decision: find the base by directory convention (`<grandparent>/base.yaml`).

- *What:* `configs/experiments/x.yaml` picks up `configs/base.yaml` automatically.
- *Why:* zero ceremony for the layout the project ships.
- *Alternative:* an explicit `base:` key inside each experiment file, or an explicit `--base` flag.
- *Cost:* **what a config file means depends on where it sits on disk.** Copy `baseline.yaml` to `/tmp/` and load it: the grandparent of `/tmp/baseline.yaml` is `/`, there's no `/base.yaml`, and the base is skipped. `baseline.yaml` alone has no `dataset:` block, so this fails loudly (`dataset` is required), which is the lucky case. A copied file that *does* declare `dataset:` would load and run silently with **code defaults** (`steps: 200`, `checkpoint_every: 50`, `batch_size: 32` for eval) instead of base.yaml's values (`1000`, `100`, `16`). The resolved config records which values were used, so it's recoverable, but nothing warns you.

Merge-then-validate is the right order, and the module docstring states it. The *merged* dict is what Pydantic sees, so a base value plus an override value are validated together. A validator like "fractions sum to one" sees the final numbers, not each file alone.

Doc vs code: `--base` doesn't exist. `configs/smolvla/so100_pickplace.yaml`'s header says:

> include its contents in an experiment file's `training:` block, or point --base at it explicitly

The only CLI caller is `cli.py:67`, `cfg = load_experiment_config(config)`, and `mtrain train` has options `--config` and `--seed` only. **There's no `--base` flag.** `base_path` is reachable only from Python. And `load_experiment_config` takes *one* base, so the "second model-specific layer" that file talks about can't be stacked on top of `base.yaml` at all. The file is effectively documentation. Grep shows nothing loads it. Its values also duplicate `base.yaml`'s exactly, so even if it were wired up it would change nothing.

`source_config_path` overwrites the user. It's assigned after merging, so a YAML `source_config_path:` is overwritten. Good: provenance can't be spoofed through this one field. But see 2.6 for the two identity fields that *can* be.

2.3 `DatasetRef` and the "identity, not a path" decision

```
class DatasetRef(BaseModel):
    """The dataset version an experiment starts from -- an identity, not a path."""
    dataset_id: str
    version: str = Field(..., description="e.g. 'clean/v1' -- the input to the intervention/build step")
```

Decision: name data by `(dataset_id, version)`, never by filesystem path.

- *Why:* a path can be overwritten in place; a catalog version is immutable (the engine's claim, inherited). This is what makes "this model was trained on `so100_pickplace@training/v6`" a stable statement.
- *Alternative:* `data_dir: /data/so100/` (the common LeRobot style).
- *Cost:* you can't train on anything that hasn't been ingested into the engine first. There's no "just point at a folder" path. That's deliberate.

The input version is `clean/v1`, not `training/v1`. `base.yaml`'s comment explains it:

```
dataset:
  dataset_id: so100_pickplace
  version: clean/v1   # the intervention/build step consumes this; NOT training/v1,
                      # which is itself an *output* of that step
```

The config names the version the intervention *starts from*. Every run then produces its own `training/vN` (Module 4). A consequence to have ready: **two runs with identical configs produce two different `training/vN` versions.** The README's lineage example is `training/v6`, which tells you at least six had been built.

2.4 `DatasetIntervention`: semantics enforced, mostly

```
kind: Literal["full", "fraction", "reduced", "imbalanced", "rebalanced", "custom"] = "full"
fraction: float | None = Field(default=None, ge=0.0, le=1.0)
variable: str | None = None
variable_value: Any = None
episode_ids: list[int] | None = None
episode_weights: dict[int, float] | None = None
oversample_factor: float = 3.0
seed: int = 0
reason: str = ""
```

This is a **tagged union flattened into one class**: `kind` is the discriminator, and which other fields matter depends on it.

- *Alternative:* Pydantic's discriminated unions, i.e. one class per kind (`FullIntervention`, `FractionIntervention(fraction: float)`, and so on), selected by `Field(discriminator="kind")`. Each class would declare *only* its fields, so `kind: full, fraction: 0.3` would be rejected as an unknown field (with `extra="forbid"`).
- *Cost of the flat form:* irrelevant fields are accepted and ignored. `kind: full` with `episode_ids: [1, 2]` is valid, and `resolve_episode_subset` returns *all* episodes for `full` before it ever looks at `episode_ids` (Module 4). It's a silent no-op on a field the user explicitly set.

The validator:

```
@model_validator(mode="after")
def _check_shape(self) -> "DatasetIntervention":
    if self.kind in ("fraction", "reduced") and self.episode_ids is None and self.fraction is None:
        if self.kind == "reduced":
            self.fraction = 0.5
        else:
            raise ValueError("intervention.kind='fraction' requires 'fraction' or an explicit 'episode_ids'")
    if self.kind == "imbalanced" and self.episode_ids is None and self.variable is None:
        raise ValueError("... requires 'variable' (from a dataset profile) or an explicit 'episode_ids' "
                         "-- the weak dimension must not be presumed")
    if self.kind == "rebalanced" and self.variable is None and not self.episode_weights:
        raise ValueError("... requires 'variable' ... or an explicit non-empty 'episode_weights'")
    if self.kind == "custom" and self.episode_ids is None:
        raise ValueError("intervention.kind='custom' requires an explicit 'episode_ids'")
    return self
```

Line by line:

- **`reduced` defaults `fraction` to 0.5.** "Reduced" means "the project-standard ~50%", so `reduced` is just `fraction` with a default. A *mutating* after-validator is unusual but legal, and Pydantic v2 allows assignment inside `mode="after"`. The cost: `0.5` appears as a magic number in **two** places. The other is `resolve_episode_subset`'s `intervention.fraction if intervention.fraction is not None else 0.5`, which is unreachable for configs that went through this validator. Two sources of one default can drift.
- **`imbalanced` requires `variable` or `episode_ids`.** This encodes the design rule "the weak dimension must not be presumed": you can't ask for an imbalance without naming *along what*. It's a methodological rule enforced as a type error. That's the strongest thing in this file, and it's tested (`test_imbalanced_requires_variable_or_episode_ids`).
- **`rebalanced` requires `variable` or non-empty `episode_weights`.** `not self.episode_weights` treats both `None` and `{}` as missing. That's exactly why the shipped `rebalanced.yaml` template, with `episode_weights: {}`, **fails to load**, as its header promises and `test_rebalanced_template_fails_loudly_until_filled_in` asserts. Doc and code agree here.
- **`custom` requires `episode_ids`, but `[]` passes** (it checks `is None`, not falsiness). That asymmetry is *deliberate*: `imbalanced.yaml` is a `custom` template with `episode_ids: []`, and its header says it is "Loadable as-is ... but resolving it ... will fail loudly with 'no episodes'". It does: `build_intervention_dataset` raises `"resolved to zero episodes"` (Module 4). So the two templates fail at *different* stages, one at load and one at resolution. The headers document this correctly, but it's an inconsistency you could be asked about. The honest answer is that they're placeholders with different shapes, and both fail before any training step.

What the validator does *not* check

1. **`imbalanced` with `variable` set but `variable_value` missing passes.** Follow it through `resolve_episode_subset` (Module 4): with `variable="task_id"`, `val=None`, it keeps `e` where `task_of.get(e) != None`, which is *every episode that has a task*. With `variable="episode_length_bucket"`, `val=None`, no episode has bucket `None`, so all are kept. The subset equals the full set, so `build_intervention_dataset` sets `clean_version = input_version` and **trains a full-data run labelled `imbalanced`.** That's exactly the "silently run as a no-op" outcome the `rebalanced.yaml` header says the design forbids. Rebalanced has a guard for this (`variable_value is not None` before resolving targets, then "resolved to empty episode_weights" raises). **Imbalanced doesn't.**
2. **`variable_value: Any` isn't coerced.** Hand-write `variable: task_id, variable_value: "0"` (a string) and the comparison `task_of.get(e) != "0"` is true for every integer task id, so again a silent full run. The CLI's `build-intervention` converts `task_id` values to `int` (`value = int(variable_value)`), so the *generated* path is safe. The *hand-written* path isn't.

3. **`oversample_factor` and `episode_weights` values are unbounded.** `oversample_factor: 0` or `-1` passes validation. What a zero or negative sampling weight does inside the engine's sampler is unverified. A `torch.multinomial`-style sampler would reject negatives. Either way, it isn't caught here.
4. **`fraction` with `kind: full` (or `rebalanced`) is accepted and ignored.**

`episode_weights: dict[int, float]` and YAML string keys. `mtrain build-intervention` writes weights as `{str(e): oversample_factor}`, so YAML holds `'3': 3.0`. Pydantic lax mode coerces a numeric string dict key to `int`, so this round-trips correctly. It's correct, but it works by coercion and not by design. A non-numeric key would be a validation error, which is the right outcome.

2.5 `SplitConfig`

```
class SplitConfig(BaseModel):
    train: float = 0.8
    val: float = 0.1
    test: float = 0.1
    seed: int = 42
    mode: Literal["quantile", "independent"] = "quantile"

    @model_validator(mode="after")
    def _sums_to_one(self) -> "SplitConfig":
        total = self.train + self.val + self.test
        if abs(total - 1.0) > 1e-6:
            raise ValueError(f"train+val+test must sum to 1.0, got {total}")
        return self
```

- **The `1e-6` tolerance** is needed because `0.8 + 0.1 + 0.1` in binary floating point isn't exactly `1.0` (it's `0.9999999999999999` or `1.0000000000000002` depending on order). An exact `== 1.0` would reject the default config. Anything looser would accept typos like `0.8/0.1/0.11`.
- **Missing: non-negativity.** `train: 1.2, val: -0.1, test: -0.1` sums to 1.0 and passes. So does `val: 0.0`. What the engine's `assign_splits` does with a negative fraction is unverified. A zero val split means the trainer's validation pass scores an empty split, `val_metric` comes back missing or `None`, no checkpoint is ever "best", and selection silently falls back to the final checkpoint (Module 6). The fix is one line: `Field(ge=0.0, le=1.0)` on each, plus `val > 0`.
- **`mode: quantile` vs `independent`.** These are the engine's split-assignment modes. `base.yaml`'s comment, "exact split sizes at this episode count (48)", plus the README's 38/5/5, suggests `quantile` assigns *exact* counts (48 × 0.8 = 38.4 → 38, and 4.8 → 5 for each of val and test). `independent` would presumably draw each episode's split independently, giving sizes that vary run to run. Both are inferred from the name and the comment; the engine's code isn't here. **Why it matters at n = 48:** with independent draws, a 10% split of 48 episodes has a standard deviation of about √(48·0.1·0.9) ≈ 2 episodes, so a 5-episode test set could easily come out as 3 or 7. Exact sizes are the right choice at this scale.
- **`seed: 42` (split seed) is separate from `training.seed: 0`.** That separation is a real decision. You can vary the training seed while holding the test split fixed, which is what makes "both SO-100 runs are scored on the identical test split" possible. The cost: there are now three seeds (split, intervention, training), and the CLI's `--seed` touches **none** of them (Module 11).

The docstring says "identical semantics to `rde.transformations.ops.assign_splits`, because it *is* that call." That's honest only partly. The fractions, seed and mode are passed straight through to `DatasetBuilder.build` (Module 4), but `SplitConfig`'s validation is *this* repo's, so the two layers could disagree about what's legal (for example, if the engine accepts fractions that don't sum to 1).

2.6 `TrainingConfig`, `EvaluationConfig`, `ExperimentConfig`

Fields validated but never consumed

I grepped every `TrainingConfig` and `EvaluationConfig` field against `src/`:

Field	Consumed?	Evidence
`training.num_workers`	**no**	only in `schema.py`
`training.mixed_precision`	**no**	only in `schema.py`
`evaluation.success_tolerance`	**no**	only in `schema.py`; never passed to `Evaluator`
`training.device`	yes	`SmolVLAPolicy(..., device=tc.device)`
everything else in `TrainingConfig`	yes	passed to `policy.fit` or used in `Trainer`

Doc vs code: `docs/API_REFERENCE.md` describes `success_tolerance` as "Tolerance band defining `Evaluator`'s `success_rate` proxy." Setting it to 0.05 changes nothing. The evaluator uses its own default (the README's "below 0.1" matches the schema default by coincidence, or because both copy the engine's value; I can't verify which). **These three fields make the config lie:** a resolved config that says `mixed_precision: true` describes a run that used full precision.

The comment pointing at a module that doesn't exist

```
#: Metric used for checkpoint selection. Must be computed on validation,
#: never test -- enforced by model_training.experiments.selection, not here.
selection_metric: str = "mse"
```

`model_training/experiments/` contains `__init__.py`, `runner.py` and `data_fraction.py`. **There's no `selection` module.** The val-only rule *is* enforced, but structurally, in `Trainer.on_step`, which hard-codes `split="val"` (Module 6). So the guarantee holds and the citation is stale. Say it exactly that way.

`selection_metric: str` is a free string. If you name a metric the evaluator doesn't produce (`"msee"`), `result.metrics.get(ec.selection_metric)` returns `None`, no checkpoint ever becomes best, and the trainer silently falls back to `final` (Module 6). A `Literal[...]` of known metric names would catch it at load.

`training.model: str`, the most dangerous free string

`model: str = "smolvla"`. The runner branches on `config.training.model == "smolvla"`, and *anything else* skips training, appends a limitation note, and still completes a "successful" run that scores baselines only. That path is intended for `mean_action` (the runner unit test uses it). But a typo, `model: SmolVLA` or `smol_vla`, produces a run directory, a report, a summary and an index row, with **no fine-tuning and exit code 0.** The only trace is one line under "Limitations" in the report. `Literal["smolvla", "mean_action", "nearest_neighbor"]` would turn this into a load-time error.

Two sources of defaults

Field	`schema.py` default	`base.yaml` value
`training.steps`	200	1000
`training.checkpoint_every` / `validate_every`	50 / 50	100 / 100
`training.warmup_steps`	0	50
`evaluation.batch_size`	32	16
`evaluation.slices`	`["per_task"]`	`["per_task", "per_length_bucket"]`

Which one applies depends on whether `base.yaml` was found (2.2). Persisting the resolved config makes it *auditable*, but a reader of `schema.py` gets the wrong mental model of a "default" run. Pick one home, ideally the YAML, with schema defaults kept only for fields that are genuinely optional.

Identity stamping

```
experiment_id: str | None = None
created_at: str | None = None
source_config_path: str | None = None

@model_validator(mode="after")
def _stamp_identity(self) -> "ExperimentConfig":
    if self.created_at is None:
        self.created_at = _dt.datetime.now(_dt.timezone.utc).isoformat()
    if self.experiment_id is None:
        basis = f"{self.experiment_name}:{self.dataset.dataset_id}:{self.dataset.version}:{self.created_at}"
        digest = hashlib.sha256(basis.encode()).hexdigest()[:10]
        self.experiment_id = f"{self.experiment_name}-{digest}"
    return self
```

- **What:** an ID like `baseline_full-3f9a1c0b2e`. The readable prefix is for humans; the hash makes it unique.
- **Why `created_at` is in the hash:** so two runs of the same experiment get different IDs. Without it, every run of `baseline_full` on `clean/v1` would collide in the index.
- **The hash covers name, dataset and version only, not the hyperparameters.** So the ID isn't content-addressed. Two configs differing only in learning rate get IDs that differ *only because of the timestamp*. It's a unique run identifier wearing a content hash's clothes. Calling it "deterministic" (as the test name `test_experiment_id_is_deterministic_given_same_inputs` does) is true only when `created_at` is pinned, which never happens from YAML.
- **Alternative:** `uuid4()` for uniqueness, plus a *separate* `config_hash = sha256(resolved_dict minus identity fields)`. The second would let you ask "have I already run exactly this config?", which nothing in the system can answer today.
- **Ten hex characters** is 40 bits. Collisions are irrelevant at this scale, so that's fine.
- **Doc vs code: "not user-editable."** The field comment says "Set once at load time; not user-editable via YAML," and API_REFERENCE repeats it. **Nothing prevents it.** `experiment_id: anything` in a YAML is accepted, the `is None` guard skips stamping, and the run is indexed under the user's string. Two runs could then share an ID. It's fixable with a `mode="before"` validator that pops these keys, or with `Field(exclude=True)` plus `init=False` semantics. Meanwhile, `data_fraction.py` *relies* on re-stamping by explicitly setting both to `None` before re-validating (Module 11). That pattern shows the author knew the guard exists.

```
def resolved_dict(self) -> dict[str, Any]:
    return self.model_dump(mode="json")
```

`mode="json"` ensures the dump contains only JSON-safe types, so `yaml.safe_dump` won't fail on, say, an `int`-keyed dict (its keys become strings). The persisted `config.resolved.yaml` therefore has **string** keys for `episode_weights`, and re-loading it coerces them back to `int`. It round-trips through coercion again.

`extra` is never forbidden, the root gap

None of the six models sets `model_config = ConfigDict(extra="forbid")`. So:

```
training:
  learning_rat: 3.0e-5     # typo
```

loads cleanly. `learning_rate` keeps its inherited `1.0e-4`, and the run trains at the *wrong* learning rate. Worse, **the resolved config drops the typo**, so the run's own record shows `learning_rate: 1e-4` with no trace that you ever asked for something else. For a project whose schema docstring opens with "Every experimental parameter that matters lives here", this is the single most important one-line fix in the codebase.

2.7 The shipped configs, read as experiments

File	Kind	Runnable as-is?	Notes from the code
`baseline.yaml`	`full`	yes	18 lines; inherits everything else from base
`baseline_extended.yaml`	`full` + `resume_from`	**only on the author's machine**	`resume_from: runs/2026-09-03/baseline_full/seed_0/checkpoints/step_1000`, relative to the *current working directory*, a dated path, and git-ignored (Module 1)
`reduced.yaml`	`reduced`, fraction 0.5, seed 0	yes	`test_reduced_config_loads_with_fraction` asserts the 0.5
`imbalanced.yaml`	`custom`, `episode_ids: []`	loads; fails at resolution	intentional template
`rebalanced.yaml`	`rebalanced`, empty weights	**fails at load**	intentional template; test asserts it
`aloha_coffee_baseline.yaml`	`full` on `aloha_coffee`	yes, if ingested	overrides `slices` to drop `per_task`

Points to slow down on:

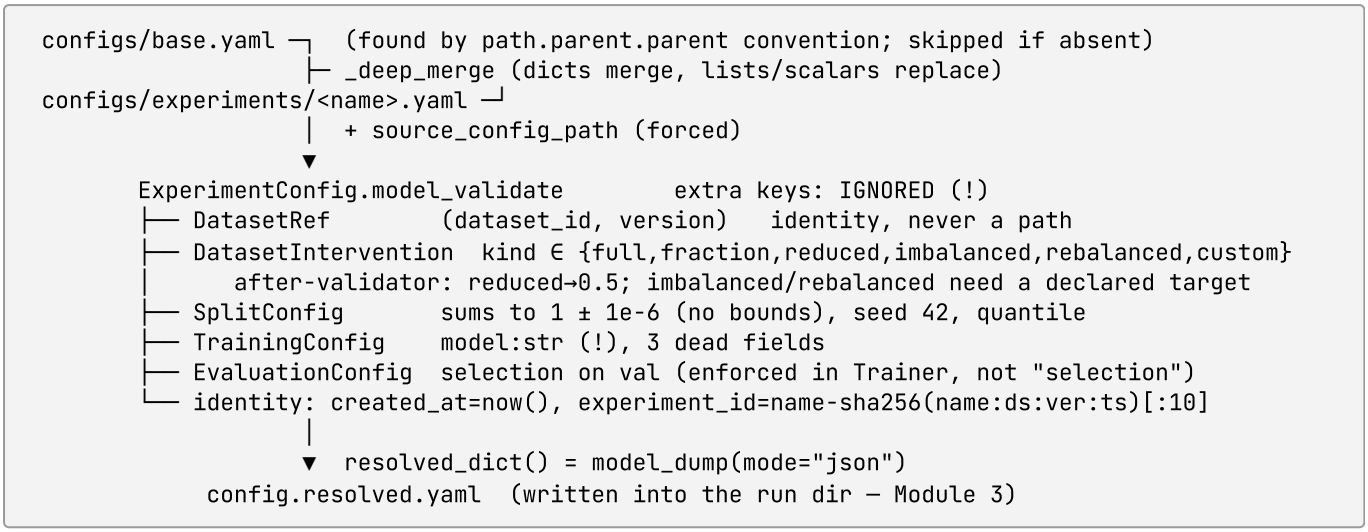
- **`baseline_extended.yaml` resumes weights, not optimizer state.** Its header says the reduced LR is "freshly-warmed-up" (`warmup_steps: 20`, `learning_rate: 3.0e-5`). That's consistent with `Trainer` writing `"optimizer_state_saved": False` into each checkpoint (Module 6). The optimizer's moment estimates are lost on resume, which is *why* a fresh warmup is needed: Adam-style optimizers with zeroed second moments take large steps at first. The comment and the code agree, and it's a genuinely good decision. Learning-rate *schedule* semantics after warmup (API_REFERENCE says "cosine decay") live in the engine and are unverified.
- **The extended run builds a *new* `training/vN`** (`intervention: kind: full`), not the same one baseline_full used. "The test set is identical" therefore depends on `DatasetBuilder` assigning the same splits given the same episodes, seed (42) and mode. That's plausible for a deterministic seeded builder, and the README asserts it, but it isn't checked by any code here. A test comparing `split_integrity.json` split membership across the two runs would prove it. The README shows split *sizes*, not membership.
- **`aloha_coffee_baseline.yaml`'s hypothesis says "generalizes without code changes".** That's a claim about the engine's adapter and `fit()`, which "read camera/action shape from the dataset contract". The smoke test `test_policy_features_concatenate_every_inline_modality_like_state_adapter_does` exists because ALOHA's two inline modalities (28 state dims) exposed a mismatch. So it generalized *after* a fix, which is a better story to tell than "without changes".
- **`baseline_extended.yaml` duplicates many base values** (`model`, `pretrained_checkpoint`, `batch_size`, `checkpoint_every`, ...) that the deep merge would supply anyway. It's harmless, but it contradicts

base.yaml's own guidance that "only the fields that differ need to appear there."

2.8 Honest gaps to own before an interviewer finds them

1. **Unknown keys are silently ignored, and the resolved config erases the evidence** (no `extra="forbid"` on any model). *Consequence:* a typo trains with the wrong hyperparameter, and the run record looks correct.
2. `training.model` **is a free string, and any value except** `"smolvla"` **silently skips training.** *Consequence:* a case typo produces a complete, exit-0 run with no fine-tuning, flagged only in the report's Limitations.
3. **Three dead fields:** `num_workers`, `mixed_precision`, `success_tolerance`. *Consequence:* the persisted config misdescribes the run, and API_REFERENCE documents an effect `success_tolerance` doesn't have.
4. `imbalanced` **with** `variable` **but no (or a wrongly typed)** `variable_value` **silently becomes a full-data run labelled imbalanced.** *Consequence:* the "never silently no-op" guarantee that rebalanced enforces doesn't hold for imbalanced.
5. `experiment_id` **and** `created_at` **are user-settable despite the "not user-editable" comment.** *Consequence:* ID collisions in the index are possible from a copied resolved config.
6. `experiment_id` **isn't content-addressed** (hyperparameters aren't in the hash). *Consequence:* there's no way to detect "already ran this exact config".
7. **The schema comment cites** `model_training.experiments.selection`**, which doesn't exist.** *Consequence:* a stale reference. The guarantee itself is real but lives in `Trainer`.
8. **Split fractions aren't bounded to [0, 1] and val can be 0.** *Consequence:* a zero val split silently disables checkpoint selection.
9. **The base-config location depends on directory convention, and there's no** `--base` **flag despite** `configs/smolvla/so100_pickplace.yaml` **saying there is.** *Consequence:* a config's meaning changes when the file moves, and the model-specific layer file is dead.
10. **Two sources of defaults (schema vs** `base.yaml`**) disagree** on steps, cadence, warmup, eval batch size and slices. *Consequence:* there's no single answer to "what's the default run?".
11. `baseline_extended.yaml` **hard-codes a dated, CWD-relative, git-ignored checkpoint path.** *Consequence:* it isn't runnable by anyone else, or by you after cleaning `runs/`.
12. **The flat intervention class accepts fields irrelevant to its** `kind` (`kind: full` plus `episode_ids`). *Consequence:* user intent is silently ignored.
13. **The 0.5 "reduced" default lives in two places.** *Consequence:* drift risk. The second copy is unreachable for validated configs.

The shape to carry into Module 3



Drill — answer from memory, files closed

1. Walk through what happens, field by field, when `configs/experiments/reduced.yaml` is loaded. Where does `fraction: 0.5` come from, and where else does 0.5 appear?
2. *Why didn't you just* use Hydra or OmegaConf?
3. *What breaks in production:* a teammate writes `learning_rat: 3e-5` in a new experiment file. What happens, and what does the run directory say?
4. Why are there three separate seeds, and which one does `mtrain train --seed 1` change?
5. *Interviewer framing:* "How do you make sure an experiment can't silently run as something other than what was intended?"

Appendix A — Model answers to the drill

A1. Loading `reduced.yaml`. "`load_experiment_config` reads the file, finds `configs/base.yaml` by going two directories up, and deep-merges the experiment file over the base. Nested dicts merge key by key; lists and scalars replace. So `dataset`, `splits`, `training` and `evaluation` all come from base, and `intervention`, `experiment_name` and `tags` come from the experiment. It forces `source_config_path`, then calls `ExperimentConfig.model_validate` once on the merged dict. In this file `fraction: 0.5` is set explicitly. If it weren't, the `DatasetIntervention` after-validator would set it, because `kind: reduced` with no fraction and no episode IDs defaults to 0.5. There's a second 0.5 fallback in `resolve_episode_subset` that validated configs can never reach, which is a small duplication I'd remove. Finally the identity validator stamps `created_at` and an ID of the form `reduced_data-` plus ten hex characters of a SHA-256 over name, dataset, version and timestamp."

Why that answer lands: it follows the real order of operations (merge, then validate, then stamp) and volunteers the duplicated default. That shows you've read the code path, not the docstring.

A2. Why not Hydra? "The surface I needed was small: one base layer, one override layer, and strong semantic validation, like 'an imbalanced run must name the dimension it skews'. Pydantic gives me the semantic validators, and the merge is ten lines. Hydra would add command-line overrides and multirun sweeps, but here every run is meant to correspond to a file, because the config file *is* the experiment record and carries a hypothesis and an objective. The cost is no `training.lr=...` overrides and a directory-convention base lookup that changes meaning if you move a file. I'd move to Hydra, or at least OmegaConf with structured configs, once hyperparameter sweeps became routine rather than data-fraction sweeps, which the code already handles by generating configs programmatically."

Why that answer lands: it justifies the choice by what the configs are *for* (an experiment log), admits the concrete cost, and names the trigger for switching.

A3. The `learning_rat` typo. "It loads without error, because none of the Pydantic models set `extra='forbid'`, so unknown keys are dropped. `learning_rate` keeps the base value of 1e-4, and the run trains at the wrong rate. The worse part is the record: `config.resolved.yaml` is a dump of the validated model, so the typo is gone. The run directory says `learning_rate: 1e-4`, which is true, and there's no trace that you asked for 3e-5. That's the most important one-line fix in the project: `model_config = ConfigDict(extra='forbid')` on every config model. Together with turning `training.model` into a `Literal`, it closes the two ways a config can silently mean something other than what was written."

Why that answer lands: it identifies both the failure and the *evidence-erasure*, which is the subtler point, and gives the exact fix. It sounds like someone who has debugged this kind of bug.

A4. Three seeds. "The split seed (42) decides which episodes are in train, val and test. Keeping it fixed is what lets two runs share an identical test set. The intervention seed decides which episodes a fraction draw keeps, so a reduced run is a reproducible random subset. The training seed decides sampling order and initialization noise during fine-tuning. Separating them lets me vary one source of randomness while holding the others fixed. Honestly, `mtrain train --seed 1` changes none of them. It only names the run directory `seed_1` and records 1 in the summary, while `training.seed` stays at 0. That's a real bug. The flag should set `training.seed`, and probably loop over the config's `seeds` list, which today ignores everything after the first entry."

Why that answer lands: it explains the design reason for separating seeds, then volunteers a real bug in your own code before being caught. That's the single most credibility-building move available.

A5. "How do you make sure an experiment can't silently run as something other than intended?" "At the config layer there are semantic validators. An imbalanced intervention can't load without naming the variable it skews, a rebalanced one can't load with empty weights, and split fractions must sum to one. The shipped templates are built to fail before any training step. Downstream, the resolved config, including every default, is persisted with the run. But I have to be straight about the holes I found reviewing it. Unknown keys are ignored, so typos vanish. The model name is a free string, and anything but `smolvla` skips training while still producing a complete run. Three fields, including `mixed_precision`, are validated but never used. And an imbalanced intervention with a variable but no value resolves to the full dataset. The fixes are `extra='forbid'`, `Literal` types for model and metric names, deleting or wiring the dead fields, and the same 'resolved to the full set' guard that rebalanced already has."

Why that answer lands: it leads with what works and why, then lists specific, ranked holes with specific fixes. The interviewer learns you can audit your own system, which is what the question is really probing.

Module 2 Summary — the one page to revise

What this module is. A two-layer YAML config (base plus experiment) deep-merged and validated once by Pydantic into an `ExperimentConfig` whose resolved dump is persisted with every run. Semantic validators enforce the project's methodology (declare the weak dimension); the type surface is too permissive to guarantee the config is truthful.

Five claims to make cold

1. Merge first, validate once. Lists replace, dicts merge, and the resolved config, defaults included, is persisted.
2. Data is named by `(dataset_id, version)` and never by path. The input is `clean/v1`, and each run builds its own `training/vN`.
3. Imbalanced and rebalanced interventions can't load without a declared target. "The weak dimension must not be presumed" is enforced as a validation error.
4. There are three seeds: split 42, intervention 0, training 0. The split seed is what keeps test sets identical across runs.
5. The biggest gap is no `extra="forbid"`. Typos vanish, and the resolved record hides them.

Decisions

Chose	Over	Because	Cost
Hand-rolled deep merge plus Pydantic	Hydra/OmegaConf	small surface; file = experiment record	no CLI overrides; location-dependent base
Version identity	filesystem path	immutability, traceable lineage	can't train on un-ingested data
Flat intervention class with a `kind` tag	discriminated union	one simple class	irrelevant fields silently accepted
Mutating after-validator (`reduced` →0.5)	explicit fraction required	ergonomic shorthand	default duplicated in resolver
`name-sha256(name:ds:ver:ts)[:10]`	`uuid4` plus config hash	readable and unique	not content-addressed; can't detect re-runs
Exact (`quantile`) splits	independent draws	n = 48 makes independent sizes vary ±2	none at this scale
Fresh warmup on resume	restoring optimizer state	optimizer state isn't saved	lost Adam moments; LR schedule restarts

Volunteer unprompted: no `extra="forbid"`; `model: str` silently skips training; `num_workers`, `mixed_precision` and `success_tolerance` are dead; imbalanced can silently no-op; `experiment_id` is user-settable; there's no `--base` flag despite a config comment; the `selection` module citation is stale; split fractions are unbounded.

Numbers and names: split 0.8/0.1/0.1, seed 42, `quantile` → 38/5/5 on 48 episodes and 32/4/4 on 40; base training 1000 steps, batch 4, lr 1e-4, wd 1e-5, warmup 50, ckpt/val every 100, `val_batches` 8, keep 3, clip 1.0; extended lr 3e-5, warmup 20; eval batch 16, tolerance 0.1; sum tolerance 1e-6; ID hash 10 hex characters (40 bits).

Sentences worth memorizing

- "The config file is the experiment record. It carries the hypothesis as well as the hyperparameters."
- "An imbalanced run that doesn't name its dimension is a validation error, not a judgement call."

- "The resolved config is persisted so a run never depends on an unwritten default. The gap is that it also can't show a key you mistyped."
- "The split seed is separate from the training seed so the test set stays fixed while training varies."

Module 3 — The run directory and the cross-run index

Files covered: `src/model_training/tracking/run_dir.py`, `src/model_training/tracking/index.py`

Where we are: Module 2 produced a validated `ExperimentConfig`. Before any data is touched, the runner gives that config a place to live: a run directory, plus snapshots of the environment and git state, plus rows in a project-wide index. Every later module writes *into* the structure defined here, so the guarantees and non-guarantees of this module are inherited by everything after it.

The one-line thesis: Run tracking is plain files: one directory per run that refuses to be overwritten, plus four append-only JSONL files queryable without this code. The design is honest and tool-agnostic, but none of the writes are atomic, the index isn't transactional with the run, and the provenance it captures (git commit of both repos) can silently come out as `null` for the repo that matters most.

3.1 Prerequisites — the absolute basics

- **Experiment tracking.** Recording, for every training run, what went in (config, data, code version, environment) and what came out (metrics, checkpoints), so runs can be compared and reproduced. Off-the-shelf tools: MLflow, Weights & Biases, TensorBoard, DVC, Aim.
- **Run directory.** A folder holding one run's artifacts. Here it's `runs/<date>/<experiment_name>/seed_<seed>/`.
- **JSON / JSONL.** JSON is one structured document. **JSONL** (JSON Lines) is one JSON object per line. Its virtue is that you can *append* a record without rewriting the file, and tools like `jq`, `pandas.read_json(lines=True)` and DuckDB `read_json_auto` read it directly.
- **Append-only log.** A file that's only ever added to, never edited. History is preserved, and "the current value" for a key is the *latest* row for that key (the event-sourcing idea in miniature).
- **Atomic write.** A write that either fully happens or doesn't happen at all, so no reader ever sees half a file. The standard recipe is to write to a temporary file in the same directory, `fsync`, then `os.replace(tmp, target)`. `os.replace` is atomic on POSIX filesystems.
- **Torn write.** What you get *without* atomicity when a process dies mid-write: a truncated file or a half line.
- **TOCTOU (time-of-check to time-of-use).** A race where you check a condition ("is this directory empty?") and then act on it, and another process changes the condition in between.
- **`O_APPEND`.** The OS flag behind Python's `open(..., "a")`. Each `write()` syscall goes to the current end of file. On a local POSIX filesystem a single `write()` of a modest size won't interleave with another process's append. Python's buffered text I/O may split one logical line into several syscalls if it exceeds the buffer (8 KiB by default), and network filesystems (NFS) don't guarantee append atomicity at all.
- **Dirty working tree.** A git checkout with uncommitted changes (`git status --porcelain` non-empty). A commit hash alone doesn't identify the code that ran from a dirty tree.
- **CUDA compute capability vs driver version.** *Compute capability* (for example 8.9) is the GPU architecture's feature level. The *driver version* is the installed NVIDIA kernel driver. `torch.version.cuda` is a third thing: the CUDA toolkit version PyTorch was *compiled* against.

3.2 `RunDir` and `make_run_dir`: the no-overwrite rule

```
@dataclass
class RunDir:
    root: Path
    experiment_name: str
    seed: int
    date: str

    _SUBDIRS = ("checkpoints", "predictions", "evaluation", "failures", "reports", "logs", "metrics")

    @property
    def path(self) -> Path:
        return self.root / self.date / self.experiment_name / f"seed_{self.seed}"
```

- `_SUBDIRS` has no type annotation, so `@dataclass` treats it as a **class attribute, not a field**. That's intentional: it's a constant of the layout, not per-instance state.
- **Layout `date / name / seed`.** It groups by day first, which suits a researcher scanning what they ran today. *Alternative:* `name / date / seed` groups all runs of one experiment together, which suits comparison. *Cost of the choice:* every reference to a run embeds its date. `baseline_extended.yaml` hard-codes `runs/2026-09-03/...` (Module 2), and `scripts/plot_results.py` needs three separate dated paths.
- **`seed` in the path.** Multi-seed runs of the same experiment sit side by side. As Module 2 showed, the `seed` here is *the CLI/summary seed*, which doesn't control `training.seed`. So `seed_1/` and `seed_0/` from `mtrain train --seed 1` vs the default **differ only in name** (Module 11 covers this in detail).

```
def make_run_dir(base_runs_dir, experiment_name, seed, date=None) -> RunDir:
    date = date or _dt.datetime.now(_dt.timezone.utc).strftime("%Y-%m-%d")
    rd = RunDir(root=Path(base_runs_dir), experiment_name=experiment_name, seed=seed, date=date)
    if rd.path.exists() and any(rd.path.iterdir()):
        raise FileExistsError(
            f"run directory already exists and is non-empty: {rd.path} -- "
            "never silently overwrite an experiment run"
        )
    rd.ensure()
    return rd
```

Decision: refuse a non-empty run directory and never overwrite.

- *What:* if `runs/<today>/<name>/seed_<n>/` has anything in it, raise.
- *Why:* overwriting would destroy checkpoints and metrics. The guarantee is tested (`test_runner_refuses_to_overwrite_an_existing_run` expects `FileExistsError` on the second run).
- *Alternative 1:* auto-suffix (`seed_0_2`, or a timestamp or UUID directory). *Alternative 2:* MLflow-style opaque run IDs, with names as tags.
- *Cost:*
  1. **A failed run blocks its own retry for the rest of the (UTC) day.** The runner creates the directory and writes `environment.json`, `git.json` and `config.resolved.yaml` *before* the intervention step (Module 11). So a typo in a dataset version, a missing GPU, anything at all that fails, leaves a non-empty directory. The next attempt raises `FileExistsError`. The message explains what to do, which is the right behaviour for a guard, but there's no `--force` and no crash marker outside the trainer (Module 6 has `crash.json`; earlier failures have nothing).
  2. **The date is UTC.** A run started at 23:50 and retried at 00:10 UTC gets a *different* directory and doesn't collide. Collision behaviour therefore depends on the wall clock, which is surprising but harmless.
  3. **Check-then-create is a TOCTOU race.** Two processes launching the same experiment and seed at once can both see "empty", both call `ensure()`, and both write into the same directory. It's unlikely

for a single researcher on one GPU. It's guaranteed to bite with a job scheduler. *Fix:* `path.mkdir(parents=True, exist_ok=False)` on the leaf, which is atomic at the filesystem level, instead of check-then-`exist_ok=True`.

- **An *empty* existing directory is reused on purpose** (`any(rd.path.iterdir())`), which is reasonable: an empty directory holds nothing to destroy.

```
def ensure(self) -> Path:
    p = self.path
    p.mkdir(parents=True, exist_ok=True)
    for sub in self._SUBDIRS:
        (p / sub).mkdir(parents=True, exist_ok=True)
    return p
```

Seven subdirectories are pre-created. **`logs/` is created and never written.** Grep for `"logs"` finds only the `_SUBDIRS` tuple, and **the package has no logging at all** (there's no `import logging`; output goes to the terminal through `rich.Console`). Training progress printed by the engine isn't captured into the run directory, so an empty `logs/` folder in every run *implies* a log that doesn't exist. Either tee stdout/stderr into `logs/run.log`, or delete the directory.

```
def write_json(self, name: str, data: Any) -> Path:
    target = self.path / name
    target.parent.mkdir(parents=True, exist_ok=True)
    target.write_text(json.dumps(data, indent=2, default=str))
    return target

def append_jsonl(self, relpath: str, record: dict[str, Any]) -> None:
    target = self.path / relpath
    target.parent.mkdir(parents=True, exist_ok=True)
    with target.open("a") as f:
        f.write(json.dumps(record, default=str) + "\n")

def sub(self, name: str) -> Path:
    p = self.path / name
    p.mkdir(parents=True, exist_ok=True)
    return p
```

- **`write_json` isn't atomic.** `write_text` truncates and then writes. A kill between the two leaves an empty or truncated `summary.json` or `training_outcome.json`. `mtrain validate` and `mtrain compare` then fail with `JSONDecodeError` and no hint of what happened. The code goes to some length to defend checkpoints against partial writes (the trainer verifies each one, Module 6) but not its own metadata.
- **`default=str` is a silent type-coercion escape hatch.** Anything not JSON-serializable (a `Path`, a `datetime`, a NumPy scalar that isn't a `float` subclass, such as `np.float32`) becomes a string instead of raising. For `Path`, that's what you want. For a metric, it's a hazard: an `np.float32` MSE would be written as the *string* `"0.0239"`, and every downstream comparison or plot would then deal in strings. Whether the engine's `Evaluator` returns Python floats or NumPy scalars is **unverified**. `np.float64` is a `float` subclass and serializes correctly, so the risk is limited to other dtypes. It's still a correctness question settled by a fallback instead of a type.
- **Not every writer uses `RunDir`.** The runner writes `config.resolved.yaml`, `evaluation/*.json`, `predictions/test_outcomes.jsonl`, `failures/feedback_artifact.json` and `reports/*.md` by building paths and calling `write_text` or `open` directly (Module 11). So the run directory has no single writer through which atomicity or a manifest could be added later. The abstraction exists but is only partly used.
- `sub()` creates on every call, so it's idempotent and cheap. Callers use it as "path to subdirectory, guaranteed to exist."

The artifacts that end up in a run directory

Collected by grepping every `write_json` / `append_jsonl` call plus the runner's direct writes:

File	Written by	When
`environment.json`, `git.json`, `config.resolved.yaml`	runner	first, before data
`dataset.json`	runner	after intervention
`split_integrity.json`, `dataset_profile.json`	runner	after the leakage check and profile
`metrics/train.jsonl`, `metrics/validation.jsonl`, `metrics/checkpoint_errors.jsonl`	trainer	during training
`resume_source.json`, `crash.json`	trainer	on resume, or on a training exception
`training_outcome.json`, `model.json`	runner	after training
`evaluation/{baselines,test_overall,stratified}.json`	runner	after evaluation
`predictions/test_outcomes.jsonl`	runner	per-frame test outcomes
`model_artifact.json`, `failures/feedback_artifact.json`	runner	after a SmolVLA evaluation
`metrics/test.jsonl`, `metrics/rollout.jsonl`	runner	end
`reports/experiment_report.md`, `reports/model_card.md`, `reports/figures/*.png`	runner	end
`summary.json`	runner	**last**, which makes it the de facto "completed" marker
`manifest.json`	**nobody**	read by `reports/comparison.py` and `cli.py export`, but never written (gap S12)

`summary.json` being written last is the closest thing to a completion marker, though no code *uses* it as one. `compare` and `export` read whatever exists. Making it explicit (for example, a `status` field written first as `running` and atomically replaced with `completed` or `failed`) would turn the directory into a reliable state machine.

3.3 Provenance capture: git and environment

```
def _git_info(repo_path: Path) -> dict[str, Any]:
    if not (repo_path / ".git").exists():
        return {"repo": str(repo_path), "commit": None, "dirty": None, "branch": None,
                "note": "not a git repository"}
    try:
        def _run(*args: str) -> str:
            return subprocess.check_output(
                ["git", "-C", str(repo_path), *args], text=True, stderr=subprocess.DEVNULL
            ).strip()
        commit = _run("rev-parse", "HEAD")
        dirty = bool(_run("status", "--porcelain"))
        branch = _run("rev-parse", "--abbrev-ref", "HEAD")
        return {"repo": str(repo_path), "commit": commit, "dirty": dirty, "branch": branch}
    except Exception as exc:  # pragma: no cover - environment-dependent
        return {..., "error": str(exc)}
```

Decision: record git identity of *both* repositories.

- *Why:* Module 1 established that the training step lives in the engine, so the engine's commit is *more* important to reproducibility than this repo's. Recording both is correct and unusual. Most projects record only their own repo.
- `.exists()` is true for a `.git` *file* too (worktrees and submodules use one), so those cases work.
- **`dirty` is recorded, but the diff isn't.** A dirty run is flagged as unreproducible-from-commit, but the uncommitted change isn't saved. *Fix:* write `git diff HEAD` to `git_<repo>.patch` when dirty.
- **`status --porcelain` counts untracked files.** That's why the `.gitignore` entries `runs` and `experiments/*.jsonl` (Module 1) matter: without them, every run after the first would mark the repo dirty because of the previous run's outputs. That's a real, non-obvious reason those ignore rules exist.
- **`stderr=DEVNULL` plus a broad `except`:** git failures (git not installed, "dubious ownership" safe-directory errors in containers) become an `"error"` string in `git.json`, and the run continues. That's the right failure mode, since provenance shouldn't block training, but it's silent at the terminal.

Where the paths come from (in `runner.py`, shown here because it decides what this captures):

```
RDE_REPO = Path(os.environ.get("RDE_REPO_PATH", str(_default_repo_root() / "episodic-data-engine")))
MT_REPO  = Path(os.environ.get("MODEL_TRAINING_REPO_PATH", str(Path(__file__).resolve().parents[3])))
```

- The engine path is a **guess from directory layout** (`parents[4]`, the parent of this repo, plus `/episodic-data-engine`), overridable by an environment variable.
- **The commit recorded isn't necessarily the code that ran.** The git info is read from wherever `RDE_REPO` points, not from where the imported `rde` package actually lives (`rde.__file__`). If you installed the engine from a different checkout, or non-editable from a wheel, `git.json` reports the commit of an unrelated directory, or `"not a git repository"`, and the run proceeds. **For the repository that owns `fit()`, provenance can be null or wrong without any warning.** *Fix:* resolve the repo from `Path(rde.__file__).resolve()` and walk up to the `.git`.
- `MT_REPO` = `parents[3]` is correct only for an editable `src/` install. For a regular wheel install it resolves to somewhere like `.../lib/python3.11/`, which isn't a git repo. The runner's default `runs/` and `experiments/` directories are derived from the same `MT_REPO` (Module 11), so they'd be created *inside the Python installation*. Because of the sibling-path dependency, the project is effectively editable-only (Module 1), so in practice this is latent.

Doc vs code: the report's git line is always `n/a`. `collect_git_info` returns keys `"episodic_data_engine"` and `"vla_finetune"`, while `reports/experiment_report.md`'s renderer reads `env.get('git', {}).get('model_training', ...)` (Module 10). The model_training commit therefore always prints `n/a` in every report, even though `git.json` has it. A one-word bug.

```
def collect_environment() -> dict[str, Any]:
    ...
    info = {
        "python_version": sys.version,
        "os": platform.platform(),
        "hostname": socket.gethostname(),
        "cpu_count": os.cpu_count(),
        "ram_total_gb": ram_total_gb,
        "packages": {
            "episodic-data-engine": _package_version("episodic-data-engine"),
            "vla-finetune": _package_version("vla-finetune"),
            "numpy": _package_version("numpy"),
            "pydantic": _package_version("pydantic"),
        },
        "torch": None, "cuda": None, "gpu": None, "lerobot": None, "notes": [],
    }
    try:
        import torch
        info["torch"] = torch.__version__
        info["cuda"] = {"available": torch.cuda.is_available(), "version": torch.version.cuda}
        if torch.cuda.is_available():
            props = torch.cuda.get_device_properties(0)
            info["gpu"] = {
                "name": torch.cuda.get_device_name(0),
                "total_memory_gb": round(props.total_memory / 1e9, 2),
                "driver_capability": f"{props.major}.{props.minor}",
            }
    except ImportError:
        info["notes"].append("torch not installed in this environment (light .venv, no `vla` extra)")
```

- **Pattern: "observe, don't fabricate."** Missing dependencies produce `None` plus a note, never a guess. That matches the project's stated principle, and it's what the runner keys its `limitations` list on (`environment.get("gpu") is None`, Module 11).
- **`driver_capability` is mislabelled.** `props.major` and `props.minor` are the GPU's **compute capability** (8.9 for an RTX 4060), not a driver version. The driver version isn't recorded anywhere. Neither is cuDNN (`torch.backends.cudnn.version()`). And `cuda.version` is `torch.version.cuda`, the *compile-time* toolkit, not what the driver supports.
- **The package list misses what actually determines model behaviour.** The versions that matter for SmolVLA numerics are `torch` (recorded), `lerobot` (recorded via `__version__`), and `transformers` plus `safetensors` (SmolVLA's backbone loader; *not recorded*). `numpy` and `pydantic` are recorded instead. `vla-finetune` is always `0.1.0` and carries no information; the git commit is the real identity. A full `pip freeze` (or `importlib.metadata.distributions()`) dump would be cheap and complete.
- **`import torch` at run start** costs a second or two and initializes CUDA. That's harmless here, since training follows immediately.
- **Only GPU 0 is described.** That's fine for a single-GPU laptop (README: RTX 4060 Laptop, 8 GB). The record would be wrong under `CUDA_VISIBLE_DEVICES` remapping on a multi-GPU box, where device 0 might not be the one used.
- **`hostname` is recorded.** That's fine for local use. Note that `mtrain export` doesn't copy `environment.json` into the paper-ready bundle (it copies configs and manifests only; Module 12), so hostnames don't leak into exported artifacts. It's unclear whether that was deliberate, but the effect is right.

3.4 `JsonlIndex` and `ExperimentIndex`: the cross-run index

```
class JsonlIndex:
    def __init__(self, path: Path):
        self.path = Path(path)
        self.path.parent.mkdir(parents=True, exist_ok=True)

    def append(self, record: dict[str, Any]) -> None:
        with self.path.open("a") as f:
            f.write(json.dumps(record, default=str) + "\n")

    def read_all(self) -> list[dict[str, Any]]:
        if not self.path.exists():
            return []
        out = []
        with self.path.open() as f:
            for line in f:
                line = line.strip()
                if line:
                    out.append(json.loads(line))
        return out

    def query(self, **filters: Any) -> Iterator[dict[str, Any]]:
        for row in self.read_all():
            if all(row.get(k) == v for k, v in filters.items()):
                yield row
```

Decision: JSONL files, not a database or a tracking server.

- *What:* four files under `experiments/`, namely `experiments.jsonl`, `metrics.jsonl`, `datasets.jsonl` and `models.jsonl`, one row per event.
- *Why:* the module docstring says "queryable with `jq`, `pandas.read_json(lines=True)`, or DuckDB's `read_json_auto` without this project running". That's the "scrapable output" principle. No server, no schema migration, readable decades later, diffable.
- *Alternative 1:* MLflow (a tracking server plus artifact store, with a UI). *Alternative 2:* SQLite, which is transactional and indexed, and is plausibly what the engine's own catalog already uses (unverified). *Alternative 3:* W&B (hosted, rich UI, network dependency).
- *Cost:*
  1. **No atomicity or crash tolerance on read.** `read_all` does `json.loads` on every line with no `try`. If a process dies mid-append, leaving a half line, **every subsequent `read_all`, `query`, `ModelRegistry.get` and `ModelRegistry.list` raises `JSONDecodeError` forever**, until someone hand-edits the file. One torn write poisons the whole index. *Fix:* skip and count undecodable lines (and warn), or write each record with a single `os.write` on an `O_APPEND` file descriptor.
  2. **Concurrent appends from several processes** are safe on local disk *only* while each line goes out in one `write()` syscall. `models.jsonl` rows carry the full observation and action schema plus both repos' git info (Module 9), so they can approach or exceed the 8 KiB buffer, at which point lines from two processes can interleave. It's unlikely on one laptop and plausible under a scheduler or NFS.
  3. **Every query is a full scan.** That's `O(rows)` per lookup, which is irrelevant at tens or hundreds of runs. It becomes painful only at tens of thousands, and by then DuckDB over the same files is the natural upgrade with no format change. That's a real strength of the format choice.
  4. **No schema.** `record_experiment(**fields)` accepts anything. Rows written by different code versions differ silently, and there's no `schema_version` field.

```
class ExperimentIndex:
    def __init__(self, root: Path):
        self.root = Path(root)
        self.experiments = JsonlIndex(self.root / "experiments.jsonl")
        self.metrics = JsonlIndex(self.root / "metrics.jsonl")
        self.datasets = JsonlIndex(self.root / "datasets.jsonl")
        self.models = JsonlIndex(self.root / "models.jsonl")

    def record_experiment(self, **fields): self.experiments.append(fields)
    def record_metric(self, **fields):     self.metrics.append(fields)
    def record_dataset(self, **fields):    self.datasets.append(fields)
    def record_model(self, **fields):      self.models.append(fields)
```

- **Pattern:** a *Façade* over four *append-only logs*, close to the *Repository* pattern without the query side. Only `ModelRegistry` (Module 9) ever reads one of these files back. **Nothing in `src/` reads `experiments.jsonl`, `metrics.jsonl` or `datasets.jsonl`.** `mtrain compare` reads run *directories*, not the index (Module 10). So the index is currently **write-only from the system's point of view**. It's a sound design, but it's a log for external tools rather than a component the system relies on.
- **The index isn't transactional with the run directory.** The runner writes `record_dataset` right after the intervention, *before* training (Module 11). `record_model`, `record_experiment` and `record_metric` come at the end. So a run that crashes in training leaves a `datasets.jsonl` row pointing at a `training/vN` with no matching `experiments.jsonl` row, and there's no row anywhere saying the run failed. **You can't list failed runs from the index.** Even orphaned dataset versions are invisible unless you join `datasets.jsonl` against `experiments.jsonl` by `experiment_id`.
- **Doc check:** the index docstring lists the four files and their row semantics ("one row per run", "one row per (run, split, metric snapshot)", ...). That matches the call sites, with one nuance: `models.jsonl` gets one row per *SmolVLA* run only, because baseline-only runs never build a model artifact.

3.5 Honest gaps to own before an interviewer finds them

1. **No write is atomic** (`write_json` truncates then writes; `append_jsonl` and `JsonlIndex.append` are plain buffered appends). *Consequence:* a kill mid-write corrupts `summary.json` or a JSONL line, and readers fail with an unexplained `JSONDecodeError`.
2. **One torn line poisons the whole index.** `read_all` has no per-line error handling. *Consequence:* `ModelRegistry.get`/`list` and every `query` fail until the file is repaired by hand.
3. **A failed run blocks its own same-day retry**, because the directory is created and populated before any validation of data or environment, and there's no `--force` or crash marker. *Consequence:* the user has to rename the experiment or delete the directory, which defeats the no-overwrite guarantee's purpose.
4. **`make_run_dir` has a TOCTOU race** (check non-empty, then `mkdir(exist_ok=True)`). *Consequence:* two concurrent identical launches share one directory.
5. **Engine provenance can be silently wrong or null.** The git info is read from a layout-guessed path, not from the imported `rde` package's location. *Consequence:* for the code that owns `fit()`, `git.json` may record nothing, or an unrelated checkout.
6. **Dirty trees are flagged but the diff isn't saved.** *Consequence:* a dirty run is unreproducible by construction.
7. **`environment.json` mislabels compute capability as `driver_capability`, omits the driver, cuDNN and `transformers` versions, and records a meaningless `vla-finetune` 0.1.0.** *Consequence:* the record misses exactly the versions most likely to explain a numerics difference.

8. **The report's git line always prints `n/a`** (key `vla_finetune` written, `model_training` read). *Consequence:* the human-readable report hides provenance that `git.json` holds.
9. **`logs/` is created in every run and never written, and there's no logging at all.** *Consequence:* engine training output isn't captured, and the empty folder implies otherwise.
10. **The index is write-only inside the system, not transactional with the run, and has no status field.** *Consequence:* failed runs and orphaned dataset versions are undiscoverable from the index.
11. **`default=str` silently stringifies non-JSON types.** *Consequence:* a non-`float64` NumPy metric would be stored as a string. Unverified whether the engine produces one.
12. **Writers bypass `RunDir`** for about half the artifacts. *Consequence:* there's no single choke point to add atomicity, a manifest or checksums later.
13. **Default `runs/` and `experiments/` locations derive from `parents[3]` of the installed file.** *Consequence:* with a non-editable install they'd land inside the Python installation (latent, because the project is editable-only).

The shape to carry into Module 4

```
runs/                                          experiments/            (both git-ignored)
└── <UTC date>/                                ├── experiments.jsonl   one row per COMPLETED run
    └── <experiment_name>/                     ├── metrics.jsonl       one row per test eval
        └── seed_<n>/   ← make_run_dir:        ├── datasets.jsonl      one row per intervention
            │              raise if non-empty   └── models.jsonl        one row per SmolVLA artifact
            ├── environment.json  (torch/cuda/gpu or None+note)       append-only, full-scan,
            ├── git.json          (engine + this repo; dirty flag)    no locking, no schema,
            ├── config.resolved.yaml                                  write-only inside system
            ├── checkpoints/ predictions/ evaluation/
            ├── failures/ reports/ metrics/
            ├── logs/            ← always empty
            └── summary.json     ← written LAST (implicit "completed")
  writes: write_text / open("a")  — none atomic
```

Drill — answer from memory, files closed

1. What happens if `mtrain train` fails during the intervention step and you immediately rerun the same command?
2. *Why didn't you just* use MLflow or Weights & Biases?
3. *What breaks in production* if the machine loses power while the runner is appending to `experiments/models.jsonl`?
4. Why does `.gitignore` excluding `runs` matter for the correctness of `git.json`, beyond keeping the repo small?
5. *Interviewer framing:* "How would I reproduce one of your runs six months from now? What would stop me?"

Appendix A — Model answers to the drill

A1. Fail-then-rerun. "The runner creates the run directory and writes `environment.json`, `git.json` and the resolved config before it touches the data. So if the intervention fails, say a wrong dataset version, the directory is already non-empty, and the rerun raises `FileExistsError` from `make_run_dir`, which never reuses a non-empty run directory. The guard is deliberate, because overwriting a real run would destroy checkpoints. But here it protects nothing, since the failed run has no results. I'd fix that by validating the dataset and environment before creating the directory, writing a status marker that a failed run can be identified by, and allowing a retry over a directory whose status is `failed`. Today you rename the experiment or delete the directory, and the error message tells you that."

Why that answer lands: it traces the exact order of writes, defends the guard's intent, and then shows that the guard misfires in one specific case, with a fix that keeps the guarantee.

A2. Why not MLflow or W&B? "The requirements were reproducibility and inspectability, not a dashboard. Run directories plus append-only JSONL mean every artifact is a plain file you can query with `jq`, pandas or DuckDB without this code, a server, a login or network access, and the format will still be readable in ten years. There's also no second source of truth: the run directory is the record. The costs are that I own the durability problems myself (none of my writes are atomic yet), there's no UI, and every query is a full scan. I'd switch, or add MLflow as a sink alongside the files, once more than one person was running experiments or once runs numbered in the thousands. At that point locking and a UI earn their keep, and the JSONL files map directly onto MLflow params and metrics."

Why that answer lands: it argues from requirements, admits the costs you inherited by rolling your own, and names a migration path that keeps the existing format, not a rewrite.

A3. Power loss during an index append. "The append is a buffered text write with no fsync, so after a power loss the last line can be truncated. The reader parses every line with `json.loads` and no error handling, so from then on every read of that file raises, including the model registry's `get` and `list`. One torn write takes out the whole registry until someone hand-edits the file. The fix has two halves: make the reader skip and report undecodable lines, and make the writer emit each record as a single `os.write` on an `O_APPEND` descriptor, with fsync if durability matters. The same class of bug exists for the run directory's JSON files, which are truncated and rewritten in place, so I'd move those to write-to-temp plus `os.replace`."

Why that answer lands: it gets the failure exactly right (a single bad line breaks all reads, not just the lost record), gives both halves of the fix, and generalizes to the sibling bug unprompted.

A4. Why ignoring `runs` matters for `git.json`. "`git.json` records a dirty flag from `git status --porcelain`, which counts untracked files. If run outputs lived unignored inside the repo, every run after the first would see the previous run's files as untracked and mark the tree dirty, so the flag would be permanently true and carry no information. Ignoring `runs` and the index files keeps 'dirty' meaning 'the code changed', which is the only meaning that's useful for reproducibility. The remaining gap is that when the tree really is dirty, I record the flag but not the diff, so I'd save `git diff HEAD` as a patch next to it."

Why that answer lands: it connects two files that look unrelated, showing systems-level reading, and ends with the next gap.

A5. "How would I reproduce one of your runs in six months?" "Start from the run directory. `config.resolved.yaml` has every parameter including defaults, `dataset.json` names the exact training version in the engine's catalog, and the engine's lineage resolves that back to raw episodes. `git.json` has the commit of both repositories, and `environment.json` has the Python, torch, lerobot and GPU details. Then check out both commits, install, and rerun the config against the same input version. What would stop you,

honestly: there's no lockfile, so transitive versions like `transformers` may differ and aren't recorded. If either tree was dirty, the diff wasn't saved. The engine's commit is read from a path guessed from directory layout, not from the package that was actually imported, so it could be wrong. And the checkpoints and run directories aren't versioned, so you need my disk, not just the repo. Even with all of that fixed, SmolVLA's flow-matching sampling is stochastic, so you'd reproduce the metrics within noise, a few percent, not bit-for-bit."

Why that answer lands: it gives the full chain first, then lists the breaks precisely, and closes with the irreducible stochasticity. That's the sign of someone who has actually tried to reproduce ML results.

Module 3 Summary — the one page to revise

What this module is. One directory per run (`runs/<UTC date>/<name>/seed_<n>/`) that's never overwritten, populated with environment and git snapshots and every artifact, plus four append-only JSONL index files that external tools can query without this code. The format is good; durability, transactionality and provenance accuracy are the weak points.

Five claims to make cold

1. A non-empty run directory is never reused: `FileExistsError`, tested.
2. Both repos' git commits and dirty flags are recorded, because the engine owns `fit()`.
3. Missing dependencies become `None` plus a note, never a guess, and the runner turns these into report limitations.
4. The index is four JSONL files, append-only, readable by `jq`, pandas or DuckDB with no server.
5. No write is atomic, and one torn JSONL line breaks every subsequent read of that file.

Decisions

Chose	Over	Because	Cost
Plain run directories plus JSONL	MLflow/W&B	inspectable, serverless, durable format	durability, locking, UI are on me
Refuse non-empty directory	auto-suffix/UUID	never destroy a run	failed runs block same-day retry; TOCTOU race
`date/name/seed` layout	`name/date/seed`	scan by day	dated paths leak into configs and scripts
Record git of both repos	own repo only	engine owns training	engine path guessed, can be null or wrong
`default=str` in JSON dumps	strict serialization	never crash on `Path` / `datetime`	silently stringifies odd types
Pre-create 7 subdirectories	create on demand	predictable layout	`logs/` always empty

Volunteer unprompted: the torn-write poisoning; the retry-blocking guard; the `driver_capability` mislabel; the report's `n/a` git line; no diff saved for dirty trees; the index is write-only and has no failed-run status; `manifest.json` is read but never written.

Numbers and names: 7 subdirectories (`checkpoints predictions evaluation failures reports logs metrics`); 4 index files; env vars `RDE_REPO_PATH` and `MODEL_TRAINING_REPO_PATH`; engine path `parents[4]/episodic-data-engine`; repo root `parents[3]`; `summary.json` written last; Python's default text buffer is 8 KiB.

Sentences worth memorizing

- "Every artifact is a plain file you can query without my code; that's the design, and durability is the price I still owe."
- "I record the engine's commit as well as mine, because the engine owns the training step."
- "One torn line in a JSONL index breaks every read after it, so the reader has to be tolerant and the writer atomic."
- "Ignoring `runs/` isn't just tidiness; it keeps the dirty flag meaningful."

Module 4 — Data interventions as lineage

Files covered: `src/model_training/data/interventions.py` (the `data/__init__.py` façade was covered in Module 1)

Where we are: Module 2 validated a `DatasetIntervention`, and Module 3 gave the run a directory. This module is the first one that *changes the world*: it turns "halve the data" or "drop the short episodes" into new, immutable dataset versions inside the engine's catalog. This is the project's central idea, the one the README's organizing principle rests on ("Even a data intervention … is executed as a lineage-tracked transform inside the engine, never as a local copy").

The one-line thesis: Every intervention becomes a real `clean/vN` (an episode filter) plus a real `training/vN` (a split assignment plus sampling weights), so the provenance is impeccable. But because the filter runs **before** the split, every excluding intervention is scored on a **different, smaller test set** than the baseline it's meant to be compared against. The lineage design quietly undermines the comparisons the experiments exist to make.

4.1 Prerequisites — the absolute basics

- **Data-centric ML / data intervention.** Holding the model and training recipe fixed and changing *the data*, to measure what the data contributes. Here the four primary interventions are *full*, *reduced* (fewer episodes, chosen at random), *imbalanced* (deliberately skewed along one attribute) and *rebalanced* (oversampling under-represented episodes).
- **Quantity vs coverage.** Two different hypotheses about why more data helps. *Quantity:* more examples of the same thing reduce variance. *Coverage:* examples of *different* situations extend what the model can handle. A random 50% subset changes quantity while preserving coverage in expectation. Removing every "short" episode changes coverage. The experiment set is designed to tell the two apart (the configs' `hypothesis: quantity_vs_coverage` tag).
- **Oversampling / importance weighting.** Instead of duplicating data, give some examples a higher probability of being drawn into a training batch. A weight of 3.0 means "drawn about three times as often as a weight-1.0 episode", in expectation, under a weighted sampler. Nothing is copied on disk.
- **Stratum / slice.** A subset of the evaluation data defined by an attribute (task, length bucket). Covered in depth in Module 7.
- **Terciles / percentiles.** The 33.3rd and 66.7th percentiles split a sorted list into three roughly equal groups. `np.percentile` uses linear interpolation between data points by default.
- **Banker's rounding.** Python's `round()` rounds halves to the nearest *even* integer: `round(2.5) == 2`, `round(3.5) == 4`. This matters whenever `fraction × count` lands on `.5`.
- **`numpy.random.default_rng(seed)`.** A modern NumPy random generator (PCG64). With the same seed and the same input array it produces the same draws on the same NumPy version. `rng.choice(ids, size=n, replace=False)` draws `n` distinct elements.
- **Nested subsets.** In a data-efficiency curve, the 25% subset being contained in the 50% subset (and that in the 75%) reduces variance between curve points, because each point adds data to the last instead of swapping it out. The standard way to get them is to take prefixes of one random permutation.
- **Orphaned version.** A dataset version created in the catalog by a run that then failed, so no experiment refers to it.

4.2 The version pipeline this module drives

```
input version (config.dataset.version, e.g. clean/v1)
     │
     ├─ kind = full / rebalanced, or a subset equal to every episode
     │        → no transform; clean_version := input_version
     │
     └─ kind = fraction / reduced / imbalanced / custom with a real subset
              → next_version(CLEAN) = clean/vK
                TransformPipeline([EpisodeSubsetFilter(subset_ids, reason)]).run(input → clean/vK)
     ▼
next_version(TRAINING) = training/vM
DatasetBuilder.build(clean_version → training/vM,
                     splits=train/val/test fractions, seed=42, split_mode=quantile,
                     episode_weights = {...} or None)
```

Decision: an intervention is two engine operations, never a local copy.

- *What:* episode selection is an engine *transform* (`EpisodeSubsetFilter`), which records `dropped_episode_ids` in lineage. Split assignment and weights are an engine *build*.
- *Why:* the README's lineage output shows exactly what this buys: `transform ['raw/v1'] -> clean/v1 episodes 50->48 dropped=[9, 31]`. Any `training/vN` can be traced back to raw episodes, with the reason for each drop. `test_reduced_intervention_selects_half_and_records_lineage` checks that the transform run's `output_episode_ids` and `dropped_episode_ids` equal what this module computed. **That test verifies the core claim end to end**, and it's the strongest test in the repo.
- *Alternative:* filter episode IDs at training time (pass `episode_ids=` to `fit`) with no new version, or copy a subset to a new directory.
- *Cost:* (1) every run writes to the shared catalog, even a `full` run, which still allocates a new `training/vN`; (2) the order "filter, *then* split" has a consequence big enough to get its own section (4.5).

4.3 `compute_length_bucket`

```python
def compute_length_bucket(episode_rows: list[dict[str, Any]]) -> dict[int, str]:
    """short/medium/long by within-dataset episode-length terciles.
    ... Same bucketing method as ``model_training.data.profile``'s coverage section, so
    the two stay comparable.
    """
    if len(episode_rows) < 3:
        return {r["episode_id"]: "medium" for r in episode_rows}
    lengths = [r["num_frames"] for r in episode_rows]
    t1, t2 = np.percentile(lengths, [33.3, 66.7])
    return {
        r["episode_id"]: ("short" if r["num_frames"] <= t1 else "long" if r["num_frames"] > t2 else "medium")
        for r in episode_rows
    }
```

Why length is a variable at all. `docs/EXPERIMENTS.md` §3 says the primary dataset `so100_pickplace` has *one* task instruction across all 48 episodes, so `task_id` has nothing to select between. Trajectory length "326 to 475 frames per episode" is the only real per-episode attribute that varies. The schema has no "variation" field (`profile.py`'s note says so too). This is an honest choice made from what the data actually contains, and it's the right story to tell: "I couldn't find a meaningful variation axis in the schema, so I used the one that exists, and I said so."

Line by line:

- `< 3` → **all** `"medium"`**.** Terciles of one or two values are meaningless, so every episode gets the neutral bucket. Without this guard `np.percentile` would still return numbers and produce a meaningless split. Fine.
- `33.3` **and** `66.7`**, not** `100/3` **and** `200/3`**.** These are rounded magic numbers. They're harmless, but they're approximations of a tercile, not a definition.
- `<= t1` → **short,** `> t2` → **long.** The asymmetric boundaries put ties at the low edge into "short". **With heavily tied lengths the buckets collapse.** The test-file comment admits the synthetic fixture's equal-length episodes all bucket to `"short"`. A real case: the README says all four ALOHA test episodes are 1,100 frames. If ALOHA's whole dataset is tied that way (I can't verify the full distribution), then `t1 = t2 = 1100`, every episode is "short", and:
  - `imbalanced, variable_value: short` drops *everything*, and the run fails loudly with "resolved to zero episodes". Good.
  - `imbalanced, variable_value: long` (or `medium`) drops *nothing*, the subset equals the full set, and **a full-data run proceeds labelled "imbalanced"** (the same silent-no-op class as Module 2 gap 4).
- **The docstring's comparability claim is false.** It says the method is the "Same bucketing method as `model_training.data.profile`'s coverage section". Grep for `percentile` or `tercile` in `profile.py` finds **nothing**. The profile reports the engine's duration *histogram*, not terciles. The only other tercile code is in `evaluation/offline.py:171`, and that one computes terciles **within the evaluated split**, not the whole dataset (`docs/EXPERIMENTS.md` Table 2 confirms: "within-split episode-length terciles"). So the `"short"` you dropped at intervention time (whole-dataset cut points) and the `length_short` slice you score at test time (test-split cut points, from 5 episodes) **are different sets with different thresholds**. Any claim that "dropping short episodes hurt the short slice" is comparing two different definitions of short.

4.4 `resolve_episode_subset`

```
kind = intervention.kind
if kind == "full":
    return sorted(all_episode_ids), "full dataset, no episode selection"
if intervention.episode_ids is not None:
    return sorted(intervention.episode_ids), intervention.reason or f"explicit episode_ids ({kind})"
```

- `full` **short-circuits before** `episode_ids`**.** `kind: full, episode_ids: [1, 2]` returns every episode. That matches the docstring "explicit `episode_ids` always wins" *only for non-full kinds*. That's a silent override of user input (Module 2 gap 12).
- **Explicit IDs aren't validated against the dataset.** `episode_ids: [999]` returns `[999]`. The caller computes `excluded = set(all_ids) - {999}` (everything) and asks the engine's filter for an episode that doesn't exist. What `EpisodeSubsetFilter` does then is **unverified**. It might raise, or it might produce an empty version that fails later in the build. Duplicates (`[3, 3, 5]`) survive `sorted` and make `subset_ids == sorted(all_ids)` false even when every episode is listed, so they'd create an unnecessary `clean/vK`.
- `reason` **is always recorded.** It's either user-supplied or generated. It travels into the engine's transform (`EpisodeSubsetFilter(subset_ids, reason=reason)`), and so into lineage. That's the auditable "why" for every dropped episode, and it's a genuinely good detail.

Fraction and reduced

```
if kind in ("fraction", "reduced"):
    frac = intervention.fraction if intervention.fraction is not None else 0.5
    rng = np.random.default_rng(intervention.seed)
    ids = sorted(all_episode_ids)
    # frac=0.0 is a legitimate, explicit request (a zero-data anchor
    # point on a data-efficiency sweep) and must be honored as 0
    # episodes, not silently floored to 1 ...
    n = round(frac * len(ids))
    if frac > 0:
        n = max(1, n)
    chosen = sorted(int(i) for i in rng.choice(ids, size=n, replace=False))
    return chosen, intervention.reason or f"seeded random {frac:.0%} draw over all episodes (seed={intervention.seed})"
```

- `sorted(all_episode_ids)` **before drawing** makes the draw independent of the order the catalog returns rows in. Without it, the same seed could select different episodes if the catalog's row order changed. This is the non-obvious line that makes `test_reduced_intervention_is_deterministic_given_seed` meaningful.
- `int(i)` converts NumPy integers to Python `int`, so they serialize cleanly into JSON and catalog records (Module 3's `default=str` would otherwise be the only safety net).
- **The 0.5 fallback is unreachable** for validated configs (Module 2 gap 13).
- **Banker's rounding:** `round(0.5 × 5) = 2`, `round(0.5 × 7) = 4`. For the real dataset (48 episodes) every multiple of 1/48 is exact, so this only matters in sweeps over small sets. It's still a "why is my 50% run using 2 of 5?" question you should be able to answer.
- **The floor at 1** (`max(1, n)` for `frac > 0`) prevents "1% of 3 episodes" from silently meaning "no data". That's sensible.

Doc and test vs code: the zero-fraction anchor can't run. The comment insists `frac=0.0` "must be honored as 0 episodes", and `test_reduced_intervention_at_zero_fraction_is_honored` asserts that `resolve_episode_subset` returns `[]`. But the only caller, `build_intervention_dataset`, does this next:

```
subset_ids, reason = resolve_episode_subset(intervention, all_ids, task_of, length_bucket_of)
if not subset_ids:
    raise ValueError(f"intervention kind={intervention.kind!r} resolved to zero episodes")
```

So a data-fraction sweep including `0.0` (the "zero-data anchor" the comment names as the use case) **raises before training**. The test passes because it tests the helper in isolation, never the path a sweep actually takes. That's a textbook unit-tests-pass, system-fails gap. The honest resolution is to decide what a zero-data point *means*. Most likely it's "evaluate the pretrained `smolvla_base` with no fine-tuning", which doesn't need a dataset version at all, and should be implemented as its own path, not as an empty filter.

Draws at different fractions aren't nested. `rng.choice(ids, size=n, replace=False)` with the same seed at `n = 12` and `n = 24` isn't guaranteed to return a 12-subset of the 24-subset. NumPy's `Generator.choice` without replacement doesn't promise prefix consistency across sizes, and I haven't checked which algorithm it selects for these sizes. *Fix:* `perm = rng.permutation(ids); chosen = sorted(perm[:n])` gives nested subsets by construction, which is the standard design for data-efficiency curves.

Imbalanced

```
if kind == "imbalanced":
    var, val = intervention.variable, intervention.variable_value
    if var == "task_id":
        keep = sorted(e for e in all_episode_ids if task_of_episode.get(e) != val)
        return keep, intervention.reason or f"drop every episode with task_id={val!r}"
    if var == "episode_length_bucket" and length_bucket_of_episode is not None:
        keep = sorted(e for e in all_episode_ids if length_bucket_of_episode.get(e) != val)
        return keep, intervention.reason or f"drop every episode in length bucket {val!r}"
    raise ValueError(f"... no automatic rule for variable={var!r}; supported: 'task_id', ...")
```

- **"Imbalanced" here means *removed entirely*, not *under-represented*.** Every episode with the chosen value is dropped. The `imbalanced.yaml` hypothesis template says "Skewing the demonstration set to *under-represent* `<variable>=<value>`". Zero coverage isn't under-representation. It turns the experiment into an *out-of-distribution* test ("can it handle a regime it never saw?"), which is a different and arguably more interesting question, but not the one the config states. Under-representation would keep, say, 20% of those episodes. Name the semantics correctly in an interview.
- **The unsupported-variable path raises**, and it's tested (`test_resolve_episode_subset_unknown_variable_raises`, plus the end-to-end `test_unknown_dataset_variation_raises` in `test_failure_modes.py`). The design rule "never guess" is enforced here.
- **The `None`/mistyped-value path doesn't raise** (Module 2 gap 4): `!= None` keeps everything.
- **The module docstring is stale.** It says imbalance today uses "`task_id`, the one grouping key this dataset's schema actually carries". The code also supports `episode_length_bucket`, and the function's own docstring two definitions down explains that `so100_pickplace` is single-task, so `task_id` imbalance has nothing to select there. The two docstrings disagree, and the code and `EXPERIMENTS.md` side with the length bucket.

4.5 `build_intervention_dataset`, and the test set that moves

```
manifest = catalog.get_manifest(dataset_id, input_version)
if manifest is None:
    raise LookupError(f"unknown version: {dataset_id}@{input_version}")
episode_rows = catalog.get_episodes(dataset_id, input_version)
all_ids = [e["episode_id"] for e in episode_rows]
task_of = {e["episode_id"]: e["task_id"] for e in episode_rows}
length_bucket_of = compute_length_bucket(episode_rows)
if not all_ids:
    raise ValueError(f"{dataset_id}@{input_version} has no episodes to build from")
```

- **The `LookupError` for an unknown version** is tested (`test_intervention_on_unknown_input_version_raises`). It's the right exception type, matching the engine's `get_dataset` behaviour asserted in `test_invalid_dataset_version_raises`.
- **Every episode in the input version is a candidate.** There's no filter on QC status here. The conftest comment says `DatasetBuilder.build()` "admits only PASSED/WARNING by default", so failed-QC episodes are presumably dropped *at build time*, after this module has computed `episode_ids` and `excluded_episode_ids`. If that's right (unverified), then `dataset.json`'s `episode_ids` can list episodes that never made it into the training version. The engine's `build_summary` would show the true count, but this module's own fields wouldn't. For `so100_pickplace`, whose `clean/v1` already dropped QC failures (episodes 9 and 31), it probably doesn't bite.

Rebalanced: weights, never exclusion

```
if intervention.kind == "rebalanced":
    weights = dict(intervention.episode_weights or {})
    if not weights and intervention.variable is not None and intervention.variable_value is not None:
        target = _target_episodes(intervention.variable, intervention.variable_value)
        if not target:
            raise ValueError(f"no episodes with {intervention.variable}={intervention.variable_value!r} to upweight")
        weights = {e: intervention.oversample_factor for e in target}
    if not weights:
        raise ValueError("rebalanced intervention resolved to empty episode_weights -- "
                         "nothing would change relative to a full-data run")
    subset_ids = sorted(all_ids)
    excluded: list[int] = []
    clean_version = input_version
```

Decision: rebalance by sampling weight, not by duplication or synthesis.

- *Why:* `docs/EXPERIMENTS.md` §4 states the constraint plainly: the project collects no new data, so a rebalanced run "can only redistribute training attention over data that already exists". Weights are honest about that; duplicating episodes on disk would just be a heavier, less traceable way to get the same effect.
- *Alternative:* physically duplicate episodes in a new `clean/vK`, which would be visible in lineage as copies, or use loss re-weighting instead of sampling re-weighting.
- *Cost:* the effect depends on the engine's weighted sampler in `fit`, which isn't verifiable here. Sampling with replacement at weight 3.0 also means those episodes' frames repeat within an epoch, which raises overfitting risk on exactly the regime you're trying to help.

Double guard against a no-op, the counterpart of the gap imbalanced has: both "`variable` set but no matching episodes" and "nothing resolved" raise. Note that `_target_episodes` returns `[]` for an *unsupported* variable too, so the error says "no episodes with camera=dim to upweight" when the truth is "camera isn't a supported variable". It's a misleading message, though it's at least loud.

Explicit weights aren't validated. Keys naming nonexistent episodes have no effect, silently. Weights ≤ 0 pass (Module 2 gap 15). Weights are also attached to *every* targeted episode in the pool, including those that end up in val and test. The runner only reads weights for `train` IDs (Module 11), so this is harmless, but it does mean `dataset.json`'s `episode_weights` overstates which episodes were actually upweighted during training.

`docs/EXPERIMENTS.md` §4 overclaims: "Every experiment report generated by the system states this limitation explicitly in its 'Limitations' section." Grep for `rebalanc` in `experiments/` and `reports/` finds **nothing**. The runner's limitations list covers only GPU, torch and non-SmolVLA model notes (Module 11). A rebalanced report would *not* contain the caveat. The caveat exists only in the YAML comment and in the docs.

The excluding path

```
else:
    subset_ids, reason = resolve_episode_subset(intervention, all_ids, task_of, length_bucket_of)
    if not subset_ids:
        raise ValueError(f"intervention kind={intervention.kind!r} resolved to zero episodes")
    excluded = sorted(set(all_ids) - set(subset_ids))
    weights = intervention.episode_weights or {}
    if subset_ids == sorted(all_ids):
        clean_version = input_version
    else:
        clean_version = next_version(catalog, dataset_id, VersionStage.CLEAN)
        TransformPipeline([EpisodeSubsetFilter(subset_ids, reason=reason)],
                          settings=settings, catalog=catalog).run(dataset_id, input_version, clean_version)

training_version = next_version(catalog, dataset_id, VersionStage.TRAINING)
build = DatasetBuilder(settings=settings, catalog=catalog).build(
    dataset_id, clean_version, training_version,
    splits=splits.as_fractions(), seed=splits.seed, split_mode=splits.mode,
    episode_weights=weights or None,
)
```

- `subset_ids == sorted(all_ids)` → **reuse the input version.** No pointless transform for a filter that drops nothing. It's sensible, and it's exactly why the silent-no-op imbalanced case (4.4) passes through invisibly: no new `clean` version, and `excluded_episode_ids == []`. The only trace is `kind: imbalanced` alongside zero exclusions in `dataset.json`.
- `weights or None` turns an empty dict into `None`, presumably so the engine skips weighting entirely rather than recording empty weights. That's inferred from the call site.
- `next_version` **then** `run` **isn't atomic.** Two concurrent runs can both be allocated `training/v7`. Whether the engine's catalog rejects the second write is **unverified**. If it doesn't, two experiments claim the same version.
- **Failures leave orphans.** If `DatasetBuilder.build` fails after the transform has succeeded, `clean/vK` exists with no experiment pointing at it. And because the runner calls this function *after* `make_run_dir` (Module 3), the run directory exists too, blocking a retry. There's no cleanup, and none is possible, since versions are immutable by design. The right fix is to make orphans *discoverable* (a run status that links to its versions), not deletable.
- **Version proliferation.** Every run allocates a fresh `training/vN`, even `full` runs with identical config and seed, which produce byte-for-byte equivalent splits if the builder is deterministic. The README's `training/v6` shows this in practice. Content-addressing a build (hash of input version, subset, splits, seed and weights) would let identical requests reuse one version.

The finding that matters most: the test set depends on the intervention

`DatasetBuilder.build` assigns splits over `clean_version`, which for any excluding intervention is the *filtered* subset. Splits are therefore assigned after filtering:

Run	Episode pool split	Test episodes (0.8/0.1/0.1, quantile)
`baseline_full`	48	5 (README: 38/5/5)
`reduced_data` (fraction 0.5)	24	about 2–3, and **not** a subset of the baseline's 5
`imbalanced` (drop "short", about 16 of 48)	about 32	about 3, with **no short episodes at all**
`rebalanced` (weights only)	48	5; plausibly identical to the baseline's if the builder is deterministic in pool and seed (unverified)

Consequences:

1. `reduced` vs `baseline_full` **compares MSE on different test episodes, of different sizes.** With 2–3 test episodes, the per-episode spread (README: SO-100 per-episode MAE 0.085–0.111 across five episodes) is of the same order as the effect you'd be trying to detect.
2. **The imbalanced run can't measure the regime it removed.** Its test split contains no short episodes, so there's no `length_short` slice for it to fail on. The experiment designed to "give the rebalanced experiment a real weak regime to target" (its `objective:`) can't observe that weak regime.
3. **The feedback artifact's recommended action**, "retrain, then compare against this run on the same fixed test slice" (Module 8), is impossible for any excluding intervention.

What the docs say. `docs/EXPERIMENTS.md` §6 *does* acknowledge this, and that's to its credit: "Comparing two different dataset interventions is, necessarily, a comparison across two different `training/vN` versions' test splits, since an intervention can change which episodes are assigned to `test`". Two corrections to that sentence:

- It isn't *necessary*. It follows from filtering before splitting. Split the full pool **once** (a frozen `training/v_base`), then apply interventions *only to its train split* (filter or weight train episodes, inherit val and test unchanged). Every intervention is then scored on the identical test set by construction. The engine's lineage model supports this shape: a transform whose input is the split training version.
- It says "each experiment's `dataset.json` records exactly which … episode identifiers were evaluated". **It doesn't.** `dataset.json`'s `episode_ids` is the *whole* selected pool (train + val + test). `split_integrity.json` stores split *sizes* and overlaps, not membership. The evaluated test episodes can only be recovered from `predictions/test_outcomes.jsonl` (one `episode_id` per frame, Module 11), or by reopening the training version.

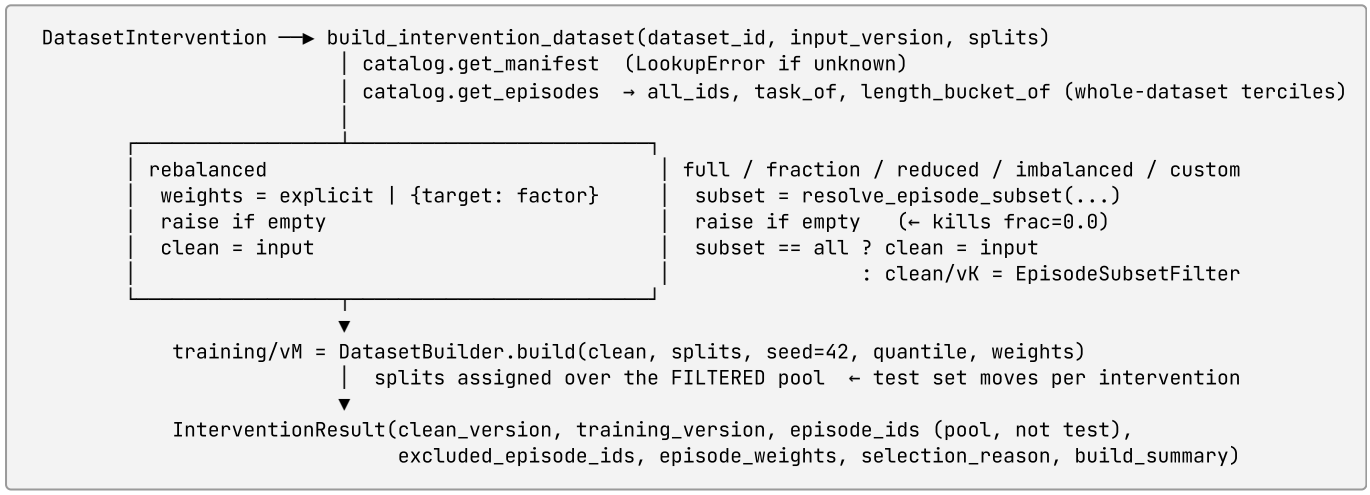
This is the gap to raise yourself in an interview. Finding it in your own experimental design, and having the "split once, intervene on train only" fix ready, is worth more than any of the features.

4.6 Honest gaps to own before an interviewer finds them

1. **Filter-then-split moves the test set.** Every excluding intervention is evaluated on a different, smaller test split than `baseline_full`, and the imbalanced run's test split doesn't contain the removed regime. *Consequence:* the core quantity-vs-coverage comparisons aren't like-for-like, and the imbalanced → rebalanced story can't be measured as designed.
2. **The zero-fraction anchor fails** (the resolver returns `[]`, the builder raises), despite the comment and a passing unit test. *Consequence:* a sweep including 0.0 crashes, and the test gives false confidence.
3. **Imbalanced means total removal, not under-representation**, contrary to the config template's hypothesis. *Consequence:* the experiment measures out-of-distribution generalization, so it must be described that way.
4. **Length buckets differ between intervention (whole-dataset terciles) and evaluation (within-split terciles)**, and the "same method as profile.py" docstring is false (profile has no tercile code). *Consequence:* "short" means different things at train time and test time.
5. **Tied lengths collapse the buckets** (`<= t1` puts ties in "short"). *Consequence:* on a dataset with uniform episode lengths, imbalancing "long" or "medium" silently runs full data.
6. **Fraction draws aren't nested across fractions.** *Consequence:* data-efficiency curves carry extra between-point variance from swapping episodes. Prefixes of a single permutation fix it.
7. **Explicit `episode_ids` and `episode_weights` aren't validated** against the dataset (unknown IDs, duplicates, non-positive weights). *Consequence:* engine-side behaviour is undefined here; duplicates create needless versions.

8. **The rebalanced "limitation in every report" claim is false** (`EXPERIMENTS.md` §4). *Consequence:* the report for a rebalanced run omits the key caveat about what the result can't show.
9. **`dataset.json` doesn't record test membership**, contrary to `EXPERIMENTS.md` §6. *Consequence:* the evaluated episodes are recoverable only from the predictions file.
10. **Version allocation isn't atomic, failed runs orphan versions, and identical requests always mint new versions.** *Consequence:* catalog clutter, plus a possible version collision under concurrency (unverified engine behaviour).
11. **The module docstring is stale** (it says `task_id` is "the one grouping key"). *Consequence:* the docs disagree with themselves; the code and `EXPERIMENTS.md` use the length bucket.
12. **A misleading error for unsupported rebalance variables** ("no episodes with X=Y"). *Consequence:* it slows down diagnosis.
13. **Weights are recorded for val and test episodes too.** *Consequence:* `dataset.json` overstates which episodes were upweighted in training.

The shape to carry into Module 5



Drill — answer from memory, files closed

1. Trace what happens to the catalog when you run `reduced.yaml` against `so100_pickplace@clean/v1`. Which versions are created, and what does the lineage record?
2. *Why didn't you just* pass a list of episode IDs to `fit()` instead of creating new dataset versions?
3. *What breaks in production:* someone runs `run_data_fraction_sweep` with `[0.0, 0.25, 0.5, 1.0]`.
4. Is `reduced_data`'s test MSE comparable to `baseline_full`'s? Explain precisely, and say what you'd change.
5. *Interviewer framing:* "Tell me about a flaw in your own experimental design."

Appendix A — Model answers to the drill

A1. Running `reduced.yaml`. "The intervention is `reduced` at fraction 0.5 with seed 0. The module reads the 48 episodes of `clean/v1` from the catalog, sorts the IDs so the draw doesn't depend on row order, and uses a seeded NumPy generator to draw 24 without replacement. Because that's a strict subset, it allocates the next clean version, say `clean/v2`, and runs the engine's `EpisodeSubsetFilter` transform from `clean/v1` to `clean/v2`. The lineage for that transform records the 24 kept and 24 dropped IDs and the reason string, 'seeded random 50% draw over all episodes (seed=0)'. Then it allocates the next training version and calls the engine's builder, which assigns train/val/test over those 24 episodes with split seed 42 in quantile mode. So the catalog gains two versions, and `rde version lineage` on the training version walks back through the filter to raw. The unit test checks that the transform's recorded kept and dropped IDs equal exactly what my code computed."

Why that answer lands: it's concrete down to the version names and the recorded reason, and it ends with how the claim is verified, not just asserted.

A2. Why not pass episode IDs to `fit()`? "Because then the experiment's data would exist only as an argument in one process. The question 'what exactly did this model train on' would depend on my run directory surviving and being read correctly. As engine versions, every intervention is an immutable, named dataset with lineage back to raw episodes and a recorded reason for every exclusion, so the data has an identity independent of my code. The cost is real. Every run writes to the shared catalog, identical requests mint duplicate versions, failures leave orphaned versions, and, the big one, because I filter before splitting, the split is recomputed over each filtered pool. If I were redoing it, I'd keep versions but change the order: split the full pool once, then express interventions as transforms on the train split only."

Why that answer lands: it defends the decision on provenance grounds, then volunteers that the same decision causes the design flaw, and gives the fix without abandoning the principle.

A3. A sweep with 0.0. "The 0.0 point fails before training. The resolver honours zero, returning an empty list, and there's a unit test that checks exactly that. But the only caller, `build_intervention_dataset`, raises 'resolved to zero episodes' on an empty subset, so the first sweep point throws and the sweep stops. The unit test passes because it tests the helper in isolation, not the path the sweep uses. Even if it got through, a zero-episode version can't be split or trained on. What a zero-data anchor should really mean is 'evaluate the pretrained base checkpoint with no fine-tuning', and that needs its own code path, not an empty filter. Separately, the other fractions' subsets aren't nested, so I'd draw them as prefixes of a single permutation."

Why that answer lands: it identifies the contradiction between the helper and the caller, explains why the test missed it, and redefines the experiment correctly. That shows judgement, not just bug-spotting.

A4. Is `reduced` comparable to `baseline_full`? "Not on a like-for-like basis. Splits are assigned after the filter, so the baseline is scored on 5 test episodes out of 48, and the reduced run on about 2 or 3 out of 24, which aren't even guaranteed to be a subset of the baseline's 5. With per-episode error varying by roughly 25% across episodes, a difference on 2–3 episodes is mostly noise about which episodes landed in test. The docs acknowledge the test sets differ, but call it necessary, and it isn't. I'd split the full 48 once and freeze that, then apply every intervention only to the train split, so every run is scored on the same 5 test episodes. And because those 5 are few, I'd report per-episode results and run several training seeds before claiming a quantity effect."

Why that answer lands: a precise "no", with the mechanism, the magnitude of the problem and the fix. Correcting your own docs' "necessarily" is exactly the kind of rigour the question is fishing for.

A5. "Tell me about a flaw in your own experimental design." "The data interventions are implemented as engine transforms that filter episodes, and the training-set builder then assigns splits over whatever's left. That gives perfect lineage, but it means each intervention gets its own test split. So 'reduced vs full' compares different test episodes, and worse, the imbalanced run, which drops every short episode, has no short episodes in its test set, so it can't show the failure it was designed to create. I found it by tracing what the builder's split input actually was, not by looking at results. The fix keeps the provenance model and changes the order: split once on the full pool, then intervene only on the train split, so every experiment shares one frozen test set. I'd also define the length buckets once, on the full dataset, and use the same cut points for the intervention and the evaluation slice, because today they're computed differently."

Why that answer lands: it's a real, non-trivial flaw, explained mechanistically, discovered by reasoning, with a fix that preserves the system's core principle. That's the most valuable answer type in a research-engineering interview.

Module 4 Summary — the one page to revise

What this module is. The translator from a declared data intervention to engine operations: an optional `EpisodeSubsetFilter` transform into `clean/vK`, then a `DatasetBuilder.build` into `training/vM` with splits and optional sampling weights. Provenance is excellent; filtering before splitting makes cross-intervention test sets incomparable.

Five claims to make cold

1. Interventions are engine transforms with recorded reasons and dropped IDs, never local copies, and a test verifies the lineage matches.
2. Rebalancing uses sampling weights over existing episodes. Nothing is duplicated or synthesized, so it can't close a coverage gap.
3. "Imbalanced" removes a regime entirely. It's an out-of-distribution test, not under-representation.
4. Splits are assigned after filtering, so every excluding intervention has its own smaller test set. The fix is to split once and intervene on train only.
5. The length variable exists because `so100_pickplace` has one task. The intervention and evaluation buckets use different cut points.

Decisions

Chose	Over	Because	Cost
Engine versions per intervention	IDs passed to `fit`	immutable identity, lineage, reasons	catalog churn, orphans, duplicate versions
Filter → build (split)	split → intervene on train	uses existing transform/build shape	test set changes per intervention
Sampling weights	duplication / synthesis	honest about no new data	repeated frames; sampler unverifiable here
Length-bucket terciles	a `task_id` variable	only varying attribute in so100	tie collapse; mismatched with eval terciles
Seeded `rng.choice` over sorted IDs	unseeded / unsorted	deterministic per seed	not nested across fractions
Reuse input when subset = all	always transform	no pointless versions	hides silent no-op imbalances

Volunteer unprompted: the moving test set; the 0.0 anchor that can't run despite its test; total removal vs under-representation; the bucket definitions don't match; `EXPERIMENTS.md` overclaims the rebalanced limitation and `dataset.json` test membership.

Numbers and names: so100 has 48 episodes, 326–475 frames, one task; terciles at the 33.3/66.7 percentiles; `< 3` episodes → all "medium"; default oversample 3.0; split seed 42, quantile; `EpisodeSubsetFilter`, `TransformPipeline`, `DatasetBuilder.build`, `next_version`, `VersionStage.CLEAN/TRAINING`; README lineage `raw/v1 → clean/v1 (50→48, dropped [9, 31]) → training/v6`.

Sentences worth memorizing

- "An intervention is a dataset version with a reason, not an argument to a training call."
- "I filter before I split, so each intervention gets its own test set. The fix is to split once and intervene on train only."
- "Dropping every short episode isn't imbalance; it's an out-of-distribution test, and its test set can't contain the regime it removed."

- "Rebalancing redistributes attention over data I already have; it can't close a coverage gap, and I don't claim it does."

Module 5 — Pre-training guards: split integrity and the dataset profile

Files covered: `src/model_training/data/split_integrity.py`, `src/model_training/data/profile.py`

Where we are: Module 4 produced a `training/vN` version. Before a single gradient step, the runner does two things to it: it re-checks that the splits don't leak, and it profiles the version (statistics, coverage, QC findings) for the report. Both are framed in the docs as *guards*, evidence that the data is fit to train on. This module asks how much each one actually guards.

The one-line thesis: The split check is honest code that checks the one property that can't fail, and passes the properties that *can* fail (unassigned episodes, empty splits). The profile is careful about saying what it can't measure, but its headline "coverage" number is normalized by the number of *occupied* bins, so it rewards a dimension that uses two bins exactly as much as one that uses all ten.

5.1 Prerequisites — the absolute basics

- **Data leakage (split leakage).** Information from the evaluation data reaching training. At episode level, that's the same episode ID in two splits. At content level, it's *different* episodes that are near-duplicates (recorded twice, or the same session and scene with trivially different motion). ID-level checks can only catch the first kind.
- **Partition.** A division of a set into disjoint subsets that together cover it. "Splits form a partition" means: no overlap, *and* nothing left over.
- **Defense in depth / independent verification.** Re-checking an invariant with code that doesn't share a path with the code that established it. It's only valuable if the invariant can actually be violated in the representation you check.
- **Tautological check.** A check whose failure condition can't be represented by the data it reads. It always passes, and it proves nothing.
- **Test double (fake).** A minimal stand-in object with just the methods a function under test calls. `_FakeView` in the tests is one.
- **Shannon entropy.** For a probability distribution $p_1 \ldots p_k$, $H = -\Sigma\, p_i \log_2 p_i$ bits. It's 0 when all mass is on one outcome, and $\log_2 k$ when spread evenly over k outcomes.
- **Normalized entropy (evenness).** H / H_{max}. The value of a normalized entropy depends entirely on **what you pick as H_max**: $\log_2(\text{number of possible bins})$ measures how much of the possible space is used evenly, while $\log_2(\text{number of occupied bins})$ measures only how even the occupied bins are (ecologists call this *Pielou's evenness*). The two answer different questions.
- **Equal-width histogram.** Split `[min, max]` into `B` equal intervals and count values per interval. Here `B = 10` (`_HIST_BINS`).
- **Coefficient of variation (CV).** `std / mean`, a scale-free measure of spread. *Not* `(max - min) / mean`. For a roughly normal sample of 48 values, the expected range is about 4.5 standard deviations, so range/mean comes out around 4.5× the CV.
- **DuckDB.** An in-process analytical SQL engine that queries Parquet and Arrow directly. The engine's `explorer.sql(...)` runs SQL against a version's `frames` table (inferred from the query text here). DuckDB **list indexing is 1-based**.

- **SQL injection.** Building SQL by string interpolation lets a crafted value change the query's meaning. Identifier quoting (`"col"` with embedded `"` doubled) is the defence for column names; parameters are the defence for values.

5.2 `verify_split_integrity`

```
class SplitLeakageError(Exception):
    """Raised when any two of train/val/test share at least one episode id."""

@dataclass
class SplitIntegrityReport:
    dataset_id: str
    version: str
    split_sizes: dict[str, int]
    train_val_overlap: list[int]
    train_test_overlap: list[int]
    val_test_overlap: list[int]
    unassigned_episodes: list[int]

    @property
    def ok(self) -> bool:
        return not (self.train_val_overlap or self.train_test_overlap or self.val_test_overlap)

    def to_dict(self) -> dict:
        return {**self.__dict__, "ok": self.ok}
```

```
def verify_split_integrity(view: DatasetView, *, raise_on_failure: bool = True) -> SplitIntegrityReport:
    train = set(view.episode_ids(split="train"))
    val = set(view.episode_ids(split="val"))
    test = set(view.episode_ids(split="test"))
    all_ids = {e.episode_id for e in view.episodes}

    report = SplitIntegrityReport(
        ...,
        split_sizes={"train": len(train), "val": len(val), "test": len(test)},
        train_val_overlap=sorted(train & val),
        train_test_overlap=sorted(train & test),
        val_test_overlap=sorted(val & test),
        unassigned_episodes=sorted(all_ids - train - val - test),
    )
    if raise_on_failure and not report.ok:
        raise SplitLeakageError(...)
    return report
```

What it does: it reads split membership back from the materialized view, intersects every pair, and raises on any overlap. The runner calls it with the default `raise_on_failure=True`, right after opening the training version (Module 11), and writes `split_integrity.json`. The README quotes that file as evidence: "Before any training step, an independent check confirms that no episode appears in more than one split, and the run aborts if one does."

Why it's framed as valuable (module docstring): the builder "already guarantees this by construction", and this check "exists to catch a *real* leak regardless of where it would come from -- including a bug this project's own intervention builder introduces -- by re-deriving the partition independently".

What it can actually catch. Look at how split membership is stored. `evaluation/offline.py` reads it as `e.attributes.get("split") == base_split` (Module 7). So **each episode carries one `split` attribute with one value.** An episode can't simultaneously have `split = "train"` and `split = "test"` in that representa-

tion. If `view.episode_ids(split=s)` filters on that attribute (very likely, though I can't verify the engine), then `train ∩ test` is *empty by construction of the data structure*, not just by the builder's correctness. The test file says as much in its fake's docstring:

> "used to prove the checker actually detects an engineered overlap, since a real `DatasetView` built by rde's own (correct) builder cannot produce one."

So **the overlap check is close to tautological**. It verifies a property the storage format makes unrepresentable. It isn't harmful, and it's cheap (three set operations over at most 48 IDs), but "independent re-verification" overstates it. The only way it could fire is if the engine stored membership as three separate ID lists, which isn't how this repository reads it.

What can actually go wrong, and passes:

1. **Unassigned episodes don't fail the check.** `ok` ignores `unassigned_episodes`. An episode with no split attribute is silently excluded from train, val *and* test, so it's lost data, and the report says `"ok": true`. The partition property ("covers the set") isn't enforced, only disjointness.
2. **Empty splits don't fail the check.** A version with `val` empty (for example from `splits.val: 0.0`, which Module 2 showed validates) or a tiny pool after a `reduced` intervention passes. An empty val split disables checkpoint selection (Module 6), and an empty test split makes the headline metric meaningless. Neither is caught here.
3. **Split sizes aren't compared with the config.** The report records `split_sizes` but never checks them against `config.splits`. A builder that put 46/1/1 for a requested 0.8/0.1/0.1 would pass.
4. **Content-level leakage is out of scope**, and that's the leak that actually matters in single-setup robot datasets. Episodes from one session in one scene are highly correlated across splits. The README itself concedes nearest-neighbour retrieval "is a strong baseline when test episodes come from the same setup". Nothing here (or anywhere) detects near-duplicate episodes across splits.

The `raise_on_failure=False` **escape hatch** is used only by tests. The production path always raises, which is right. The `test_split_leakage_report_can_be_inspected_without_raising` test exists to show the report is introspectable.

The tests use a hand-written fake (`_FakeView`, which exposes only `episodes` and `episode_ids`). That's good test design: the only way to exercise the failure branch is to construct an impossible view. It also proves the point above. **The failure branch is reachable only with a fake.**

What I'd change. Keep the disjointness check (it's free) and make `ok` mean the property that can fail:

```
ok = no_overlaps and not unassigned and all(split_sizes[s] > 0 for s in ("train", "val", "test"))
```

Also compare sizes against the requested fractions within ±1 episode for `quantile` mode, and frame the README sentence as "verifies the version is a complete, non-empty, disjoint partition". Content-level leakage would need episode embeddings or state-trajectory hashing. That's out of scope; name it as a known limitation.

5.3 `build_dataset_profile`, part 1: reuse, don't recompute

```
explorer = DatasetExplorer(settings=settings, catalog=catalog)

overview = explorer.overview(dataset_id, version)
durations = explorer.duration_distribution(dataset_id, version)
tasks = explorer.task_summary(dataset_id, version)
modalities = explorer.modality_summary(dataset_id, version)
dims = explorer.dimensionality(dataset_id, version)
qc = explorer.qc_summary(dataset_id, version)
lineage = explorer.lineage(dataset_id, version)
```

Decision: package the engine's explorer outputs instead of recomputing.

- *Why:* the module docstring says "this module's only real job is packaging those into the profile shape and adding the two things `DatasetExplorer` does not compute: coverage/diversity scores and the train/val/test split distribution". It's a single source of truth for statistics, so the profile and `rde`'s own CLI can't disagree.
- *Cost:* the profile's correctness is the explorer's correctness, and the profile's *shape* depends on undocumented dict keys (`durations["duration_s"]["max"]`, `dims[k]["per_dimension"][i]["min"]`, `lineage["chain"][-1]["inputs"]`). An engine change to any of those keys raises `KeyError` here, and nothing pins the engine version (Module 1).
- `explorer.overview` on an unknown version raises `LookupError` (`test_profile_of_unknown_version_raises`), so that error surfaces before any of the fallbacks below are reached.

```
parent_version = lineage["chain"][-1]["inputs"][0] if lineage["chain"] and lineage["chain"][-1]["inputs"] else None
```

The parent is the *first* input of the *last* lineage step. That's correct for this project's single-input transforms and builds. `test_profile_to_dict_is_json_shaped` asserts `parent_version == "clean/v1"` for the fixture's training version. For a multi-input operation (a merge), it would silently report only one parent.

```
manifest = catalog.get_manifest(dataset_id, version)
contract = manifest.attributes.get("training_contract") if manifest else None
if not contract:
    from rde.dataset_builder.builder import build_training_contract
    contract = build_training_contract(manifest) if manifest else {"observation": {}, "action": {}}
```

- **Fallback contract:** a `clean/vN` version has no stored training contract (only builds create one, by inference), so the profile derives one on the fly. That's why `mtrain profile` works on `clean/v1`, which is exactly what `EXPERIMENTS.md`'s workflow needs before choosing an imbalance variable.
- The `if manifest else {...}` empty-contract branch is effectively **unreachable**: a missing manifest means an unknown version, and `explorer.overview` already raised `LookupError` above.

```
act_key = contract.get("action", {}).get("key")
state_keys = [k for k, v in contract.get("observation", {}).items() if v.get("plane") == "inline"]
image_keys = [k for k, v in contract.get("observation", {}).items() if v.get("plane") == "external"]
```

Inline vs external is the modality test used everywhere (Module 1 vocabulary). Note that "state" means *every* inline modality. For ALOHA that's `state.joint` *and* `force.effort` (the smoke test's 14 + 14 = 28 dimensions). So "state-space coverage" for ALOHA includes effort sensors, not just joint positions.

Splits: "unassigned" isn't a split

```
if manifest is not None:
    engine = DataEngine(settings)
    try:
        with engine.get_dataset(dataset_id, version) as view:
            raw_splits = view.splits()
        if set(raw_splits) - {"unassigned"}:
            splits = raw_splits
    except Exception as exc:
        notes.append(f"could not resolve splits for {dataset_id}@{version}: {exc}")
if not splits:
    notes.append(f"{version!r} carries no split assignment (it is not a training/vN version) -- "
                 "splits reported empty, not fabricated")
```

- **The `"unassigned"` bucket is the subtle line.** The engine's `view.splits()` reports episodes with no split attribute under the key `"unassigned"`. A clean version is *entirely* unassigned, which means "no split exists yet", not "a split called unassigned". Without this line, the profile of `clean/v1` would report `{"unassigned": 48}` as if it were a real partition. It's tested (`test_profile_on_clean_version_has_no_splits`).
- **But on a training version with *some* unassigned episodes, `raw_splits` keeps the `"unassigned"` key**, so the profile reports it as a fourth split alongside train, val and test, while `split_integrity.json` reports it separately and still says `ok`. That's a second place unassigned episodes would show up and still not stop anything.
- **The broad `except Exception` plus a note** means profiling never fails because of the split lookup. Then the "no split assignment (it is not a training/vN version)" note is *also* appended, which is false if the cause was an exception on a training version. Two notes, one of them misleading.
- **The `with ... as view`** context manager is used here, while the runner and CLI use explicit `try/finally: view.close()`. Both are correct; the codebase just isn't consistent.

5.4 `build_dataset_profile`, part 2: coverage

```
_HIST_BINS = 10

def _shannon_entropy_bits(counts: list[int]) -> float | None:
    total = sum(counts)
    if total <= 0:
        return None
    probs = [c / total for c in counts if c > 0]
    return -sum(p * math.log2(p) for p in probs)

def _normalized_entropy(counts: list[int]) -> float | None:
    """0 (all mass in one bucket) .. 1 (uniform across every non-empty bucket)."""
    h = _shannon_entropy_bits(counts)
    if h is None:
        return None
    k = sum(1 for c in counts if c > 0)
    if k <= 1:
        return 0.0
    return round(h / math.log2(k), 4)
```

The normalization is by `k`, the number of *occupied* bins, not by `_HIST_BINS`. Work two examples through:

Dimension's 10-bin counts	H (bits)	k	Normalized entropy reported	$\log_2(10)$-normalized
`[50, 0, 0, 0, 0, 0, 0, 0, 0, 50]` (bimodal, e.g. a gripper that's open or closed)	1.0	2	**1.0**	0.30
`[10]*10` (uniform across the whole range)	3.32	10	**1.0**	1.0
`[90, 10, 0, …]`	0.47	2	0.47	0.14

A dimension using two of ten bins scores the same "perfect coverage" as one spread across all ten. The metric is *evenness among occupied bins* (Pielou's J), not coverage. Range-filling, which is what "coverage" means in ordinary speech and in the report's label "action-space coverage", is ignored, because the empty bins *are* the gaps and they're excluded from the denominator.

What the docs say:

- The function's docstring is accurate: "uniform across every non-empty bucket".
- The methodology string written into every profile is half accurate: "0 = all mass in one bin, 1 = uniform across every non-empty bin. **Measures how evenly the *observed* range is used**". The first sentence is right. The second is wrong: the observed range is `[min, max]` split into ten bins, and empty interior bins are exactly unused range.
- The module docstring says coverage is "a real, computable proxy for 'how much of the observed range gets used'". That's the same overclaim.

Fix (one token): `return round(h / math.log2(_HIST_BINS), 4)` for per-dimension coverage. Keep the evenness variant if you want it, under its real name. For *task* entropy, `k`-normalization is arguably fine, because there's no "possible task" denominator.

Edge behaviour, for completeness:

- `k <= 1 → 0.0`: a single task (so100) gives task entropy and instruction diversity of `0.0`. That's correct and honest, and it's how the profile *shows* that `task_id` can't be the imbalance variable.
- `total <= 0 → None`: an empty version produces `None`, not a fake number.
- `round(..., 4)`: four decimals, display precision only.

The per-dimension histogram query

```
def _dim_coverage(explorer, dataset_id, version, column, index, lo, hi):
    if hi <= lo:
        return {"normalized_entropy": None, "reason": "zero-range (constant) dimension"}
    col = '"' + column.replace('"', '""') + '"'
    sql = (
        f"SELECT LEAST({_HIST_BINS - 1}, GREATEST(0, "
        f"CAST(FLOOR({_HIST_BINS} * ({col}[{index + 1}] - {lo}) / ({hi} - {lo})) AS INTEGER))) "
        f"AS bucket, COUNT(*) AS n FROM frames GROUP BY 1"
    )
    try:
        rows = explorer.sql(dataset_id, version, sql)
    except Exception as exc:  # pragma: no cover
        return {"normalized_entropy": None, "reason": f"histogram query failed: {exc}"}
```

Every non-obvious piece:

- **`hi <= lo` guard:** without it, a constant dimension divides by zero inside DuckDB. It's reported as `None` with a reason, which is the project's "unavailable, not fabricated" pattern applied well.

- **Identifier quoting** (`'"' + column.replace('"', '""') + '"'`): column names like `action.joint` contain a dot, which DuckDB would otherwise parse as `table.column`. Doubling embedded quotes is the correct SQL escape. This is *necessary for correctness*, not just security.
- **`[index + 1]`:** DuckDB lists are 1-indexed; Python's `enumerate` is 0-indexed. Without the `+ 1`, every dimension would be histogrammed as its neighbour, and the last would read out of bounds (NULL).
- **`LEAST(9, GREATEST(0, …))` clamp:** a value exactly equal to `hi` computes bucket 10, and the clamp folds it into bucket 9. It also silently folds anything *outside* `[lo, hi]` into the edge bins. If the explorer's min/max were ever computed on a different population than the `frames` table (for example, raw vs normalized values), every value would pile up in bins 0 and 9 with no error.
- **`lo` and `hi` are interpolated as Python float literals.** That's safe from injection because they're numbers from the engine. A `nan` or `inf` would produce invalid SQL and fall into the `except`. Parameter binding would be cleaner.
- **NULL values** produce `bucket IS NULL`, and `if b is not None` skips them. Missing values are silently excluded from coverage. That's defensible, but it isn't reported.
- **Cost:** one full scan of `frames` per dimension. For ALOHA that's 14 action plus 28 state dimensions, so 42 scans per profile. That's seconds at this data size. The video plane isn't touched, which matters: only the inline columns are read.

Two smaller mislabels

```
"episode_length_cv": (
    round(length_frames.get("mean", 0) and
          (float(durations["duration_s"]["max"] - durations["duration_s"]["min"])
           / max(durations["duration_s"]["mean"], 1e-9)), 4)
    if durations else None
),
```

- **It isn't a coefficient of variation.** It's `(max - min) / mean` of duration. For a roughly normal sample of 48 episodes, range ≈ 4.5σ, so this reports about **4.5× the true CV**. The label invites exactly the wrong reading.
- **`length_frames.get("mean", 0) and (...)`** uses Python's `and` as a guard: if the frames mean is 0 (or missing), the value is `0`; otherwise it's the ratio. It guards on the **frame** mean but divides by the **duration** mean. That works only because both are zero together. The `max(..., 1e-9)` guard is then redundant. It's convoluted for what should be `statistics.pstdev(lengths) / mean(lengths)`.
- **`task_coverage.normalized_entropy` and `instruction_diversity.diversity_normalized_entropy` are the same number**, computed twice from `task_episode_counts`. The profile presents one fact as two metrics.

The parts that are genuinely good

```
"visual_diversity": {"status": "unavailable",
                     "reason": "requires an image embedding/similarity model; ..."},
"episode_diversity": {"status": "partial", ..., "reason_partial": "limited to task-id and length variability; ..."},
```

```
missing_data = {
    "source": "rde QC finding records (qc_records) for this version -- only failing/warning "
              "checks are persisted as rows (passing checks leave no row), so a check's "
              "absence below means either it found nothing or has not been run against "
              f"{dataset_id}@{version} specifically; cross-check with `rde obs quality`",
    ...
}
```

- **Refusing to compute visual diversity** without an embedding model, and saying so in the data, is the right call. A pixel-variance "diversity score" would look rigorous and mean nothing.
- **The QC caveat is the best-written line in the module.** It names a real ambiguity in absence-of-evidence ("no row" means "passed *or* never ran") that most tools hide. Quote it in an interview as an example of honest metadata.
- `variation_note` states that the schema has no variation field and that `task_id` and length are proxies "rather than an invented 'variation' axis". It's consistent with Module 4.

What population is profiled

The runner profiles `training_version`, the whole post-intervention version, *including val and test episodes* (Module 11). So the report's "action-space coverage" and "Dataset statistics (from profile)" describe train ∪ val ∪ test, not what the model trained on. For a "quantity vs coverage" study, you'd want coverage of the **train split**, ideally next to coverage of the **test split**, so you can say whether test states fall inside the trained range.

`to_dict` **lists all 20 fields by hand** instead of `dataclasses.asdict(self)`. Add a field and forget to add it to `to_dict`, and it silently vanishes from `dataset_profile.json`. `asdict` would deep-copy nested dicts, which is a cost here, but `{f.name: getattr(self, f.name) for f in fields(self)}` would avoid both problems.

5.5 Honest gaps to own before an interviewer finds them

1. **The overlap check is near-tautological.** Split is a single attribute per episode, so an overlap can't be represented, and the failure branch is reachable only with a fake view. *Consequence:* "an independent check confirms no leakage" overstates what's verified.
2. `ok` **ignores unassigned episodes, empty splits, and sizes vs config.** *Consequence:* the properties that *can* actually fail (lost episodes, an empty val split that disables selection) pass with `"ok": true`.
3. **No content-level leakage detection.** *Consequence:* near-duplicate episodes from the same session can sit in train and test. This is the realistic leak in single-setup datasets, and it's consistent with nearest-neighbour retrieval being so strong.
4. **"Coverage" is normalized by occupied bins.** *Consequence:* a two-bin bimodal dimension scores 1.0, the same as a uniform one. "Action-space coverage" in every report measures evenness, not range use, and the methodology string misdescribes it.
5. `episode_length_cv` **is range/mean, about 4.5× the true CV at n = 48.** *Consequence:* a mislabelled number in every profile.
6. **The profile covers train ∪ val ∪ test, not the train split.** *Consequence:* it can't answer "did training cover the test states?", which is what a coverage study needs.
7. **Dependence on undocumented explorer dict keys, with no engine pin.** *Consequence:* an engine refactor breaks profiling with `KeyError`.
8. **The histogram clamp silently folds out-of-range values into the edge bins, and NULLs are dropped unreported.** *Consequence:* a min/max mismatch produces confident but wrong coverage.
9. **Duplicate metric** (task entropy = instruction diversity). *Consequence:* one fact is presented as two.
10. **A misleading note after a split-lookup exception** ("it is not a training/vN version"). *Consequence:* it misdiagnoses.
11. `to_dict` **is hand-listed.** *Consequence:* new fields can silently disappear from the persisted profile.
12. `parent_version` **takes the first input only.** *Consequence:* wrong for future multi-input operations.

The shape to carry into Module 6

```
training/vN (DatasetView)
  │
  ├─ verify_split_integrity(view)                 runner: raise_on_failure=True
  │     train/val/test = view.episode_ids(split=…)
  │     overlaps  → SplitLeakageError   (unrepresentable if split is one attribute/episode)
  │     unassigned / empty / size-vs-config → NOT checked → ok=true
  │     → split_integrity.json
  │
  └─ build_dataset_profile(dataset_id, training_version)     (all splits pooled)
        DatasetExplorer: overview · durations · tasks · modalities · dims · qc · lineage
        contract = stored | build_training_contract(manifest)
        splits   = view.splits() minus pure-"unassigned"
        coverage = per-dim DuckDB 10-bin histogram → H / log2(#occupied bins)   (evenness!)
        visual_diversity = "unavailable" (honest)   episode_length_cv = range/mean (mislabel)
        → dataset_profile.json → report + task/length figures
```

Drill — answer from memory, files closed

1. What exactly does `verify_split_integrity` check, and what would have to be true about the storage format for it to ever fail on a real dataset?
2. *Why didn't you just* trust the builder's guarantee and skip the check, or, alternatively, why not make the check stronger?
3. Compute the profile's reported normalized entropy for a gripper dimension whose values fall half in the lowest bin and half in the highest. What's wrong with that number?
4. *What breaks in production:* a builder bug leaves three episodes with no split attribute. What happens across the pipeline?
5. *Interviewer framing:* "How do you know your test set isn't leaking into training?"

Appendix A — Model answers to the drill

A1. What the split check verifies. "It reads the train, val and test episode IDs back from the materialized training version, intersects each pair, and raises `SplitLeakageError` if any intersection is non-empty. It also records split sizes and any episode in none of the three. For the overlap branch to fire on real data, the engine would have to be able to store an episode in two splits at once. But split is a single attribute on each episode, and the evaluation code reads it as `attributes['split']`, so an overlap isn't representable. The test file says so itself: it uses a fake view because the real builder can't produce an overlap. So the overlap check is cheap insurance against a representation change, not a meaningful independent verification."

Why that answer lands: it describes the check precisely, then shows you've reasoned about *whether it can fail*. That's the move that separates real validation from ceremony.

A2. Skip it or strengthen it? "I'd keep it, since three set intersections over 48 IDs are free and it documents the invariant in the run directory, but I'd make it check what can actually go wrong. Today `ok` ignores unassigned episodes and empty splits, and those are the realistic failures. An empty val split silently disables checkpoint selection, and unassigned episodes silently vanish from training and evaluation. So `ok` should mean a complete, non-empty partition whose sizes match the configured fractions within one episode. The harder leak, near-duplicate episodes from the same recording session ending up on both sides, needs content-level checks like trajectory hashing or embedding similarity. I'd list that as a known limitation rather than imply the ID check covers it."

Why that answer lands: it rejects both extremes, specifies the stronger invariant exactly, and distinguishes ID-level from content-level leakage. That's a sign of real data-quality experience.

A3. The gripper example. "Two occupied bins with equal mass give 1 bit of entropy. The code normalizes by the log of the number of *occupied* bins, $\log_2 2 = 1$, so it reports 1.0, the same score as a dimension spread uniformly across all ten bins. Normalized by the ten possible bins it would be about 0.30. So the number measures evenness among occupied bins, not how much of the range is used. The empty bins are exactly the coverage gaps, and they're excluded from the denominator. The function's docstring says 'uniform across every non-empty bucket', which is accurate, but the methodology text and the report label call it coverage of the observed range, which it isn't. The fix is to normalize by $\log_2$ of the bin count."

Why that answer lands: you do the arithmetic out loud, find the conceptual error, separate the accurate docstring from the inaccurate label, and give a one-token fix. Interviewers love a worked number.

A4. Three unassigned episodes. "The split check computes them as unassigned but still reports `ok: true`, so nothing stops. The training loop asks for train IDs and never sees them. Validation and test ask for their splits and never see them. So three episodes silently drop out of the experiment. The profile, which runs over the whole version, would include them in its statistics, and would also show an extra 'unassigned' split, so the profile and the actual training data disagree. `dataset.json` would still list them among the selected episodes. The only place you'd notice is comparing split sizes against the episode count. The fix is to make unassigned episodes fail the integrity check, because a partition that doesn't cover the set isn't a partition."

Why that answer lands: it traces the failure through every downstream consumer, including the inconsistency between artifacts, and ties the fix to a crisp definition.

A5. "How do you know your test set isn't leaking into training?" "At the episode level: splits are assigned per episode, never per frame, so neighbouring near-identical frames can't straddle the boundary, and each episode carries exactly one split label. A check before training reads the splits back and confirms they're disjoint. Selection uses validation only; the trainer hard-codes the val split for scoring. The test set is

touched once, after a checkpoint is chosen. At the content level I can't claim it isn't leaking. Both datasets are single-setup recordings, so test episodes are close relatives of training episodes, and you can see it in the results: frame-level nearest-neighbour retrieval ties fine-tuned SmolVLA on SO-100. That's the honest framing. These are in-distribution held-out episodes, not a generalization test, and I'd add a held-out-session or held-out-scene split before claiming generalization."

Why that answer lands: it gives the real protections, then volunteers the limitation, backed by evidence from your own results, and names the fix that would justify a stronger claim.

Module 5 Summary — the one page to revise

What this module is. Two pre-training steps over the new `training/vN`: a split-disjointness check that raises on overlap, and a profile that packages the engine's explorer statistics and adds per-dimension histogram "coverage" plus honest "unavailable" markers. Both are useful records, and both are weaker guards than the docs imply.

Five claims to make cold

1. The split check runs before training and raises on overlap. Overlap is essentially unrepresentable because split is one attribute per episode.
2. `ok` doesn't cover unassigned episodes, empty splits, or sizes vs config. Those are the failures that can actually happen.
3. ID-level checks can't detect content-level leakage. Single-setup data makes held-out episodes in-distribution, and nearest-neighbour's strength shows it.
4. Profile coverage is entropy normalized by *occupied* bins, so it's evenness, not range use. The fix is to normalize by $\log_2(10)$.
5. The profile refuses to fabricate visual diversity, and documents that missing QC rows mean "passed or never ran".

Decisions

Chose	Over	Because	Cost
Re-check splits from the view	trust the builder	defense in depth, recorded evidence	checks an unrepresentable failure; misses real ones
Explorer reuse	recomputing stats	single source of truth with `rde` CLI	coupled to undocumented dict keys
10 equal-width bins plus entropy	no coverage metric / k-NN coverage	cheap, one DuckDB pass per dimension	occupied-bin normalization; clamp hides range mismatch
"unavailable" visual diversity	pixel-variance proxy	no embedding model; refuse to fake	profile is numeric-only
Fallback contract for clean versions	require a training version	profile before choosing an intervention	none
Profile `training_version`	profile train split only	one call, the whole version	coverage includes test episodes

Volunteer unprompted: the tautological overlap branch; `ok` ignores unassigned and empty splits; the occupied-bin normalization; `episode_length_cv` is range/mean; coverage pools all splits; there's no content-level leak check.

Numbers and names: `_HIST_BINS = 10`; DuckDB 1-based list index (`[index + 1]`); clamp `LEAST(9, GREATEST(0, …))`; `k <= 1 → 0.0`; ALOHA state = 28 dimensions (joint 14 + effort 14), so 42 histogram queries; range of n = 48 normal samples ≈ 4.5σ; README split 38/5/5, all `overlap: []`, `ok: true`.

Sentences worth memorizing

- "A check you can only make fail with a fake object isn't verifying anything the data can do."
- "A partition has to be disjoint *and* complete; I only checked the half that couldn't break."
- "My 'coverage' metric was really evenness: dividing by occupied bins hides exactly the gaps."
- "These are held-out episodes from the same setup, which is why retrieval is such a strong baseline. It's not a generalization test."

Module 6 — The training loop around `fit`

Files covered: `src/model_training/training/trainer.py` (`training/__init__.py` was covered in Module 1)

Where we are: Modules 4 and 5 produced a verified, profiled `training/vN`. This module spends the GPU budget. Module 1 established that the gradient step itself, `SmolVLAPolicy.fit`, lives in the engine. What's here is everything *around* the step: the callback that validates, checkpoints, selects and prunes, plus crash recording and memory release. This is the code you can honestly say you own in "the training loop."

The one-line thesis: `Trainer` is a careful *Template-Method-by-callback* wrapper. It runs validation only on the val split, verifies every checkpoint before it can become "best," never prunes the best one, and always saves a final checkpoint. But the checkpoint it selects is chosen by **32 validation frames of a stochastic policy**. That's small enough that the README's validation MSE is about 13× lower than its test MSE. The crash handler also catches fewer failures than its comment claims.

6.1 Prerequisites — the absolute basics

- **Training step.** One forward pass, loss, backward pass and optimizer update on one batch. "1,000 steps at batch size 4" means 4,000 sampled frames (or sample windows), not 1,000 epochs.
- **Gradient accumulation (`grad_accum_steps`).** Sum the gradients of `k` micro-batches before one optimizer update. The effective batch is `k × batch_size`. Here `k = 1`.
- **Gradient clipping (`max_grad_norm = 1.0`).** If the gradient vector's L2 norm exceeds the threshold, rescale it down. This stops one bad batch from blowing up the weights.
- **Weight decay (`1e-5`).** A small pull toward zero on the weights each step (L2-style regularization).
- **Learning-rate warmup and schedule.** Start the LR near zero and ramp it up linearly over `warmup_steps`. A sudden full LR on pretrained weights, with an optimizer whose statistics are still empty, can wreck them. `docs/API_REFERENCE.md` says the schedule is "warmup then cosine decay". That lives in the engine and is **unverified** here.
- **Optimizer state.** Adam-family optimizers keep two running averages per parameter (first and second moments). Restarting without them behaves differently from continuing, which is why resuming needs a fresh warmup (Module 2).
- **Checkpoint cadence.** Save every `checkpoint_every` steps. **Validation cadence** means scoring on held-out data every `validate_every` steps.
- **Model selection / early stopping.** Pick the checkpoint with the best validation score instead of the last one. That's a guard against overfitting.
- **Winner's curse (selection bias).** If you take the minimum of `m` noisy measurements, the selected value is biased *low*. The more candidates and the noisier the estimate, the bigger the bias. The selected checkpoint's validation score then overstates how good it is.
- **Effective sample size.** Consecutive frames of one episode are highly correlated, so 32 adjacent frames carry roughly the information of a handful of independent samples, not 32.
- **Template Method pattern.** A fixed algorithm skeleton (here, `fit`'s loop) with hook points that subclasses or callers fill in (here, `on_step`). When the hook is a function argument rather than a subclass override, it's often called *inversion of control* or a *callback hook*.
- **Closure / `nonlocal`.** A nested function that captures variables from its enclosing function. `nonlocal x` lets the nested function rebind `x`.

- **`BaseException` vs `Exception`.** `KeyboardInterrupt` (Ctrl-C) and `SystemExit` inherit from `BaseException`, *not* `Exception`, so `except Exception` doesn't catch them. SIGTERM's default action terminates the process without raising anything. SIGKILL (the Linux OOM-killer) can't be caught at all.
- **PyTorch CUDA memory.** `torch.cuda.max_memory_allocated()` reports the peak bytes *handed out by the caching allocator to tensors*. It excludes the allocator's cached-but-free blocks and the CUDA context (often several hundred MB), so it's lower than what `nvidia-smi` shows. `empty_cache()` returns *unreferenced* cached blocks to the driver. It can't free memory that something still references.

6.2 Shape of `Trainer.train`

```
train(train_view, val_view, episode_ids, episode_weights)
  ├─ guard: model must be "smolvla"
  ├─ build SmolVLAPolicy(resume_from or pretrained); load(contract or None)
  ├─ reset CUDA peak stats; resolve train ids (raise if empty)
  ├─ define on_step(record)            ← the hook: log · validate · checkpoint · verify · select · prune
  ├─ try:   fit_result = policy.fit(..., on_step=on_step)      ← engine owns the loop
  │  except Exception: write crash.json; del policy; empty_cache; raise
  ├─ save checkpoints/final (always)
  ├─ peak GPU; del policy; empty_cache
  └─ return TrainingOutcome(best_checkpoint = best or final, ...)
```

Decision: drive the engine's `fit` loop through a callback rather than writing the loop here.

- *What:* `fit` owns the iteration. This module injects behaviour at each step through `on_step(record)`.
- *Why:* the step mechanics (batching against the dataset contract, weighted sampling, the optimizer) are engine-coupled (Module 1). The run *policy* (when to validate, what to keep) is experiment-coupled. The callback is the seam between them.
- *Alternative:* `fit(steps=100)` called in a loop of chunks, returning control between chunks. That's simpler to reason about, but it needs optimizer and schedule state carried across calls (`fit` does accept `optimizer_state=` per `docs/API_REFERENCE.md` line 792, which is unverified). The other alternative is a framework trainer (PyTorch Lightning, HF `Trainer`) with its own callback system.
- *Cost:* the callback runs **inside** the engine's loop, so validation, checkpoint I/O and pruning all happen on the training thread, between steps, and their time is counted in `wall_clock_s` (the README's "Training time 324 s" includes them). An exception inside `on_step` (a full disk during `save_checkpoint`) propagates out of `fit` and is recorded as a training crash.

6.3 Setup: model construction and resume

```
if tc.model != "smolvla":
    raise ValueError(f"Trainer implements real gradient-based fine-tuning for model='smolvla' only ...")
```

This duplicates the runner's own branch, which already only constructs a `Trainer` when `model == "smolvla"` (Module 11). It's defensive and correct, and it makes `Trainer` safe to call directly.

```
checkpoint_dir = self.run_dir.sub("checkpoints")
policy = SmolVLAPolicy(checkpoint=tc.resume_from or tc.pretrained_checkpoint, device=tc.device)
# Resuming one of our own checkpoints: it is already configured (shapes
# and normalization stats) for this exact dataset -- reload as-is. Only
# a *fresh* pretrained checkpoint (which may target a different
# embodiment than train_view's) needs its shapes/stats overridden from
# the dataset contract ...
policy.load(contract=None if tc.resume_from else train_view.contract)
if tc.resume_from:
    meta = SmolVLAPolicy.verify_checkpoint(tc.resume_from)
    self.run_dir.write_json("resume_source.json", meta)
```

Decision: pass the dataset contract only for a fresh start.

- *Why:* `smolvla_base` was pretrained with *its own* input and output shapes and normalization. The SO-100 run needs 6-dimensional action, and ALOHA needs 14-dimensional action plus 28-dimensional state (the smoke test's cross-embodiment cases). Loading with the contract rewrites the features and normalization stats from *this* dataset. A resumed checkpoint already has them. Passing the contract again could overwrite the checkpoint's stats with the new version's (identical here, but not in general).
- *Cost:* **a resume isn't checked against the dataset.** `resume_source.json` records the source checkpoint's `dataset_id` and `dataset_version` (written into every checkpoint's `extra`, see below) but never compares them with `train_view`. Resume an SO-100 checkpoint against ALOHA by mistake and the shapes disagree. The failure would surface somewhere inside the engine's batching, as a tensor-shape error far from the cause. `if meta["dataset_id"] != train_view.dataset_id: raise` is one line.
- **Order:** `load` runs *before* `verify_checkpoint`, so a corrupt resume directory fails inside `load` with whatever error the engine raises, and the clear "incomplete checkpoint" message from `verify_checkpoint` (asserted by `test_corrupted_checkpoint_directory_rejected`) is never reached. Swap the two lines.

Optimizer state isn't saved, and it could be. Every periodic checkpoint is written with `"optimizer_state_saved": False`, and resume passes no optimizer state. Module 2 framed the fresh warmup on resume as the consequence. Per `docs/API_REFERENCE.md`, the engine's `fit` *returns* `"optimizer_state"` and *accepts* `optimizer_state=`. So this is a choice, not an engine limitation (unverified, since I can't read the engine). The likely reason is disk: Adam's two moments are twice the parameter count, roughly tripling checkpoint size. That's a reasonable choice on a laptop. It isn't recorded as a reason anywhere, and the API doc's §9.2 presents the LR restart as inherent.

```
import torch  # safe: policy.load() above already required it to succeed
```

This is the lazy import that keeps the module importable without torch (Module 1), but `trainer.py`'s *top-level* `from rde.model_adapters.models import SmolVLAPolicy` means importing this file still loads the engine's model module. The comment is accurate about torch specifically.

```
if torch.cuda.is_available():
    torch.cuda.reset_peak_memory_stats()
peak_rss = proc.memory_info().rss if proc else None
```

Peak stats are reset *after* the model is loaded, so `peak_gpu_mb` measures training, not the load. The README's "Peak GPU memory 2.2 GB / 3.0 GB" is therefore allocator-reported peak during training. It's lower than `nvidia-smi` would show (context plus cache excluded), so it understates what you'd need to provision on a GPU.

```
ids = episode_ids if episode_ids is not None else train_view.episode_ids(split="train")
if not ids:
    raise ValueError("no training episodes resolved -- nothing to fine-tune on")
```

The fallback to `split="train"` is for direct callers. The runner always passes train IDs. The empty guard runs **after** the model has been loaded onto the GPU, so an empty-train failure costs a full model load first. Moving it to the top is free.

6.4 The hook: `on_step`

```
val_adapter = VisionLanguageAdapter(val_view.contract)
evaluator = Evaluator()
max_val_frames = tc.val_batches * tc.batch_size     # 8 × 4 = 32 with base.yaml

checkpoints: list[dict[str, Any]] = []
best: dict[str, Any] = {"metric": None, "checkpoint": None}
last_step = 0

def on_step(record: dict[str, Any]) -> None:
    nonlocal peak_rss, last_step
    last_step = record["step"] + 1
    self.run_dir.append_jsonl("metrics/train.jsonl", {"experiment_id": self.config.experiment_id, **record})
    if proc is not None:
        peak_rss = max(peak_rss or 0, proc.memory_info().rss)

    step = last_step  # steps completed so far, 1-indexed
    do_ckpt = bool(tc.checkpoint_every) and step % tc.checkpoint_every == 0
    do_val = bool(tc.validate_every) and step % tc.validate_every == 0
    if not (do_ckpt or do_val):
        return
```

- **`best` is a dict, not two variables,** so the closure can *mutate* it without `nonlocal`. `peak_rss` and `last_step` are rebound, so they need `nonlocal`. Mixing both styles in one closure is legal but inconsistent.
- **`record["step"] + 1`** assumes the engine reports 0-indexed step numbers. With `steps = 1000` and `checkpoint_every = 100`, saves land at 100, 200, ..., 1000, which matches the README's "step_1000". With `grad_accum_steps > 1` it's unknown whether `record` fires per micro-batch or per optimizer step. If per micro-batch, every cadence would be off by a factor of `k`. That's unverifiable here and unexercised (`grad_accum_steps: 1` everywhere).
- **`bool(tc.checkpoint_every) and …`** makes `0` mean "never" and avoids `step % 0` (a `ZeroDivisionError`). That's the defensive branch, and it has a purpose.
- **One JSONL append per step** means 1,000 file open/write/close cycles. That's negligible next to a SmolVLA step (around 300 ms per step at the README's 324 s per 1,000 steps), but it's an fsync-free append, so the Module 3 torn-write caveat applies.
- **Peak RSS is sampled once per step.** It misses transient spikes between samples, which is fine as a rough number.

Validation: what "best" is chosen by

```
    val_metric = None
    if do_val:
        policy.eval_mode()
        result = evaluator.evaluate(
            val_view, policy, val_adapter, split="val", batch_size=tc.batch_size,
            max_frames=max_val_frames, persist=False,
        )
        policy.train_mode()
        val_metric = result.metrics.get(ec.selection_metric)
        self.run_dir.append_jsonl("metrics/validation.jsonl",
                                  {"experiment_id": self.config.experiment_id, "step": step, **result.metrics})
```

- **`split="val"` is hard-coded here,** and *this* is where "selection never touches test" is actually enforced. It isn't in the nonexistent `model_training.experiments.selection` that the schema comment cites (Module 2). Say it that way: it's enforced structurally, by the only call site.
- **`persist=False`** stops the evaluator from writing these interim results into the engine's catalog (inferred from the flag name). Contrast this with the feedback step, which uses `persist=True` (Module 8).
- **`eval_mode()` / `train_mode()` toggling** switches dropout and normalization behaviour. If `evaluate` raises, `train_mode()` is never called, but the exception aborts training anyway, so it doesn't matter.

The validation sample is tiny. `max_val_frames = val_batches × batch_size = 8 × 4 = 32` frames. The SO-100 val split is 5 episodes. The README's test split of 5 episodes has 1,908 frames, so val is about 1,900 frames. **Validation scores roughly 1.7% of the validation set.** Which 32 frames is decided by the engine's `Evaluator` with `max_frames`, and I can't verify that. If it takes the first `N` frames in iteration order (the most common implementation), the 32 frames are **the first ~1 second (at 30 fps) of the first validation episode**.

Circumstantial evidence that this matters, from the README itself:

- README §2.2 says SO-100 error "is lowest at the start (0.051) … of an episode and peaks in the middle … at 0.144". That's mean absolute error by progress decile on test.
- README result 2 says validation MSE "reached its minimum (**0.0018**) at step 1,800", while the selected checkpoint's **test** MSE is **0.0239**. **Test is about 13× worse than validation.**
- An MAE of about 0.05 corresponds to an MSE on the order of 0.003, the same order as the validation numbers. Validation on start-of-episode frames would produce exactly this pattern.

I can't *prove* the first-N behaviour without the engine. But the 13× val–test gap is a real number in the README, and nothing else in the code explains it as naturally. (Distribution shift between val and test episodes is possible with 5 episodes each, but 13× is large.) Even if the frames were drawn at random, the setup has three compounding problems:

1. **Correlated frames.** 32 frames from one or two episodes have a tiny effective sample size.
2. **A stochastic policy.** README §2.3 says flow-matching sampling moves MSE by up to 5% when re-scoring the *same* frames.
3. **Selection over 10–20 noisy estimates** (every 100 steps across 1,000–2,000 steps) means a winner's-curse bias in `best_val_metric`. The reported "selection value" is optimistic by construction.

Decision: a fixed small validation budget instead of the full val split.

- *Why (inferred):* a full validation pass over about 1,900 frames at the README's roughly 47–50 ms per frame is about 90 seconds per pass, about 15 minutes over ten passes. That would triple a 5-minute training run on a laptop GPU.
- *Cost:* checkpoint selection is close to noise on SO-100, where validation MSE was still falling. It probably picked the late checkpoint correctly *because* the trend dominated the noise.

- *The fix that keeps the budget:* choose the 32 (or 128) validation frames **once**, stratified across every val episode and across episode progress, with a fixed seed. Score every checkpoint on that **same frozen subset**, and average two stochastic samples per frame. The cost stays the same, the noise drops sharply, and every checkpoint is compared on identical inputs.

Checkpoint, verify, select, prune

```
    if do_ckpt:
        ckpt_path = checkpoint_dir / f"step_{step}"
        policy.save_checkpoint(ckpt_path, extra={
            "experiment_id": ..., "step": step,
            "dataset_id": train_view.dataset_id, "dataset_version": train_view.version,
            "optimizer_state_saved": False,
        })
        # A process killed mid-write (disk full, OOM-killer, Ctrl-C)
        # can leave a partial checkpoint directory; verifying before
        # trusting it as `best` (and therefore protected from
        # pruning) keeps a corrupt save from silently displacing a good one.
        try:
            SmolVLAPolicy.verify_checkpoint(ckpt_path)
            verified = True
        except (FileNotFoundError, ValueError) as exc:
            verified = False
            self.run_dir.append_jsonl("metrics/checkpoint_errors.jsonl", {...})
        checkpoints.append({"step": step, "path": str(ckpt_path), "val_metric": val_metric, "verified": verified})
        if verified and val_metric is not None:
            better = (best["metric"] is None
                      or (ec.selection_mode == "min" and val_metric < best["metric"])
                      or (ec.selection_mode == "max" and val_metric > best["metric"]))
            if better:
                best.update(metric=val_metric, checkpoint=str(ckpt_path))
        self._prune_checkpoints(checkpoint_dir, checkpoints, tc.keep_last_n_checkpoints, best)
```

- `extra` **metadata** stamps every checkpoint with its experiment, step and dataset version. That's what lets `build_model_artifact` later trust the checkpoint's own claim about its training data (Module 9). It's a good provenance detail.
- **The verify-before-trust comment describes a scenario its mechanism can't handle.** "A process killed mid-write (disk full, OOM-killer, Ctrl-C)": if the process is *killed*, the verification line never runs in that process, and there's no subsequent `best` to protect. A disk-full error during `save_checkpoint` would *raise*, not return, so it would crash training instead of producing an unverified checkpoint. **Verification only catches a `save_checkpoint` that returns normally but wrote an incomplete directory.** What `verify_checkpoint` checks is, per `test_corrupted_checkpoint_directory_rejected`, that all required files exist ("only one of five required files" gives "incomplete"). That's a presence check, not an integrity check: a truncated `model.safetensors` would pass. The mechanism is sound as cheap insurance, but the comment oversells the threat model. The case where partial checkpoints really bite, resuming from one after a kill, goes through the resume path (6.3), where `load` runs first.
- **Validation and checkpoint cadences must coincide for selection to work.** A checkpoint saved at a step with no validation has `val_metric = None` and can never become `best`. A validation at a step with no checkpoint has no weights to select. With `checkpoint_every: 50, validate_every: 30`, **no checkpoint is ever eligible**, and selection silently falls back to `final` (below). Nothing validates that `checkpoint_every % validate_every == 0`. Both are 100 in `base.yaml`, so the shipped configs are fine.
- **A NaN validation metric can lock selection.** `val_metric is not None` admits `float("nan")`. If the *first* validated checkpoint has a NaN metric, `best["metric"] is None` makes it `better`, so `best = NaN`. After that, `x < NaN` is always `False`, and **no later checkpoint can ever displace it.** Whether the engine's `fit` raises on a NaN loss (the crash-handler comment says "a NaN loss" as if it raises) or keeps going is unverified. If it keeps going, this is a real path. Guard with `math.isfinite(val_metric)`.
- `selection_mode` **is checked with string equality.** `Literal["min", "max"]` at the schema level (Module 2) makes the third case impossible. Good.

`_prune_checkpoints`

```
@staticmethod
def _prune_checkpoints(checkpoint_dir, checkpoints, keep_last_n, best) -> None:
    """Drop periodic checkpoints beyond ``keep_last_n``, always keeping the
    best-by-validation one regardless of recency."""
    if keep_last_n <= 0:
        return
    import shutil

    periodic = [c for c in checkpoints if not c.get("final")]
    periodic.sort(key=lambda c: c["step"])
    keep_paths = {best.get("checkpoint")} | {c["path"] for c in periodic[-keep_last_n:]}
    for c in periodic[:-keep_last_n] if len(periodic) > keep_last_n else []:
        if c["path"] in keep_paths:
            continue
        p = Path(c["path"])
        if p.exists():
            shutil.rmtree(p, ignore_errors=True)
        c["pruned"] = True
```

Walk it for `keep_last_n = 3`, with saves at 100...500 and the best at 200: `periodic[-3:]` = {300, 400, 500}, `keep_paths` = those plus 200, and the candidates `periodic[:-3]` = {100, 200}. So 100 is deleted and 200 is kept. That's correct.

- **`keep_last_n <= 0` means "keep everything".** That's a reasonable reading of "keep the last 0", read as "pruning disabled", but it's undocumented and opposite to the literal meaning.
- **Unverified (possibly corrupt) checkpoints count toward the last-N.** A corrupt `step_500` occupies a slot and pushes a good `step_200` out, unless 200 is best.
- **`ignore_errors=True` plus `c["pruned"] = True` unconditionally** means a deletion that fails (permissions, a file handle still open on network storage) is marked pruned anyway. The record and the disk disagree, silently.
- **Pruned entries stay in `checkpoints`** with `pruned: True`, so `training_outcome.json` lists paths that no longer exist. They're marked, so a reader can tell. That's fine.
- **This is a pure function with no I/O except `rmtree`, and it has no unit test.** Grep for `_prune` in `tests/` finds nothing. The only coverage is the smoke test (torch required), which sets `keep_last_n_checkpoints: 1` with checkpoints at steps 2 and 4 and doesn't assert what was pruned. It's the easiest function in the trainer to test with `tmp_path`, and it's untested.

6.5 Calling `fit`, the crash path, and the final checkpoint

```
started = time.perf_counter()
try:
    fit_result = policy.fit(
        train_view, episode_ids=ids, episode_weights=episode_weights,
        steps=tc.steps, batch_size=tc.batch_size, learning_rate=tc.learning_rate,
        weight_decay=tc.weight_decay, grad_accum_steps=tc.grad_accum_steps,
        max_grad_norm=tc.max_grad_norm, warmup_steps=tc.warmup_steps,
        seed=tc.seed, on_step=on_step,
    )
```

What's passed and what isn't: `num_workers`, `mixed_precision` and `device` (device went to the constructor) **aren't passed to `fit`**. This confirms Module 2's dead-field finding at the only call site. `seed=tc.seed` means the training seed comes from config only. The CLI `--seed` never reaches here (Module 11).

```
except Exception as exc:
    # A crash here (OOM, a NaN loss, SIGTERM) previously left the run
    # directory in an undetectable half-finished state ...
    self.run_dir.write_json("crash.json", {
        "experiment_id": ..., "error_type": type(exc).__name__, "error": str(exc),
        "steps_completed": last_step, "checkpoints": checkpoints,
        "wall_clock_s": time.perf_counter() - started,
    })
    del policy
    if torch.cuda.is_available():
        torch.cuda.empty_cache()
    raise
```

Decision: record a crash marker and re-raise; never swallow.

- *Why:* the comment narrates a real bug that was fixed. A crashed run used to be indistinguishable from one that never started. `crash.json` with `steps_completed` and the surviving checkpoint list makes a crashed run diagnosable and partly salvageable, since a verified `step_N` can be resumed from.
- **What it actually catches, versus what the comment claims:**

Failure	Caught by `except Exception`?	`crash.json` written?
CUDA out-of-memory (`torch.OutOfMemoryError`, a `RuntimeError` subclass)	yes	yes
NaN loss	only if the engine *raises* on NaN (unverified)	conditional
Exception inside `on_step` (full disk during save, JSON error)	yes	yes
Ctrl-C (`KeyboardInterrupt`, a `BaseException`)	**no**	**no**
SIGTERM (scheduler or `kill`)	**no**: default handler terminates without an exception	**no**
Host OOM-killer (SIGKILL)	**no**: uncatchable	**no**

The comment's "SIGTERM" is wrong, and the run-directory status gap from Module 3 remains for the two most common ways a long job dies on a shared machine. *Fix:* `except BaseException`, for Ctrl-C, plus a `signal.signal(SIGTERM, …)` handler that raises, and a `status: running` marker written *before* `fit`, so that a SIGKILL'd run is at least identifiable as "running with no heartbeat".

- **`del policy; empty_cache()` frees less than it appears to.** At this point the exception object `exc` holds a traceback whose frames include `fit`'s frame, and `fit`'s `self` *is* the policy. That's a live reference to every parameter and optimizer tensor until the exception is fully handled upstream. So `empty_cache()` here can't return that memory. The `del` is harmless, but the stated purpose, "release the GPU memory this policy was holding rather than leaving it for the caller's next model", isn't achieved while the exception propagates. The runner doesn't load another model after a training exception anyway (it re-raises out of `run`), so the practical impact is nil. The comment just misdescribes the effect.

```
wall_clock_s = time.perf_counter() - started

# Always checkpoint the final step, even off-cadence, so resume and
# deployment never require rerunning training to get a usable artifact.
final_path = checkpoint_dir / "final"
policy.save_checkpoint(final_path, extra={..., "step": fit_result["steps"], ..., "final": True})
checkpoints.append({"step": fit_result["steps"], "path": str(final_path), "val_metric": None, "final": True})
```

- **`wall_clock_s` excludes the final save** but includes every periodic validation and save. It's "training plus in-loop overhead", not pure optimization time.

- **The final checkpoint is never verified** (no `verify_checkpoint` call, unlike the periodic ones). It's verified later only if it becomes the selected checkpoint, inside `build_model_artifact` (Module 9).
- **The final checkpoint is never validated** (`val_metric: None`), so it can never compete for `best`. When `steps % checkpoint_every == 0` (the shipped configs: 1000 % 100), the last periodic checkpoint `step_1000` and `final` are **the same weights saved twice**: one is validated and selectable, the other isn't. SmolVLA is reported publicly as about 450M parameters, so each save is plausibly around 1–2 GB (I didn't measure this). Under the shipped configs it's a duplicate on every run.
- **Placing the final save outside the `try`** means a failure while saving it (disk full) raises *without* a `crash.json`, though training itself finished.

```
peak_gpu_mb = torch.cuda.max_memory_allocated() / 1e6 if torch.cuda.is_available() else None
del policy
if torch.cuda.is_available():
    torch.cuda.empty_cache()

return TrainingOutcome(
    steps_completed=fit_result["steps"],
    final_train_loss=fit_result["final_loss"],
    best_checkpoint=best["checkpoint"] or str(final_path),
    best_val_metric=best["metric"],
    ...
)
```

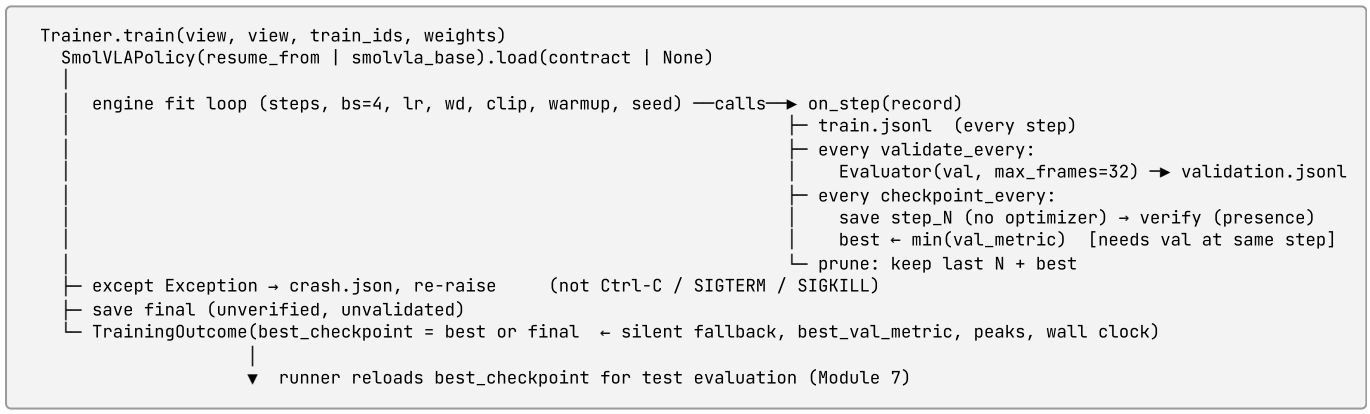
- **Releasing memory before return** is correct here: the runner immediately loads a second `SmolVLAPolicy` from the selected checkpoint for evaluation (Module 11), and two copies resident at once is exactly what an 8 GB laptop GPU can't afford. On this path there's no traceback pinning the model. `fit_result` may still hold `optimizer_state` (if the engine returns it, per the API doc) until `train` returns, which is moments later.
- **The `/ 1e6` is megabytes in SI units**, not MiB. The README's "2.2 GB" is consistent with SI.
- **`best["checkpoint"] or str(final_path)` is a silent fallback.** If no checkpoint was ever eligible (cadence mismatch, a wrong `selection_metric` name, a zero-size val split from Module 5, every checkpoint unverified), the run reports `best_checkpoint = final` with `best_val_metric = None`. The runner writes that into `model.json` as `selection_value: null`, and **no limitation is recorded**. The run looks selected, but it was just last-step. Add a limitation string whenever the fallback fires.

6.6 Honest gaps to own before an interviewer finds them

1. **Checkpoint selection uses 32 validation frames** (`val_batches × batch_size`), probably the opening second of one episode (unverified engine behaviour), on a stochastic policy, with a min over 10–20 candidates. *Consequence:* the README's selected val MSE (0.0018) is about 13× lower than test (0.0239). Selection is near-noise, and `best_val_metric` is optimistically biased.
2. **The crash handler misses Ctrl-C, SIGTERM and the OOM-killer**, contrary to its comment. *Consequence:* the most common ways long jobs die leave no `crash.json`.
3. **A silent fallback to `final`** when no checkpoint is eligible, with no limitation recorded. *Consequence:* "selected checkpoint" can mean "last step" with no indication.
4. **A NaN first validation locks `best` forever** (NaN passes `is not None`, and every later comparison is false). *Consequence:* selection is pinned to a broken checkpoint (if `fit` doesn't raise on NaN).
5. **Validation and checkpoint cadences aren't checked for alignment.** *Consequence:* misaligned settings silently disable selection.

6. **A resume isn't checked against the training dataset, and `load` precedes `verify_checkpoint`.** *Consequence:* a wrong resume fails as an obscure tensor error, and the clear verification message is unreachable.
7. **Optimizer state is never saved or restored, though the engine's `fit` reportedly supports it.** *Consequence:* resume restarts warmup and schedule. It's a defensible disk trade-off, but undocumented as a choice.
8. **`final` duplicates the last periodic checkpoint, is never verified or validated, and is saved outside the `try`.** *Consequence:* about one wasted model-size copy per run, and a save failure without a crash marker.
9. **`verify_checkpoint` is a presence check, and the comment's threat model (process killed mid-write) can't be caught in-process.** *Consequence:* the verification is narrower than described.
10. **`_prune_checkpoints` is untested; failed deletions are marked pruned; corrupt checkpoints occupy keep slots.** *Consequence:* disk state and records can diverge silently.
11. **`del policy; empty_cache()` in the crash path can't free memory pinned by the traceback.** *Consequence:* a comment that misdescribes the effect (no practical harm today).
12. **`peak_gpu_mb` is allocator-peak, measured after load.** *Consequence:* the README's GPU figures understate what you'd need to provision.
13. **The empty-train-IDs guard runs after the model load.** *Consequence:* a wasted GPU load on a trivially detectable error.
14. **Step semantics under `grad_accum_steps > 1` are unknown.** *Consequence:* the cadence could be off by a factor of `k` (unexercised).

The shape to carry into Module 7



Drill — answer from memory, files closed

1. Walk through exactly what happens at step 300 of a run with `checkpoint_every: 100`, `validate_every: 100`, `keep_last_n_checkpoints: 3`.
2. *Why didn't you just* validate on the whole validation split every time?
3. *What breaks in production:* the job scheduler sends SIGTERM at a time limit, 70% through training. What's on disk, and what can you recover?
4. The README reports validation MSE 0.0018 and test MSE 0.0239 for the same checkpoint. Explain the gap.
5. *Interviewer framing:* "How did you choose which checkpoint to evaluate, and how confident are you that it was the right one?"

Appendix A — Model answers to the drill

A1. Step 300. "The engine's `fit` finishes optimizer step 300 and calls my `on_step` callback. It appends the step's loss, learning rate and grad norm to `train.jsonl` and samples resident memory. 300 is a multiple of both cadences, so it switches the policy to eval mode, runs the evaluator on the val split capped at 32 frames, switches back to train mode, and logs the full metrics to `validation.jsonl`. Then it saves `checkpoints/step_300` with metadata (experiment, step, dataset version, and a flag that optimizer state wasn't saved), and verifies the directory has all required files. If it's verified and its val MSE beats the running best, it becomes best. Finally pruning keeps the last three periodic checkpoints plus the best. At step 300 there are only three, so nothing is deleted yet. At step 400, step 100 would be deleted unless it was the best."

Why that answer lands: it's the exact sequence, including the eval/train toggle and the conditions for becoming best, plus a correct worked pruning example. It shows you could reimplement it.

A2. Why not validate on the full split? "Cost. The val split is about 1,900 frames, and SmolVLA inference ran at roughly 50 ms per frame on the laptop GPU, so a full pass is about a minute and a half. Ten of them would roughly triple a five-minute run. So I capped it at `val_batches × batch_size`, 32 frames. That turned out to be the weak point of the design. Thirty-two correlated frames, probably all from the start of one episode, scored with a stochastic flow-matching sampler, and then a minimum taken over ten or twenty checkpoints. The selected validation number is noisy and biased low. The fix that keeps the cost is to choose a fixed, stratified subset once (frames spread across every val episode and across episode progress), score every checkpoint on that same subset, and average a couple of samples per frame."

Why that answer lands: it gives the real budget arithmetic, owns the consequence precisely, and offers a fix that respects the original constraint rather than just saying "use more data."

A3. SIGTERM at 70%. "Python's default SIGTERM handler terminates the process without raising, so my `except Exception` never runs. No `crash.json`, no final checkpoint, no `training_outcome.json` or `summary.json`. On disk you'd have `train.jsonl` and `validation.jsonl` up to about step 700, and the last three periodic checkpoints plus the best, each verified when saved, so you can resume from one of them in a new run with `resume_from`. You'd lose the optimizer state, so the resume restarts warmup. The run directory itself is now non-empty, so retrying the same experiment the same day is blocked. The fix is a SIGTERM handler that raises, catching `BaseException` so Ctrl-C is recorded too, and a 'running' status file written before training, so a hard kill is at least identifiable."

Why that answer lands: it knows that SIGTERM doesn't raise by default (a classic gap), inventories exactly what survives, and connects it to the retry-blocking behaviour from another module.

A4. 0.0018 val vs 0.0239 test. "Several things push validation down, and I think one dominates. Validation scores only 32 frames, and if the evaluator takes the first frames in order, that's the first second of one validation episode. My own per-phase analysis shows error is lowest at the start of episodes and peaks mid-episode. Second, the selected value is the minimum over about twenty noisy checkpoint evaluations, so it's biased low by selection alone. Third, the val and test splits are five episodes each, so episode-to-episode variation is large. I can't fully separate these without the engine code in front of me, but I wouldn't report that validation number as an estimate of anything. The test number is the honest one, and the fix is a frozen, stratified validation subset."

Why that answer lands: it gives a mechanistic hypothesis tied to your own data, then two other contributing causes, and states the uncertainty explicitly. That's the posture of a careful experimentalist.

A5. "How did you choose which checkpoint, and how confident are you?" "Checkpoints every 100 steps, each validated on the val split only (the trainer hard-codes val, so test never influences selection) and verified for completeness before it's eligible. The lowest validation MSE wins, and pruning never deletes it. If nothing is eligible, it falls back to the final checkpoint, which I'd now flag explicitly in the report, because today it's silent. Confidence: moderate on SO-100, low as a general method. On SO-100, validation was still decreasing at the end of training, so 'pick a late checkpoint' was almost certainly right regardless of noise. But the selection signal itself is 32 frames of a stochastic policy, and the validation minimum is about 13× below test, so it's not a precise estimator. If the curve had been flat, I'd have been picking among checkpoints at random."

Why that answer lands: it separates the *mechanism's* correctness (no test leakage, verified, protected) from the *estimate's* quality. It also explains why the result was probably right anyway, which is more convincing than either defending or disowning it.

Module 6 Summary — the one page to revise

What this module is. A wrapper around the engine's `SmolVLAPolicy.fit` that injects run policy through an `on_step` callback: val-only validation, checkpointing with presence verification, best-by-validation selection, pruning that protects the best, a forced final checkpoint, a crash marker, and GPU release before evaluation.

Five claims to make cold

1. The gradient step is the engine's. The trainer owns cadence, selection, pruning, crash recording and memory release.
2. Selection never sees test, because the only validation call hard-codes `split="val"`.
3. Only verified checkpoints with a validation score can be best, and the best is never pruned.
4. Selection uses 32 frames of a stochastic policy, so the val minimum (0.0018) is about 13× below test (0.0239). The fix is a frozen, stratified subset.
5. The crash marker covers in-process exceptions, not Ctrl-C, SIGTERM or SIGKILL.

Decisions

Chose	Over	Because	Cost
Callback into engine `fit`	own loop / Lightning	step mechanics are contract-coupled	callback time counted as training; `on_step` errors look like training crashes
Contract only on fresh start	always pass contract	resumed checkpoint already has its stats	resume not checked against dataset
No optimizer state in checkpoints	save moments	about 3× smaller checkpoints (inferred)	resume restarts warmup and schedule
32-frame validation	full val split	about 90 s per full pass vs a 5-minute run	near-noise selection, winner's curse
Verify before eligible	trust every save	a corrupt save can't become a protected best	presence check only
Keep last N plus best	keep all / last N only	bounded disk, best preserved	untested; failed deletes marked pruned
Always save `final`	rely on periodic	a usable artifact even off-cadence	duplicate of `step_N` when aligned; unverified
Record `crash.json` and re-raise	swallow / no marker	diagnosable crashes	misses BaseException and signals

Volunteer unprompted: the 13× val–test gap and its likely cause; the silent fallback to `final`; NaN locking `best`; the signals not caught; `_prune_checkpoints` has no test; `final` duplicates `step_1000`.

Numbers and names: base config 1,000 steps, batch 4, lr 1e-4, warmup 50, wd 1e-5, clip 1.0, ckpt and val every 100, `val_batches` 8 (so 32 frames), keep 3; extended lr 3e-5, warmup 20; README val min 0.0018 at step 1,800 vs test 0.0239; SO-100 324 s and 385 s, ALOHA 576 s; peak GPU 2.2 / 3.0 GB (allocator peak, SI); about 50 ms per frame for SO-100 inference; files `train.jsonl`, `validation.jsonl`, `checkpoint_errors.jsonl`, `crash.json`, `resume_source.json`, `checkpoints/step_N`, `checkpoints/final`.

Sentences worth memorizing

- "The engine owns the step; I own what happens between steps."
- "Test can't influence selection because the only validation call in the trainer hard-codes the val split."

- "My validation signal was 32 frames of a stochastic policy, which is why val sits 13× below test. Selection was right on SO-100 because the curve was still falling, not because the estimate was good."
- "`except Exception` doesn't see Ctrl-C, and SIGTERM doesn't raise at all; the crash marker covers less than its comment says."

Module 7 — Offline evaluation: baselines, tails, time and slices

Files covered: `src/model_training/evaluation/offline.py` (`evaluation/__init__.py` was covered in Module 1)

Where we are: Module 6 produced a selected checkpoint. This module turns it into the numbers the project is judged by. It scores two baselines through the same evaluator as SmolVLA, then breaks the test error down by tail, by episode, by phase of the episode, and by named slice. Everything in README §2, the headline results, comes out of these five functions plus the engine's `Evaluator`.

The one-line thesis: The decompositions are the right ones to report. Baselines are scored through the same call, tails come before means, error is broken down by phase, and metrics that can't be computed are refused. But each slice **re-runs a stochastic model** instead of slicing the predictions already made. The slices are tiny (1–2 episodes), and nothing carries an uncertainty estimate, so the finer-grained claims in the README are mostly noise.

7.1 Prerequisites — the absolute basics

- **Per-frame error, as defined here.** For one frame, `FrameOutcome.error` is the **mean absolute error across action dimensions** (per `docs/METRICS.md` §4 and the `error_distribution` docstring). Each frame produces one scalar.
- **Split-level MSE vs "per-episode MSE".** The split's `mse` squares *each dimension's* error and then averages. This module's per-episode `"mse"` squares the *per-frame mean absolute error* and averages. Those are different quantities: by Jensen's inequality `(mean |e|)² ≤ mean e²`, so the per-episode number is systematically smaller. `METRICS.md` §4 states this honestly.
- **Percentiles / tail metrics.** p90 is the error below which 90% of frames fall. Tails matter for control because one wildly wrong action can end a task even when the average is low.
- **Normalized episode progress.** `frame_index / (episode_length - 1)`, from 0 at the start to 1 at the end, so episodes of different lengths line up by phase.
- **Stratified evaluation.** Reporting metrics separately on predefined subsets (strata or slices) instead of only in aggregate. Aggregates can hide a subgroup that fails.
- **Mean-action baseline.** It always predicts the training-set mean action. In normalized units its MSE is about the variance, roughly 1.0 (README: 0.908 SO-100, 1.010 ALOHA). Any model must beat it.
- **Nearest-neighbour baseline (behaviour retrieval).** Per `docs/API_REFERENCE.md`, `NearestNeighborPolicy` paired with `StateAdapter`: find the training frame whose **robot state** is closest to the query frame's state, and return that frame's action. **It uses no images and no language.** It's described there as "a genuinely strong baseline on single-task data". (This is from the docs, because the engine code isn't here, but the integration test's `StateAdapter` plus `nearest_neighbor` pairing corroborates it.)
- **Stochastic inference.** SmolVLA samples from noise each call (flow matching, Module 1). Scoring the same frames twice gives different errors.
- **Sampling uncertainty.** With 5 test episodes, a metric's spread over *which* episodes happened to be in test is large. A **bootstrap over episodes** (resampling episodes with replacement and recomputing the metric) is the standard way to put an interval on it.

7.2 `evaluate_baselines`: same call, different inputs

```
def evaluate_baselines(view, names, *, split="test", train_split="train",
                       batch_size=32, max_frames=None) -> dict[str, EvaluationResult]:
    """Fit and score each baseline through the exact call used for the model
    under test, so the comparison is apples-to-apples."""
    ev = Evaluator()
    out: dict[str, EvaluationResult] = {}
    for name in names:
        if name not in MODEL_REGISTRY:
            raise KeyError(f"unknown baseline model {name!r}; known: {sorted(MODEL_REGISTRY)}")
        model, adapter = build_model(name, view.contract)
        model.fit(view, split=train_split)
        out[name] = ev.evaluate(view, model, adapter, split=split, batch_size=batch_size,
                                max_frames=max_frames, persist=False)
    return out
```

Decision: score baselines through the identical `Evaluator.evaluate` call, in every run.

- *Why:* the metric code, frame selection, normalization and `max_frames` cap are shared, so a ratio to the baseline is meaningful. `METRICS.md` says it directly: "a model that cannot outperform the training-set mean has, by definition, learned nothing that this metric can detect."
- *What "apples-to-apples" does and doesn't mean:* the **evaluation** is identical; the **inputs** aren't. `build_model` returns each model's own adapter: `StateAdapter` for both baselines, `VisionLanguageAdapter` for SmolVLA. That's correct, since a baseline can't use images, but it changes what the comparison *says* (next point).
- *Fitted on the same `train` split of the same `training/vN`,* so for an intervention run, the baselines see the same reduced or filtered training pool as SmolVLA. They don't receive `episode_weights`, so in a rebalanced run the baselines are unweighted. That's a small asymmetry, worth knowing.
- *The `KeyError` for an unknown name* is tested (`test_evaluate_baselines_rejects_unknown_model`). It's raised *before* any work, with the list of known names.
- *Refit every run* (k-NN index built over all training frames each time). That's cheap on CPU and avoids stale caches.

What the headline result actually means. README Table 2: SO-100 test MSE is SmolVLA **0.0239** and nearest-neighbour **0.0239**. ALOHA: SmolVLA **0.2157**, nearest-neighbour **0.1062**. Since the nearest-neighbour baseline looks up the closest *robot state* and replays its action, the tie says:

> On these single-setup datasets, **robot state alone nearly determines the next action**, and a 450M-parameter vision-language-action model, fine-tuned for 2,000 steps, adds nothing measurable *offline* over a state-space lookup table, and does worse on the harder bimanual task.

That isn't a failure of the evaluation; it's what the evaluation is *for*. Offline, though, it can't tell you what the image and language inputs buy in **closed loop**, where retrieval typically breaks down once the robot leaves the demonstrated states. That's the strongest argument for the rollout evaluation the project can't run. Have this paragraph ready; it's the most interesting sentence the project can say.

Doc vs code: `METRICS.md` §2 says baseline-relative metrics (`mse_ratio`, `mae_ratio`) "are computed by `rde.evaluation.Evaluator.compare` and by `model_training.evaluation.offline.evaluate_baselines`". **`evaluate_baselines` computes no ratio**; it returns raw results. The only ratio in this repo is calculated in the report renderer (`m["mse"] / mean_mse`, Module 10). Whether `Evaluator.compare` exists in the engine is unverifiable here, and nothing in `src/` calls it.

7.3 `error_distribution` and `episode_metrics`

```
def error_distribution(result: EvaluationResult) -> dict[str, Any]:
    """Median / p90 / p95 / p99 of per-frame mean-absolute error."""
    if not result.outcomes:
        return {"status": "no outcomes"}
    errs = np.array([o.error for o in result.outcomes])
    return {"median": float(np.median(errs)), "p90": float(np.percentile(errs, 90)),
            "p95": float(np.percentile(errs, 95)), "p99": float(np.percentile(errs, 99)),
            "min": float(errs.min()), "max": float(errs.max())}
```

- **Tails, not just means.** README §2.1 uses these: continued training cut p99 by 7% and the worst frame by 32% (0.553 → 0.375), while the median moved less. That's a genuinely useful observation, because tail frames are where control fails.
- **`float(...)` casts** turn NumPy scalars into Python floats, so the JSON is exact. This line is what keeps Module 3's `default=str` stringification hazard from applying here.
- **The empty guard** returns a status dict instead of crashing `np.percentile` on an empty array. `plot_error_distribution` then skips plotting because the keys are missing (Module 10). That's consistent.
- **The p99 of 1,908 correlated frames** is set by about 19 frames, which may all come from one difficult moment in one episode. `METRICS.md` §3 says so ("statistically noisy; the frame count should always be reported"). The returned dict doesn't include the frame count, though, so the caveat relies on the reader.

```
def episode_metrics(result: EvaluationResult) -> list[dict[str, Any]]:
    by_ep: dict[int, list[float]] = {}
    for o in result.outcomes:
        by_ep.setdefault(o.episode_id, []).append(o.error)
    out = []
    for ep, errs in sorted(by_ep.items()):
        a = np.asarray(errs)
        out.append({"episode_id": ep, "frames": len(a), "mae": float(a.mean()),
                    "mse": float((a ** 2).mean()), "rmse": float(np.sqrt((a ** 2).mean())),
                    "max_error": float(a.max())})
    return out
```

- **The `"mse"` and `"rmse"` keys are misnamed.** They're computed from per-frame *mean absolute* error, so they're "mean squared MAE", not MSE in the sense every other `mse` in the system uses. `METRICS.md` §4 discloses it, which is to its credit, but a JSON consumer or a `compare --metric mse` reader won't see the doc. Rename to `mean_sq_frame_mae` or compute true per-episode MSE from `per_dim_error` (which `FrameOutcome` carries). README §2.2 quotes per-episode **MAE** only, which is the correctly named one.
- **`sorted(by_ep.items())`** makes the output order deterministic by episode ID, so diffs between runs are stable.
- **Per-episode is the right grain for a 5-episode test set.** README: SO-100 per-episode MAE spans 0.085–0.111. That spread (about 25%) is the honest measure of how much a single test-split number depends on which episodes are in test. Use it as your uncertainty argument (7.6).

7.4 `temporal_error`: error by phase of the episode

```
def temporal_error(result, view, buckets: int = 10) -> dict[str, Any]:
    if not result.outcomes:
        return {"status": "no outcomes"}
    episodes_touched = {o.episode_id for o in result.outcomes}
    frame_index: dict[int, int] = {}
    ep_len: dict[int, int] = {}
    for ep_id in episodes_touched:
        table = view.get_episode_frames(ep_id, columns=["frame_id", "frame_index"])
        d = table.to_pydict()
        ep_len[ep_id] = max(1, len(d["frame_id"]))
        for fid, fidx in zip(d["frame_id"], d["frame_index"]):
            frame_index[int(fid)] = int(fidx)

    bucket_errors: list[list[float]] = [[] for _ in range(buckets)]
    unresolved = 0
    for o in result.outcomes:
        fidx = frame_index.get(o.frame_id)
        if fidx is None:
            unresolved += 1
            continue
        length = ep_len[o.episode_id]
        progress = fidx / (length - 1) if length > 1 else 0.0
        b = min(buckets - 1, max(0, int(progress * buckets)))
        bucket_errors[b].append(o.error)
    return {"buckets": buckets, "progress_bucket_edges": [...],
            "mean_error_by_progress_bucket": [round(float(np.mean(b)), 6) if b else None for b in bucket_errors],
            "n_by_bucket": [len(b) for b in bucket_errors], "unresolved_frames": unresolved}
```

- **Why the re-read.** The docstring explains it: `FrameOutcome` has `frame_id` but not `frame_index`, so the function fetches the small `frame_id → frame_index` table once per touched episode. It reads two narrow columns, not images. That's the right cost profile.
- **`columns=[...]`** projects only the needed columns. Without it, the engine might materialize full frames (state, action and image references) for every episode.
- **`length = len(d["frame_id"])` assumes `frame_index` runs from 0 to length−1 with no gaps.** If an episode has dropped frames (the profile reports `episodes_with_missing_or_discontinuous_frames`, Module 5), `frame_index` can exceed `length - 1`, so `progress > 1`, and the clamp `min(buckets - 1, …)` silently piles those frames into the last bucket. Using `max(frame_index)` instead of the row count would be robust to gaps.
- **`max(1, …)` and `if length > 1 else 0.0`** guard single-frame episodes against division by zero.
- **The `min/max` clamp** handles `progress == 1.0` exactly (the last frame), which would otherwise compute bucket 10.
- **`unresolved_frames` is reported rather than hidden.** That's the honest-metadata pattern.
- **Weighting:** every bucket pools frames across episodes, so longer episodes contribute more frames per bucket. With SO-100's 326–475-frame spread, that's mild.
- **With a `max_frames` cap and first-N frame selection (Module 6's open question), later buckets would be empty.** The function correctly returns `None` and `n = 0` for them instead of a fabricated mean.

What it found: README §2.2 says SO-100 error is lowest at the start (0.051) and end (0.049), and peaks mid-episode (0.144 at 50–60% progress). That's plausible physically, since the middle is the grasp and transport phase. It's also the evidence behind Module 6's hypothesis that start-of-episode validation frames are unrepresentatively easy.

7.5 `stratified_evaluate` : slices that re-run the model

```
ev = Evaluator()
out: dict[str, EvaluationResult] = {
    "overall": ev.evaluate(view, model, adapter, split=base_split, batch_size=batch_size,
                           max_frames=max_frames, persist=False)
}
for name in slices or []:
    if name == "per_task":
        for task_id in sorted({e.task_id for e in view.episodes if e.task_id is not None}):
            ep_ids = view.episode_ids(split=base_split, task_id=task_id)
            if not ep_ids:
                continue
            out[f"task_{task_id}"] = ev.evaluate(
                view, model, adapter, split=f"{base_split}:task_{task_id}",
                episode_ids=ep_ids, batch_size=batch_size, max_frames=max_frames, persist=False)
    elif name == "per_length_bucket":
        lengths = {e.episode_id: e.num_frames for e in view.episodes
                   if e.attributes.get("split") == base_split}
        if len(lengths) >= 3:
            t1, t2 = np.percentile(list(lengths.values()), [33.3, 66.7])
            groups = {"short": [], "medium": [], "long": []}
            for eid, n in lengths.items():
                groups["short" if n <= t1 else "long" if n > t2 else "medium"].append(eid)
            for bucket_name, ids in groups.items():
                if ids:
                    out[f"length_{bucket_name}"] = ev.evaluate(..., episode_ids=ids, ...)
    else:
        raise ValueError(f"unknown evaluation slice {name!r}; supported: 'per_task', 'per_length_bucket' ...")
return out
```

Decision: define slices from real per-episode metadata only, and reject anything else.

- *Why:* the docstring says "Slice names are resolved from real per-episode metadata, never invented". Asking for `"variation"` raises (`test_stratified_evaluate_rejects_unknown_slice`). This is the same "refuse to fabricate" discipline as the profile.
- *The split label* `f"{base_split}:task_{id}"` is passed to `evaluate` with explicit `episode_ids` . Per `docs/API_REFERENCE.md` , `episode_ids` "takes precedence over `split` for frame selection" and `split` becomes a label. That's subtle and correct: the slice is restricted to test episodes by construction, because `ep_ids` came from `episode_ids(split=base_split, ...)` . `test_stratified_evaluate_per_task_and_length_bucket` asserts that every slice's episodes are a subset of overall's.

Problem 1: every slice is a fresh stochastic inference pass. Each `ev.evaluate(...)` calls `model.predict` again on frames that `"overall"` already predicted. For SmolVLA, whose outputs are sampled, **a slice's frames get different predictions from the same frames inside `"overall"` .** Consequences:

- **Slices don't reconcile with overall.** A frame-weighted average of the three length slices won't equal the overall MSE, beyond rounding.
- **The README's noise estimate *is* this effect.** README §2.3: "The stratified pass re-scores all 1,908 SO-100 test frames, and its MSE differs from the main pass by up to 5% (0.0305 against 0.0290 ...)". SO-100 has a single task, so `task_0` is *every* test episode, which makes it a full re-scoring of overall with fresh noise. That's actually a lucky natural experiment: it's the only measurement of inference variance in the project. But it was produced by redundancy, not design.
- **Cost.** With `base.yaml` 's `slices: [per_task, per_length_bucket]` on single-task SO-100, test inference runs over overall (1×) + task_0 (1×) + three length slices (about 1× together), so **about 3× the necessary inference**. At about 50 ms per frame over 1,908 frames, that's about 95 s per pass, or about 3 minutes extra per run.

- **Config inconsistency.** `aloha_coffee_baseline.yaml` drops `per_task` because "a 'per_task' slice would be an exact duplicate of 'overall' -- pure redundant video-decode cost". **SO-100 is also single-task** (`EXPERIMENTS.md` §3), but `base.yaml` keeps `per_task` for it. Same reasoning, opposite configuration.
- **The fix:** predict once, then slice the `outcomes` of `"overall"` by `episode_id` in Python. Every slice then becomes a partition of one set of predictions, reconciles exactly with overall, and costs zero extra inference. If you want an inference-noise estimate, measure it *deliberately* (score overall twice with different seeds), not by accident.

Problem 2: the length slices hold 1–2 episodes each. The test split has 5 episodes (SO-100) or 4 (ALOHA). The 33.3/66.7 percentiles of 5 lengths, with `<= t1` short and `> t2` long, give groups of about 2/1/2. README §2.2 claims "long SO-100 episodes are easier (MSE 0.0206) than short (0.0290) and medium (0.0311) ones". That's a comparison of **one or two episodes per group**, each scored once by a stochastic model whose re-scoring moves MSE by about 5%, among episodes whose per-episode MAE already varies by about 25% (0.085–0.111). **That claim isn't supported by the data's resolution.** It might be true, but you can't tell from this. In an interview, don't repeat it as a finding. Say that the slices exist and that the test set is too small for them to be informative.

- For ALOHA, the README correctly notes the length slice is uninformative because "all four of its test episodes are 1,100 frames long and fall into one bucket". With ties, `<= t1` puts everything into "short" (the Module 4 tie collapse). That's a correct caveat in the README.
- `len(lengths) >= 3` **silently skips** the length slices for test splits under 3 episodes. A `reduced` run with 2–3 test episodes (Module 4's moving test set) would silently get **no length slices, with no note**.

Problem 3: within-split cut points. Here terciles are computed over the *test split's own* 5 lengths; in Module 4's intervention they're computed over the *whole dataset*. So `length_short` at evaluation time and `"short"` at intervention time are different sets (Module 4 gap 4). They also differ *between runs* whenever the test split changes, which is every excluding intervention. A "short slice" isn't a fixed slice.

`docs/EXPERIMENTS.md` **§6 claims** "A slice is a frozen episode-identifier list, produced identically on every invocation". That's true for *one* `training/vN` (same test episodes, same lengths, same cut points), and false across interventions, and the cut points themselves depend on the 5 test lengths. "Fixed evaluation slices" in the docs means "deterministic given the version", not "the same slice across experiments".

7.6 What isn't here: uncertainty

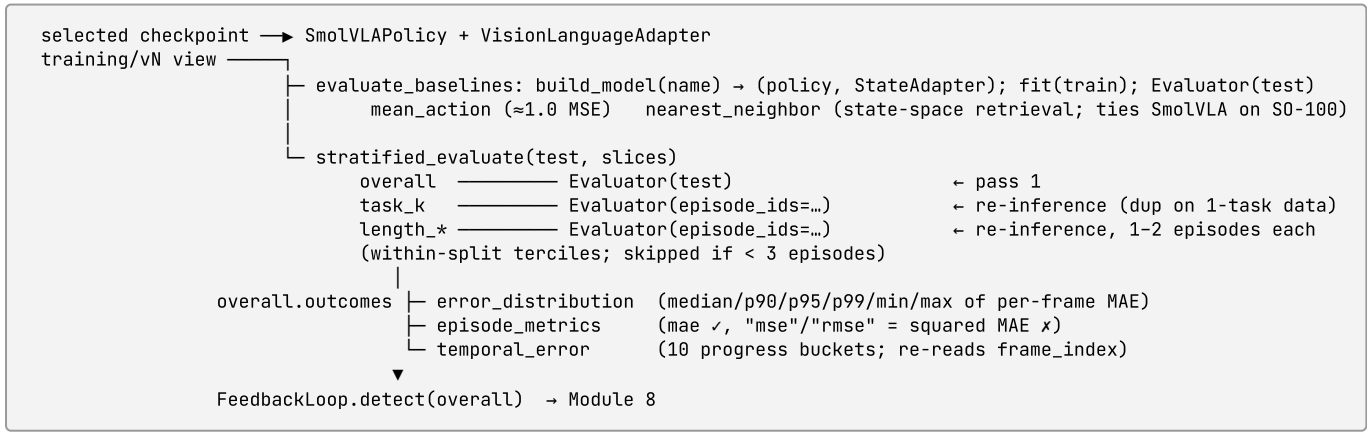
Nothing in this module (or anywhere in the repo) produces a confidence interval, a standard error, or a multi-seed aggregate. With 5 test episodes, one training seed and a stochastic policy, a single test MSE has three variance sources: *which episodes are in test*, *the training seed*, and *inference sampling*. The project measured the third once (about 5%), by accident. The README's "18% improvement from continued training" sits outside that 5% inference band, but it isn't tested against episode variance or seed variance.

The cheapest honest addition is an **episode-level bootstrap**. Resample the 5 test episodes with replacement 1,000 times, and recompute MSE from the stored per-frame `per_dim_error` (already written to `predictions/test_outcomes.jsonl` in Module 11). That gives an interval at zero inference cost. A **paired** comparison between two runs on the *same* test episodes (only possible for full and rebalanced runs, Module 4) is stronger still: per-episode differences cancel the episode-difficulty variance.

7.7 Honest gaps to own before an interviewer finds them

1. **Slices re-run stochastic inference instead of slicing one prediction set.** *Consequence:* slices don't reconcile with overall, cost about 3× the necessary inference on SO-100, and the project's only inference-noise estimate is an accident of a redundant slice.
2. **Length slices contain 1–2 test episodes, so the README's "long episodes are easier" is below the data's resolution.** *Consequence:* a claim that can't survive an interviewer's "how many episodes?"
3. **There's no uncertainty estimate anywhere** (no bootstrap, no seeds, no intervals). *Consequence:* no difference in the README is statistically qualified except against the accidental 5% noise band.
4. **The nearest-neighbour baseline is state-only retrieval, and it ties or beats SmolVLA.** *Consequence:* this isn't a bug. It's the key interpretive fact, and you must frame it as "offline, state nearly determines action on single-setup data".
5. **Per-episode `"mse"` / `"rmse"` are mean squared per-frame MAE, not MSE.** *Consequence:* a misnamed key that's disclosed only in `METRICS.md`.
6. **`base.yaml` keeps `per_task` for single-task SO-100**, which the ALOHA config explicitly removes as redundant. *Consequence:* about 1 wasted full test pass per SO-100 run.
7. **`METRICS.md` §2 says `evaluate_baselines` computes ratios.** It doesn't. *Consequence:* a doc overclaim. The ratio lives only in the report renderer.
8. **Length slices are silently skipped below 3 test episodes**, with no note. *Consequence:* small-pool intervention runs lose slices invisibly.
9. **`temporal_error` assumes contiguous `frame_index`.** *Consequence:* episodes with gaps pile frames into the last bucket.
10. **Baselines don't receive `episode_weights` in rebalanced runs.** *Consequence:* a minor asymmetry in "apples-to-apples".
11. **Tail percentiles don't carry their frame count**, despite `METRICS.md` saying it should always be reported. *Consequence:* the noise caveat depends on the reader.
12. **`success_tolerance` is accepted by the engine's `Evaluator.evaluate` (per the API doc) but never passed.** *Consequence:* this confirms that Module 2's dead config field could be wired in one line.

The shape to carry into Module 8



Drill — answer from memory, files closed

1. What exactly does the nearest-neighbour baseline do, and what does it mean that it ties SmolVLA on SO-100?
2. *Why didn't you just* compute slice metrics by filtering the overall predictions?
3. The README says long SO-100 episodes are easier than short ones. Would you defend that claim?
4. *What breaks in production:* you add a third slice type and run on a dataset with 40 tasks. What happens to evaluation time, and why?
5. *Interviewer framing:* "Your model ties a nearest-neighbour lookup. Why should I believe it learned anything useful?"

Appendix A — Model answers to the drill

A1. The nearest-neighbour baseline. "It's behaviour retrieval over robot state. During fit it indexes every training frame's state vector and action. At test time it finds the training frame whose state is closest to the query's state and replays that action. No images, no language. It's scored through exactly the same evaluator call as SmolVLA. On SO-100 both reach 0.0239 test MSE, so offline, on a single-setup, single-task dataset, the robot's current joint configuration almost determines the demonstrated next action, and the vision-language model adds nothing measurable on top of a lookup table. On ALOHA, retrieval is twice as good. That's the honest result. What offline evaluation can't show is closed-loop behaviour, where retrieval usually fails as soon as the robot drifts into states the demos never visited. That's where vision should earn its keep, and it's exactly the experiment I couldn't run without a robot or simulator."

Why that answer lands: it explains the mechanism precisely, states the uncomfortable result without spin, and identifies exactly what experiment would distinguish the models.

A2. Why not slice the overall predictions? "I should have. Each slice calls the evaluator again, which reruns SmolVLA, and since SmolVLA samples actions from noise, the same frame gets a different prediction inside a slice than inside overall. So slices don't reconcile with the overall number, and on SO-100, which has a single task, the per-task slice is a complete second pass over the test set. Together with the length slices, that's about three times the inference I needed. Funnily enough, that redundant pass is the only place I measured inference noise (about 5%). The right design predicts once, slices the per-frame outcomes by episode ID, and measures inference noise deliberately by scoring twice with different seeds."

Why that answer lands: it admits the flaw directly, explains the stochastic mechanism that makes it more than a performance issue, and turns the accidental noise estimate into a point about deliberate measurement.

A3. "Long episodes are easier"? "No, not as a finding. The test split has five episodes, and the length terciles of five values put about two episodes in short, one in medium and two in long. Each is scored once by a stochastic model whose re-scoring moves MSE by about five percent, and per-episode error already varies by about twenty-five percent across the five episodes. A difference between one and two episodes at that noise level isn't evidence. I'd say the slicing infrastructure works, and that this test set is too small for slice-level conclusions. To make a real claim, I'd want a bootstrap interval over episodes at minimum, and ideally a larger held-out set or several seeds."

Why that answer lands: it retracts the overreach yourself, with arithmetic, before the interviewer can. Volunteering a downgrade buys credibility for every claim you keep.

A4. A third slice type on a 40-task dataset. "Evaluation time scales with the number of slices, because every slice is a separate inference pass. `per_task` alone would add 40 passes. They're disjoint subsets, so together about one more full pass, plus 40 rounds of evaluator overhead and model batching inefficiency on small slices. Add a third slice type and every new axis costs another pass. And because the passes are stochastic, the slices wouldn't sum to overall. If slices ever overlapped, like task crossed with length, the cost would grow with the number of combinations. The fix is the same: one inference pass, with slicing as a cheap group-by over stored per-frame outcomes. Then any number of slice definitions is essentially free."

Why that answer lands: it reasons about scaling from the code's structure, catches the stochastic-reconciliation issue again, and gives a fix that changes the complexity class.

A5. "Your model ties a nearest-neighbour lookup. Why believe it learned anything?" "Offline, I'd say it learned the demonstrated state-to-action mapping about as well as memorizing it does. It's at 2.6% of the mean-action error, so it clearly learned the task distribution, but it isn't better than retrieval here, and on

ALOHA it's worse. I wouldn't claim more than that. What would justify more is a test where retrieval should fail: held-out scenes or object positions, or closed-loop rollouts where the policy's own errors push it off the demonstrated states. That's where a model using vision and language should beat a state-space lookup. Without a robot or simulator, I made the system report rollout as unavailable rather than imply it. The value of the project at this stage is that the evaluation harness told me the truth, including that the baseline is this strong."

Why that answer lands: it concedes exactly what the evidence supports, names the experiment that would change the conclusion, and reframes the project's value around rigorous measurement, which is defensible, rather than model performance, which isn't yet.

Module 7 Summary — the one page to revise

What this module is. Five functions over the engine's `Evaluator`: score baselines through the identical call, then decompose test error into tails, per-episode, per-phase and named slices, each resolved only from real metadata. The decompositions are right; the slicing re-runs inference, and nothing carries uncertainty.

Five claims to make cold

1. Baselines go through the same `Evaluator.evaluate` call. Mean-action is about 1.0 MSE in normalized units; nearest-neighbour is state-only retrieval.
2. SmolVLA ties nearest-neighbour on SO-100 (0.0239 vs 0.0239) and loses on ALOHA (0.216 vs 0.106). Offline, state nearly determines action on single-setup data.
3. Tails and phases are reported, not just means. Continued training cut the worst frame by 32%, and mid-episode error is about 3× start and end error.
4. Each slice is a fresh stochastic pass, so slices don't reconcile with overall. The fix is to predict once and slice outcomes.
5. With 5 test episodes and 1–2 per length slice, slice-level claims are below resolution, and there's no bootstrap or multi-seed interval.

Decisions

Chose	Over	Because	Cost
Same evaluator for baselines	separate baseline scoring	comparable ratios	inputs still differ (state vs vision and language)
Slices from real metadata only; reject others	proxy "variation" slices	refuse to fabricate	only task and length available
Slice by re-calling `evaluate`	slice stored outcomes	reuses the evaluator's metric code	about 3× inference; non-reconciling stochastic slices
Within-split length terciles	dataset-level cut points	adapts to the split	slices move across runs; differ from intervention buckets
Per-frame MAE as the error scalar	per-dimension MSE	one scalar per frame	per-episode "mse" is misnamed
Re-read `frame_index` per episode	add `frame_index` to outcomes	no evaluator change	assumes contiguous indices

Volunteer unprompted: the redundant `per_task` on SO-100 and the accidental 5% noise estimate; the "long episodes are easier" overreach; no intervals; the misnamed per-episode MSE; `METRICS.md`'s ratio claim.

Numbers and names: SO-100 test 1,908 frames from 5 episodes; ALOHA 4,400 frames from 4 episodes (all 1,100 frames); SmolVLA/mean-action 3.2% → 2.6% (SO-100), 21.4% (ALOHA); NN 0.0239 / 0.1062; mean-action 0.908 / 1.010; per-episode MAE 0.085–0.111 (SO-100), 0.296–0.405 (ALOHA); temporal 0.051 start / 0.144 mid / 0.049 end; re-scoring noise up to 5%; 10 progress buckets; terciles at the 33.3/66.7 percentiles, `>= 3` episodes required; NN confidence–error correlation ≈ −0.79 (METRICS.md).

Sentences worth memorizing

- "Offline, on single-setup data, the robot's state almost determines the next action; that's why a state-space lookup ties a VLA."
- "The harness's job was to tell me the truth, and it did: the baseline was that strong."

- "Every slice re-ran a stochastic model, so my slices didn't add up to my overall number. Predict once, slice the outcomes."
- "Five test episodes can't support slice-level claims; I'd want a bootstrap over episodes before saying long episodes are easier."

Module 8 — The feedback artifact: closing the loop

Files covered: `src/model_training/feedback/artifact.py` (`feedback/__init__.py` was covered in Module 1)

Where we are: Module 7 produced `test_overall`, a per-frame record of how the selected checkpoint did on the test split. This module turns that into the project's "flywheel" output: a structured artifact that ranks failing episodes, offers candidate explanations, and recommends the next data intervention. README Figure 1's loop arrow ("the feedback artifact's recommended action informs the next data intervention") begins here.

The one-line thesis: The artifact is a thin, honest-sounding wrapper over the engine's `FeedbackLoop`. Its percentile-relative failure threshold guarantees roughly 10% of frames always "fail", so with this repo's test sizes it **can't recommend "no action"**, and it **essentially can't produce its "data defect" explanation**. Worse, it runs on the **test split**, so following its advice and re-scoring on the same test set turns test into a tuning set.

8.1 Prerequisites — the absolute basics

- **Data flywheel.** The loop deploy (or evaluate) → find failures → fix the data → retrain → re-evaluate. Each turn is supposed to improve the model through better data.
- **Failure detection by threshold.** A frame "fails" if its error exceeds a threshold. **Absolute** threshold: error > 0.1. **Relative (percentile)** threshold: error above the 90th percentile of *this run's* errors. A relative threshold always flags about the same *fraction* of frames, by definition, however good or bad the model is.
- **Episode nomination.** Aggregating frame-level failures into a per-episode review: failure rate (flagged frames ÷ evaluated frames in that episode), mean error, and a priority score.
- **Data defect vs model limitation.** Two different causes of failure. A *defect* is a corrupted or mislabelled demonstration, so fix the data. A *limitation* is a regime the model hasn't learned, so get more coverage or a better model. The project's rule, stated in the code, is "a model failure does not automatically imply bad data."
- **Adaptive overfitting to the test set (the "garden of forking paths").** Every decision you make *by looking at* test results, such as which intervention to run next, makes subsequent test results optimistically biased. The test set has become part of your tuning loop. The textbook rule: decisions use validation, and test is touched once, at the end.
- **Quarantine.** Marking an episode for human review without deleting it.

8.2 The code

```
def build_feedback_artifact(experiment_id, model_id, dataset_id, dataset_version, result, *,
                            settings=None, catalog=None,
                            error_percentile: float = 90.0, min_failures_per_episode: int = 3) -> dict:
    """Detect failures via the real flywheel, then package the structured artifact."""
    loop = FeedbackLoop(settings=settings, catalog=catalog)
    report = loop.detect(result, error_percentile=error_percentile,
                         min_failures_per_episode=min_failures_per_episode, persist=True)
    selection = loop.build_retraining_selection(report)
```

Decision: wrap the engine's `FeedbackLoop` instead of reimplementing detection.

- *Why:* the module docstring says the engine's detection and nomination logic is reused, and this module adds "what that module deliberately leaves to a human: candidate explanations … and a concrete recommended next action". It's a single implementation of detection.
- *Cost:* every important number (the threshold, `failure_rate`, priority) is computed in code I can't read here. The behaviour below is derived from `docs/FAILURE_ANALYSIS.md` §2 and the explanation strings, so it's partly **unverified**.

`persist=True`: evaluation writes to the shared catalog. Unlike every `Evaluator` call in Modules 6 and 7 (`persist=False`), this one *persists* the failure report into the engine. So every SmolVLA run mutates the engine's state as a side effect of evaluation. That's intended (it's how a curator later finds flagged episodes with `rde` tools), but it means:

- re-running an experiment writes a second set of failure records for the same episodes;
- a crashed run after this point leaves engine records with no completed run directory pointing at them (Module 3's non-transactional index, again);
- `mtrain evaluate` re-scores a run *without* re-running feedback (Module 12), so after a re-evaluation, `feedback_artifact.json` and the engine's records describe the *old* predictions.

`error_percentile=90.0` and `min_failures_per_episode=3` are hard-coded defaults. No config field reaches them. The runner calls this with defaults only (Module 11).

Why a relative threshold, and what it costs

`FAILURE_ANALYSIS.md` §2 defends the choice: "against percentiles computed from the current evaluation run, not a fixed absolute threshold; a fixed threshold would flag either everything or nothing the moment the model or the normalization scheme changes." That's a real argument: an absolute 0.1 would flag about half the frames for SmolVLA on SO-100 (README: 49.8–56.9% within tolerance) and nearly all of them on ALOHA (0.05%).

The cost is arithmetic. At the 90th percentile, *by definition* about 10% of evaluated frames are flagged, in every run, for every model.

- SO-100 test: 1,908 frames, so about 190 flagged frames spread over 5 episodes of about 380 frames each.
- An episode is reviewed if it has at least 3 flagged frames. With about 190 flagged frames across 5 episodes, **every episode will almost certainly have 3 or more**. So `report.reviews` is non-empty in essentially every real run.
- Therefore the `else` branch ("no episode crossed the failure-concentration threshold … no evidence") and the recommendation `"no action indicated by this evaluation."` are **effectively unreachable** on this project's datasets. **The artifact will recommend an intervention for a perfect model too**, because a perfect model's top 10% of errors are still "failures".

The two explanations, and why one almost never fires

```
if report.reviews:
    concentrated = [r for r in report.reviews if r.failure_rate > 0.5]
    if concentrated:
        candidate_explanations.append(
            f"{len(concentrated)} episode(s) fail on more than half their evaluated frames -- "
            "concentrated, episode-local failure is evidence worth checking for a data/collection "
            "defect ... (a model failure does not automatically imply bad data).")
    scattered = [r for r in report.reviews if r.failure_rate <= 0.5]
    if scattered:
        candidate_explanations.append(
            f"{len(scattered)} episode(s) fail on a minority of their evaluated frames -- more "
            "consistent with a model/coverage limitation ... than an episode-specific data defect.")
```

The epistemic framing is good: concentrated failure suggests a defect, scattered failure suggests a limitation, and neither is asserted. `FAILURE_ANALYSIS.md` §5 says "Neither pattern is presented as a conclusion". That's the right posture, and it's worth quoting.

But combine `failure_rate > 0.5` with a 90th-percentile threshold. If `failure_rate` is flagged ÷ evaluated frames *for that episode* (which the explanation text, "fail on more than half their evaluated frames", implies), then the *maximum* failure rate any episode can reach is about

`min(1, 0.10 × total_frames / episode_frames)` ≈ `0.10 × 1,908 / 380` ≈ **0.50** on SO-100,

and that only happens if *every* flagged frame in the whole test set lands in that one episode. On ALOHA (4 episodes of 1,100 frames), it's `0.10 × 4,400 / 1,100` = **0.40**. **So "concentrated" (> 0.5) is mathematically unreachable on ALOHA and needs a perfect worst case on SO-100.** The "data defect" explanation can, in practice, never be produced here. The artifact will say "scattered → model/coverage limitation" every time. That isn't evidence. It's a consequence of choosing p90 and 0.5 together. (It's reachable only when an episode is much shorter than average, or when there are many episodes and one is truly anomalous.)

Confirmed by the README's own numbers (added during the Data Provenance module). README §2.4 reports that on the 2,000-step SO-100 run the p90 rule flags **191 of 1,908** test frames (that's 10.0%, as the arithmetic predicts), "every test episode contains some of them", and **no episode exceeds 13.8%** (range 6.5–13.8%), so the artifact says "scattered". On ALOHA, 440 of 4,400 are flagged, and the worst episodes reach 22.6% and 12.1%. Both datasets sit far below the 0.5 cut, exactly as the bound says they must. The README presents "scattered → model or coverage limitation" as a *finding*. It's the only output these thresholds can produce at these sizes.

The two thresholds should be expressed relative to each other. For example, call an episode concentrated if its failure rate exceeds k× the expected base rate of `(100 - p)/100`, or test it with a binomial test against that base rate. That's the one-line fix that makes both branches reachable.

The recommendation is a template

```
recommended = (
    "no action indicated by this evaluation."
    if not report.reviews else
    "inspect the top-ranked episode(s) below for corruption/ambiguity before assuming a data fix "
    "will help; if the pattern holds and traces to a real shared attribute (task, length bucket), "
    "build a targeted correction with "
    "`mtrain build-intervention --kind rebalanced --variable <attribute> --variable-value <value>` "
    "and retrain, then compare against this run on the same fixed test slice."
)
```

- **It's one fixed string with placeholders.** Nothing analyses the reviews to see whether failures share a task or length bucket, or names the attribute and value. `FAILURE_ANALYSIS.md` §6 says the field "names a

concrete next step whenever the evidence supports one". It names the *same* step, with `<attribute>` and `<value>` left blank, whenever any episode is reviewed, which (above) is always. The recommendation is good *advice*; it isn't *output*.

- **"Compare against this run on the same fixed test slice" is impossible** for any excluding intervention (Module 4: the test set moves). For `rebalanced`, the recommended kind, the test set plausibly *is* identical, because rebalancing excludes nothing. So the specific advice is internally consistent. That's worth noticing: rebalancing is the only intervention the loop can evaluate fairly.
- **"Inspect ... before assuming a data fix will help"** puts a human gate on the loop, matching README Figure 1's dashed "decision taken by a person" arrow. Good.

The returned artifact

```
return {
    "experiment_id": ..., "model_id": ..., "dataset_id": ..., "dataset_version": ...,
    "affected_slice": {"split": result.split, "frames_evaluated": result.frames_evaluated},
    "failure_summary": report.summary(),
    "ranked_episode_reviews": [r.to_dict() for r in report.reviews],
    "candidate_explanations": candidate_explanations,
    "recommended_next_action": recommended,
    "retraining_selection": selection,
    "rde_artifacts": report.artifacts,
}
```

- **`affected_slice` is always the whole evaluated split** (`"test"`), never a slice. The name promises a localization the code doesn't do.
- **`retraining_selection`** comes from the engine's `build_retraining_selection(report)`. By `FAILURE_ANALYSIS.md` §5, flagged episodes are "upweighted -- never dropped -- via the `episode_weights` mechanism", so this is presumably a weights proposal. Nothing in this repo *consumes* it. `mtrain build-intervention` computes weights from `--variable` and `--variable-value`, not from this field (Module 12). The engine's own nominated selection is computed, stored and ignored.
- **The integration test** (`test_prediction_to_evaluator_to_metrics_to_feedback_artifact`) checks shape only: every ranked episode ID is real, and `recommended_next_action` exists. It doesn't check that either explanation branch can fire, which is exactly the gap above.

8.3 The methodological problem: the loop runs on test

The runner calls `build_feedback_artifact(..., test_overall, ...)` (Module 11), on the **test** split's outcomes. The loop the docs describe (`FAILURE_ANALYSIS.md` §6) is:

```
train → evaluate on TEST → feedback picks a weak regime FROM TEST → rebalance → retrain → compare on TEST
```

Each turn of that loop **chooses the next dataset by looking at test errors**, then **judges the choice on the same test errors**. With 5 test episodes, an intervention tuned to the specific episodes that happened to fail in test will tend to look like an improvement on those same episodes, whether or not it generalizes. This is adaptive overfitting, and it's the single most important methodological correction in the project. Training-time selection is strict about val (Module 6), but data-curation selection isn't.

The fix keeps everything else: run `build_feedback_artifact` on a **validation** evaluation (a full val pass of the selected checkpoint, scored once). Drive interventions from val failures, and keep test untouched until a final comparison. If the val split is too small to localize failures (5 episodes), that's itself the finding: the dataset is too small to run a data flywheel on and also keep an honest test set.

8.4 Docs vs code in `FAILURE_ANALYSIS.md`

Claim	Reality in the code
§4: "predictions are deterministic given the checkpoint and the frame, they are reproducible on demand"	**False for SmolVLA.** README §2.3 says flow-matching sampling makes re-scoring differ by up to 5%. The doc's justification for *not* storing predicted and ground-truth action vectors rests on a false premise. Re-running `mtrain evaluate` gives *different* predictions.
§3: per-action-dimension error is "surfaced per episode via `episode_metrics`"	`episode_metrics` returns `mae/mse/rmse/max_error` per episode, with **no per-dimension output** (Module 7). Per-dimension error is in each frame's `per_dim_error` in `test_outcomes.jsonl`, but nothing aggregates it per episode.
§3: highest-error episodes come from "`episode_metrics`, sorted by mean squared error"	`episode_metrics` sorts by **episode ID** (`sorted(by_ep.items())`).
§3 Table 1 lists stratification dimensions "this project adds"	None of them appear in the feedback artifact. They're separate fields of `test_overall.json`.
§6: the recommendation "names a concrete next step whenever the evidence supports one"	A fixed template with blank placeholders, emitted whenever any episode is reviewed (effectively always).
§2: SmolVLA's constant confidence gives `confidence_used: false`	Engine-side; unverifiable here. It's consistent with `model_card.py`'s "confidence is an uncalibrated constant".

8.5 Honest gaps to own before an interviewer finds them

1. **The loop runs on test.** Failures that drive the next intervention come from the test split, and the intervention is then judged on the same test split. *Consequence:* adaptive overfitting. Data curation tunes on test.
2. **"No action" is effectively unreachable.** A 90th-percentile threshold always flags about 10% of frames, which spread across 4–5 episodes always exceed 3 per episode. *Consequence:* the artifact recommends an intervention for any model, including a perfect one.
3. **The "concentrated / data defect" branch is mathematically near-unreachable** (maximum per-episode failure rate ≈ 0.50 on SO-100 and 0.40 on ALOHA, versus a > 0.5 rule). *Consequence:* the artifact always says "scattered → model limitation", by construction, not by evidence.
4. **The recommendation is a static template** with unfilled `<attribute>` / `<value>` placeholders. *Consequence:* "names a concrete next step" overclaims; no analysis links reviews to attributes.
5. **`FAILURE_ANALYSIS.md` claims predictions are deterministic.** They're stochastic. *Consequence:* the stated reason for not persisting predicted and target vectors is false, and "re-run to reproduce" doesn't reproduce.
6. **Several `FAILURE_ANALYSIS.md` §3 claims don't match `episode_metrics`** (no per-dimension output, sorted by ID). *Consequence:* doc overclaims.
7. **`persist=True` writes failure records into the engine on every run**, with no dedupe and no refresh on `mtrain evaluate`. *Consequence:* engine records go stale or duplicate relative to the run directory.
8. **`retraining_selection` is computed and never consumed.** *Consequence:* the engine's nominated weights are dead output, and `build-intervention` ignores them.
9. **`affected_slice` is always the whole split.** *Consequence:* the field name overclaims localization.
10. **Thresholds (`error_percentile`, `min_failures_per_episode`) aren't configurable.** *Consequence:* you can't tune them per dataset without a code change.

11. **The integration test checks shape, not that either branch is reachable.** *Consequence:* gaps 2–3 are invisible to the suite.

The shape to carry into Module 9

```
test_overall (EvaluationResult on TEST)  ← should be VAL for curation decisions
       │
       ▼
rde FeedbackLoop.detect(p90, min 3/episode, persist=True)  ──► engine failure records (side effect)
       │   ~10% of frames flagged by definition
       ▼
report.reviews (failure_rate, mean error, priority)    loop.build_retraining_selection → (unused)
       │
       ├─ failure_rate > 0.5 → "data defect?"     ← max ≈ 0.50 (SO-100) / 0.40 (ALOHA): ~never
       ├─ failure_rate ≤ 0.5 → "model/coverage limitation"   ← always
       └─ recommended = fixed template "build-intervention --kind rebalanced --variable <…>"
       ▼
failures/feedback_artifact.json ──► model card "Known failure modes", report, human decision
                                        │ (dashed arrow: a person picks the next intervention)
                                        ▼
                                Module 4: next training/vN
```

Drill — answer from memory, files closed

1. Why does the feedback loop use a percentile threshold instead of an absolute one, and what does that choice make impossible?
2. Show, with numbers, why the "concentrated failure" explanation essentially never appears on SO-100 or ALOHA.
3. *Why didn't you just* run failure analysis on the test set? That's where the final model is evaluated.
4. *What breaks in production:* you run the loop five times (feedback, rebalance, retrain, compare) and report the fifth model's test MSE. What's wrong with that number?
5. *Interviewer framing:* "Walk me through your data flywheel. Does it work?"

Appendix A — Model answers to the drill

A1. Percentile vs absolute. "An absolute threshold depends on the model and the normalization. At 0.1 mean absolute error, SmolVLA would flag about half the SO-100 frames and nearly all of ALOHA's, so the flag would carry no information about *which* frames are unusually bad. A 90th-percentile threshold adapts to the run and always isolates the worst tenth. The cost is that it can never say 'nothing is wrong'. Ten percent of frames fail by definition, for a terrible model and a perfect one alike. With 1,900 test frames over five episodes, every episode gets well over the three-failure minimum, so the 'no action' branch is effectively dead and the artifact always recommends an intervention. I'd keep the relative threshold for *ranking*, and add an absolute or baseline-relative gate for *whether to act*, for example only recommending action if the flagged frames' error is worse than the nearest-neighbour baseline's on the same frames."

Why that answer lands: it defends the original choice with real numbers, then names the exact capability the choice destroys, and proposes a two-threshold fix that keeps the benefit.

A2. Concentrated failure is unreachable. "At p90, about 10% of all test frames are flagged: about 190 of SO-100's 1,908. An episode is about 380 frames. Even if every one of the 190 flagged frames landed in a single episode, its failure rate would be 190 / 380, which is 0.5, and the rule needs strictly more than 0.5. On ALOHA it's 440 flagged frames over 1,100-frame episodes, so at most 0.4. So the explanation that points at a data defect can't fire, and every run reports 'scattered failure, consistent with a model limitation'. That sounds like evidence, but it's a consequence of picking p90 and 0.5 independently. The fix is to define 'concentrated' relative to the 10% base rate, for example with a binomial test per episode."

Why that answer lands: it's a clean back-of-envelope proof that a code path is dead. Few things are more convincing in an interview than showing a branch can't execute.

A3. Why not run it on test? "Because the feedback artifact *decides* what to do next, and anything that makes decisions has to use data you're not going to report on. If test failures pick which regime to upweight and test then judges whether the upweighting helped, the test set has become a tuning set. With five test episodes, an intervention fit to the particular episodes that failed will look like an improvement on exactly those episodes. My training loop is strict about this, since checkpoint selection only ever sees val, but the data-curation loop isn't, and that's the most important methodological fix in the project. Run the feedback on a full val evaluation and keep test for the final comparison. If val is too small to localize failures, that tells me the dataset is too small for a flywheel *and* an honest test set."

Why that answer lands: it identifies adaptive overfitting precisely, contrasts it with the place the project gets it right, and turns the small-data constraint into an insight instead of an excuse.

A4. Five turns of the loop. "The fifth model's test MSE is optimistically biased, because each of the five interventions was chosen by looking at test failures and kept or rejected by test MSE. That's five rounds of selection on a five-episode test set, and the reported number is the winner of a search. It isn't an estimate of performance on new episodes. On top of that, the test set may not even be the same across those runs: rebalancing keeps it fixed, but any intervention that excludes episodes reshuffles the split. To report honestly, I'd either hold out a fresh test set that the loop never saw, or report the fifth model alongside the first on a split none of the decisions touched, with a bootstrap interval over episodes."

Why that answer lands: it names selection bias over repeated rounds, connects it to the moving-test-set issue from another module, and gives an actionable reporting protocol.

A5. "Walk me through your data flywheel. Does it work?" "Mechanically, yes: every stage is a real persisted artifact. A run's test predictions go through the engine's failure detector, which flags the top 10% of frames, rolls them up into ranked episode reviews, and writes them to the catalog for curators. My layer

adds candidate explanations, careful not to equate model failure with bad data, plus a recommended next step: rebalance toward a shared attribute and retrain. That becomes a new, lineage-tracked dataset version. As a *decision procedure*, it doesn't work yet, for three reasons. It always recommends action, because the percentile threshold always flags 10%. The 'data defect' explanation can't fire at these test sizes. And it runs on test, so following it overfits the test set. I'd move it onto validation, add an absolute or baseline-relative gate for whether to act, and make the concentration test relative to the base rate. The plumbing is the hard part, and that's done; the decision logic needs those three changes."

Why that answer lands: it separates "the plumbing works" from "the decisions are sound", gives specific reasons for the second, and ends with a clear, bounded fix list. That's credible, not defensive.

Module 8 Summary — the one page to revise

What this module is. A wrapper that sends test-set outcomes through the engine's `FeedbackLoop` (p90 threshold, at least 3 failures per episode, persisted to the catalog), then attaches templated explanations ("concentrated → check data" or "scattered → model limitation") and a fixed recommendation to rebalance and retrain. The plumbing is sound; the decision logic is degenerate at these data sizes and runs on the wrong split.

Five claims to make cold

1. Detection is the engine's. This layer adds explanations framed so that "model failure does not imply bad data."
2. A p90 threshold flags about 10% of frames by definition, so "no action" is effectively unreachable.
3. Concentrated (> 0.5) can't occur: the maximum is about 0.50 (SO-100) and 0.40 (ALOHA).
4. Feedback runs on test, so the curation loop tunes on test. Move it to val.
5. Rebalancing is the one intervention that keeps the test set fixed, so it's the only one the loop can evaluate fairly.

Decisions

Chose	Over	Because	Cost
Wrap engine `FeedbackLoop`	reimplement	single detection implementation	thresholds and metrics unverifiable here
Percentile threshold (p90)	absolute threshold	adapts to model and normalization	can never say "nothing is wrong"
Fixed 0.5 concentration cut	base-rate-relative test	simple	unreachable at p90 with 4–5 episodes
Templated explanation and recommendation	attribute analysis of reviews	honest hedging, a human in the loop	no concrete attribute or value produced
`persist=True`	in-memory only	curators find flags via `rde`	side effects, duplicates, stale after re-evaluation
Run on test outcomes	run on val	test is what's evaluated at the end	adaptive overfitting of test

Volunteer unprompted: the test-set loop; both dead branches, with the arithmetic; `FAILURE_ANALYSIS.md`'s "deterministic predictions" claim contradicts README §2.3; `retraining_selection` is unused.

Numbers and names: `error_percentile = 90.0`, `min_failures_per_episode = 3`, concentration `> 0.5`; SO-100 1,908 frames, 5 episodes, about 190 flagged, max rate ≈ 0.50; ALOHA 4,400 frames, 4 × 1,100, max rate 0.40; priority = failure_rate × (1 + mean_error) per `FAILURE_ANALYSIS.md`; output at `failures/feedback_artifact.json`.

Sentences worth memorizing

- "A percentile threshold is good for ranking failures and useless for deciding whether there are any."
- "My training loop never selects on test; my data-curation loop did, and that's the fix I'd make first."
- "At p90 with five test episodes, 'concentrated failure' can't happen. The branch is dead by arithmetic."
- "The plumbing of the flywheel is real and persisted; the decision logic on top of it needs to move to validation and get an absolute gate."

Module 9 — Model artifact, registry and deployment loader

Files covered: `src/model_training/deployment/artifact.py`, `src/model_training/deployment/registry.py`, `src/model_training/deployment/loader.py` (`deployment/__init__.py` was covered in Module 1)

Where we are: Modules 6–8 produced a selected checkpoint, its test metrics and a feedback artifact. This module answers "what exactly is this checkpoint, and how would I run it?" `build_model_artifact` packages its identity, `ModelRegistry` is meant to let you look it up, and `DeployedModel` is meant to load and serve it with health checks and latency timing. The README and `docs/DEPLOYMENT.md` present this as the project's deployment layer.

The one-line thesis: The artifact is a good identity record, and it prefers the checkpoint's own claims about its training data. But the "deployment" layer is **never executed by the pipeline**, its health check **never compares the checkpoint to the contract** despite its docstring, it checks normalization on the object inference doesn't use, and it serves **dataset `Sample` objects, not robot observations**. It's a replay harness, not a deployment.

9.1 Prerequisites — the absolute basics

- **Model artifact.** A trained model packaged with the metadata needed to use and trust it: weights, configuration, input/output schema, training data identity, code version, dependencies and metrics.
- **Model registry.** A catalogue of artifacts that answers "which model is this?" and, in mature systems (MLflow Model Registry, SageMaker), "which version is *promoted* to staging or production?", through stages, aliases and transitions. Lookup alone is a catalogue. Promotion is what makes it a registry.
- **Pre- and postprocessor.** The input normalization and tokenization before the network, and the action de-normalization after it. In LeRobot-style checkpoints these are saved as separate files (`policy_preprocessor.json` and `policy_postprocessor.json`, per `docs/DEPLOYMENT.md` Table 1), because the normalization statistics live there, not in the weights.
- **Health check / readiness probe.** Verifying, before serving, that a model can actually serve the inputs it will get. The valuable checks are *compatibility* checks (does this model's expected input match what I'll feed it?), not just *presence* checks.
- **Training/serving skew.** Any difference between how inputs were processed at training time and at serving time (different normalization stats, camera order, image size). It's the most common silent production ML failure.
- **Action chunking.** SmolVLA predicts a *sequence* of future actions per call. A robot controller executes some of the chunk, then re-queries.
- **Latency measurement on a GPU.** CUDA calls are asynchronous: Python returns before the GPU finishes. Timing with `time.perf_counter()` measures real GPU work only if something forces a synchronization (`torch.cuda.synchronize()`, or copying results to the CPU). The first call also includes one-off warm-up costs (kernel selection, memory allocation).

9.2 `build_model_artifact`

```
@dataclass
class ModelArtifact:
    model_id: str; model_version: str; artifact_path: str; architecture: str
    pretrained_model: str; training_run_id: str; experiment_id: str
    dataset_id: str; dataset_version: str
    observation_schema: dict; action_schema: dict; embodiment: str | None
    dependencies: dict; git_commit: dict; created_at: str; metrics: dict

def build_model_artifact(*, checkpoint_path, contract, run_dir_path, experiment_id, tags,
                         dataset_id, dataset_version, environment, git_info, metrics) -> ModelArtifact:
    """Assemble a ModelArtifact from a checkpoint already saved by
    SmolVLAPolicy.save_checkpoint -- verified, not assumed complete."""
    meta = SmolVLAPolicy.verify_checkpoint(checkpoint_path)
    model_id = f"{tags.get('model', 'smolvla')}-{experiment_id}"
    return ModelArtifact(
        model_id=model_id,
        model_version=str(meta.get("step", "final")),
        artifact_path=str(checkpoint_path),
        architecture="smolvla",
        pretrained_model=meta.get("checkpoint_source", ""),
        training_run_id=str(run_dir_path),
        experiment_id=experiment_id,
        dataset_id=meta.get("dataset_id", dataset_id),
        dataset_version=meta.get("dataset_version", dataset_version),
        observation_schema=contract.get("observation", {}),
        action_schema=contract.get("action", {}),
        embodiment=tags.get("embodiment"),
        dependencies={"torch": environment.get("torch"), "lerobot": environment.get("lerobot")},
        git_commit=git_info,
        created_at=_dt.datetime.now(_dt.timezone.utc).isoformat(),
        metrics=metrics,
    )
```

What's good:

- **It verifies the selected checkpoint.** This is the only place the `final` checkpoint is ever verified, when it's the fallback selection (Module 6). So an incomplete selected checkpoint fails *here*, before the artifact is registered.
- **The dataset identity comes from the checkpoint's own metadata first.** `meta.get("dataset_id", dataset_id)` prefers what `Trainer` stamped into the checkpoint at save time (`extra={"dataset_id": ..., "dataset_version": ...}`, Module 6) over what the caller says. The checkpoint is the authority on what it was trained on. That's the right provenance direction, and it would catch a caller passing the wrong version, though only by silently preferring the checkpoint's answer, not by flagging the disagreement.
- **Keyword-only arguments** (`*`) make an eleven-field constructor call impossible to get wrong by position.

What's weaker:

- **`metrics` are the *test* metrics** (`test_overall.metrics` in the runner, Module 11). `docs/DEPLOYMENT.md` Table 2 and the registry docstring both say `row["metrics"]` answers "**What metrics justified selection?**". **Selection was justified by the *validation* metric** (Module 6), which isn't in the artifact at all. Putting test metrics under that label invites exactly the confusion the project otherwise works hard to avoid.
- **`model_id = f"{tags['model']}-{experiment_id}"`** takes its prefix from a free-form tag. `experiment_id` is unique per load (Module 2), so IDs don't collide, but the prefix is cosmetic and can disagree with `architecture` (hard-coded `"smolvla"`).

- **`model_version = str(meta.get("step", "final"))`.** The `final` checkpoint is saved with `"step": fit_result["steps"]` (Module 6), so this is always a step number and the `"final"` default is unreachable for this codebase's checkpoints. "Version" here means "training step", not a registry version.
- **`pretrained_model = meta["checkpoint_source"]`.** For a resumed run (`baseline_extended`), this is plausibly the *resume path* (the source the policy was constructed from), not `lerobot/smolvla_base`. That's engine-side and unverified. If so, the lineage of weights is one hop deep: you'd follow `resume_source.json` (Module 6) by hand to get to the base model.
- **`observation_schema` and `action_schema` come from the *view's* contract**, not the checkpoint, so the artifact can't show a mismatch between them either.
- **`dependencies` records only torch and lerobot.** It misses `transformers` and `safetensors`, as Module 3 found for `environment.json`.

9.3 `ModelRegistry`

```
class ModelRegistry:
    def __init__(self, path: Path):
        self._index = JsonlIndex(Path(path))

    def register(self, artifact: ModelArtifact) -> None:
        self._index.append(artifact.to_dict())

    def get(self, model_id: str) -> dict[str, Any] | None:
        """The most recent record for ``model_id``. ... returning the first match
        returned the oldest, stalest one. Later rows supersede earlier ones for the same id."""
        row = None
        for row in self._index.query(model_id=model_id):
            pass
        return row

    def list(self) -> list[dict[str, Any]]:
        return self._index.read_all()

    def for_experiment(self, experiment_id: str) -> list[dict[str, Any]]:
        return list(self._index.query(experiment_id=experiment_id))
```

- **"Last write wins" over an append-only log** is the correct read semantics, and the docstring records the bug it fixed (the earlier version returned the first match, which was the stalest). The `for row in ...: pass` idiom is terse: it leaves the loop variable bound to the last yielded row, or `None` if there were none. Some reviewers will find it too clever. `rows = list(...); return rows[-1] if rows else None` reads more plainly.
- **It isn't wired in.** `docs/DEPLOYMENT.md` §7 says so honestly: "`ExperimentRunner.run` writes each model record directly through `ExperimentIndex.record_model` ... and never calls `ModelRegistry.register`." Grep confirms that `ModelRegistry` appears only in `deployment/__init__.py`, `registry.py` and `test_deployment.py`. It's *compatible* with the runner's file, because `record_model(**model_artifact.to_dict())` writes the same row shape, so `ModelRegistry("experiments/models.jsonl").get(...)` would work. It's just never constructed outside the test.
- **It's a catalogue, not a registry.** There's no stage, alias, promotion, or "which model is deployed" field. The question a registry exists to answer, "what's in production?", has no representation. For a project with no production, that's honest scope. Just don't call it a registry in the MLflow sense.

- **It inherits every `JsonlIndex` weakness** (Module 3): a full scan per lookup, no locking, and one torn line breaks `get` and `list` for good.
- **The one test** (`test_model_registry_round_trips`) covers register, get, for_experiment and the missing-ID case. It doesn't cover the "most recent wins" fix that the docstring narrates. A two-row test would pin it.

9.4 `DeployedModel`: what it checks, and what it doesn't

```
class DeploymentHealthError(Exception):
    """Raised by DeployedModel.health_check on any checkpoint/contract
    mismatch, detected before inference rather than surfacing as a confusing
    tensor-shape error mid-prediction."""
```

```
def health_check(self) -> dict[str, Any]:
    """Everything that must be verified before inference. ..."""
    meta = SmolVLAPolicy.verify_checkpoint(self.checkpoint_path)  # raises on incomplete/foreign dir

    action = self.contract.get("action", {})
    if not action.get("shape"):
        raise DeploymentHealthError("dataset contract has no action shape -- cannot verify compatibility")

    obs = self.contract.get("observation", {})
    images = [k for k, v in obs.items() if v.get("plane") == "external"]
    if not images:
        raise DeploymentHealthError("dataset contract has no image modality -- SmolVLA requires at least one camera")
    for k in images:
        shape = obs[k].get("shape")
        if not shape or len(shape) != 3:
            raise DeploymentHealthError(f"camera {k!r} has no valid (H, W, C) shape in the contract: {shape}")

    state_keys = [k for k, v in obs.items() if v.get("plane") == "inline"]
    for k in state_keys:
        if not obs[k].get("normalization"):
            raise DeploymentHealthError(
                f"state modality {k!r} carries no normalization statistics in the dataset contract -- "
                "the checkpoint's preprocessor expects normalized state and per-run normalization is missing")

    return {"status": "ok", "checkpoint_meta": meta, "cameras": images, "state_keys": state_keys}
```

Read it adversarially. `meta` is loaded, and then **never used again** except to be returned. Every other check looks at `self.contract` alone. So:

Promised (docstrings, `DEPLOYMENT.md` §4)	Actually checked
"any checkpoint/contract **mismatch**"	**No comparison is made.** It checks the checkpoint for completeness and the contract for well-formedness, independently.
"cannot verify compatibility" (the action-shape error text)	Only that a shape *exists*, not that it equals the checkpoint's action dimension, which `rde_checkpoint_meta.json` carries per `DEPLOYMENT.md` Table 1 ("action dimensionality").
SmolVLA needs cameras	That at least one camera exists with a 3-dimensional shape. **Not** that the camera *count* or names match what the checkpoint was trained with.
state normalization is present	Present **in the contract**. But see below: inference doesn't use it.

A concrete passing mismatch. Load an ALOHA checkpoint (14-dimensional action, 4 cameras, 28-dimensional state) with the SO-100 contract (6-dimensional action, 2 cameras, 6-dimensional state). Every check passes: the checkpoint is complete, and the contract has an action shape, 3-dimensional cameras and normalized state. `health_check` returns `{"status": "ok"}`. Then `predict` fails mid-inference with *exactly* the "confusing tensor-shape error mid-prediction" the exception class says it exists to prevent. Or, worse, if the shapes happen to line up, it runs and produces nonsense. The test suite proves the gap: `_make_checkpoint`

writes metadata with `"dataset_id": "d"` and an **empty** `model.safetensors` (`write_bytes(b"")`), and `test_health_check_passes_on_well_formed_checkpoint_and_contract` passes it against an SO-100-shaped contract.

The normalization check is on the wrong object. `load()` constructs the policy with `SmolVLAPolicy.from_checkpoint(...)`, which per `docs/API_REFERENCE.md` "loads with `contract=None`". The checkpoint uses **its own saved preprocessor statistics**. `DEPLOYMENT.md` §5 confirms that `predict` "applies the checkpoint's own preprocessor (normalization and tokenization)". So the contract's normalization statistics, the only thing the health check inspects, **aren't used at inference at all.** The real training/serving-skew risk is the opposite one: the checkpoint's stats differing from the live data's distribution. That would need a comparison of the checkpoint preprocessor's mean and std against the contract's, and that comparison isn't made.

The fix is cheap, because the metadata already exists:

```
if meta.get("action_dim") not in (None, action["shape"][0]):
    raise DeploymentHealthError(f"checkpoint action_dim={meta['action_dim']} != contract {action['shape']}")
if meta.get("dataset_id") not in (None, expected_dataset_id): ...
# and compare camera keys/count and the preprocessor's state mean/std against the contract's
```

(The exact metadata key names are engine-side and unverified. `DEPLOYMENT.md` Table 1 says the metadata includes "action dimensionality, chunk size".)

```
def load(self) -> "DeployedModel":
    self.health_check()
    t0 = time.perf_counter()
    self._policy = SmolVLAPolicy.from_checkpoint(self.checkpoint_path, device=self.device)
    self.load_time_ms = (time.perf_counter() - t0) * 1000.0
    return self
```

- **Verification runs twice**, once in `health_check` and again inside `from_checkpoint`. It's cheap (file-presence checks) and harmless.
- **`load_time_ms` excludes the health check**, and it includes weight transfer to the device. It's measured once.
- **Returning `self`** enables `DeployedModel(...).load()` chaining, as in the `DEPLOYMENT.md` example.
- **No fallback model** on failure. `DEPLOYMENT.md` §4 makes "no silent substitution" a stated rule. That's good.

`predict` and the latency breakdown

```
def predict(self, samples: Sequence[Sample]) -> tuple[Prediction, LatencyBreakdown]:
    if self._policy is None:
        raise RuntimeError("call load() before predict()")
    t0 = time.perf_counter()
    model_input = self._adapter.adapt(samples)
    t1 = time.perf_counter()
    prediction = self._policy.timed_predict(model_input)
    t2 = time.perf_counter()

    preprocessing_ms = (t1 - t0) * 1000.0
    inference_ms = prediction.latency_ms
    postprocessing_ms = max(0.0, (t2 - t1) * 1000.0 - inference_ms)
    latency = LatencyBreakdown(model_load_ms=self.load_time_ms, preprocessing_ms=preprocessing_ms,
                               inference_ms=inference_ms, postprocessing_ms=postprocessing_ms,
                               total_ms=preprocessing_ms + inference_ms + postprocessing_ms)
    return prediction, latency
```

- **The input is `Sequence[Sample]`, an engine dataset object.** Samples come from `DatasetView.iter_samples`, as in the integration test. A robot controller has camera frames, a joint-state vector and an instruction string, not engine `Sample`s. **So `DeployedModel` can only replay recorded data.** It can't be dropped into a control loop without an adapter from raw observations to `Sample`. That's the most important framing point in this module: this is an *offline inference harness with timing*, not deployment.
- **"`postprocessing_ms`" isn't postprocessing.** Per `DEPLOYMENT.md` §5, `SmolVLAPolicy.predict` applies the postprocessor (action de-normalization) *inside* the predict call, so it's already included in `prediction.latency_ms`. What's left in `(t2 - t1) - inference_ms` is the overhead of the `timed_predict` wrapper itself, which is near zero. The label promises a stage that this subtraction doesn't isolate.
- **`max(0.0, …)` hides clock disagreement.** If `inference_ms` (measured inside the engine) exceeds the outer interval, the negative value is clamped to 0, and `total_ms` (a *sum* of parts, not a measured interval) can then **exceed the real wall time**. Measuring `total_ms = (t2 - t0) × 1000` directly would make the parts auditable against the whole.
- **GPU synchronization is unverified.** If `timed_predict` returns tensors still on the GPU without synchronizing, `inference_ms` measures kernel *launch* time. The integration test's `prediction.actions.shape` suggests NumPy arrays, which would force a device-to-host copy and therefore a sync, but I can't confirm that from here.
- **No warm-up, no repetition, no percentiles.** `DEPLOYMENT.md` §6 explicitly leaves p50, p95 and p99 to the caller, as "a benchmark-design decision". That's a defensible scope line, but nothing in the repo *is* that caller.
- **The `LatencyBreakdown.note`** (preprocessing isn't split into decode, resize and tensor construction) is the project's honest-metadata habit applied well. But it names `VisionStateAdapter`, while this class uses `VisionLanguageAdapter`, which is related but not the same class.
- **Action chunks:** whether `Prediction.actions` is the first action of each chunk or the whole chunk isn't visible here. A control loop needs to know.

Nothing calls it

Grep shows `DeployedModel` used only in `deployment/__init__.py`, `tests/unit/test_failure_modes.py` and `tests/unit/test_deployment.py` (health checks only, never `load` or `predict`), and as a string inside `reports/model_card.py`. **`DeployedModel.predict` has never run in any test, any script or the pipeline.** The README's latency figures ("47–50 ms per frame on SO-100 and 86 ms per frame on ALOHA") come from `Evaluator`'s `latency_ms_per_frame` during test evaluation (Module 7). The README correctly says they "are not a deployment latency measurement". **No deployment latency has ever been measured by this code.**

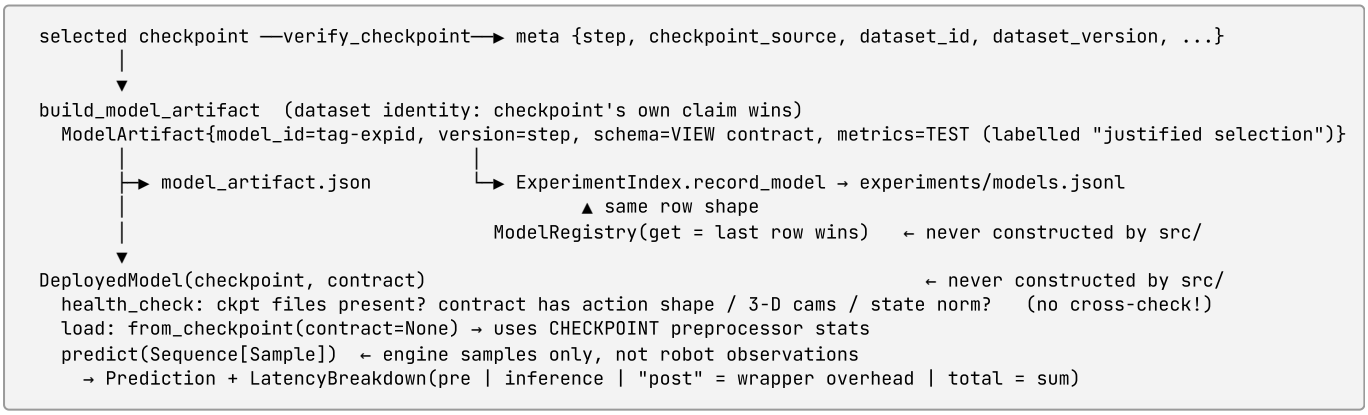
Doc vs doc: `DEPLOYMENT.md` §5 says `SmolVLAPolicy.predict` "invokes `select_action`". `docs/API_REFERENCE.md` §7.4 says predict "Uses `predict_action_chunk` … **not** `select_action` (which maintains a stateful action queue … reusing it across independent evaluation batches silently replays stale actions)". That's the bug the smoke test's comment describes as fixed. `DEPLOYMENT.md` was written before the fix and never updated. Since you can't read the engine here, cite API_REFERENCE and the smoke test as the current behaviour.

9.5 Honest gaps to own before an interviewer finds them

1. **`health_check` never compares the checkpoint to the contract**, despite "raised on any checkpoint/contract mismatch". *Consequence:* an ALOHA checkpoint against an SO-100 contract passes and then fails, or silently misbehaves, mid-inference. That's the exact failure the class exists to prevent.

2. **It checks normalization on the contract, but inference uses the checkpoint's own preprocessor stats.** *Consequence:* the check guards an unused object, and real training/serving skew between checkpoint stats and live data is unchecked.
3. **`DeployedModel` consumes engine `Sample`s, not robot observations.** *Consequence:* it's a replay harness. Deploying to a robot needs an observation adapter that doesn't exist.
4. **`DeployedModel.load` and `predict` have never been executed** by any test, script or pipeline. *Consequence:* the deployment path is unverified code, and no deployment latency exists.
5. **`postprocessing_ms` measures wrapper overhead, not postprocessing; `total_ms` is a sum, not an interval; negatives are clamped.** *Consequence:* the breakdown can misattribute time and overstate the total.
6. **The registry is a catalogue** (no stages, promotion or "deployed" concept) and **isn't wired in** (honestly documented). *Consequence:* "model registry" overclaims relative to the industry meaning.
7. **The artifact's `metrics` are test metrics, labelled as "what metrics justified selection".** *Consequence:* the validation metric that actually justified selection is absent from the artifact.
8. **`model_version` is always a step number, `pretrained_model` may be a resume path (unverified), and `model_id`'s prefix comes from a free tag.** *Consequence:* identity fields are less meaningful than their names suggest.
9. **`DEPLOYMENT.md` §5 says predict uses `select_action`.** API_REFERENCE and the smoke test say `predict_action_chunk`. *Consequence:* a stale doc describing the pre-fix bug as current behaviour.
10. **Health-check tests use an empty `model.safetensors` and mismatched metadata, and pass.** *Consequence:* the suite encodes the presence-only behaviour as correct.
11. **The registry's "most recent wins" fix isn't tested.** *Consequence:* a regression to first-match wouldn't be caught.
12. **Dependencies recorded: torch and lerobot only.** *Consequence:* the artifact can't explain a `transformers`-driven behaviour change.

The shape to carry into Module 10



Drill — answer from memory, files closed

1. What does `DeployedModel.health_check` actually verify, and name one incompatible checkpoint/contract pair it would pass.
2. *Why didn't you just* build a FastAPI server around the model?
3. *What breaks in production:* you hand `DeployedModel` to a robotics engineer and say "here's the deployment layer." What do they hit first?

4. Your model card says the registry answers "what metrics justified selection." What's in that field, and is the label right?
5. *Interviewer framing:* "How did you deploy this model, and what was its inference latency?"

Appendix A — Model answers to the drill

A1. What `health_check` verifies. "Four things: the checkpoint directory has all its required files; the contract declares an action shape; it has at least one camera with a three-dimensional shape; and every state modality in the contract has normalization stats. What it doesn't do is compare the two. It reads the checkpoint's metadata and then never uses it. So an ALOHA checkpoint with 14-dimensional actions and four cameras, checked against the SO-100 contract with 6-dimensional actions and two cameras, passes, then fails inside inference with a shape error, which is exactly what the exception class says it prevents. And the normalization check looks at the contract's stats, while inference loads the checkpoint's own preprocessor stats, so it guards something that isn't used. The fix is to compare the checkpoint's recorded action dimension, camera set and normalization stats against the contract's. All of that metadata is already saved."

Why that answer lands: it states what's checked precisely, constructs a concrete passing counterexample, and spots the wrong-object normalization check. That's an adversarial reading of your own safety code.

A2. Why not a FastAPI server? "The brief was to avoid infrastructure that doesn't earn its keep, and there's no consumer for a network endpoint. No robot or simulator is connected, so a server would only add serialization, a process boundary and failure modes. The deployment layer is deliberately a function call. If a network interface were needed, it would be a thin wrapper around `predict`. The honest limitation isn't the missing server, though. It's that `predict` takes the engine's dataset `Sample` objects, so even in-process it can only replay recorded data. Before any server, I'd add an adapter from raw observations (camera frames, a joint vector, an instruction) to model input, and a closed-loop harness that executes action chunks. I'd add a server only once a separate process, like a robot controller, needed to call it."

Why that answer lands: it defends the minimalism with a reason, then volunteers the real gap, which is input type, not transport, and orders the next steps correctly.

A3. A robotics engineer gets `DeployedModel`. "First, it doesn't accept their inputs. `predict` wants a sequence of engine `Sample` objects from a dataset view, and they have camera images and joint states from a live robot. Second, if they wrap that, they'll find it returns predictions per call with no handling of action chunks or control frequency, and SmolVLA plans in chunks. Third, the health check would pass a checkpoint trained on a different embodiment, so a wrong model reaches inference. And fourth, none of it has ever been executed: `load` and `predict` have no test, and no latency was ever measured through this class. I'd tell them it's a replay and timing harness, and that a deployment needs an observation adapter, a chunk-execution loop, a real compatibility check and a warm-up-plus-percentile benchmark."

Why that answer lands: it ranks the failures in the order they'd actually be hit, and closes with a precise, honest reclassification of what the code is.

A4. "What metrics justified selection." "That field holds the test-split metrics, the MSE and so on of the selected checkpoint on the test set. But selection was justified by the validation metric, because the trainer only ever compares checkpoints on val, and that number isn't in the artifact at all. So the label is wrong, and it's wrong in the most sensitive direction, because it invites someone to think test was used for selection. I'd store both, under 'selection: val MSE at step N' and 'reported: test metrics', and rename the registry's question."

Why that answer lands: it catches a labelling error that touches the project's core validity principle, and fixes it in a way that reinforces the val/test separation.

A5. "How did you deploy this model, and what was its inference latency?" "I didn't deploy it to a robot. There was no robot or simulator connected, and the system says so rather than reporting a rollout number. What exists is a local loader class with a health check and a per-stage latency breakdown, but that path has never been run end to end, and its health check doesn't cross-check checkpoint and contract, which I'd fix first. The latency numbers I can give are evaluation-harness numbers: about 47–50 milliseconds per frame on SO-100 and 86 on ALOHA, which has four cameras. That's prediction time divided by batch size during batched offline evaluation on a laptop RTX 4060, excluding data loading. That isn't control-loop latency. A real number would be single-sample, warmed up, synchronized, reported as p50 and p99, and measured inside a control loop that executes action chunks."

Why that answer lands: it refuses the overclaim the question invites, gives the real numbers with their exact provenance, and describes what a correct measurement would look like. Precision here is what interviewers test.

Module 9 Summary — the one page to revise

What this module is. An identity record for the selected checkpoint (provenance taken from the checkpoint's own metadata), a JSONL "registry" that's a correct but unwired catalogue, and a `DeployedModel` loader/predictor that's never executed, doesn't cross-check checkpoint and contract, and only accepts dataset samples.

Five claims to make cold

1. The artifact verifies the checkpoint and prefers the checkpoint's own dataset identity over the caller's.
2. The registry reads last-write-wins over an append-only log, isn't wired into the runner (and the docs admit it), and has no promotion concept.
3. `health_check` checks presence and well-formedness, not compatibility, and its normalization check targets the contract, which inference doesn't use.
4. `DeployedModel` takes engine `Sample`s, so it's a replay harness, and it has never run.
5. README latency (about 50 ms/frame SO-100, 86 ms ALOHA) is evaluator-batched, not deployment latency.

Decisions

Chose	Over	Because	Cost
Function-call deployment	network service	no consumer; avoid infrastructure	none yet, but the input type is the real gap
Checkpoint metadata as identity authority	caller-supplied identity	the checkpoint knows its own data	disagreement silently resolved, not flagged
JSONL registry	MLflow registry	consistent with the index; no server	no stages or promotion; inherits JSONL fragility
Load with checkpoint preprocessor	re-derive stats from contract	avoids training/serving skew from new stats	the contract check is then irrelevant
Single preprocessing latency bucket	fabricated sub-stage split	can't measure inside the adapter	"post" bucket mislabelled instead
No fallback model	load a default	never silently substitute	none

Volunteer unprompted: no cross-check in `health_check`; the wrong-object normalization check; `Sample`-only input; never executed; the test-metrics-as-selection label; `DEPLOYMENT.md` §5's stale `select_action` claim.

Numbers and names: 5 checkpoint files (`config.json`, `model.safetensors`, `policy_preprocessor.json`, `policy_postprocessor.json`, `rde_checkpoint_meta.json`); `model_id = <tag model>-<experiment_id>`; latency fields `model_load_ms`, `preprocessing_ms`, `inference_ms`, `postprocessing_ms`, `total_ms`; README evaluator latency 47–50 ms/frame (SO-100), 86 ms/frame (ALOHA), batch 16.

Sentences worth memorizing

- "My health check verified that the checkpoint and the contract each existed, not that they agreed."
- "It takes dataset samples, not robot observations, so it's a replay harness with timing, not a deployment."
- "The latency I can quote is evaluator throughput per frame, not control-loop latency, and I say that every time."

- "The registry labelled test metrics as the selection justification; selection was on validation, and the artifact should say so."

Module 10 — Reports, figures and cross-run comparison

Files covered: `src/model_training/reports/experiment_report.py`, `src/model_training/reports/model_card.py`, `src/model_training/reports/comparison.py`, `src/model_training/reports/figures.py`, `scripts/plot_results.py` (`reports/__init__.py` was covered in Module 1)

Where we are: Modules 3–9 wrote every artifact a run produces. This module turns them into what people read: an experiment report, a model card, per-run figures, a cross-run comparison table, and the README's results figure. If an earlier module is honest and this one misreports it, the honesty is lost at the last step. So this module checks presentation against source.

The one-line thesis: The report and model card follow a strong discipline: every section always renders, and anything missing says "not run" or "unavailable" instead of disappearing. But the report is built from in-memory state despite claiming otherwise, it drops a real number (this repo's git commit, SmolVLA's MAE), the comparison table has no seed aggregation despite the docs, and the README figure script hard-codes the very selection points it claims to read from disk.

10.1 Prerequisites — the absolute basics

- **Experiment report.** A human-readable record of one run: hypothesis, data, configuration, results, limitations, conclusion. Here it's `reports/experiment_report.md`.
- **Model card.** A short document attached to a trained model describing intended use, training data, evaluation results and limitations. The convention comes from Mitchell et al. (2019). Here it's `reports/model_card.md`.
- **"Absent vs not run".** In a report, an omitted section is ambiguous: was the thing not done, or did it fail silently? Always rendering the section, with an explicit "not run", removes that ambiguity. It's the reporting version of the project's "never fabricate" rule.
- **Matplotlib backend.** The rendering engine. `Agg` is a non-interactive raster backend that works without a display (servers, CI). `matplotlib.use("Agg")` sets it *process-wide*.
- **Figure lifecycle.** Each `plt.subplots()` allocates a figure that stays in memory until `plt.close(fig)`. Long loops that never close figures leak memory, and matplotlib warns after 20 open figures.
- **Reproducible figure.** A figure generated by a script from stored data, so it can be regenerated identically, as opposed to a hand-edited plot. The test is whether the script contains *any* hard-coded number that belongs to the data.
- **Seed aggregation.** Reporting mean ± standard deviation (or a confidence interval) over runs that differ only in random seed.

10.2 `render_experiment_report`

```
def render_experiment_report(ctx: dict[str, Any]) -> str:
    """``ctx`` carries everything the report cites; see ExperimentRunner for exactly what it
    assembles. Fields that were not run/available render as stated text
    ("not run", "unavailable") rather than being omitted silently."""
```

Decision: fixed sections, with explicit "not run" text. Every section heading (Objective, Dataset, Model, Training configuration, Compute environment, Test metrics, Baselines, Stratified, Failure analysis, Rollout, Limitations, Conclusion, Data feedback recommendation) renders whether or not it has data. A baseline-only run still shows "Test metrics … *not run*" and "Closed-loop / rollout evaluation: unavailable". The runner test asserts `"unavailable" in report_text` (rollout must be stated, not omitted). *Why:* a reader can tell "absent" from "failed". *Alternative:* a template engine (Jinja2) with conditionals that hide empty sections. *Cost of the choice:* a longer report with some empty sections, which is the right trade.

Doc vs code: "assembled entirely from saved run artifacts". The module docstring says the report is "assembled entirely from saved run artifacts, never from mutable live state, so re-running this function against a finished run's directory always reproduces the same report". In fact:

- the function takes a `ctx` **dict built in memory by the runner** from live objects (`intervention_result`, `training_outcome`, `test_overall`, `profile`, `feedback_artifact`, Module 11), not from files;
- **no function exists that rebuilds `ctx` from a run directory**, and no CLI command regenerates a report;
- `ctx["generated_at"]` is `now()`, so even an identical `ctx` produces a different report.

The *values* in `ctx` match what was written to disk, because they come from the same objects at the same moment. So the report is *consistent with* the artifacts, but it isn't *derived from* them. Say "built from the same in-memory results the artifacts were written from", and say that a regenerate-from-disk command is missing.

Line-level findings

```
lines.append(f"- git (episodic-data-engine): {env.get('git', {}).get('episodic_data_engine', {}).get('commit', 'n/a')}")
lines.append(f"- git (model_training): {env.get('git', {}).get('model_training', {}).get('commit', 'n/a')}\n")
```

This is the confirmed bug from Modules 3 and S2. `collect_git_info` writes `"vla_finetune"`, so the second line *always* prints `n/a`. The engine line works. Every report ever generated says this repo's commit is unknown, while `git.json` next to it has the commit.

```
lines.append(f"- selected checkpoint: `{model.get('selected_checkpoint', 'n/a')}`")
```

The runner always *sets* `selected_checkpoint` (to `None` for baseline-only runs), so `.get(..., 'n/a')` returns `None` and the report prints `` `None` `` instead of `n/a`. The default only applies to a missing key, not to a `None` value. It's the same pattern as `_fmt`'s own `None → "n/a"`, which isn't used here. Cosmetic, but it's the "absent vs not run" discipline slipping.

```
if ctx.get("model_test_mse") is not None and mean_mse:
    lines.append(f"| **{cfg['training']['model']} (this run)** | {_fmt(ctx['model_test_mse'])} | "
                 f"n/a | {_fmt(ctx['model_test_mse'] / mean_mse)} |")
```

SmolVLA's MAE is printed as `n/a` in the baseline table, though `test_metrics["overall"]["mae"]` is right there in `ctx`. Each baseline row shows its MAE, and the model under test doesn't. That's a hard-coded `n/a` for a number that exists.

```
def _fmt(v, digits=4):
    if v is None: return "n/a"
    if isinstance(v, float): return f"{v:.{digits}g}"
    return str(v)
```

- **4 significant figures** (`g` format), so 0.023942 becomes `0.02394` and 1.0e-4 becomes `0.0001`. That's sensible for MSEs that range over orders of magnitude.

- A **NumPy** `float32` isn't a Python `float`, so it falls through to `str(v)` with full repr. Module 3's `default=str` concern again: harmless here, inconsistent display.

```
def _metrics_table(metrics):
    rows = [(k, v) for k, v in metrics.items() if not isinstance(v, (list, dict))]
```

Per-dimension MAE is a list, so it's dropped from the report. README Table 4 (per-joint MAE, "error grows from the base of the arm toward the end effector") is one of the project's most interpretable findings. It isn't in any generated report. It must have been read out of `test_overall.json` by hand. That's a reporting gap that hides a real result.

- **Limitations** come only from the runner's list (no GPU, no torch, non-SmolVLA model). **The rebalanced caveat `EXPERIMENTS.md` §4 promises is never added** (Module 4 gap 8), and neither is the silent `best → final` fallback (Module 6 gap 3).
- **Conclusion** is always `None` from the runner, so every report says "*not yet written -- fill in after reviewing the metrics above.*". That's honest (the code doesn't pretend to conclude), but it means every report is a draft by construction.
- **"Data feedback recommendation"** repeats `recommended_next_action`, which was already printed under Failure analysis. It's a duplicated section, and Module 8 showed the text is the same template every run.
- **No escaping.** `hypothesis` and `objective` text go into Markdown raw. A `|` in a hypothesis would break the following table only if it were inside one, and they aren't. Harmless in practice.

10.3 `render_model_card`

```
"""``model_card.md`` generation. Claims only what evaluation
actually measured -- no capability beyond what ``test_metrics``/``rollout`` carry is stated."""
```

- **The "claims only what was measured" promise is mostly kept.** The card lists architecture, base model, dataset version, embodiment, input modalities, four training hyperparameters, raw test metrics, and two hard-coded limitations: confidence is an uncalibrated constant, and metrics are offline, not task success. There's no capability language. That's to its credit.
- **"Known failure modes" is the feedback artifact's `candidate_explanations`.** Module 8 showed those are templates, and at these data sizes it's always the "scattered → model/coverage limitation" sentence. So **every model card's "Known failure modes" section is the same canned sentence with a count**, presented under a heading that implies analysis of *this* model.
- **"Deployment requirements: loader: `model_training.deployment.DeployedModel` (local, no server)"** names a loader that has never run (Module 9 gap 4).
- **Metrics are printed raw** (`f"- {k}: {v}"`, no `_fmt`), so you get 17-digit floats next to the report's 4-significant-figure ones.
- **The confidence limitation is hard-coded.** It's correct for SmolVLA, and only SmolVLA runs produce a model card (the runner builds one only when there's a selected checkpoint). It would be wrong if a second model family with real confidence got a card.
- **Embodiment comes from a free-form tag.** "unspecified" if the tag is absent.

10.4 `comparison.py`

```
def load_run_summary(run_dir):
    run_dir = Path(run_dir)
    if not run_dir.exists():
        raise FileNotFoundError(f"no run directory at {run_dir}")
    summary = {"run_dir": str(run_dir)}
    for name in ("dataset.json", "model.json", "manifest.json", "summary.json"):
        p = run_dir / name
        if p.exists():
            summary[name[:-5]] = json.loads(p.read_text())
    ...  # test_overall.json, baselines.json, stratified.json likewise
```

- **This one genuinely *is* "reproducible from disk"**, as its docstring says. It reads only saved files. Good.
- `manifest.json` **is never written by anything** (Module 3, gap S12), so `summary["manifest"]` is always absent. It's dead code that implies a file exists.
- **Missing files are silently skipped**, so a crashed run (no `summary.json`, no `test_overall.json`) yields a row of `None`s with no indication of *why*. There's no `status` to read (Module 3 gap 10).
- **A partially written JSON file** (Module 3 gap 1) raises `JSONDecodeError` and aborts the whole comparison.

```
def build_comparison_table(run_dirs, metric="mse"):
    for rd in run_dirs:
        s = load_run_summary(rd)
        ...
        row = {"run_dir": ..., "model": model.get("architecture"), "dataset_id": ...,
               "training_version": ..., "intervention": dataset.get("kind"),
               metric: overall.get(metric)}
        for name, m in (s.get("baselines") or {}).items():
            row[f"{name}_{metric}"] = m.get(metric)
        rows.append(row)
```

- **There's no aggregation across seeds.** `docs/EXPERIMENTS.md` §7: "`mtrain compare` can then report the mean and standard deviation across the resulting run directories." **No mean, standard deviation or grouping code exists** (I grepped for `std`, `mean(` and `stdev` in `reports/` and `experiments/` and found nothing). Each run is one row. That's a documented feature that isn't implemented (gap logged in Module 4).
- **There's no check that the runs share a test set.** Rows from `baseline_full` and `reduced_data` sit side by side as if comparable, while Module 4 showed they're scored on different test episodes. The table should at least show each run's test-episode IDs or a hash of them.
- **Baseline columns are added per row**, so runs with different baseline sets produce heterogeneous rows. `write_comparison_csv` handles that correctly (union of keys, in first-seen order, via `csv.DictWriter`). The terminal table in `cli.py` doesn't. It builds headers from `rows[0].keys()` and adds cells *by position*. Checking rich's source shows `Table.add_row` appends headerless extra columns instead of raising. **So with heterogeneous rows, values land under the wrong headers, silently.** That's a Module 12 issue, flagged here because it's this function's output shape that triggers it.
- `metric` **names a key in** `test_overall.metrics`**.** `--metric mse` gets the split-level MSE (correct). There's no confusion with the per-episode misnamed "mse" (Module 7), because that lives in a different structure.

```
def build_model_dataset_matrix(rows, metric="mse"):
    """The model x dataset table: one row per (model, dataset/intervention) pair."""
    return [{"model": r.get("model"), "dataset": r.get("training_version"),
             "intervention": r.get("intervention"), metric: r.get(metric)} for r in rows]
```

- **It's not a matrix and it's not called.** It's a projection of rows, one per run, and nothing in `src/`, `scripts/` or `tests/` calls it. It also keys "dataset" on `training_version`, which is *unique per run* (Module 4: every run mints a new `training/vN`), so even a real pivot on it would never group two runs together. Pivoting on `dataset_id` plus `intervention` is what "model × dataset" means.

10.5 `figures.py`

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt  # noqa: E402

def _save(fig, path):
    path.parent.mkdir(parents=True, exist_ok=True)
    fig.savefig(path, dpi=150, bbox_inches="tight")
    plt.close(fig)
    return path
```

- **`matplotlib.use("Agg")` at import time** makes figure rendering work headless, so training boxes and CI need no display. *Cost:* it's a **process-wide side effect of importing the package**. `reports/__init__.py` imports `figures`, and the runner imports `reports`, so importing `model_training.experiments` in a Jupyter notebook switches the notebook's backend to Agg. Inline plots then stop appearing until you reset it. The fix is to create figures with the object API (`matplotlib.figure.Figure()` plus `FigureCanvasAgg`), which needs no global backend.
- **`plt.close(fig)` in `_save`** prevents the figure leak. Without it, a sweep over many runs would accumulate open figures. It's the non-obvious line that matters.
- **`dpi=150, bbox_inches="tight"`** are sensible defaults for PNGs in Markdown.

The "never a fabricated placeholder chart" contract (module docstring) is kept. Every function returns `None` or `[]` when there's nothing to plot (no records, missing keys, fewer than 2 fraction points). That's consistent with Module 7's `{"status": "no outcomes"}` returns.

Per-function notes:

- `plot_training_curves`: loss always; LR and grad-norm only if present. **`r["loss"]` is a hard key access**, so an engine change to the record shape is a `KeyError` that aborts the *run* (the call isn't guarded in the runner). Plotting can crash the whole run after training succeeded.
- `plot_validation_curve`: filters rows that have the metric. That's robust.
- `plot_error_distribution`: **a bar chart of four quantiles** (median, p90, p95, p99). It shows four numbers as bars, which is a weak encoding. A histogram or empirical CDF of the per-frame errors (all available in `test_overall.outcomes`) would show the actual tail shape.
- `plot_episode_length_distribution`: expects the engine's histogram entries with `lo`, `hi` and `count` (another undocumented-shape coupling, as in Module 5). The x-label says duration in seconds.
- `plot_task_distribution`: for single-task SO-100, a one-bar chart in every run.
- **`plot_model_comparison` and `plot_data_fraction_curve` are never called** (grep finds only their definitions). The data-efficiency curve, the deliverable `run_data_fraction_sweep` exists to produce, has no

caller and no CLI path.

10.6 `scripts/plot_results.py`: the README figure

```
"""Render the README results figure from completed run directories.

Reads only artifacts every run writes (``metrics/validation.jsonl``,
``evaluation/test_overall.json``, ``evaluation/baselines.json``), so the figure
is reproducible from disk: ..."""
```

It *reads* only those artifacts, but it also **hard-codes facts that belong to the data**:

```
first = _validation(a.so100)
cont = [(1000 + s, m) for s, m in _validation(a.so100_continued)]   # resume offset hard-coded
_curve(ax, first + cont, selected_step=1800)                         # selection hard-coded
ax.text(1030, 0.12, "resumed,\nlr 1e-4 → 3e-5", ...)                  # LRs hard-coded as text
ax.set_ylim(6e-4, 0.3)
...
_curve(ax, _validation(a.aloha), selected_step=800)                  # selection hard-coded
ax.set_ylim(0.09, 1.6); ax.set_yticks([0.1, 0.2, 0.5, 1.0])
```

- **The "selected checkpoint" ring is drawn at a hard-coded step** (1800 and 800), not read from `model.json` / `training_outcome.json`'s `best_checkpoint`, which the script has access to. If you regenerate it from different runs, it either raises `KeyError` (`dict(points)[selected_step]` with no such step) or rings the wrong point under the label "selected checkpoint". For *these* runs it agrees with the README ("validation MSE reached its minimum (0.0018) at step 1,800"), so the published figure is right. The script just isn't the general, from-disk generator its docstring describes.
- **The resume offset** `1000 + s` should be the base run's final step (`training_outcome.steps_completed`). **The LR annotation** should come from the two runs' `config.resolved.yaml`.
- **The axis limits are tuned to these numbers**, so other runs may clip.
- **Panel (c) uses** `summary.mse` **from** `test_overall.json` for SmolVLA and `baselines.json` for the baselines, both from the same run directory. That's the right source, the same test split for all three bars.
- `"svg.fonttype": "path"` renders text as outlines, so the SVG looks identical without the font installed, at the cost of non-selectable text. That's a deliberate portability choice for a README image.
- **It reads** `r["mse"]` **from validation rows**, so it's tied to MSE regardless of `selection_metric`.

Honest phrasing: "the figure is generated by a script from the runs' metric files; the selection markers and annotations are set by hand for these runs."

10.7 Honest gaps to own before an interviewer finds them

1. **The report's model_training git commit always prints** `n/a` (key mismatch). *Consequence:* every report hides this repo's provenance.
2. **The report claims to be "assembled entirely from saved run artifacts"**, but it's built from in-memory `ctx`, with no regenerate-from-disk path, and it's timestamped. *Consequence:* an overclaim, and reports can't be rebuilt after a template fix.
3. **Per-dimension MAE is dropped from the report** (lists are filtered out). *Consequence:* the per-joint finding (README Table 4) is invisible in generated reports.

4. **SmolVLA's MAE is a hard-coded `n/a` in the baseline table.** *Consequence:* the model under test shows less than the baselines.
5. **`mtrain compare` has no seed aggregation**, despite `EXPERIMENTS.md` §7. *Consequence:* a documented feature is missing.
6. **The comparison doesn't check or show whether runs share a test set.** *Consequence:* cross-intervention rows look comparable when they aren't (Module 4).
7. **The CLI comparison table misaligns heterogeneous rows** (positional cells under first-row headers; rich silently adds columns). *Consequence:* wrong numbers under wrong headers.
8. **`plot_results.py` hard-codes the selected steps, resume offset, LR text and axis limits**, contrary to "reproducible from disk". *Consequence:* regenerating it from other runs mislabels or crashes.
9. **`build_model_dataset_matrix`, `plot_model_comparison` and `plot_data_fraction_curve` are uncalled.** The "matrix" pivots on per-run-unique versions. *Consequence:* the data-efficiency curve has no path to exist.
10. **`matplotlib.use("Agg")` at import is a process-wide side effect.** *Consequence:* importing the runner breaks inline plotting in notebooks.
11. **The model card's "Known failure modes" is a templated sentence** (Module 8), and it names an unexecuted loader. *Consequence:* the card implies model-specific analysis it doesn't contain.
12. **The rebalanced and selection-fallback limitations are never added to reports.** *Consequence:* the two caveats that matter most are missing.
13. **A plot `KeyError` (`r["loss"]`) can abort a run after successful training.** *Consequence:* a presentation bug destroys an expensive result's completion (no `summary.json`).
14. **`manifest.json` is read but never written; missing files are silently skipped.** *Consequence:* dead code, and crashed runs appear as rows of `None`.
15. **No unit tests for any reports module** (the only coverage is the runner test's existence and substring checks). *Consequence:* every bug above shipped unnoticed.

The shape to carry into Module 11

```
runner (in-memory ctx: config, dataset, profile, model, outcome, env+git, metrics, baselines,
        stratified, feedback, limitations, conclusion=None)
  │
  ├─ render_experiment_report(ctx) ──▶ reports/experiment_report.md   (all sections always; "not run")
  │        bugs: git(model_training)=n/a · SmolVLA MAE n/a · per-dim MAE dropped · `None` checkpoint
  ├─ render_model_card(artifact, {...}) ──▶ reports/model_card.md      (failure modes = template)
  └─ figures.* ──▶ reports/figures/*.png  (Agg at import; close every fig; None when nothing to plot)

later, from disk only:
  mtrain compare --runs A B ──▶ load_run_summary (reads manifest.json: never exists)
                                └ build_comparison_table → one row per run, no seed aggregation,
                                  no shared-test-set check → rich table (positional!) / CSV (by key)
  scripts/plot_results.py ──▶ docs/figures/results.svg   (selected steps 1800/800 hard-coded)
```

Drill — answer from memory, files closed

1. What does "absent vs not run" mean in this report generator, and why does it matter?
2. *Why didn't you just* use Jinja2 templates, or log everything to W&B and screenshot it?

3. *What breaks in production:* you fix a typo in the report template and want every past run's report regenerated. What happens?
4. Is the README's results figure reproducible? Be precise.
5. *Interviewer framing:* "How do you compare experiments, and how do you know a difference between two runs is real?"

Appendix A — Model answers to the drill

A1. Absent vs not run. "Every section of the report always renders. If something didn't happen (no GPU, no fine-tuning, no rollout), the section says 'not run' or 'unavailable' with a reason instead of disappearing. That matters because an omitted section is ambiguous: it could mean 'skipped on purpose' or 'crashed silently', and in a results document that ambiguity is where overclaiming hides. Rollout is the clearest case. Every report states there's no simulator or robot, and the runner test asserts the word 'unavailable' is in the report. I'll admit the discipline slips in a couple of places: a baseline-only run prints a literal `None` for the selected checkpoint, and SmolVLA's MAE is hard-coded to 'n/a' even though it exists."

Why that answer lands: it explains the principle in terms of what it prevents, gives the tested example, and volunteers where your own code doesn't live up to it.

A2. Why not Jinja2 or W&B? "The report is linear and small, so f-strings in one function keep it readable and dependency-free. Jinja would mostly add conditionals I deliberately don't want, because sections never hide. W&B gives you a dashboard, not a durable document. The report is meant to sit in the run directory next to the artifacts it cites, readable in ten years without a service. The real weakness isn't the templating choice, though. It's that the report is rendered from in-memory state at the end of the run, and there's no command to rebuild it from a run directory. I'd fix that by writing a `ctx` builder that reads only the saved artifacts, which `compare` already does for its own table."

Why that answer lands: it justifies the choice from the document's purpose, then redirects to the actual architectural flaw, with a fix grounded in code that already exists.

A3. Regenerating past reports. "You can't, with what exists. The report is built from a context dict the runner assembles in memory at the end of each run, and there's no function or CLI command that reconstructs that dict from a run directory. So a template fix only affects future runs. The artifacts needed are all on disk (resolved config, dataset, profile, training outcome, evaluation, feedback), so the fix is a `build_report_ctx(run_dir)` function and an `mtrain report --run` command, with a test that regenerating a fresh run's report reproduces it apart from the timestamp. That would also make the docstring's 'assembled from saved artifacts' claim true."

Why that answer lands: a precise "no", the reason, and a fix with its own acceptance test. It also turns a doc overclaim into a concrete deliverable.

A4. Is the README figure reproducible? "Partly. The data in it is: the curves and bars are read from each run's validation log and test and baseline JSON, so regenerating from the same run directories redraws the same data. But the script hard-codes things that belong to the data: the selected-checkpoint markers at steps 1,800 and 800, the 1,000-step resume offset, the learning-rate annotation and the axis limits. For these runs, those values are correct (the validation minimum really was at 1,800), but point the script at different runs and it either crashes looking up step 1,800 or rings the wrong point as 'selected'. The fix is to read the selection from each run's model.json, the offset from the base run's steps completed, and the LRs from the resolved configs."

Why that answer lands: it separates data reproducibility from annotation reproducibility, confirms that the published figure is correct, and gives the exact source each hard-coded value should come from.

A5. "How do you compare experiments, and how do you know a difference is real?" "Comparison reads only saved artifacts, so `mtrain compare` works on runs from any session and gives one row per run: model, dataset version, intervention, test MSE, and each baseline's MSE on the same split. Knowing a difference is real is where the project is weakest, and I'd say so. Each experiment is a single seed. The test split is five episodes. The policy is stochastic: re-scoring moves MSE by up to five percent. The docs say compare re-

ports a mean and standard deviation over seeds, but that isn't implemented. And runs with different data interventions are scored on different test episodes. So the one difference I'd defend is the 18% improvement from continued training, which is on the identical test split and well outside the measured inference noise. For anything else, I'd want a paired comparison on a shared test set, a bootstrap over episodes and at least three seeds."

Why that answer lands: it describes the mechanism, then gives an honest, specific accounting of uncertainty, picks the one claim that survives and explains why, and names the minimum evidence needed for the rest.

Module 10 Summary — the one page to revise

What this module is. A Markdown experiment report and model card rendered at the end of each run, per-run PNG figures, a disk-only cross-run comparison table, and a script for the README figure. The "never omit, say not run" discipline is good; several values are lost or mislabelled on the way to the page, and two documented capabilities (seed aggregation, rebuild from disk) don't exist.

Five claims to make cold

1. Every report section always renders, and missing work reads "not run" or "unavailable". Rollout is always stated as unavailable, and that's tested.
2. Comparison reads only saved artifacts, so it's reproducible across sessions, with one row per run.
3. The report is rendered from in-memory state. There's no regenerate-from-disk path, despite the docstring.
4. There's no seed aggregation and no shared-test-set check in `compare`. Only continued training (same test split, 18%, outside about 5% noise) is defensible.
5. The README figure's data comes from disk, but its selection markers (1,800 and 800) are hard-coded.

Decisions

Chose	Over	Because	Cost
f-string Markdown with fixed sections	Jinja / dashboards	readable, durable, no hidden sections	no regenerate path; manual escaping
Render from runner `ctx`	build from run directory	data already in memory	can't rebuild; overclaimed docstring
`Agg` at import	object-oriented figures	headless rendering	process-wide backend side effect
Return `None` when nothing to plot	placeholder charts	never fabricate	none
Quantile bar chart	histogram / CDF	simple	weak view of the tail
Hand-tuned README figure	fully data-driven	publication polish	hard-coded selection and annotations

Volunteer unprompted: the `n/a` git line; per-dimension MAE dropped; seed aggregation missing; the positional rich-table misalignment; `plot_results.py`'s hard-coded markers; no tests for the reports package.

Numbers and names: `_fmt` uses 4 significant figures; figures at dpi 150; README figure size 13.2×4.1 in, three panels, selected steps 1,800 (SO-100) and 800 (ALOHA), resume offset 1,000; test MSE bars SO-100 0.908 / 0.0239 / 0.0239, ALOHA 1.010 / 0.106 / 0.216; files `reports/experiment_report.md`, `reports/model_card.md`, `reports/figures/{training_loss,learning_rate,gradient_norm,validation_mse,error_distribution,episode_length_distribution,task_distribution}.png`.

Sentences worth memorizing

- "A section that says 'not run' is honest; a section that's missing is ambiguous."
- "My comparison table reads only what's on disk, but it can't tell you two runs used different test sets, and that's the column I'd add first."
- "The only difference I'd defend is continued training: same test split, 18%, well outside the 5% inference noise."
- "The README figure's data is regenerated from disk; its annotations were placed by hand."

Module 11 — Orchestration: `ExperimentRunner` and the data-fraction sweep

Files covered: `src/model_training/experiments/runner.py`, `src/model_training/experiments/data_fraction.py` (`experiments/__init__.py` was covered in Module 1)

Where we are: Modules 2–10 each described one stage. This module is the conductor that calls them in order and threads results between them: `ExperimentRunner.run`, plus a sweep driver that calls `run` once per data fraction. Most of the cross-module bugs found so far *happen* here. This is where a bad config meets an expensive GPU run, where seeds are or aren't applied, and where partial failure leaves its traces. So this module is mostly about **ordering and identity**.

The one-line thesis: The runner is a clear, linear *Transaction Script* that makes every stage leave a named artifact. But it validates late (config errors surface *after* training), it applies `seed` only to directory names (so a "multi-seed" study trains identical seeds under one shared `experiment_id`), and it isn't transactional. The sweep driver inherits all of that, and adds a curve whose points are scored on different test sets.

11.1 Prerequisites — the absolute basics

- **Orchestrator / pipeline driver.** Code whose job is ordering calls to other components and passing results along, with little logic of its own.
- **Transaction Script (Fowler).** One procedure that runs a whole business operation step by step. It's simple and readable, and it grows long. *Alternatives:* a pipeline of stage objects (Chain of Responsibility), or a DAG framework (Airflow, Prefect, Kedro) that adds caching, retries and resume.
- **Fail fast.** Validate everything you can *before* doing anything expensive, so errors surface in seconds, not after a GPU hour.
- **Transactional vs best-effort side effects.** A transaction commits all of its effects or none. This runner writes to four places (the run directory, the engine catalog, the JSONL index and engine failure records) with no coordination, so a failure midway leaves some writes done and others not.
- **Seed vs replicate.** A *replicate* is a run that differs only in its random seed. A study of seed variance needs the seed to reach every source of randomness (initialization, data order, dropout, sampling). A run whose "seed" changes only its folder name isn't a replicate.
- **Data-efficiency (learning) curve.** Test performance plotted against training-set size. To be valid, every point must be scored on the *same* test set, and ideally trained on *nested* subsets.

11.2 `ExperimentRunner.__init__` and module constants

```
RDE_REPO = Path(os.environ.get("RDE_REPO_PATH", str(_default_repo_root() / "episodic-data-engine")))
MT_REPO = Path(os.environ.get("MODEL_TRAINING_REPO_PATH", str(Path(__file__).resolve().parents[3])))

class ExperimentRunner:
    def __init__(self, settings=None, catalog=None, runs_dir=None, index_dir=None):
        self.settings = settings or get_settings()
        self.catalog = catalog or Catalog(self.settings.paths.catalog_db)
        self.runs_dir = Path(runs_dir) if runs_dir else MT_REPO / "runs"
        self.index = ExperimentIndex(Path(index_dir) if index_dir else MT_REPO / "experiments")
```

- **Constructor injection with defaults.** Every collaborator can be passed in, and the tests do exactly that (`ExperimentRunner(settings=..., catalog=..., runs_dir=tmp_path/..., index_dir=...)`). That's *Dependency Injection* done simply, without a container, and it's the reason the runner is testable against a temporary catalog.
- **Construction has side effects.** `ExperimentIndex(...)` creates the `experiments/` directory (each `JsonlIndex.__init__` calls `mkdir`), so instantiating a runner touches disk.
- **The path constants are read at import time.** Setting `RDE_REPO_PATH` after `model_training.experiments` has been imported has no effect. Module 3 covered the provenance consequences of guessing these paths.

11.3 `run`, in order, with what can fail where

```
def run(self, config: ExperimentConfig, seed: int | None = None) -> ExperimentRunResult:
    seed = seed if seed is not None else config.seeds[0]
    run_dir = make_run_dir(self.runs_dir, config.experiment_name, seed)
```

Seed: the identity bug

`seed` is used in exactly two places: `make_run_dir(..., seed)`, which names the directory, and `summary["seed"]`. **It never reaches `config.training.seed`, the intervention seed or the split seed.** `Trainer` reads `tc.seed` (Module 6). So:

- `mtrain train --config X --seed 1` trains with `training.seed = 0`, like the default run, and writes to `seed_1/`.
- `docs/EXPERIMENTS.md` §7 says: "Running the same configuration across several seeds is a matter of calling `run` once per seed; `mtrain compare` can then report the mean and standard deviation". Following that literally yields **replicates that share a training seed**. They'd differ only through GPU nondeterminism and flow-matching sampling, so the "seed variance" you measured would **underestimate the real variance**. That's arguably worse than having no replicates, because it looks like evidence.
- **The replicates also share one `experiment_id`.** The config is loaded once, and `experiment_id` is stamped at load (Module 2). Calling `run(cfg, seed=s)` for several `s` reuses it. Downstream, `model_id = f"{tag}-{experiment_id}"` (Module 9) is identical across seeds, so `ModelRegistry.get(model_id)` returns only the *last* seed's model, and the index's `experiments.jsonl`, `datasets.jsonl` and `metrics.jsonl` rows can't be told apart by `experiment_id`.
- `config.seeds[1:]` is never read by anything (Module 2).

The fix: `cfg = config.model_copy(update={"training": config.training.model_copy(update={"seed": seed})})`, re-stamp the identity, and loop over `config.seeds` in the CLI. Whether the split seed should vary across replicates is a design choice. Usually it shouldn't, so the test set stays fixed.

Stage 0: identity and environment, before anything can fail usefully

```
environment = collect_environment()
git_info = collect_git_info(RDE_REPO, MT_REPO)
run_dir.write_json("environment.json", environment)
run_dir.write_json("git.json", git_info)
(run_dir.path / "config.resolved.yaml").write_text(yaml.safe_dump(config.resolved_dict(), sort_keys=False))

limitations: list[str] = []
if environment.get("gpu") is None:
    limitations.append("no GPU detected in this environment at run time -- training ran on CPU if it ran at all")
if environment.get("torch") is None:
    limitations.append("torch not installed in this environment -- this run could not perform real fine-tuning")
```

- **Writing provenance first is a defensible choice.** Even a crashed run records what it ran on. The cost (Module 3) is that the directory is now non-empty, so a same-day retry is blocked.
- `sort_keys=False` keeps the resolved YAML in schema field order, so it reads like the source config. That's a small, deliberate readability touch.
- **The `limitations` list starts here** and is threaded through to the report and model card. It's the right mechanism, but underused: it never receives the rebalanced caveat (Module 4), the `best → final` fallback (Module 6) or the shared-seed issue above.
- **CPU training isn't blocked.** With no GPU, a SmolVLA run proceeds on CPU (the limitation text admits this). `docs/IMPLEMENTATION_PLAN.md` records that CPU-only fine-tuning was "impractical". A `--require-gpu` guard would turn hours of wasted CPU time into an instant error.

Stage 1: intervention, the first irreversible write

```
intervention_result = build_intervention_dataset(
    config.dataset.dataset_id, config.dataset.version, config.intervention,
    config.splits, settings=self.settings, catalog=self.catalog,
)
run_dir.write_json("dataset.json", {...})
self.index.record_dataset(experiment_id=..., training_version=..., kind=..., episode_count=len(...), recorded_at=...)
```

- **This creates engine versions** (Module 4): `clean/vK` and `training/vM`. From here on, a failure orphans catalog versions.
- **`record_dataset` is written to the index *before* training**, so a later crash leaves an index row for a run that never finishes (Module 3 gap 10).
- **`episode_count = len(episode_ids)`** is the whole selected pool (train + val + test), not the training count. The column name invites misreading.

Stage 2: open the view, check, profile

```
training_version = intervention_result.training_version
engine = DataEngine(self.settings)
view = engine.get_dataset(config.dataset.dataset_id, training_version)
try:
    split_report = verify_split_integrity(view)  # raises on leakage
    run_dir.write_json("split_integrity.json", split_report.to_dict())
    profile = build_dataset_profile(config.dataset.dataset_id, training_version, settings=..., catalog=...)
    run_dir.write_json("dataset_profile.json", profile.to_dict())
    figures.plot_task_distribution(...); figures.plot_episode_length_distribution(...)
```

- **`try/finally: view.close()`** guarantees the view is released on every exit path, including exceptions. Without it, a crashed run would leak file handles (and presumably a DuckDB connection) for the life of the process. That matters in a sweep (11.5), which runs many experiments in one process. It's correct, and it's the most important resource-management line in the file.

- The profile opens its *own* second view internally (Module 5). Two views of one version are open at once, which is harmless and slightly wasteful.

Stage 3: training

```
train_ids = view.episode_ids(split="train")
episode_weights = {
    eid: float((view.episode(eid).attributes or {}).get("sampling_weight", 1.0))
    for eid in train_ids
}
episode_weights = episode_weights if any(w != 1.0 for w in episode_weights.values()) else None
```

- **Weights are read back from the *built* version**, not from `intervention_result.episode_weights`. That's a good choice: the trainer uses what the engine actually stored (the `sampling_weight` attribute), so a disagreement between what was requested and what was built can't silently change training. It's the same "trust the materialized artifact" stance as Module 5's split check. Here it's actually load-bearing.
- **Only train episodes' weights are passed**, which resolves Module 4's "weights recorded for val and test too" at the point that matters.
- **All-ones collapses to `None`**, so unweighted runs take the engine's unweighted sampling path (inferred).

```
training_outcome = None
selected_checkpoint = None
if config.training.model == "smolvla":
    from ..training.trainer import Trainer
    trainer = Trainer(run_dir, config)
    training_outcome = trainer.train(view, view, episode_ids=train_ids, episode_weights=episode_weights)
    run_dir.write_json("training_outcome.json", training_outcome.to_dict())
    figures.plot_training_curves([json.loads(l) for l in (run_dir.path / "metrics" / "train.jsonl").read_text().splitlines()]
                                 if (run_dir.path / "metrics" / "train.jsonl").exists() else [], ...)
    ...
    selected_checkpoint = training_outcome.best_checkpoint
else:
    limitations.append(f"training.model={config.training.model!r} has no gradient-based fit; "
                       "this run only scored baselines, no fine-tuning occurred")
```

- **`trainer.train(view, view, ...)`.** The same view is passed as `train_view` and `val_view`. The trainer's two-view signature anticipates separate datasets for validation (for example, a held-out scene), and the runner never uses that.
- **The lazy `Trainer` import** is the import-light idea. Module 1 showed it saves nothing on the engine side, because `deployment.artifact` already imports `SmolVLAPolicy` at module top.
- **The `else` branch is Module 2's typo trap.** Any `model` string other than exactly `"smolvla"` lands here, and the run completes with exit 0.
- **Re-reading `train.jsonl` from disk for plotting** (instead of using `fit_result["history"]`, which the API doc says exists) is consistent with "figures from saved artifacts" (Module 10), and it costs one file read.

Stage 4: baselines, then the model under test

```
baseline_results = evaluate_baselines(view, config.evaluation.baseline_models, split=config.evaluation.split,
                                      batch_size=config.evaluation.batch_size, max_frames=config.evaluation.max_frames)
...
if selected_checkpoint:
    from rde.model_adapters.models import SmolVLAPolicy
    policy = SmolVLAPolicy.from_checkpoint(selected_checkpoint)
    adapter = VisionLanguageAdapter(view.contract)
    strat_results = stratified_evaluate(view, policy, adapter, base_split=config.evaluation.split, ...,
                                        slices=config.evaluation.slices)
    test_overall = strat_results.pop("overall")
```

Validation happens after the expensive part. Two config errors are detected only here, *after* fine-tuning has finished:

- an unknown baseline name raises `KeyError` inside `evaluate_baselines` (Module 7);
- an unknown slice name raises `ValueError` inside `stratified_evaluate` (Module 7).

A typo like `baseline_models: [nearest_neighbour]` (British spelling) costs a full training run (5–10 minutes on the README's hardware, or hours at scale) and then fails, leaving a run directory with checkpoints and no evaluation, and no `summary.json`. Both checks are pure name lookups that could run in stage 0 (or in the Pydantic schema with `Literal`). That's the clearest *fail-fast* violation in the codebase.

- **`config.evaluation.split` is used for the "test" evaluation.** It defaults to `"test"`. Setting it to `"val"` would make the headline "test metrics", the feedback artifact and the model card all describe validation, with every label still saying "test". It's a sharp edge, and arguably the hook for Module 8's fix: feedback should run on val *in addition to* test.
- **`from_checkpoint(selected_checkpoint)` is called without `device=`.** `training.device` is honoured for training but not for evaluation, which auto-selects. Minor.
- **The evaluation policy is never released.** After stratified evaluation, `policy` stays referenced until `run` returns. In a sweep (11.5), memory is reclaimed only when the next `run` frame replaces it. That's fine on one GPU, because the next run's trainer allocates after this frame is gone. It's worth an explicit `del` for symmetry with the trainer's careful release.

Stage 5: artifacts, feedback, model card

```
predictions_dir = run_dir.sub("predictions")
with (predictions_dir / "test_outcomes.jsonl").open("w") as f:
    for o in test_overall.outcomes:
        f.write(json.dumps({"run_id": str(run_dir.path), "experiment_id": ..., "dataset_id": ...,
                            "dataset_version": training_version, "frame_id": o.frame_id,
                            "episode_id": o.episode_id, "error": o.error, "confidence": o.confidence,
                            "per_dim_error": o.per_dim_error}, default=str) + "\n")
```

- **Per-frame outcomes with full provenance.** This file is quietly the most valuable artifact in the run. It's the only place the evaluated test episodes are recorded (Module 4), the input for the bootstrap Module 7 recommends, and the base for per-joint analysis. `FAILURE_ANALYSIS.md` §4 is right to call it the traceability anchor.
- **`run_id` is the absolute path of the run directory**, so it breaks if the runs directory is moved or copied to another machine. A relative path, or the `experiment_id` plus seed, would be portable.
- **Predicted and target action vectors aren't stored**, only the errors. Module 8 showed the stated justification ("deterministic, reproducible on demand") is false for SmolVLA.

```
model_artifact = build_model_artifact(checkpoint_path=selected_checkpoint, contract=view.contract, ...,
                                      metrics=test_overall.metrics)
self.index.record_model(**model_artifact.to_dict())
feedback_artifact = build_feedback_artifact(config.experiment_id, model_artifact.model_id, ..., test_overall, ...)
(run_dir.sub("failures") / "feedback_artifact.json").write_text(json.dumps(feedback_artifact, ...))
run_dir.write_json("model_artifact.json", model_artifact.to_dict())
(run_dir.sub("reports") / "model_card.md").write_text(render_model_card(...))
```

- **`record_model` goes into the index *before* feedback runs.** A failure in feedback (an engine error inside `FeedbackLoop.detect`) leaves a registered model in `models.jsonl` with no `model_artifact.json` in its run directory, because that's written *after* feedback. The index then points at a run that has no artifact file. Writing the local artifact first and indexing last would keep the run directory authoritative.
- **Feedback runs on `test_overall`.** That's Module 8's central methodological issue, and this is the line to change.

Stage 6: closing writes

```
if test_overall is not None:
    run_dir.append_jsonl("metrics/test.jsonl", {"experiment_id": ..., **test_overall.metrics})
run_dir.append_jsonl("metrics/rollout.jsonl", {"experiment_id": ..., "status": "unavailable",
                                               "reason": "no simulator or physical robot connected in this environment"})
report_md = render_experiment_report(report_ctx)
(run_dir.sub("reports") / "experiment_report.md").write_text(report_md)
summary = {..., "test_mse": ..., "baseline_mean_action_mse": baseline_json.get("mean_action", {}).get("mse"),
           "steps_completed": ..., "wall_clock_s": ...}
run_dir.write_json("summary.json", summary)
self.index.record_experiment(run_dir=..., created_at=..., **summary)
if test_overall is not None:
    self.index.record_metric(experiment_id=..., run_dir=..., split=config.evaluation.split, **test_overall.metrics)
```

- **The rollout "unavailable" record is written in every run.** That's the refusal-to-fabricate principle made into data, so a downstream tool reading `rollout.jsonl` can't mistake absence for zero.
- `summary.json` **records the mean-action baseline but not nearest-neighbour**, which is the baseline that actually matters (Module 7's tie). So the index's `experiments.jsonl`, built from `summary`, can't show the project's key result without opening each run's `baselines.json`.
- `wall_clock_s` **is training only.** Total run time (including about 3× test inference from the slices, Module 7) isn't recorded anywhere.
- `summary.json` **is written last** and is the implicit completion marker (Module 3).
- **There's no top-level** `try/except`**.** Any exception after stage 1 propagates out of `run` with partial state in four stores. The trainer's `crash.json` covers only `fit` (Module 6).

What `run` writes, and where, in order

#	Write	Store	Irreversible?
1	run directory, env, git, config	run directory	blocks same-day retry
2	`clean/vK`, `training/vM`	**engine catalog**	**yes** (immutable versions)
3	`datasets.jsonl` row	index	append-only
4	checkpoints, metrics, outcome	run directory	—
5	`models.jsonl` row	index	append-only
6	failure records (`persist=True`)	**engine**	yes
7	artifacts, report, `summary.json`	run directory	—
8	`experiments.jsonl`, `metrics.jsonl` rows	index	append-only

There's no coordinator across the three stores, and no status. Module 3's state-machine fix (`status: running → completed/failed`, written first and atomically replaced) is the minimal remedy.

11.4 Why a Transaction Script, and when it stops being right

Decision: one linear `run` **method of about 250 lines.**

- *Why:* every stage's inputs are visible in one place, the order is obvious, and the README's Figure 1 ("seven ExperimentRunner stages") maps directly onto the code. For a research harness with one user, that readability is worth a lot.
- *Alternative:* stage objects with a common interface (`Stage.run(ctx) -> ctx`), or a DAG tool (Kedro, Prefect, Airflow) with per-stage caching.

- *Cost:* no stage can be re-run alone. If evaluation fails, you can't resume from the trained checkpoint through `run`; you'd use `mtrain evaluate` (Module 12), which re-implements a *subset* of stage 4 with different settings. There's no caching (a re-run rebuilds versions and retrains), and the method mixes orchestration with artifact formatting (the inline `json.dumps` for predictions and evaluation files).
- *When it flips:* once you need **resume-from-stage** (multi-hour runs, preemptible GPUs) or **sweeps of more than a handful of runs**. Both want stage-level checkpointing and caching, which a DAG framework or a simple "skip stage if its output artifact exists and is complete" rule would give.

11.5 `run_data_fraction_sweep`

```
"""... Fractions are not executed unless explicitly requested -- calling this function *does* run them
(the framework makes them possible; they do not run by default); the caller decides which fractions and how many."""

def run_data_fraction_sweep(base_config, fractions, runner=None) -> list[dict[str, Any]]:
    """Run ``base_config`` once per fraction, varying only the data quantity.

    Every other knob (model, hyperparameters, splits, seed) is held fixed, so
    the resulting curve isolates the quantity effect -- ..."""
    runner = runner or ExperimentRunner()
    results = []
    for frac in fractions:
        data = base_config.model_dump()
        data["experiment_name"] = f"{base_config.experiment_name}_frac{int(round(frac * 100))}"
        data["intervention"] = {"kind": "fraction", "fraction": frac, "seed": base_config.intervention.seed}
        data["experiment_id"] = None
        data["created_at"] = None
        cfg = ExperimentConfig.model_validate(data)
        outcome = runner.run(cfg)
        results.append({"fraction": frac, **outcome.summary})
    return results
```

- **The module docstring contradicts itself** ("not executed unless explicitly requested -- calling this function *does* run them ... they do not run by default"). What it means is that nothing calls this automatically. The prose reads like two edits merged. `docs/API_REFERENCE.md` Table 8 states the same idea cleanly.
- **Re-stamping identity** (`experiment_id = None`, `created_at = None`) is deliberate and correct: each fraction gets a fresh ID. This is the pattern Module 2 noted as relying on the "not user-editable" guard.
- **"Isolates the quantity effect" is false, because of Module 4.** Each fraction builds its own `training/vN` *from the filtered pool*, so **every point on the curve is scored on a different test set**, of a different size (at 25% of 48 episodes the test split is about 1 episode). The curve conflates "less training data" with "different, smaller test set". The fix is the same: split once, and subsample only the train split.
- **Non-nested subsets** (Module 4 gap 6): every fraction draws with the same seed independently, so the 25% subset isn't inside the 50% subset.
- **`fraction = 0.0` raises** (Module 4 gap 2), and **there's no `try` per fraction**. One failing fraction aborts the sweep, and `results` for the completed fractions is **lost from the return value**. Only the run directories survive. A sweep that took hours returns nothing. *Fix:* catch per fraction, record the failure, continue, and persist `results` incrementally (for example, to `experiments/sweeps/<name>.jsonl`).
- **Name collisions:** `int(round(frac * 100))` maps 0.125 and 0.12 both to `_frac12`, and Python rounds $0.125 \times 100 = 12.5$ to 12. The second fraction's `make_run_dir` raises `FileExistsError` the same day. Use `f"{frac:g}"` or permille.
- **The base intervention is discarded.** A sweep over a `rebalanced` or `imbalanced` base config silently becomes a sweep over plain random fractions. That's only acceptable if documented.
- **A single seed per fraction, run sequentially.** A curve from one seed per point at these test sizes is mostly noise (Module 7).

- **The output isn't wired to its plot.** `plot_data_fraction_curve(rows, metric, ...)` exists (Module 10). The rows here carry `test_mse` (from `summary`), so `metric="test_mse"` would work, but nothing calls it.
- **Untested and uncalled.** No test or CLI command references `run_data_fraction_sweep` (grep). Module 1's table has been updated accordingly. Its first real use would hit the 0.0 crash and the moving test set.

11.6 Honest gaps to own before an interviewer finds them

1. `seed` **only names the directory.** Replicates share `training.seed` and `experiment_id`. *Consequence:* a multi-seed study following `EXPERIMENTS.md` §7 produces fake replicates that underestimate variance, plus colliding `model_id`s (the registry returns only the last seed).
2. **Config errors are detected after training** (unknown baseline names and slice names raise in stage 4). *Consequence:* a typo costs a full GPU run and leaves an incomplete run directory.
3. **Not transactional across four stores** (run directory, engine catalog, index, engine failure records), with no status marker. *Consequence:* partial failures leave orphan versions, index rows without runs, and runs without artifacts.
4. `record_model` **is indexed before feedback and before** `model_artifact.json` **is written.** *Consequence:* a feedback failure leaves an indexed model whose run lacks the artifact file.
5. `evaluation.split` **silently relabels.** Setting it to `"val"` makes "test" reports describe validation. *Consequence:* a sharp edge on the project's central val/test discipline.
6. **No GPU guard.** SmolVLA training proceeds on CPU with only a limitation note. *Consequence:* hours lost to an impractical configuration.
7. `summary.json` **and** `experiments.jsonl` **omit the nearest-neighbour baseline**, and total run time isn't recorded. *Consequence:* the key comparison isn't queryable from the index.
8. `run_id` **in predictions is an absolute path.** *Consequence:* outcomes don't survive moving the runs directory.
9. **No stage-level resume or caching** (Transaction Script). *Consequence:* any failure after training means re-training, or using `mtrain evaluate`'s divergent re-implementation.
10. **The sweep scores each fraction on a different test set and uses non-nested subsets.** *Consequence:* the data-efficiency curve doesn't isolate quantity, contrary to its docstring.
11. **The sweep aborts on the first failure (0.0 always fails) and loses completed results.** *Consequence:* hours of sweep produce an exception and no rows.
12. **Sweep name collisions** (`_frac12` for 0.12 and 0.125) and **a discarded base intervention**. *Consequence:* a `FileExistsError` mid-sweep, and silently changed semantics.
13. **The sweep is untested and has no CLI entry point.** *Consequence:* a headline capability that has never run.
14. `training.device` **isn't used for evaluation, and the evaluation policy isn't released explicitly.** *Consequence:* minor inconsistency.
15. **The same view serves as train and val; the trainer's two-view seam is unused.** *Consequence:* none today. It's the natural place for a held-out-scene val set later.

The shape to carry into Module 12

```
ExperimentRunner.run(config, seed)                      seed → dir name + summary ONLY
 0  make_run_dir · environment.json · git.json · config.resolved.yaml · limitations[]
 1  build_intervention_dataset ──► ENGINE: clean/vK, training/vM      (irreversible)
    dataset.json · index.datasets
 2  view = get_dataset(training/vM)  ┐ try
    verify_split_integrity · profile │
 3  weights ← view attrs (train only)│
    Trainer.train(view, view) ──────┘──► checkpoints, metrics, training_outcome.json
 4  evaluate_baselines (KeyError on bad name ← AFTER training)
    SmolVLA.from_checkpoint(best) → stratified_evaluate (ValueError on bad slice ← AFTER training)
 5  predictions/test_outcomes.jsonl · index.models (before feedback!)
    FeedbackLoop on TEST ──► ENGINE failure records · feedback_artifact.json · model_artifact.json · model_card.md
 6  metrics/test.jsonl · rollout.jsonl("unavailable") · experiment_report.md · summary.json (last)
    index.experiments · index.metrics          ┘ finally view.close()

run_data_fraction_sweep(base, fractions): for f: cfg(kind=fraction, new id) → run()
    different test set per f · not nested · aborts on first failure · results lost · untested
```

Drill — answer from memory, files closed

1. List, in order, every place `ExperimentRunner.run` writes, and say which writes are irreversible.
2. *Why didn't you just* use Airflow, Prefect or Kedro for the pipeline?
3. *What breaks in production:* you run `mtrain train --config baseline.yaml --seed 0`, then `--seed 1`, then `--seed 2`, and report mean ± std. What's wrong?
4. A teammate sets `baseline_models: [nearest_neighbour]`. Walk through what happens.
5. *Interviewer framing:* "How would you produce a trustworthy data-efficiency curve with this system?"

Appendix A — Model answers to the drill

A1. The write order. "First the run directory itself, with environment, git info and the fully resolved config. Then the intervention, which creates new dataset versions in the engine's catalog. That's irreversible, since versions are immutable, and a dataset row goes into the index. Then the training artifacts in the run directory: checkpoints, train and validation metrics, the training outcome. Then evaluation files and the per-frame test outcomes, and a model row in the index. Then the feedback step, which persists failure records into the engine, so that's a second irreversible write. Then the feedback artifact, model artifact, model card, report and `summary.json`, and finally the experiment and metric rows in the index. Three stores, with no coordination between them, so a failure halfway leaves some stores updated and others not. The minimal fix is a status file written first and flipped atomically at the end."

Why that answer lands: it's complete, it's in the right order, and it identifies the two engine writes as the irreversible ones. That's a sign you've thought about failure, not just the happy path.

A2. Why not a DAG framework? "For one researcher running a handful of experiments, a single linear method is the most readable form. The stages, their order and their artifacts are all on one screen, and they map one-to-one onto the architecture diagram. A DAG tool would add a scheduler, a metadata store and a deployment surface, mostly to buy caching and retries I didn't need at five-minute runs. Where that stops being true is resume-from-stage and large sweeps. Today, if evaluation fails after training, I can't resume through the runner, I'd retrain. So the first step wouldn't be Airflow. It'd be making each stage skip itself if its output artifact already exists and is complete, which gives resume with no new infrastructure. I'd adopt Prefect or Kedro once runs got long or numerous enough that caching mattered."

Why that answer lands: it argues from scale, admits the concrete gap (no resume), and proposes an intermediate step before reaching for a framework. That's proportionate engineering.

A3. Three seeds, mean ± std. "Those aren't replicates. The `--seed` value only names the run directory and goes into the summary; it never reaches `training.seed`, which stays at the config's zero. All three runs train with the same seed, so any differences come from GPU nondeterminism and the stochastic action sampler, and the standard deviation would understate real seed variance, which is worse than reporting nothing. They also share one experiment ID, because the config was loaded once, so their model IDs collide, and the registry would return only the last one. The fix is for the runner to copy the config with `training.seed` set, re-stamp the identity, and for the CLI to loop over the config's `seeds` list, keeping the split seed fixed so all replicates share a test set."

Why that answer lands: it explains why the numbers are *misleadingly* small, not just wrong, catches the identity collision, and specifies the fix including the split-seed subtlety.

A4. `nearest_neighbour`. "Nothing checks baseline names until the evaluation stage. So the runner builds the dataset versions, runs the integrity check and profile, fine-tunes SmolVLA for the full budget, writes the checkpoints and training outcome, and only then calls `evaluate_baselines`, which raises `KeyError` because the registry name is `nearest_neighbor`. The run ends with checkpoints, no evaluation, no `summary.json`, orphaned dataset versions in the catalog, and a run directory that blocks a same-day retry. The same applies to an unknown slice name. Both are pure lookups that belong in config validation, as `Literal` types or a stage-zero check, so the error appears in under a second instead of after training."

Why that answer lands: it traces the exact sequence and its debris, then classifies it correctly as a fail-fast violation with a one-line fix.

A5. "How would you produce a trustworthy data-efficiency curve with this system?" "Not with the sweep function as it stands. It filters episodes and then re-splits, so each fraction is scored on a different and shrinking test set: at 25% of 48 episodes, the test split is about one episode. The subsets aren't nested, it fails outright at 0%, and it loses completed results if any fraction fails. I'd change four things. Split the full dataset once and freeze val and test. Subsample only the train split, as prefixes of one seeded permutation, so the subsets are nested. Make the zero point 'evaluate the pretrained base with no fine-tuning'. And run at least three training seeds per point, with real seed propagation, reporting a bootstrap interval over test episodes. Then each point differs only in training data quantity, which is what the curve claims to show."

Why that answer lands: it starts with a frank "not as it stands" and the precise reasons, then gives a protocol that addresses every confound. That's what an interviewer probing experimental design wants to hear.

Module 11 Summary — the one page to revise

What this module is. A linear Transaction Script that calls every stage in order (intervention, integrity, profile, train, baselines, stratified evaluation, artifact, feedback, reports, summary, index) with injectable collaborators and a `finally`-closed view. Plus a sweep driver that loops it over data fractions. The ordering and identity handling are where the project's cross-cutting bugs concentrate.

Five claims to make cold

1. One method, seven-plus stages, every stage leaves a named artifact. Collaborators are constructor-injected for tests.
2. `seed` names the directory only. `training.seed` is unchanged and replicates share `experiment_id`, so multi-seed studies are fake.
3. Baseline and slice names are validated after training. That's the clearest fail-fast violation.
4. Writes span the run directory, the engine catalog (irreversible), the index and engine failure records, with no transaction and no status.
5. The sweep's points use different test sets and non-nested subsets, abort on the first failure, and have never been run.

Decisions

Chose	Over	Because	Cost
Transaction Script `run`	stage objects / DAG tool	readable, maps onto Figure 1	no resume or caching; mixed concerns
Constructor DI with defaults	globals / container	testable against a temporary catalog	construction touches disk
Provenance written first	write at end	crashed runs keep provenance	blocks same-day retry
Weights read back from the built version	from the intervention request	train on what the engine stored	none
`try/finally` view close	rely on GC	no handle leaks across a sweep	none
Sweep = loop over `run`	dedicated sweep pipeline	reuse everything	inherits moving test set; no fault isolation

Volunteer unprompted: fake seeds and the ID collision; late name validation; the four-store non-transactional writes; `record_model` before the artifact file; the sweep's test-set confound and lost results.

Numbers and names: stage order 0–6 above; `summary.json` keys `experiment_id, experiment_name, dataset_id, training_version, intervention, tags, seed, model, test_mse, baseline_mean_action_mse, steps_completed, wall_clock_s`; sweep names `{name}_frac{round(f×100)}`; env vars `RDE_REPO_PATH`, `MODEL_TRAINING_REPO_PATH` (read at import).

Sentences worth memorizing

- "My `--seed` flag renamed a folder; it didn't change a seed. Replicates that share a seed understate variance, which is worse than no replicates."
- "Every config error my runner can detect, it should detect before it touches the GPU. Two of them it detects after."
- "The runner writes to three stores without a transaction; the minimum fix is a status file written first and flipped atomically last."

- "A learning curve is only a learning curve if every point is scored on the same test set."

Module 12 — The CLI surface: `mtrain` and the script wrappers

Files covered: `src/model_training/cli.py`, `scripts/train.py`, `scripts/evaluate.py`, `scripts/test.py`, `scripts/rollout.py`, `scripts/compare.py`, `scripts/analyze_failures.py`, `scripts/export.py` (`scripts/plot_results.py` was covered in Module 10)

Where we are: Module 11 was the programmatic entry point (`ExperimentRunner.run`). This module is the human one, the eleven `mtrain` commands a researcher actually types. Most commands are thin views over run-directory artifacts. Three of them *change* state: `train` runs the pipeline, `evaluate`/`test` re-score and overwrite, and `build-intervention` rewrites a config file. Those three are where the CLI can undo the guarantees the rest of the system builds.

The one-line thesis: The CLI's structure is right: lazy imports, a dry run by default for config rewriting, and honest "unavailable" commands. But `mtrain evaluate` is a divergent reimplementation of the runner's evaluation stage. It **overwrites the published test numbers in place**, with different slices and batch settings, and leaves the rest of the run directory describing the old numbers. Its own reference doc recommends using it with `--split val`, which writes validation results into `test_overall.json`.

12.1 Prerequisites — the absolute basics

- **Typer.** A library that builds a CLI from Python function signatures. `@app.command()` registers a function as a sub-command, and each parameter becomes an option via `typer.Option(default, "--flag")`. `...` (Ellipsis) as the default means *required*. `list[Path]` options are given by **repeating the flag** (`--runs a --runs b`), not as space-separated values. `"--dry-run/--apply"` declares a boolean with paired on and off flags.
- **`typer.Exit(code)`.** Ends the command with an exit code, and no traceback.
- **`rich`.** A terminal-formatting library: `Console.print` (with `[yellow]…[/yellow]` markup), `print_json` and `Table`.
- **Command–query separation (CQS).** Distinguishing commands that *change state* from queries that only *read* it. CLIs that mix the two in one verb (e.g. "evaluate" that also overwrites) surprise users.
- **Single source of truth.** Each behaviour implemented in exactly one place. Two implementations of "evaluate a run" will drift.
- **Idempotence.** Running a command twice has the same effect as running it once. For a stochastic model, re-scoring is *not* idempotent: each run produces new numbers.
- **YAML round-trip loss.** `yaml.safe_load` followed by `yaml.safe_dump` discards comments, and reflows quoting and folded strings. Preserving them needs a round-trip parser such as `ruamel.yaml`.

12.2 Structure: lazy imports and thin views

```
app = typer.Typer(no_args_is_help=True, add_completion=False)
console = Console()

def _print_json(data) -> None:
    console.print_json(json.dumps(data, indent=2, default=str))
```

- **Only `typer`, `yaml` and `rich` load at import.** Every `rde` and `model_training` import is inside a command function. So `mtrain --help`, `mtrain rollout` and `mtrain analyze-failures` start without loading the engine or torch. This is the one place where the import-light discipline (Module 1) is applied *fully*, and it's where it matters most, because CLI startup latency is user-facing.
- `no_args_is_help=True` prints help instead of an error for a bare `mtrain`. `add_completion=False` hides Typer's shell-completion install commands, so there's less surface.
- **The name `mtrain`** avoids colliding with the engine's `rde model` (module docstring, Module 1).

Read-only views

Command	Reads	Writes	Notes
`inspect --dataset --version`	engine explorer `overview`	—	`--version` defaults to `"training/v1"`, an arbitrary version that may not exist for a given dataset. That's the only CLI default that names data.
`profile --dataset --version [--out]`	Module 5's `build_dataset_profile`	optional JSON file	the read-only profile the imbalance choice must come from
`validate --run`	`training_outcome.json`, `metrics/validation.jsonl`	—	see below
`analyze-failures --run`	`failures/feedback_artifact.json`	—	prints the Module 8 artifact verbatim
`compare --runs … [--metric] [--out]`	Module 10 `build_comparison_table`	optional CSV	terminal-table misalignment (Module 10 gap 7)

`validate` misdiagnoses.

```
outcome_path = run / "training_outcome.json"
if not outcome_path.exists():
    console.print(f"[yellow]no training_outcome.json in {run} -- this run had no gradient-based training[/yellow]")
    raise typer.Exit(1)
```

A missing `training_outcome.json` means *either* a baseline-only run *or* a run that crashed in training (Module 6: `crash.json` is written, `training_outcome.json` isn't). The message asserts the first. A one-line check for `crash.json` would give the right diagnosis. `analyze-failures` has the same pattern: "this run had no test-set model evaluation" could also mean "crashed before evaluation".

`compare` usage is documented two ways. `scripts/compare.py`'s docstring says `python scripts/compare.py --runs a b c`. With a Typer `list[Path]` option, that form fails: after the first value, `b` and `c` are unexpected extra arguments. The README (line 186, "`--runs` repeated once per run") and `docs/API_REFERENCE.md` §4.7 show the correct repeated-flag form. The script wrapper's docstring is wrong.

12.3 `train`

```
@app.command()
def train(config: Path = typer.Option(..., "--config"), seed: Optional[int] = typer.Option(None, "--seed")):
    """Run one experiment end to end: intervention -> train -> evaluate -> report."""
    cfg = load_experiment_config(config)
    result = ExperimentRunner().run(cfg, seed=seed)
    console.print(f"run complete: {result.run_dir}")
    _print_json(result.summary)
```

- It's a thin wrapper over Modules 2 and 11, which is correct: one implementation.
- `--seed` **is the Module 11 bug at its entry point.** The help text gives no hint that it only names the directory.
- **No** `--base` (Module 2 gap 9), **no seed loop** over `config.seeds`, and **no** `--force` for a failed same-day run (Module 3 gap 3).
- **Exceptions surface as raw tracebacks.** For a research CLI, that's defensible: the traceback is the diagnosis. But a `LookupError("unknown version: so100_pickplace@clean/v9")` would be more readable as a one-line error with exit code 2.

12.4 `evaluate` and `test`: the divergent re-implementation

```
@app.command()
def evaluate(run: Path = typer.Option(..., "--run"), split: str = typer.Option("test", "--split")):
    """Re-run stratified + baseline offline evaluation for an existing run's
    selected checkpoint against SPLIT, overwriting its evaluation/ artifacts."""
    from rde.catalog import Catalog
    ...
    from model_training.evaluation import episode_metrics, error_distribution, evaluate_baselines, stratified_evaluate, temporal_error

    dataset_json = json.loads((run / "dataset.json").read_text())
    model_json = json.loads((run / "model.json").read_text())
    checkpoint = model_json.get("selected_checkpoint")
    if not checkpoint:
        console.print(f"[yellow]no selected_checkpoint recorded for run {run}[/yellow]")
        raise typer.Exit(1)

    settings = get_settings()
    view = DataEngine(settings).get_dataset(dataset_json["dataset_id"], dataset_json["training_version"])
    try:
        policy = SmolVLAPolicy.from_checkpoint(checkpoint)
        adapter = VisionLanguageAdapter(view.contract)
        strat = stratified_evaluate(view, policy, adapter, base_split=split, slices=["per_task"])
        overall = strat.pop("overall")
        payload = {"summary": overall.summary(), "metrics": overall.metrics,
                   "error_distribution": error_distribution(overall),
                   "episode_metrics": episode_metrics(overall),
                   "temporal_error": temporal_error(overall, view)}
        eval_dir = run / "evaluation"
        eval_dir.mkdir(exist_ok=True)
        (eval_dir / "test_overall.json").write_text(json.dumps(payload, indent=2, default=str))
        (eval_dir / "stratified.json").write_text(json.dumps({...}, indent=2, default=str))
        _print_json(payload["summary"])
    finally:
        view.close()
```

What's right: it reopens the *exact* `training_version` recorded in `dataset.json` (data by identity, never by path), reloads the *selected* checkpoint, and closes the view in `finally`. Re-scoring the same version on the same split scores the same episodes.

What diverges from the runner's stage 4 (Module 11):

Aspect	Runner	`mtrain evaluate`
Baselines	re-fit and scored	**not run.** `evaluate_baselines` is imported and never called, despite the docstring's "Re-run stratified + baseline"
Slices	`config.evaluation.slices` (`[per_task, per_length_bucket]` from `base.yaml`)	**hard-coded `["per_task"]`**
`batch_size` / `max_frames`	from config (16 / `null`)	`stratified_evaluate` defaults (32 / `None`)
Predictions file	`predictions/test_outcomes.jsonl` written	**not rewritten**, so it now describes the *old* predictions
`metrics/test.jsonl`, `summary.json`, `model_artifact.json`, report, model card, feedback, index	written	**none updated**

Consequences:

1. **It overwrites the reported numbers in place.** SmolVLA is stochastic (Module 7: up to 5% between re-scorings), so running `mtrain test` on a published run *changes* `test_overall.json`. The README's numbers can no longer be traced to the file they came from, and there's no backup or history. The rest of the system treats a run directory as write-once (`make_run_dir` refuses to overwrite). This command quietly breaks that rule for the most important file in it.
2. **It destroys the slices.** The original `stratified.json` (task plus three length slices) is replaced by per-task only. That's information loss, not just a re-score.
3. **It leaves the directory internally inconsistent.** `test_overall.json` disagrees with `summary.json`, `metrics/test.jsonl`, `model_artifact.json["metrics"]`, the report, the model card, `predictions/test_outcomes.jsonl` and the feedback artifact computed from the old outcomes.
4. **`--split val` writes validation results into `test_overall.json`.** `docs/API_REFERENCE.md` §4.5 and §9.3 *recommend* `mtrain evaluate --run <dir> --split val`. The doc honestly says it overwrites `test_overall.json`, but the *filename* then lies, and `mtrain compare`, `plot_results.py` and `load_run_summary` will all read validation metrics as test metrics. On a project whose central discipline is val/test separation, this is the most dangerous documented workflow in the repo.
5. **Unused imports** (`Catalog`, `evaluate_baselines`) are leftovers from the fuller version the docstring still describes.

What it should be: a call into the *same* evaluation function the runner uses, factored out of `ExperimentRunner.run` (a `run_evaluation_stage(run_dir, config, view, checkpoint)` that takes its settings from the run's own `config.resolved.yaml`), writing to a **new, versioned** directory (`evaluation/rescore_<timestamp>_<split>/`) and never over the original. That gives one implementation, no overwrite, and filenames named for the split.

```
@app.command()
def test(run: Path = typer.Option(..., "--run")):
    """Alias for `evaluate --split test` -- the offline test-set score."""
    evaluate(run=run, split="test")
```

Calling a Typer command function directly works, because it's still a plain Python function. Note that the alias inherits everything above. "`mtrain test`" sounds like a harmless check, but it overwrites the published test numbers.

12.5 `rollout`: honesty as a command

```
@app.command()
def rollout(run: Path = typer.Option(..., "--run")):
    """Closed-loop rollout evaluation. No simulator or robot is connected in
    this environment, so this reports 'unavailable' rather than fabricating a
    success rate -- never conflated with offline evaluation."""
    payload = {"status": "unavailable", "reason": "no simulator or physical robot connected in this environment"}
    (run / "metrics").mkdir(exist_ok=True)
    with (run / "metrics" / "rollout.jsonl").open("a") as f:
        f.write(json.dumps(payload) + "\n")
    _print_json(payload)
```

Decision: a command whose only job is to record that something can't be measured. *Why:* `docs/API_REFERENCE.md` §4.6 says "absence is a recorded, honest fact rather than a silently missing file". It also gives the interface a slot where a real rollout harness would plug in. *Cost:* it appends a duplicate record each time it's run (the runner already wrote one), with no timestamp to distinguish them. `mkdir(exist_ok=True)` without `parents=True` raises on a mistyped run path whose parent doesn't exist. That's arguably the right failure, though it's a raw traceback.

12.6 `export`: "paper-ready" with gaps

```
output.mkdir(parents=True, exist_ok=True)
for sub in ("figures", "tables", "metrics", "configs", "manifests"):
    (output / sub).mkdir(parents=True, exist_ok=True)
# figures ← reports/figures/*; metrics ← evaluation/*.json; configs ← config.resolved.yaml;
# manifests ← manifest.json, dataset.json, model.json, dataset_profile.json (if present)
(output / "experiment_summary.md").write_text(
    "... Every file under figures/, tables/, metrics/, configs/, manifests/ traces back to the "
    f"run directory above. Regenerate with `mtrain export --run {run} --output {output}`.\n")
```

- **`tables/` is created and never filled.** No code copies anything into it.
- **`manifest.json` never exists** (Module 3, S12).
- **`git.json` and `environment.json` aren't exported.** The bundle that claims everything "traces back" omits the commits and environment, which are exactly what a reader of a paper artifact needs to reproduce it. (It does keep the hostname out, as Module 3 noted, but it drops the commits along with it.)
- **`reports/experiment_report.md`, `reports/model_card.md` and `predictions/test_outcomes.jsonl` aren't exported**, so neither the narrative nor the per-frame evidence is included.
- **It overwrites an existing export silently** (`exist_ok=True` plus `copy2`). That's the opposite policy to `make_run_dir`. For a derived bundle that can be regenerated, it's defensible, but stale files from a previous export into the same directory remain.
- **"Nothing is recomputed or summarized lossily"** (API_REFERENCE §4.9) is true for what *is* copied, but the selection of what to copy is lossy in the most important places.

12.7 `build-intervention`: resolving the design rule

```
@app.command("build-intervention")
def build_intervention_cmd(dataset, version, kind, variable=None, variable_value=None,
                           oversample_factor: float = 3.0, target_config: Path = ...,
                           reason: str = "", dry_run: bool = typer.Option(True, "--dry-run/--apply")):
    """... Deliberately does not build the dataset version itself -- `mtrain train`
    does that as part of a normal run, with real lineage. This command only
    resolves *which* episodes/weights the intervention names, from the
    dataset's actual per-episode metadata, and never presumes them."""
```

What's right:

- **Dry run by default.** It prints the resolved block, and you must pass `--apply` to write. For a command that rewrites a config file, that's exactly the right default.
- **Resolution and materialization are separate.** It *names* episodes; `train` *builds* the version, so lineage is created once, by the normal path. This is the command that operationalizes "the weak dimension must not be presumed" (Module 2): it reads the real catalog and resolves from real metadata.
- **`task_id` values are converted to `int`** (`value = int(variable_value)`). That avoids Module 2's string-vs-int silent no-op on the generated path.
- **An empty result exits with an error** for rebalanced ("resolved zero episodes").

What's wrong:

- **`--apply` destroys the config's comments.** `yaml.safe_load` followed by `yaml.safe_dump(data, sort_keys=False)` rewrites the whole file. The shipped `imbalanced.yaml` and `rebalanced.yaml` carry 15–20-line header comments explaining *why* they're templates and what the result must and mustn't claim ("do not claim a fixed weak regime unless the intervention and the evidence both support it"). **Applying the resolved intervention deletes that reasoning** and reflows the `>` folded hypothesis strings. The safeguards written for the human are erased at the moment they matter. `ruamel.yaml` round-trip mode, or writing the block to a separate `*.intervention.yaml` that the loader merges, fixes this.
- **Imbalanced is written as `kind: custom`.** `block = {"kind": "custom", "episode_ids": ..., "variable": ..., "variable_value": ..., "reason": ...}`. The run's `dataset.json` will then say `kind: custom`, and `summary.intervention`, the index, and `mtrain compare`'s "intervention" column all lose the fact that this was the *imbalanced* experiment. Only `tags.intervention: imbalanced_data` preserves it, and only because the template sets that tag.
- **Imbalanced inherits the silent no-op** (Module 2 gap 4). `--kind imbalanced --variable episode_length_bucket` with **no `--variable-value`** passes `None`, `resolve_episode_subset` keeps every episode, and the command prints and (with `--apply`) writes a `custom` block listing **all** episode IDs. The rebalanced path checks for an empty result; the imbalanced path doesn't check for a *full* result.
- **`_target_episodes` is copied from `interventions.py`.** The CLI defines its own `_target_episodes(var, val)` with the same body as the closure inside `build_intervention_dataset`. That's two implementations of "which episodes have attribute = value", which will drift. It should be one public function in `data/interventions.py`.
- **A missing target config produces a file with only `intervention:`.** `data = yaml.safe_load(...) if target_config.exists() else {}`, then writing it gives a config with no `experiment_name` or `dataset`. That fails later at load, which is acceptable, but it's an odd thing to allow.
- **It resolves against `--version`, which may differ from the config's `dataset.version`.** Nothing checks that the version the episodes were resolved against matches the version the config will train from. If they differ, the episode IDs may not exist in the training input.

12.8 The script wrappers

```
#!/usr/bin/env python
"""Thin wrapper: `python scripts/train.py --config configs/experiments/baseline.yaml`.
Identical to `mtrain train`; provided for environments where the console script is not on the path."""
import sys
from model_training.cli import app

if __name__ == "__main__":
    app(["train", *sys.argv[1:]])
```

- **Seven 8–9-line files, one per command**, each prepending the sub-command name. They're all identical in shape. *Why:* environments where `mtrain` isn't on `PATH`. *Cost:* they still `import model_training`, so the package must be installed. They help only when the virtualenv's `bin/` isn't on `PATH`, and `python -m model_training.cli train ...` already covers that, because `cli.py` has `if __name__ == "__main__": main()`. So they're redundant convenience. That's harmless, but it's seven more files whose docstrings can drift, and `compare.py`'s already has.
- **There are no wrappers for `inspect`, `profile`, `validate` or `build-intervention`**, so the set is partial.
- **`scripts/test.py`'s name.** Running `python scripts/test.py` puts `scripts/` first on `sys.path`, which shadows the standard library's `test` package for that process. It's harmless here (nothing imports `test`), but it's the sort of name that bites later.

12.9 Honest gaps to own before an interviewer finds them

1. **`mtrain evaluate`/`test` overwrites `test_overall.json` and `stratified.json` in place**, for a stochastic model, with no history. *Consequence:* published numbers can't be traced to their file after a re-score, and a write-once run directory becomes mutable.
2. **`--split val` writes validation metrics into `test_overall.json`, and the docs recommend it.** *Consequence:* `compare`, `plot_results.py` and `load_run_summary` will read val as test.
3. **`evaluate` diverges from the runner**: no baselines (despite its docstring), hard-coded `per_task` slices, default batch settings, no update to predictions, summary, metrics, report, card, artifact or feedback. *Consequence:* a re-scored run directory is internally inconsistent.
4. **`build-intervention --apply` erases the template's explanatory comments** and reflows text. *Consequence:* the written-down safeguards disappear at the moment of use.
5. **Imbalanced is written as `kind: custom`.** *Consequence:* the intervention label is lost from `dataset.json`, the summary, the index and the comparison.
6. **`build-intervention --kind imbalanced` without `--variable-value` silently selects every episode.** *Consequence:* a "resolved" imbalance that's a full run.
7. **`_target_episodes` is duplicated between the CLI and `interventions.py`.** *Consequence:* drift risk in the core selection rule.
8. **The resolution version isn't checked against the config's training input version.** *Consequence:* episode IDs may not exist where training happens.
9. **`export` omits `git.json`, `environment.json`, reports and predictions; `tables/` is always empty; `manifest.json` never exists.** *Consequence:* the "paper-ready" bundle lacks the provenance it claims.
10. **`validate` and `analyze-failures` misdiagnose crashed runs** as "no training" or "no evaluation". *Consequence:* misleading triage.
11. **`scripts/compare.py`'s docstring shows `--runs a b c`, which Typer rejects.** *Consequence:* a wrong usage example.

12. `--seed` **help doesn't say it only names the directory; there's no** `--base`**,** `--force`**, seed loop,** `report` **or** `sweep` **command.** *Consequence:* the CLI can't express multi-seed studies, report regeneration or sweeps.
13. `inspect --version` **defaults to** `training/v1`**.** *Consequence:* an arbitrary data default that may not exist.
14. **Script wrappers are redundant with** `python -m model_training.cli`**, incomplete, and one shadows stdlib** `test`**.** *Consequence:* maintenance surface for no capability.
15. **No CLI tests** (grep finds none). *Consequence:* every CLI-only behaviour above is unexercised.

The shape to carry into Module 13

```
mtrain (Typer; only typer/yaml/rich at import — rde loaded per command)
├─ STATE-CHANGING
│   train --config [--seed≠seed]        → load_experiment_config → ExperimentRunner.run   (one impl ✓)
│   evaluate --run [--split] / test     → OWN re-implementation: no baselines, slices=[per_task],
│                                          default batch → OVERWRITES evaluation/test_overall.json
│                                          (val → "test_overall.json") · leaves rest of run stale
│   build-intervention ... [--apply]    → catalog episodes → block → yaml.safe_dump (comments lost)
│                                          imbalanced → kind: custom · dup _target_episodes
│   rollout --run                       → append "unavailable"  (honest placeholder)
│   export --run --output               → copy figs/eval/config/manifests (no git/env/reports; tables/ empty)
└─ READ-ONLY
    inspect · profile · validate · analyze-failures · compare (rich table positional; CSV by key)
scripts/*.py = app(["<cmd>", *argv])  (7 of 11 commands; == python -m model_training.cli)
```

Drill — answer from memory, files closed

1. Which `mtrain` commands change state, and what does each one change?
2. *Why didn't you just* have `mtrain evaluate` call the runner's evaluation code?
3. *What breaks in production:* a colleague runs `mtrain test --run <published run>` to "check it still works" the day before a paper deadline. What has happened?
4. What does `build-intervention --apply` do to `configs/experiments/imbalanced.yaml`, exactly?
5. *Interviewer framing:* "Walk me through how a researcher uses your tool from a fresh dataset to a comparison table."

Appendix A — Model answers to the drill

A1. State-changing commands. "Four. `train` runs the whole pipeline, so it creates a run directory, engine dataset versions, index rows and engine failure records. `evaluate`, and its alias `test`, reload the selected checkpoint and overwrite `test_overall.json` and `stratified.json` in that run's evaluation directory. `build-intervention --apply` rewrites the target config file's intervention block. And `rollout` appends an 'unavailable' record to the run's rollout log. Everything else (inspect, profile, validate, analyze-failures, compare, and export apart from writing its own output directory) only reads. The one I'd change is `evaluate`, because it's the only command that mutates an existing run's results."

Why that answer lands: it's a complete inventory organized by command–query separation, and it ends on the problem command.

A2. Why not reuse the runner's evaluation? "I should have. The evaluation stage is inline in `ExperimentRunner.run`, so there was no function to call, and `evaluate` re-implemented a subset of it. The result drifted. It doesn't re-run baselines even though its docstring says it does, it hard-codes the per-task slice instead of reading the run's configured slices, it uses default batch settings, and it only rewrites two of the files the runner writes, so the predictions, summary, report and feedback all describe the old numbers. The fix is to factor stage 4 out into one function that takes the run's own resolved config, call it from both places, and have re-scores write to a new versioned directory instead of overwriting."

Why that answer lands: it gives the root cause (no factored function), the concrete drift, and the fix, including the overwrite policy. That shows you know why duplication happened, not just that it did.

A3. `mtrain test` the day before a deadline. "They've changed the published numbers. `test` is an alias for `evaluate --split test`, which reloads the selected checkpoint and overwrites `test_overall.json` and `stratified.json`. SmolVLA samples actions stochastically, so the new MSE differs by up to a few percent from what's in the paper's tables, and there's no backup of the old file. The stratified file has also lost its length slices, because the command only computes per-task. Meanwhile `summary.json`, the report, the model card and the predictions file still have the original numbers, so the run directory now contradicts itself. Recovery is from git if the paper-ready export happened earlier, but the export doesn't include the predictions or the report. The fix is that re-scoring never overwrites, writing a timestamped sibling directory instead."

Why that answer lands: it gives the precise mechanism, the stochastic amplification, the internal inconsistency and the incomplete recovery path. It shows real operational thinking.

A4. `--apply` on `imbalanced.yaml`. "It loads the YAML, replaces the whole `intervention` block with the resolved one, a `custom` block listing the kept episode IDs plus the variable, value and reason, and dumps the file back with PyYAML. Two side effects. First, PyYAML doesn't preserve comments, so the twenty-line header explaining that it's a template, why the variable must come from a profile and what the result can't claim is deleted, and the folded hypothesis strings are reflowed. Second, the kind becomes `custom`, so the run records the intervention as custom, not imbalanced, everywhere except the tags. I'd use a round-trip YAML library, or write the resolved block to a separate file the loader merges, and record `kind: imbalanced` with explicit episode IDs, which the schema already allows."

Why that answer lands: it's exact about what happens and why (PyYAML semantics), identifies both the human-facing and the data-facing consequences, and offers fixes that fit the existing schema.

A5. "From a fresh dataset to a comparison table." "The dataset has to be ingested and cleaned in the data engine first, giving something like `clean/v1`. Then `mtrain profile` on that version to see what actually varies. For SO-100 that showed a single task, which is why episode length became the imbalance variable.

`mtrain train` with the baseline config builds a training version with lineage, checks the splits, fine-tunes, evaluates against both baselines, and writes the report, model card and feedback artifact. For a targeted experiment, `mtrain build-intervention` in dry-run mode resolves the episodes from the real profile; you inspect them and then `--apply`. Train that config, then `mtrain compare` with `--runs` repeated for each run. Three cautions: excluding interventions are scored on different test splits, `--seed` doesn't change the training seed yet, and `mtrain test` overwrites results, so I'd tell a new user to treat run directories as read-only."

Why that answer lands: it's a correct end-to-end workflow, grounded in the real data story, and it volunteers the three operational traps a new user would hit.

Module 12 Summary — the one page to revise

What this module is. An eleven-command Typer CLI with lazy engine imports. `train` delegates to the runner, `build-intervention` resolves template configs from real metadata with a dry run by default, `rollout` records "unavailable", and the rest are read-only views. `evaluate` / `test` is a divergent re-implementation that overwrites published results.

Five claims to make cold

1. Only `typer`, `yaml` and `rich` load at import, so `--help` and the read-only commands never load the engine or torch.
2. `build-intervention` is dry-run by default and resolves without building. `train` creates the version and its lineage.
3. `evaluate` / `test` overwrite `test_overall.json` for a stochastic model, with different slices and no baselines, and leave the run inconsistent.
4. `--split val` writes validation into `test_overall.json`, and the API doc recommends it.
5. `--apply` deletes the template's explanatory comments and relabels imbalanced as `custom`.

Decisions

Chose	Over	Because	Cost
Lazy imports per command	top-level imports	fast startup; no torch for views	none
`train` delegates to the runner	CLI-side logic	single implementation	`--seed` inherits the runner bug
`evaluate` re-implemented inline	shared stage function	runner stage wasn't factored	drift; overwrite; inconsistency
Dry run by default	apply by default	config rewrites need review	none
PyYAML round trip	ruamel round-trip	already a dependency	comments and formatting lost
`rollout` placeholder command	omit until real	absence is recorded	duplicate untimestamped records
Script wrappers	rely on `python -m`	`PATH` convenience	redundant, drifting docs

Volunteer unprompted: the overwrite plus stochasticity; val into `test_overall.json`; comment erasure; the `custom` relabel; `export` missing git and env; the `compare.py` usage error; no CLI tests.

Numbers and names: 11 commands (`inspect, profile, train, validate, evaluate, test, rollout, compare, analyze-failures, export, build-intervention`); 7 script wrappers; `--dry-run/--apply` default dry; `--oversample-factor 3.0`; `inspect --version` default `training/v1`; `evaluate` slices `["per_task"]`, batch 32; export subdirectories `figures, tables (empty), metrics, configs, manifests`.

Sentences worth memorizing

- "Every state-changing command should be obvious from its name; `mtrain test` overwrites published numbers, and that's the first thing I'd fix."
- "Two implementations of evaluation drifted within one codebase. Factor the stage once, call it twice."
- "A re-score of a stochastic model should be a new artifact, never an overwrite."
- "My config rewriter deleted the comments that told the user what not to claim."

Module 13 — The test suite, and the constraint → decision map

Files covered: `tests/conftest.py`, `tests/unit/test_config.py`, `tests/unit/test_deployment.py`, `tests/unit/test_evaluation_offline.py`, `tests/unit/test_experiment_runner.py`, `tests/unit/test_failure_modes.py`, `tests/unit/test_interventions.py`, `tests/unit/test_profile.py`, `tests/unit/test_split_integrity.py`, `tests/integration/test_full_chain.py`, `tests/smoke/test_smolvla_smoke.py`

Where we are: Modules 1–12 read the code adversarially. This module reads the tests the same way: what they prove, what they assume, and which of the gaps found so far a test *should* have caught. It then closes the code modules with the **capstone**, a table tracing every non-obvious design choice back to the constraint that forced it, plus the choices no constraint explains.

The one-line thesis: The suite's distinctive strength is that it tests against a **real engine on a real synthetic dataset**, not mocks, so the integration seams are exercised for real. But the fixture is too uniform to exercise the logic that matters (length buckets), the most important guarantees (val-only selection, seed propagation, no-overwrite of results) have **no test at all**, and several tests pin the presence-only behaviour that later modules showed to be wrong.

13.1 Prerequisites — the absolute basics

- **pytest fixture.** A function decorated `@pytest.fixture` whose return value is injected into any test that names it as a parameter. Fixtures can depend on other fixtures, forming a graph. Default scope is per test: rebuilt for each test function.
- **`tmp_path`.** A built-in fixture giving each test a fresh temporary directory.
- **`pytest.importorskip("torch")`.** Import the module, or skip the whole test file with a reason if it's missing.
- **Mocks vs fakes vs real collaborators.** A *mock* records calls and returns canned answers. A *fake* is a small working substitute (`_FakeView` in Module 5). A *real collaborator* is the production object. Tests with real collaborators catch integration bugs and are slower and coupled to the collaborator's behaviour.
- **Test pyramid.** Many fast unit tests, fewer integration tests, very few end-to-end tests. In practice, a project's "unit" folder may contain integration tests in disguise.
- **Characterization test.** A test that pins *current* behaviour, whether or not that behaviour is correct. It's useful for refactoring, and dangerous when it pins a bug as a contract.
- **Coverage vs assurance.** A line being executed by a test (coverage) is different from its *outcome* being checked (assurance). A test that only asserts a file exists covers a lot and assures little.
- **Global state in tests.** `set_settings(s)` mutates a process-wide singleton. Tests sharing it can't safely run in parallel (`pytest-xdist`).

13.2 Inventory, verified

File	Tests	Needs torch?	What it exercises
`unit/test_config.py`	8	no	merge, templates, sum-to-one, identity, resolved dump
`unit/test_deployment.py`	6	no	`health_check` (5), registry round-trip (1)
`unit/test_evaluation_offline.py`	6	no	baselines, error distribution, episode metrics, temporal, stratified, unknown slice
`unit/test_experiment_runner.py`	3	no	baseline-only full run, no-overwrite, reduced recorded
`unit/test_failure_modes.py`	10	1 self-skips if torch *is* present	unknown dataset/version/variation, zero fraction, corrupt checkpoint, bad contract
`unit/test_interventions.py`	9	no	full/reduced/imbalanced/rebalanced against a real catalog, lineage, determinism
`unit/test_profile.py`	3	no	clean vs training profile, JSON-shaped
`unit/test_split_integrity.py`	4	no	real view OK, fake leaky view raises or reports
`integration/test_full_chain.py`	3	no	view → adapter → model → Prediction → Evaluator → feedback
`smoke/test_smolvla_smoke.py`	4	**yes** (`importorskip` at module top)	three contract-to-feature builders, one 4-step SmolVLA end-to-end run
Total	**56**		52 run in the light venv (README line 282 says "52 unit + integration": **verified**)

I couldn't run any of them (Module 1: the engine isn't in this checkout). Everything below is from reading.

13.3 `conftest.py`: the real-engine fixture graph

```
"""Shared fixtures: a real synthetic dataset taken through raw -> clean ->
training, exactly the way `rde run` would, so every layer of model_training is
tested against real rde objects rather than mocks."""

@pytest.fixture
def settings(tmp_path) -> Settings:
    s = Settings(paths=Paths(tmp_path / "rde_data"), row_group_size=64, batch_size=32)
    s.paths.ensure()
    set_settings(s)
    return s

@pytest.fixture
def source_dataset(tmp_path) -> Path:
    # Two tasks (not the single-task default) so episode_id-by-task_id
    # selection -- what the imbalanced/rebalanced interventions use -- has
    # something real to select between.
    return make_lerobot_dataset(tmp_path / "source", num_episodes=8, frames_per_episode=20, fps=10,
        tasks=("Pick up the cube and place it in the box.", "Push the block to the target."))

@pytest.fixture
def raw(settings, source_dataset, catalog):
    return ingest(source_dataset, "so100_test", "lerobot", "raw/v1", settings=settings, catalog=catalog)

@pytest.fixture
def clean_version(settings, catalog, raw) -> str:
    TransformPipeline([NormalizeVector("state.joint"), NormalizeVector("action.joint")], ...).run(
        "so100_test", "raw/v1", "clean/v1")
    # `rde run` always validates before a version is considered usable
    # downstream; without this, every episode sits at quality_status=UNKNOWN
    # and DatasetBuilder.build() (which admits only PASSED/WARNING by
    # default) would refuse to build a training version from it.
    QCEngine(settings=settings, catalog=catalog).run("so100_test", "clean/v1")
    return "clean/v1"

@pytest.fixture
def training_version(settings, catalog, clean_version) -> str:
    DatasetBuilder(...).build("so100_test", clean_version, "training/v1",
                              splits={"train": 0.6, "val": 0.2, "test": 0.2}, seed=7)
    return "training/v1"
```

Decision: real engine objects, not mocks.

- *Why:* this repository's risk is almost entirely at the boundary with the engine (Module 1's import table). A mocked `DatasetBuilder` would pass while the real one changed. `test_reduced_intervention_selects_half_and_records_lineage` can check *real* lineage records only because the catalog is real. That's the strongest test in the suite.
- *Alternative:* mock `rde`, so tests run without the engine and fast.
- *Cost:* (1) **the suite can't run without the sibling engine checkout**, so nothing runs in this checkout; (2) every test pays for ingest, normalize, QC and build per test (function scope, rebuilt each time); (3) tests are coupled to engine behaviour, and an engine change can break a test here without any change here, with no version pin to warn you (Module 1).
- **The comments are excellent.** They explain *why* each fixture step exists: two tasks so task-based selection has something to select; QC so the builder admits episodes. Those are the non-obvious facts a new contributor needs.
- **`set_settings(s)` mutates global state.** It's correct for sequential runs. With `pytest -n auto` (xdist), workers are separate processes, so it's actually safe there too, but any thread-based parallelism or test that forgets the `settings` fixture would pick up the previous test's settings.

The fixture's shape limits what can be tested:

- **8 episodes × 20 frames, all the same length.** Every length-bucket computation over the fixture collapses (all "short", by the `<=` tie rule from Module 4). The test file itself says so: "The synthetic fixture's episodes are all the same length, so every episode buckets to 'short'". Only the pure `test_compute_length_bucket_splits_into_terciles` exercises real terciles.
- **A 0.6/0.2/0.2 split of 8 episodes** gives a test split of about 2 episodes (exact sizes depend on the engine's quantile rounding). `stratified_evaluate`'s length slices require `>= 3` test episodes (Module 7), so **`test_stratified_evaluate_per_task_and_length_bucket` almost certainly never executes the `per_length_bucket` branch**. Its assertions (an `overall` key exists, some `task_` key exists, slices ⊆ overall) all pass with the length branch silently skipped. The test's name claims coverage it doesn't get.
- **Two tasks** is good for task selection, but the real primary dataset has *one* task, so the code path SO-100 actually runs (single task, so `task_0` duplicates overall, Module 7) isn't the one the fixture tests.

13.4 What the tests prove well

1. **Interventions create real lineage** (`test_reduced_intervention_selects_half_and_records_lineage`). The transform run's kept and dropped IDs equal the computed ones. This is *the* claim of the project, and it's verified end to end.
2. **Weights reach the built version** (`test_rebalanced_intervention_keeps_all_episodes_and_sets_weights`): `manifest.statistics["weighted_episodes"]` is truthy, so the request survives into the engine.
3. **Templates fail as documented** (`test_rebalanced_template_fails_loudly_until_filled_in`, `test_imbalanced_template_loads_but_is_empty`).
4. **The no-overwrite rule** (`test_runner_refuses_to_overwrite_an_existing_run`).
5. **Failure modes raise specific exceptions** (`test_failure_modes.py`): unknown dataset or version raises `LookupError`, an unsupported variation raises `ValueError` with "no automatic rule", a corrupt checkpoint raises `ValueError` with "incomplete". The module docstring states the standard well: "every one of these must raise a specific, actionable error -- never degrade silently".
6. **The runner completes without torch** (`test_runner_completes_baseline_only_experiment`). It uses `model: mean_action` to exercise every non-training stage in the light venv, and asserts `"unavailable" in report_text`, so the rollout honesty rule is tested.
7. **Cross-artifact consistency, once:** `test_split_integrity_then_profile_then_stratified_eval` asserts `profile.splits == report.split_sizes`. That's a good pattern, and it's used only here.
8. **Cross-embodiment feature shapes** (smoke): the ALOHA-shaped contract gives 28-dimensional state (14 + 14) and 14-dimensional action, with normalization stats concatenated in feature order. This guards a real bug class (Module 2's note that ALOHA "generalized *after* a fix").
9. **The SmolVLA chain runs end to end** (smoke): 4 real optimizer steps, checkpoint reload, all artifacts present, loss not NaN (`final_train_loss == final_train_loss`, the classic NaN self-inequality idiom). The non-divisor `batch_size: 3` was chosen deliberately to reproduce the action-queue bug ("predict() must never reuse a cached action chunk across unrelated batches"). That's a regression test written from a real incident, which is the best kind.

13.5 What the tests miss, mapped to the gaps they would have caught

Gap (module)	Test that would have caught it	Exists?
`--seed` doesn't reach `training.seed`; replicates share `experiment_id` (M11-1)	run twice with different seeds, assert `training.seed` and the `experiment_id`s differ	**no**
Selection never uses test (the core guarantee, M6)	spy on `Evaluator.evaluate` during `Trainer.train`, assert every `split == "val"`	**no**. It's enforced only by reading the code
Unknown YAML keys silently ignored (M2-1)	load a config with `learning_rat:`, expect an error	**no**
`model: SmolVLA` silently skips training (M2-2)	expect a validation error	**no**
Imbalanced with no value becomes a full run (M2-4)	assert raise or non-empty `excluded_episode_ids`	**no**
`frac=0.0` fails in the builder (M4-2)	call `build_intervention_dataset` with 0.0	**no**. The existing test checks the helper and *asserts the opposite*
Filter-then-split moves the test set (M4-1)	assert reduced-run test IDs ⊆ baseline test IDs	**no**
Overlap check can't fail on real views (M5-1)	— (the fake view test is all that's possible)	n/a
Unassigned or empty splits pass (M5-2)	build with `val: 0.0`, expect integrity failure	**no**
Coverage normalization (M5-4)	two-bin bimodal counts → expect < 1.0	**no** (only asserts `0 ≤ x ≤ 1`)
`_prune_checkpoints` (M6-10)	pure `tmp_path` test	**no**
`crash.json` on exception; signals (M6-2)	make `fit` raise, assert `crash.json`	**no**
NaN locks best (M6-4)	feed a NaN val metric	**no**
Slices reconcile with overall (M7-1)	frame-weighted slice MSE ≈ overall	**no** (stochastic models only; baselines are deterministic, so it would pass for the wrong reason)
Feedback branches reachable (M8-2/3)	synthetic outcomes concentrated in one episode	**no**. The integration test checks shape
`health_check` cross-check (M9-1)	mismatched action dimension, expect failure	**no**. The existing test *passes* a mismatched, empty checkpoint
`DeployedModel.load` / `predict` (M9-4)	any call	**no**
Report content (git `n/a`, MAE `n/a`; M10)	assert the commit appears in the report	**no** (only substring checks for name and "unavailable")
`mtrain compare` alignment, `evaluate` overwrite, `--apply` comments (M12)	CLI tests via `typer.testing.CliRunner`	**no CLI tests at all**
`run_data_fraction_sweep` (M11-10–13)	any call	**no**
Light venv stays light (M1-7)	import the runner, assert `"torch" not in sys.modules`	**no**
Registry "most recent wins" (M9-11)	register two rows, `get` returns the second	**no**

Pattern: the suite tests that **each component works on the happy path and fails loudly on bad identity** (unknown versions, missing files). It doesn't test **cross-stage invariants** (seed flows through, test never selects, slices partition overall, artifacts agree with each other), and that's where every serious bug in this manual lives.

Tests that pin wrong behaviour (characterization hazards)

- `test_reduced_intervention_at_zero_fraction_is_honored` asserts the helper returns `[]` for 0.0, encoding a contract the system can't honour. Its docstring argues 0.0 is a "legitimate, explicit request".
- `test_health_check_passes_on_well_formed_checkpoint_and_contract` passes an *empty* `model.safetensors` and metadata naming `dataset_id: "d"` against an SO-100 contract. Fixing Module 9's cross-check would turn this test red, correctly.
- `test_invalid_deployment_contract_rejected` (failure modes) uses `rde_checkpoint_meta.json = "{}"`, so the same presence-only stance.
- `test_experiment_id_is_deterministic_given_same_inputs` is true only with a pinned `created_at`, which no YAML path provides. Its name overstates the property.

Smoke-test realities

- **It needs network on first run** (it downloads `lerobot/smolvla_base` from the Hugging Face Hub) and a torch plus lerobot install. It's skipped otherwise, "never replaced with a stub" (docstring). That's honest. It also means **no CI could run it without Hub access and a GPU or a patient CPU**.
- **The three feature-builder tests are skipped in the light venv** because `importorskip("torch")` is at module top, even though they test pure contract → shape logic. If those builders return lerobot `PolicyFeature` objects (likely, given `.shape`), lerobot is genuinely required. It's unverifiable here.
- **It imports two private engine functions** (`_policy_features_from_contract`, `_dataset_stats_from_contract`), which is Module 1's private-API coupling.
- **The end-to-end smoke asserts existence, not values.** `"mse" in test_overall["metrics"]` and files exist. With 4 steps that's appropriate (no learning is expected), but a sanity bound (for example, SmolVLA MSE finite and positive) would catch a NaN in evaluation, not just in training.

13.6 The capstone: constraint → decision map

Each row traces a non-obvious choice to the **named constraint that forced it**, with the evidence for the constraint. The second table lists the choices **no constraint explains**. Those are the ones to call arbitrary, rather than invent a justification for.

Constraints present in this project (with evidence)

ID	Constraint	Evidence
C1	**One laptop GPU, 8 GB VRAM; about 10 GB host RAM**	README §2 (RTX 4060 Laptop, 8 GB); `IMPLEMENTATION_PLAN.md` (about 10 GB VM RAM, engine's 6 GB cap)
C2	**No robot or simulator**	`rollout` text, README §2 point 4, `METRICS.md` §8
C3	**Tiny datasets** (48 / 40 episodes; 5 / 4 test episodes)	README Table 1
C4	**Data schema without a variation field; SO-100 single-task**	`profile.py` `variation_note`; `EXPERIMENTS.md` §3
C5	**A separately owned data engine with immutable, lineage-tracked versions** (the untrusted-by-default component, from this repo's point of view)	`pyproject.toml` sibling path; `ARCHITECTURE.md`
C6	**A stochastic policy** (flow matching) **that predicts action chunks**	README §2.3; smoke-test action-queue comment; API_REFERENCE `predict_action_chunk`
C7	**Heavy, platform-specific deps** (torch with CUDA, lerobot 0.4.4)	`IMPLEMENTATION_PLAN.md` (CPU torch reinstall); two-venv tables
C8	**Pretrained base for a different embodiment**	`trainer.py` load comment; smoke cross-embodiment tests
C9	**Team of one or two researchers, no infrastructure, short runs (5–10 min)**	README §2 timings (324 / 385 / 576 s); `DEPLOYMENT.md` "avoid unnecessary infrastructure"
C10	**Project-specification rules**: never fabricate, failure ≠ bad data, don't presume the weak dimension, no silent model substitution	`EXPERIMENTS.md` §2–3, `FAILURE_ANALYSIS.md` §5, `DEPLOYMENT.md` §4, `IMPLEMENTATION_PLAN.md`
C11	**An unreliable runtime** (OOM, kills, partial writes on a laptop)	`trainer.py` crash and verify comments
C12	**Offline evaluation must not be mistaken for task success** (a corollary of C2 plus C10)	`offline.py` docstring; model-card limitations

Decision → constraint

Decision (module)	Forced by	How the constraint forces it	Would flip when
Batch size 4, 1,000 steps, no grad accumulation (M2)	**C1** (motivated, but see note)	8 GB VRAM for a ~450M-param VLA plus 2–4 cameras. *Correction from Module 15:* measured peak GPU is only 2.2–3.0 GB of 8 GB, so batch 4 is conservative rather than forced, and 1,000 steps is about 0.26 (SO-100) or 0.11 (ALOHA) of an epoch	larger GPU, or multi-GPU; or simply trying batch 8–16, which the measurements suggest fits
Validation capped at `val_batches × batch_size` = 32 frames (M6)	**C1, C9**	a full val pass (~90 s) × 10 would triple a ~5-minute run	longer runs make validation relatively cheap; use a frozen stratified subset regardless
`del policy; empty_cache()` before the evaluation load (M6)	**C1**	two SmolVLA copies don't fit in 8 GB	GPU with headroom
No optimizer state in checkpoints (M6)	**C1** (disk), inferred	Adam moments about triple checkpoint size	preemptible long runs need exact resume
Keep last 3 checkpoints plus best (M6)	**C1** (disk)	each save is about one model in size	cheap storage; the number 3 itself is arbitrary
Peak RSS and GPU recorded per run (M6)	**C1**	knowing headroom on a constrained box	—
Offline-only evaluation; `rollout` writes "unavailable" (M7, M12)	**C2, C10, C12**	nothing to roll out on; fabrication forbidden	a simulator or robot is connected
`success_rate` labelled a proxy everywhere (M7)	**C12**	an offline tolerance band isn't task success	rollouts exist
`DeployedModel` is a function call over `Sample`s (M9)	**C2, C9**	no consumer process or robot; avoid infrastructure	a controller needs to call it
Episode-level, `quantile` (exact-size) splits (M2, M5)	**C3**	independent draws at n = 48 vary ±2 episodes per split	thousands of episodes
Separate split seed 42 vs training seed (M2)	**C3**	the test set must stay fixed while training varies	—
Episode-length bucket as the imbalance variable (M4)	**C4**	the only per-episode attribute that varies on SO-100	the schema gains a variation or condition field
"Visual diversity: unavailable" (M5)	**C4, C10**	no embedding model; fabrication forbidden	an image embedding model is added
Interventions as engine transforms; data named by version (M2, M4)	**C5**	lineage and immutability live in the engine	— (the project's core principle)
Re-verify splits from the materialized view (M5)	**C5**	don't trust the producing code path	— (but it checks an unrepresentable failure)

Decision (module)	Forced by	How the constraint forces it	Would flip when
Read training weights back from the built version (M11)	**C5**	train on what the engine stored	—
Git info of *both* repos (M3)	**C5**	the engine owns `fit()`	single repository
Model adapter and `fit` living in the engine (M1)	**C5, C8**	features and normalization come from the engine's contract	a second model family arrives
`predict_action_chunk`, never `select_action`, in evaluation (engine; M9)	**C6**	the stateful action queue replays stale chunks across batches	—
Noise caveat on small differences (README §2.3)	**C6**	re-scoring varies up to 5%	deterministic sampling or averaging
Two venvs; lazy imports; baseline-model tests; `importorskip` (M1, M12, M13)	**C7**	GBs of CUDA torch for 52 tests that don't need it	CI on a GPU image anyway
Contract passed on fresh load, `None` on resume (M6)	**C8**	base checkpoint shapes and stats belong to another embodiment	same-embodiment pretraining
JSONL index plus run directories, not MLflow (M3)	**C9**	one or two users, serverless, durable files	more users, thousands of runs
Transaction-Script runner, no DAG (M11)	**C9**	short runs make caching and resume unnecessary	multi-hour or preemptible runs, big sweeps
f-string Markdown reports, fixed sections (M10)	**C9, C10**	durable docs next to artifacts; never hide a section	—
Validators: imbalanced or rebalanced require a declared target (M2)	**C10**	"the weak dimension must not be presumed"	—
`build-intervention` dry run by default; resolves from the real catalog (M12)	**C10**	the choice must follow from a real profile	—
Feedback explanations hedge "data defect vs limitation" (M8)	**C10**	"model failure ≠ bad data"	—
No fallback model in `DeployedModel` (M9)	**C10**	no silent substitution	—
No-overwrite run directories (M3)	**C11**	never destroy a run's checkpoints and metrics	— (but add status plus `--force` for failed runs)

Decision (module)	Forced by	How the constraint forces it	Would flip when
Verify each checkpoint before it's eligible as best; `crash.json` (M6)	**C11**	partial saves and crashes on a laptop	—
`yaml.safe_load` everywhere (M2)	**C11/C5** (untrusted input hygiene)	YAML tags can execute code	—

Choices no constraint explains (call these arbitrary)

Choice	Where	Honest description
Failure threshold p90, `min_failures_per_episode = 3`, concentration `> 0.5`	M8	Arbitrary, and mutually inconsistent at these sizes (the concentration branch is unreachable)
Length terciles at 33.3 / 66.7, `<= t1` tie rule	M4, M7	A conventional tercile; the tie rule is arbitrary and collapses buckets on uniform lengths
10 histogram bins	M5	A conventional default; the occupied-bin normalization is a mistake, not a choice
`oversample_factor = 3.0`	M2, M12	Arbitrary; no sweep or rationale anywhere
`keep_last_n_checkpoints = 3`	M2, M6	Arbitrary within C1's disk motive
Split seed 42, training seed 0, `val_batches = 8`	M2	Arbitrary values (the *separation* of seeds is principled; the numbers aren't)
`checkpoint_every = validate_every = 100`	M2	Arbitrary, but they must be equal for selection to work, and nothing enforces that
`round(frac × n)` with banker's rounding	M4	Incidental
`date / name / seed` directory layout	M3	Preference
10-hex-character experiment-ID digest	M2	Arbitrary and sufficient
Learning-rate 3e-5 and warmup 20 for the continuation	M2	A reasonable heuristic ("lower LR, shorter warmup on resume"), not tuned

Constraints that *should* have forced a decision, but didn't

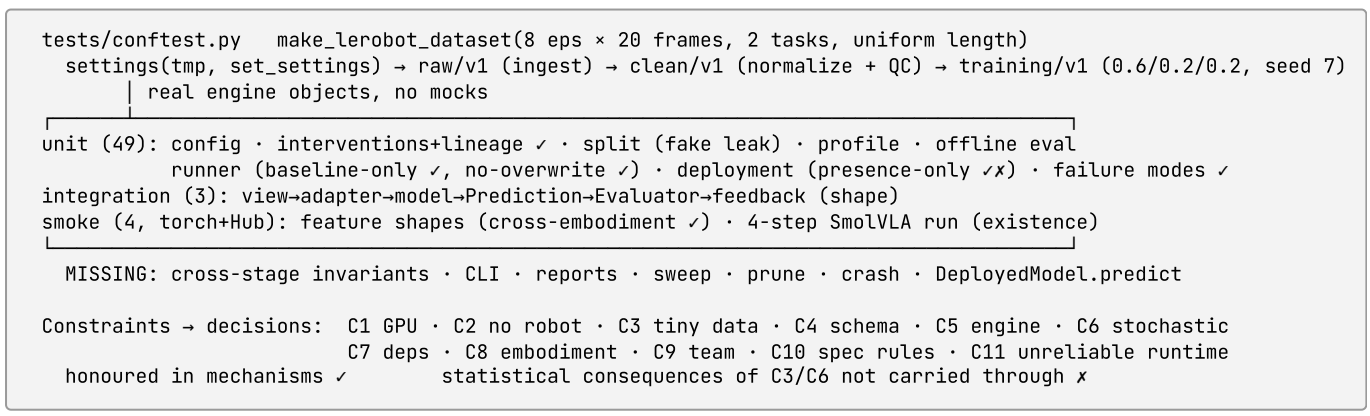
Constraint	Decision it implies	What the code does instead
C3 tiny test set	bootstrap intervals; paired comparisons on one frozen test split; multiple real seeds	single numbers; per-intervention test sets; `--seed` doesn't seed (M4, M7, M11)
C6 stochastic policy	predict once and slice; average samples during validation; store predictions	slices re-infer; 32-frame single-sample validation; docs claim determinism (M6, M7, M8)
C10 never presume / never fabricate	fail fast on config typos; forbid unknown keys	extra keys ignored; free-string model name silently skips training (M2, M11)
C11 unreliable runtime	atomic writes; status marker; signal handling	truncate-and-write; no status; SIGTERM uncaught (M3, M6)
C2 + C10 offline vs real	keep feedback decisions off the test split	the curation loop runs on test (M8)

That last table is the most important thing you can say about this project's design: **the constraints were correctly identified, and most were honoured in the *mechanism* layer (data versions, honesty markers, memory). The statistical consequences of C3 and C6, though, were not carried through into evaluation and reporting.**

13.7 Honest gaps to own before an interviewer finds them

1. **There's no test for any cross-stage invariant**: val-only selection, seed propagation, slices partitioning overall, artifacts agreeing with each other. *Consequence:* the bugs in Modules 6, 7, 8 and 11 were all invisible to the suite.
2. **The fixture's uniform episode lengths and a 2-episode test split mean length-bucket logic is never exercised on real data**, and `test_stratified_evaluate_per_task_and_length_bucket` almost certainly skips its length branch. *Consequence:* the test's name claims coverage it doesn't get.
3. **Some tests pin wrong behaviour** (zero fraction "honoured", presence-only health checks with empty weights, "deterministic" IDs). *Consequence:* fixing the bugs will break these tests, which is correct, but they currently read as specifications.
4. **There are no tests for the CLI, the reports, the sweep, `_prune_checkpoints`, `crash.json`, `DeployedModel.load` / `predict` or registry ordering.** *Consequence:* large surfaces are unexercised.
5. **The suite can't run without the sibling engine, and the smoke test needs Hub network plus torch.** *Consequence:* no CI is possible as configured, and "52 pass" is a local, dated claim.
6. **Function-scoped fixtures rebuild ingest, QC and build per test.** *Consequence:* a slow suite (unmeasured here); `scope="session"` for read-only fixtures would help.
7. **Global `set_settings` in a fixture.** *Consequence:* hidden coupling between tests that forget the fixture.
8. **The smoke test asserts existence, not value sanity, in evaluation.** *Consequence:* a NaN MSE in evaluation would pass.
9. **The private-engine-API import in the smoke test.** *Consequence:* brittle against legitimate engine refactors.

The shape to carry into the cross-cutting modules



Drill — answer from memory, files closed

1. Why does the suite use a real engine instead of mocks, and what does that cost?

2. Name three bugs from this manual that a test would have caught, and the test you'd write for each.
3. *Why didn't you just* write more unit tests with mocks so the suite runs without the engine?
4. *What breaks in production:* you set up CI on a standard GitHub runner tomorrow. What happens?
5. *Interviewer framing:* "Which of your design decisions were forced by constraints, and which were just choices?"

Appendix A — Model answers to the drill

A1. Real engine vs mocks. "This package is mostly glue around the data engine. It decides which episodes to keep, asks the engine to build versions, reads them back, and scores through the engine's evaluator. So the risk sits almost entirely at that boundary, and mocks would hide exactly the bugs I care about. With a real engine on a synthetic dataset, I can assert that the engine's own lineage record shows the same kept and dropped episodes my code computed, which is the project's core claim. The costs: the suite can't run without the sibling engine checkout, each test rebuilds ingest, QC and build, so it's slower, and an engine change can break my tests with no warning because the engine isn't version-pinned. I'd keep the real-engine tests, scope read-only fixtures to the session for speed, and pin an engine version range."

Why that answer lands: it justifies the choice by where the risk is, cites the specific test it enables, and owns all three costs with mitigations.

A2. Three bugs a test would have caught. "First, the seed bug: run the same config with `seed=0` and `seed=1` and assert the resolved training seed differs. Today it doesn't, so replicates are fake. Second, the zero-fraction sweep point: call `build_intervention_dataset` with fraction 0.0. The existing test only calls the helper and asserts it returns an empty list, which passes while the real path raises. Third, the health check: give it an ALOHA-shaped checkpoint and an SO-100 contract and expect `DeploymentHealthError`. Today the test suite actually passes an empty weights file with mismatched metadata. The pattern is that my tests check each component in isolation and miss invariants that span stages, so I'd add a small set of cross-stage invariant tests: seed flows through, selection only ever calls the evaluator on val, slices partition the overall predictions."

Why that answer lands: it gives three concrete tests tied to real bugs, explains why the existing tests missed them, and generalizes to a testing strategy.

A3. Why not mock everything? "Because the bugs that matter here live in how my code and the engine agree, and a mock encodes my *assumption* about the engine, so it agrees with my code by construction. For example, a mocked builder would never reveal that splits are assigned after filtering, which is the cause of the moving test set. Where mocks *would* help is the pure logic: pruning, threshold arithmetic, report rendering, CLI argument handling. None of that needs the engine, and none of it is tested today. So the answer is both: real-engine integration tests for the boundary, and fast, engine-free unit tests for the pure functions I currently don't test at all."

Why that answer lands: it explains the specific epistemic weakness of mocks with an example from your own system, then identifies where they *would* add value. A nuanced answer, not a dogmatic one.

A4. CI on a GitHub runner tomorrow. "Installation fails first. The engine dependency is a bare name resolved only through a uv-specific sibling-path source, so the runner would need to check out both repositories side by side and install with uv. If you fix that, the 52 light tests should run, though I've never measured how long, since fixtures rebuild a dataset per test. The smoke test would skip, because there's no torch. If you installed torch, it would try to download the SmolVLA base checkpoint from the Hugging Face Hub and run on CPU, so it would be slow and network-dependent. I'd run the light suite on every push and the smoke test nightly on a GPU runner with a cached Hub download, and add one job that imports the runner and asserts torch wasn't loaded."

Why that answer lands: it predicts the failure sequence in order, separates the fast and slow tiers, and proposes a realistic CI layout.

A5. "Which decisions were forced by constraints, and which were just choices?" "Forced: the small batch, the 32-frame validation cap and freeing GPU memory before evaluation all come from an 8-gigabyte laptop GPU. Offline-only evaluation and the 'unavailable' rollout come from having no robot or simulator. Exact-size episode splits and the separate split seed come from having 48 episodes. Using episode length as the imbalance variable comes from SO-100 being single-task with no variation field. Interventions as versioned transforms come from the data engine owning immutable versions. Just choices: the 90th-percentile failure threshold, the 0.5 concentration cut, 3× oversampling, keeping three checkpoints, and seed 42. Those are arbitrary, and in the threshold case they're inconsistent. And the interesting part is a third list, constraints that should have forced decisions but didn't. A tiny test set and a stochastic policy should have forced bootstrap intervals, real multi-seed runs, a frozen test split across interventions and predict-once slicing. I honoured the constraints in the mechanisms and didn't carry them through to the statistics."

Why that answer lands: it gives a clean three-way classification with evidence, labels arbitrary choices as arbitrary, and ends with a sharp self-critique that shows design maturity. It's the single answer that best summarizes the project.

Module 13 Summary — the one page to revise

What this module is. A 56-test suite (52 torch-free) built on a real engine and a real synthetic dataset, strong at boundary integration and loud failure modes, absent on cross-stage invariants. It's followed by the capstone map tracing each design choice to one of twelve constraints, and naming the arbitrary choices and the constraints whose statistical consequences weren't carried through.

Five claims to make cold

1. Real engine, no mocks: the lineage test proves interventions are real engine transforms.
2. 49 unit + 3 integration = 52 light tests (README verified), plus 4 smoke tests needing torch and Hub network.
3. The fixture's uniform lengths and 2-episode test split mean length-slice logic is effectively untested.
4. No test checks val-only selection, seed propagation, slice reconciliation, CLI, reports, sweep or pruning.
5. Constraints were honoured in mechanisms (memory, lineage, honesty) but not in statistics (C3 tiny data, C6 stochastic policy).

Decisions

Chose	Over	Because	Cost
Real engine fixtures	mocks	risk is at the engine boundary	no engine means no tests; slow; unpinned coupling
Synthetic 8 × 20, 2 tasks	real data sample	fast, deterministic, task selection possible	uniform lengths; tiny test split
`importorskip` smoke	stubbed SmolVLA	never fake the model	needs torch plus Hub; private-API import
Fake view for leak detection	none	only way to reach the failure branch	proves the check is near-tautological
Baseline-model runner test	skip runner in light venv	full pipeline without torch	training path untested in the light venv
Function-scoped fixtures	session scope	isolation	rebuild cost per test

Volunteer unprompted: no cross-stage invariant tests; the length-branch test that doesn't exercise its branch; tests pinning bugs; no CLI or report tests; CI impossible as configured; the capstone's "should have forced" table.

Numbers and names: 56 tests (8 / 6 / 6 / 3 / 10 / 9 / 3 / 4 unit, 3 integration, 4 smoke); fixture `so100_test`, 8 episodes × 20 frames at 10 fps, 2 tasks, splits 0.6/0.2/0.2 with seed 7; smoke: 4 steps, batch 2, ckpt and val every 2, keep 1, eval batch 3 (non-divisor, deliberate), `max_frames` 20; constraints C1–C12; RTX 4060 8 GB, about 10 GB RAM, lerobot 0.4.4, torch 2.10.

Sentences worth memorizing

- "I test against the real engine because a mock would agree with my assumptions by construction."
- "My tests check components; my bugs live between them."
- "Every non-obvious decision traces to a constraint, except the thresholds, which are arbitrary, and I say so."
- "I honoured the constraints in the mechanisms and failed to carry them through to the statistics."

Module 14 — Low-Level Design

Files covered: all of `src/model_training/` (cross-cutting). No new files; every claim here points back to a code module.

Where we are: Modules 1–13 walked the code one stage at a time. This module looks at the *same* code as a design: how it's layered, which patterns it actually uses (with exact locations), where SOLID holds or strains, which boundaries are typed and which are bare dicts, how a run moves through its states, how errors travel, and what's safe to share or repeat. It's the material for the "draw me the class and sequence diagrams" part of an interview.

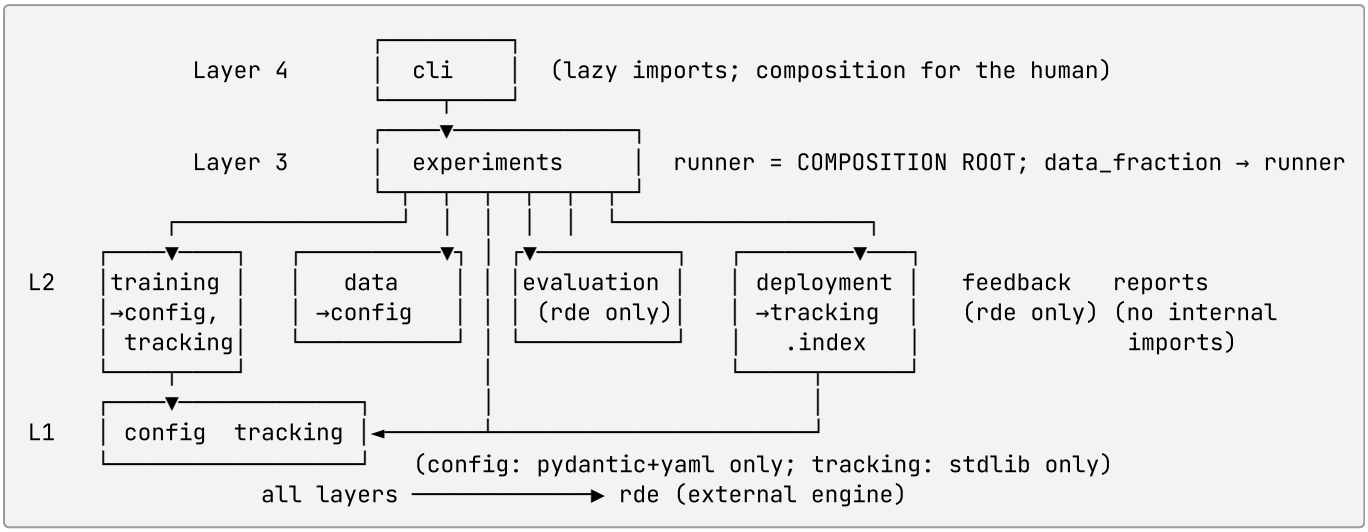
The one-line thesis: The package has a clean, acyclic, strictly downward dependency graph with a single composition root. But almost every boundary *between* stages is an **untyped** `dict[str, Any]`, the only real state machine is implicit in which files exist, and the error model is "raise built-in exceptions and let them escape". So the structure is sound, and the contracts are weak.

14.1 Prerequisites — the absolute basics

- **Layering / dependency rule.** Code is arranged in layers, and each layer may import only layers *below* it. Cycles are forbidden. That's what lets you change a high layer without touching low ones.
- **Composition root.** The one place where concrete collaborators are created and wired together. Everything else receives its collaborators.
- **Design pattern.** A named, reusable solution shape (Gang of Four, Fowler's *Patterns of Enterprise Application Architecture*). Naming one is useful only if you can point to where it is and say what it buys.
- **SOLID.**
  - **S**ingle responsibility: one reason to change.
  - **O**pen/closed: extend without editing.
  - **L**iskov substitution: subtypes are usable wherever their base type is.
  - **I**nterface segregation: no client depends on methods it doesn't use.
  - **D**ependency inversion: depend on abstractions, not concretions.
- **Typed contract.** A boundary whose data shape is declared and checked (a Pydantic model, a dataclass with type hints checked by mypy, a `TypedDict`). `dict[str, Any]` is an *untyped* contract: the shape lives only in the code that reads it.
- **Sequence diagram.** The order of calls between components for one scenario.
- **State machine.** The set of states an entity can be in and the legal transitions between them.
- **Idempotent operation.** Doing it twice has the same effect as doing it once.
- **Thread safety.** Correct behaviour when called from several threads at once. **Process safety** is the same across processes, which matters for files and catalogs.

14.2 Layering and the dependency rule

Every internal import, gathered with grep:



Verified properties:

- **There are no cycles.** No module imports `experiments` or `cli` except the CLI itself.
- **The leaves are leaves.** `config` imports only `pydantic` and `yaml`; `tracking` imports only the standard library (plus optional `torch`, `lerobot` and `psutil` for observation).
- **`reports` imports nothing internal.** It takes plain dicts. That's maximal decoupling, which is also why its inputs are untyped (14.5).
- **`evaluation` and `feedback` import only `rde`.** They're pure adapters over engine services.
- **`deployment.registry` depends on `tracking.index`**, the one lateral, cross-feature reuse, and it's a reasonable one (shared JSONL mechanism).

Where the rule bends:

- **The CLI reaches below the façade.** It imports `model_training.config.schema.DatasetIntervention` and `model_training.data.interventions.resolve_episode_subset` / `compute_length_bucket` directly, and re-implements both `_target_episodes` and evaluation (Module 12). The CLI depends on internals instead of a service API.
- **Everything depends directly on `rde`, a concrete external package**, with no abstraction layer of our own. `rde` is effectively *layer 0 for every layer*. That's a deliberate choice (Module 1: the model layer "never owns the data"), with the cost that there's no seam for substituting the engine. Tests use the real engine (Module 13) partly because there's nothing to mock *against*.
- **Import-time side effects cross layers.** `reports.figures` sets the matplotlib backend globally (Module 10), and `deployment.artifact` imports the engine's model module eagerly (Module 1). Layers are separated by imports, but not by *effects*.

Composition root: `ExperimentRunner.__init__` / `run` builds `Catalog`, `DataEngine`, `ExperimentIndex`, `Trainer`, `Evaluator` (via helpers), `SmolVLAPolicy` and `VisionLanguageAdapter`. The CLI builds `ExperimentRunner()` with defaults. That's a good shape: one place to change wiring.

14.3 Catalogue of patterns actually present

Pattern	Exact location	What it buys *here*	Alternative passed over
Façade (package level)	each `*/__init__.py` with `__all__`	stable import paths; file layout can change	deep imports everywhere
Façade (service level)	`ExperimentIndex` over 4 `JsonlIndex`es (`tracking/index.py`)	one object to record any run fact	four loose files
Transaction Script	`ExperimentRunner.run` (`experiments/runner.py`)	whole lifecycle readable in one method, matching README Fig. 1	stage objects / DAG (Module 11)
Composition Root + constructor DI with defaults	`ExperimentRunner.__init__`	tests inject a temporary catalog and directories	globals or a DI container
Template Method via callback (Hollywood principle / inversion of control)	`Trainer.train` passes `on_step` into engine `fit` (`training/trainer.py`)	run policy injected into an engine-owned loop	own training loop; Lightning callbacks
Closure-captured state	`on_step` with `nonlocal peak_rss, last_step` and mutable `best` dict	per-run state without a class	a `TrainingState` object
Adapter	`evaluation/offline.py`, `feedback/artifact.py` wrap `Evaluator` / `FeedbackLoop`	engine results shaped into project artifacts	reimplement scoring or detection
Value Object	`InterventionResult`, `SplitIntegrityReport`, `DatasetProfile`, `TrainingOutcome`, `ModelArtifact`, `LatencyBreakdown` (all dataclasses)	named, typed results between stages	tuples / dicts
Specification (validation rules on a model)	`@model_validator` on `DatasetIntervention`, `SplitConfig`, `ExperimentConfig`	methodology rules as load-time errors	runtime checks scattered across stages
Layered configuration	`_deep_merge(base, override)` (`config/loader.py`)	shared defaults plus per-experiment deltas	Hydra / OmegaConf
Append-only log / event-sourcing-lite	`JsonlIndex.append`, `ModelRegistry.get` takes the last match	history preserved; latest wins	mutable DB rows
Repository (read side only)	`ModelRegistry.get/list/for_experiment`	query models by identity	raw file scans by callers
Guard clause / fail loudly	`make_run_dir`, `verify_split_integrity`, schema validators, `health_check`	refuse bad states early	best-effort continue
Null Object / explicit "unavailable"	`{"status": "unavailable", "reason": ...}` in profile, rollout, environment, figures that return `None`	absence is data, never fabricated	omit the field / fake a value
Registry (lookup by name)	engine `MODEL_REGISTRY` used by `evaluate_baselines`	baselines named in config	`if name == ...` chains
Strategy (implicit)	`selection_mode: min/max`, `split_mode: quantile/independent`, `intervention.kind`	behaviour chosen by config	— but implemented as `if/elif`, not polymorphism
Builder-ish function	`build_model_artifact`, `build_feedback_artifact`, `build_dataset_profile`	assemble a large record from many sources	constructors with many arguments

Patterns named in docs or comments that aren't really there: a "registry" in the MLflow sense (no promotion, Module 9), a "flywheel" as a decision loop (Module 8), and a "matrix" (`build_model_dataset_matrix`, Module 10).

14.4 SOLID, and where each principle is strained

Principle	Where it holds	Where it's strained
S	`split_integrity.py` (one check), `JsonlIndex`, `_deep_merge`, `figures._save`	`ExperimentRunner.run` (orchestration plus JSON formatting plus artifact naming plus limitation text, about 250 lines); `cli.evaluate` (re-scoring plus persistence plus formatting); `DatasetProfile` builder (stats, coverage, QC, splits, notes)
O	new baselines need only an engine `MODEL_REGISTRY` entry; new figures are new functions	new intervention kinds need edits in **three** places (schema `Literal` plus validator, `resolve_episode_subset`, `build_intervention_dataset` branch) plus the CLI; new slice types need an `elif` in `stratified_evaluate`; a new model family needs the runner's `if model == "smolvla"` changed
L	baselines and SmolVLA are substitutable behind the engine's `Model` / `ModelInputAdapter` for evaluation (the same `Evaluator.evaluate` call)	`Trainer` only accepts SmolVLA (it raises otherwise), so "model" isn't substitutable for *training*. `train_view` and `val_view` are typed as `DatasetView` but used as the same object
I	`reports` take only the dict fields they render	`ModelArtifact` carries 16 fields, and the registry and model card each use a subset. `RunDir` exposes `write_json` / `append_jsonl` / `sub`, but half the writers bypass it (Module 3)
D	runner collaborators injectable; `settings` and `catalog` passed down rather than fetched where possible	every module depends on **concrete** `rde` classes (`Catalog`, `DataEngine`, `SmolVLAPolicy`, `Evaluator`); there's no project-owned protocol, and `SmolVLAPolicy` is constructed directly inside `Trainer` and the runner, so it can't be injected

The honest summary: SRP and DIP are the strained ones, and both for the same reason. The runner is the only seam, and it does too much, while the engine is depended on concretely everywhere.

14.5 Contract map: every boundary, and whether it's enforced

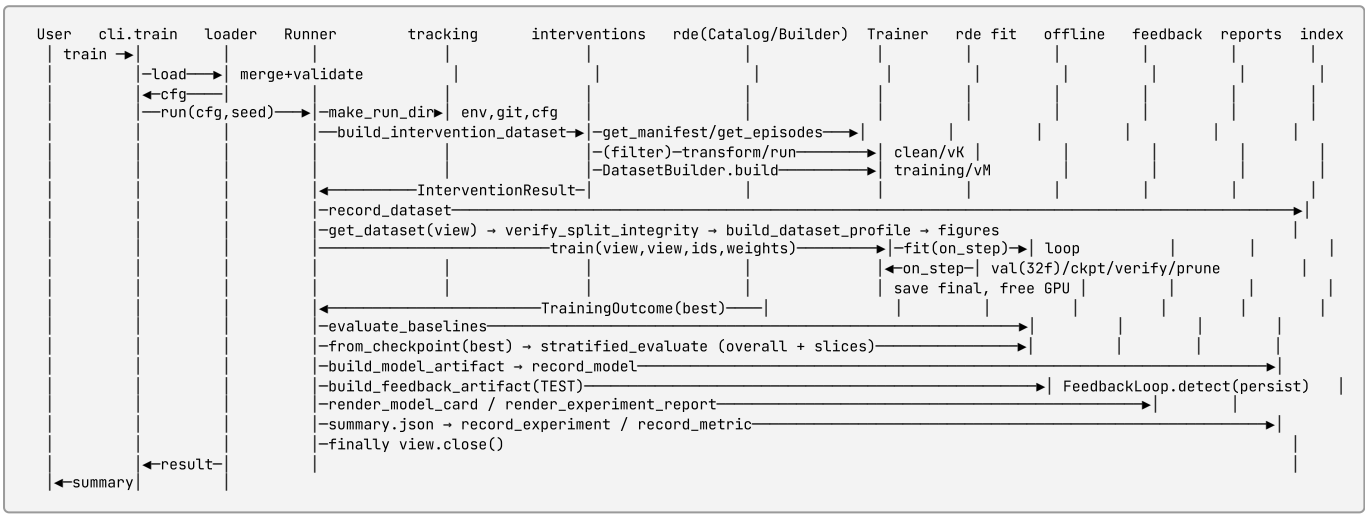
Boundary	Type	Enforced?	Notes
YAML → `ExperimentConfig`	Pydantic models	**partly**	types and cross-field rules yes; **unknown keys silently ignored** (no `extra="forbid"`); free strings for `model`, `selection_metric`, `slices`, `baseline_models` (Module 2)
CLI options → functions	Typer annotations	yes, for type	semantic validity (does the version exist?) is checked only later
intervention → runner	`InterventionResult` dataclass	type hints only (no runtime check)	its `build_summary: dict[str, Any]` is the engine's untyped blob
runner → `dataset.json`	`dict` literal	**no**	shape exists only in the runner and its readers (CLI `evaluate`, comparison)
split check → runner	`SplitIntegrityReport`	hints only	`ok` semantics too weak (Module 5)
profile → report and figures	`DatasetProfile.to_dict()` → `dict[str, Any]`	**no**	consumers index `profile["episode_length"]["histogram"][i]["lo"]` etc.
engine → profile	explorer dicts (`durations["duration_s"]["max"]`, `dims[k]["per_dimension"]`)	**no**	undocumented engine shapes; a change means `KeyError` (Module 5)
engine `fit` → `on_step(record)`	`dict` with `step`, `loss`, `lr`, `grad_norm`	**no**	`record["step"]` and `r["loss"]` are hard key accesses (Modules 6, 10)
trainer → runner	`TrainingOutcome` dataclass	hints only	`checkpoints: list[dict]` is untyped
checkpoint directory → everything	5 files plus `rde_checkpoint_meta.json`	**presence only** (`verify_checkpoint`)	no content or integrity check; metadata keys untyped
`EvaluationResult` → offline helpers	engine object (`.metrics` dict, `.outcomes` list of `FrameOutcome`)	engine-side	`metrics` keys untyped; `FrameOutcome.error` semantics (per-frame MAE) in docs only
offline helpers → JSON	`dict`	**no**	per-episode `"mse"` misnamed (Module 7)
feedback → artifact	`dict`	**no**	`report.summary()` and `artifacts` are engine blobs
artifact → index / registry	`ModelArtifact.to_dict()`	**no** on read	`ModelRegistry.get` returns `dict`
runner → report	`ctx: dict[str, Any]` with about 15 keys	**no**	the documented shape is "see `ExperimentRunner`" (docstring)
run directory → `compare` / CLI	JSON files	**no schema, no version**	missing files mean silent `None`s; partial files mean `JSONDecodeError`
checkpoint ↔ contract (deployment)	—	**no**	the cross-check is missing (Module 9)

Pattern: the *edges* into the system (YAML, CLI) are typed. The *interior* stage-to-stage handoffs are value objects that immediately degrade to dicts, and everything *persisted* is schemaless JSON. A Pydantic model per persisted artifact (`DatasetRecord`, `ModelRecord`, `SummaryRecord`) with a `schema_version` field would type

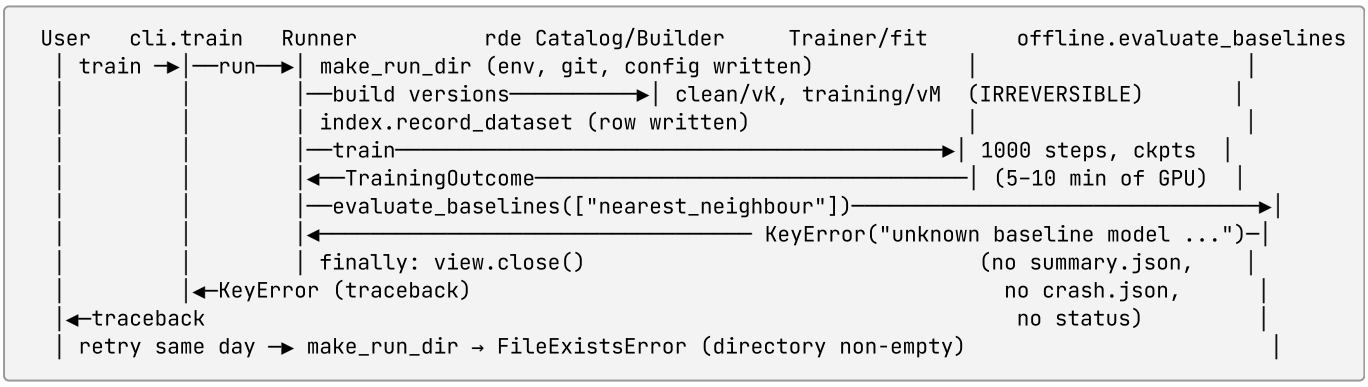
both the write and the read path, and it would have caught the `vla_finetune` / `model_training` key bug (Module 3) at once.

14.6 Sequence diagrams

Happy path: `mtrain train --config baseline.yaml`



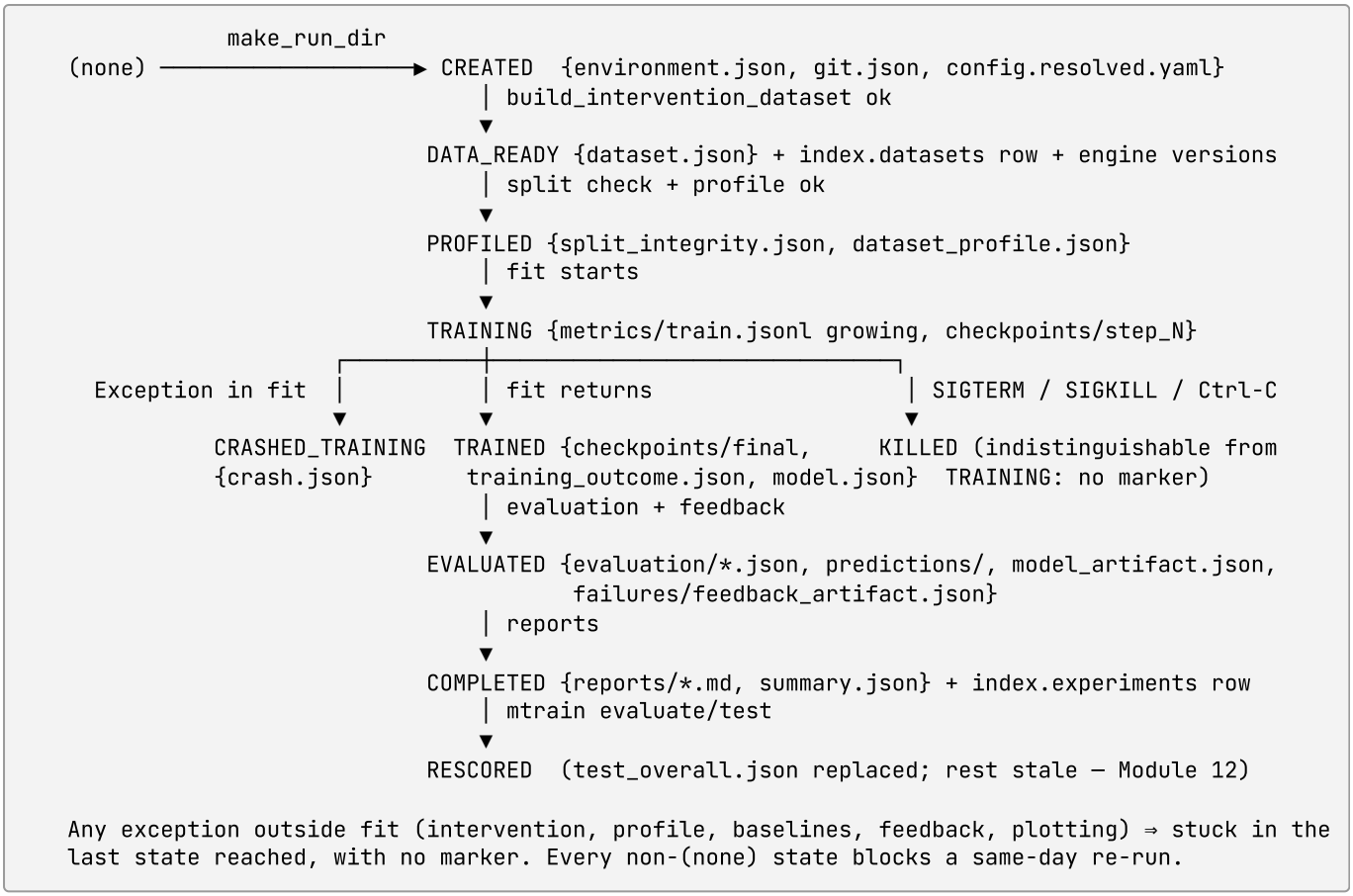
The interesting failure path: a typo in `baseline_models`, discovered after training



This one path exercises four separate gaps: late validation (Module 11), irreversible catalog writes before the risky step (Module 4), no status marker (Module 3) and the retry block (Module 3).

14.7 State machine of a run (implicit, reconstructed from which files exist)

There's no explicit `status` field. The states below are what you can *infer* from the run directory.



Missing transitions: CRASHED/KILLED → RETRY (only possible by renaming or deleting), and RESCORED → consistent (no recompute of dependents). **The fix:** write `status.json = {"state": ..., "updated_at": ...}` atomically at each arrow, and let `make_run_dir` accept `--force` only from a terminal failure state.

14.8 Error taxonomy

Error	Raised where	Caught where	What the user sees
`FileNotFoundError("no config file at …")`	`config/loader.py` `load_yaml`	nowhere	CLI traceback
`ValueError("must contain a YAML mapping")`	`loader.py`	nowhere	traceback
Pydantic `ValidationError` (wrapping validator `ValueError`s: sums, imbalanced or rebalanced target, custom IDs, fraction)	`config/schema.py`	nowhere	traceback with field paths, the clearest errors in the system
`FileExistsError("run directory already exists…")`	`tracking/run_dir.py` `make_run_dir`	nowhere	traceback; message explains the fix
`LookupError("unknown version")`	`data/interventions.py`, engine `get_dataset`, explorer	nowhere	traceback
`ValueError` (no episodes / zero resolved / empty weights / no automatic rule / no episodes to upweight)	`data/interventions.py` (6 sites)	nowhere	traceback, *after* run directory creation
`SplitLeakageError`	`data/split_integrity.py`	nowhere	traceback (practically unreachable, Module 5)
profile split lookup `Exception`	engine	`profile.py` (broad `except`)	**swallowed** into a note (sometimes a misleading one)
histogram SQL `Exception`	engine DuckDB	`profile.py`	**swallowed** into `{"reason": ...}`
git subprocess `Exception`	`run_dir._git_info`	same	**swallowed** into `{"error": ...}` in `git.json`
`ImportError` torch / lerobot / psutil	`run_dir`, `trainer`	same	**converted** to `None` plus a note (by design)
`ValueError` (trainer model guard, empty train IDs)	`training/trainer.py`	nowhere	traceback
`FileNotFoundError` / `ValueError` from `verify_checkpoint`	engine	`trainer.on_step`	**converted** to `verified: false` plus `checkpoint_errors.jsonl`
any `Exception` in `fit` (CUDA OOM, NaN if raised, `on_step` errors)	engine / trainer	`trainer.train`	`crash.json` written, then **re-raised** to a traceback
`KeyboardInterrupt`, SIGTERM, SIGKILL	OS / user	**nothing**	process ends; no marker
`KeyError("unknown baseline model")`	`evaluation/offline.py`	nowhere	traceback **after training**
`ValueError("unknown evaluation slice")`	`evaluation/offline.py`	nowhere	traceback **after training**
`KeyError` on engine dict shapes (`r["loss"]`, `durations[...]`)	profile, figures, cli	nowhere	traceback, possibly after training
`DeploymentHealthError`	`deployment/loader.py`	nowhere (only tests call it)	—
`RuntimeError("call load() before predict()")`	`deployment/loader.py`	nowhere	—
`JSONDecodeError` on torn or partial files	`JsonlIndex.read_all`, `load_run_summary`, CLI readers	nowhere	traceback with no hint of cause (Module 3)
`typer.Exit(1)` with a yellow or red message	`cli.py` (6 sites)	Typer	friendly one-line message

Shape of the error model: only **two** custom exception types (`SplitLeakageError`, `DeploymentHealthError`). Everything else is a built-in, nothing is caught at the top level, and the CLI turns *some* missing-file conditions into friendly exits but lets every pipeline error surface as a raw traceback. **Swallowing is confined to observation code** (git, environment, profile extras), and that's the right place for it. **The conversion of errors into data** (`verified: false`, `"unavailable"`, `crash.json`) is the system's best error-handling idea, and it's applied inconsistently: there's no equivalent for failures outside `fit`.

14.9 Lifetime, sharing and thread safety

Object	Created by	Lifetime	Shared across	Thread / process safety
`Settings`	`get_settings()` / test fixture `set_settings`	process (global singleton)	everything	global mutable state; tests rely on fixture order
`Catalog`	runner `__init__` (one per runner)	runner	interventions, profile, feedback	**engine-side, unverified**; process-level concurrency on the catalog file is unknown (version allocation race, Module 4)
`DataEngine` / `DatasetView`	runner `run`, profile, CLI	one `run`; closed in `finally`	trainer, evaluation, feedback (same view)	single-threaded use; two views of one version open at once (runner plus profile)
`ExperimentIndex` / `JsonlIndex`	runner `__init__`	runner	all runs in the process	appends are small writes; **not safe** for large rows across processes, **no locking** (Module 3)
`RunDir`	`make_run_dir`	one run	runner, trainer	single writer by convention; **TOCTOU** on creation
`SmolVLAPolicy` (train)	`Trainer.train`	`train()` call; explicitly freed	closure `on_step`	GPU-resident; freed before return (and not fully freed on the crash path)
`SmolVLAPolicy` (eval)	runner `run`	rest of `run`	evaluation, feedback	not explicitly freed
`Evaluator`	per helper call	call	—	stateless (inferred)
`console` (rich)	`cli.py` module level	process	all commands	fine
matplotlib backend	`figures.py` import	**process**	every importer	**global side effect**
`RDE_REPO` / `MT_REPO`	`runner.py` import	process	runner	frozen at import

No threads are created anywhere in the package. Concurrency risk is entirely **multi-process**: two `mtrain` invocations sharing `runs/`, `experiments/*.jsonl` and the engine catalog.

14.10 Idempotency table

Operation	Idempotent?	Second invocation does…
`load_experiment_config`	**no**	new `created_at`, so a new `experiment_id`
`mtrain train` (same day)	**no**, guarded	`FileExistsError`
`mtrain train` (next day)	**no**	new run directory **plus new engine versions** plus new index rows
`build_intervention_dataset`	**no**	mints new `clean/vK` and `training/vM` even for identical input
`verify_split_integrity`, `build_dataset_profile`	yes (apart from `generated_at`)	same report
`Trainer.train`	no (by construction)	new checkpoints; stochastic
`evaluate_baselines`	yes, for deterministic baselines	same metrics
`stratified_evaluate` (SmolVLA)	**no**	stochastic re-inference, numbers move up to 5%
`build_feedback_artifact`	**no**	duplicate engine failure records (`persist=True`)
`JsonlIndex.append` / `record_*`	**no**	duplicate rows
`ModelRegistry.get`	yes	the latest row
`mtrain evaluate` / `test`	**no**	overwrites with new stochastic numbers
`mtrain rollout`	**no**	appends another identical record
`mtrain build-intervention --apply`	**yes for the block**, **no for the file**	same block; comments already gone after the first run
`mtrain export`	yes (overwrite)	same copy; stale extra files remain
`mtrain compare`	yes	same table

Takeaway: the read paths are idempotent and the write paths are not, and nothing *dedupes* (versions, index rows, failure records). Content-addressing builds (Module 4) and a run status (14.7) are the two changes that would make re-execution safe.

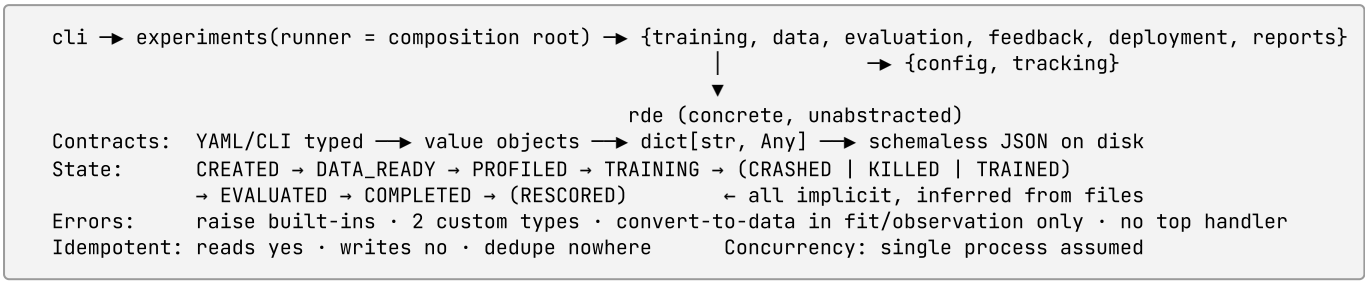
14.11 Patterns deliberately absent, and the trigger to add each

Absent pattern	Why absent is fine today	Trigger to add it
Plugin / Strategy registry for interventions and slices	6 kinds and 2 slices, edited in place	a third slice type, or a fourth new intervention kind (O/C strain)
Project-owned engine protocol (`typing.Protocol` for `Catalog` / `View` / `Evaluator`)	one engine, real-engine tests	a second backend, or the need for engine-free unit tests
Stage pipeline / DAG with caching	5–10 minute runs	multi-hour or preemptible runs, sweeps over 10 runs
Unit of Work / transaction across stores	one user, rare failures	concurrent users, or any automation that retries
Schema-versioned persisted records	one code version produced every run	the first breaking change to an artifact shape
Observer / event bus for run progress	a terminal and files suffice	a dashboard or remote monitoring
Retry with backoff	local disk and GPU errors aren't transient	network-backed storage or Hub downloads in CI
Locking (`fcntl` / sqlite) on the index	single process	two concurrent `mtrain train`
Dependency-injection container	one composition root suffices	multiple entry points wiring differently
Factory for models	one trainable model	a second trainable model family
Server (FastAPI / gRPC)	no consumer process	a robot controller in another process

14.12 Honest gaps to own before an interviewer finds them

1. **Interior contracts are untyped dicts**, and persisted artifacts have no schema or version. *Consequence:* key-mismatch bugs (`vla_finetune` / `model_training`) and engine shape changes surface as `KeyError` or silent `n/a`, never at write time.
2. **There's no explicit run state machine.** State is inferred from file existence, and killed, crashed and in-progress runs are indistinguishable. *Consequence:* retries are blocked, and compare and export can't tell complete from partial.
3. **The runner violates SRP** (orchestration plus formatting plus naming), and **the CLI duplicates runner internals.** *Consequence:* the `evaluate` drift (Module 12), and no reusable stage functions.
4. **Concrete dependence on `rde` everywhere, with no project-owned protocol.** *Consequence:* no seam for engine-free unit tests or a second backend (DIP strained).
5. **Open/closed is strained for interventions** (3–4 edit sites per new kind) and slices (an `elif` chain). *Consequence:* extension means editing core functions.
6. **Only two custom exceptions, no top-level handler, and late validation.** *Consequence:* expensive failures, and raw tracebacks for user errors.
7. **Write operations aren't idempotent, and nothing dedupes.** *Consequence:* re-execution multiplies versions, index rows and engine failure records.
8. **Process-level concurrency is unsafe** (TOCTOU, unlocked appends, non-atomic version allocation). *Consequence:* two concurrent runs can corrupt shared state.
9. **Import-time global side effects** (matplotlib backend, path constants). *Consequence:* importing the package changes process behaviour.

The shape to carry into the High-Level Design module



Drill — answer from memory, files closed

1. Draw the package dependency graph. Where's the composition root, and which edge breaks the layering?
2. Name four design patterns in this codebase, with exact locations and what each buys.
3. *Why didn't you just* define a typed `Protocol` for the data engine?
4. *What breaks in production:* two researchers launch `mtrain train` at the same moment on a shared machine. Walk through every shared resource.
5. *Interviewer framing:* "What's the weakest part of your low-level design, and how would you fix it?"

Appendix A — Model answers to the drill

A1. Dependency graph. "Four layers. At the bottom, `config` (Pydantic and YAML only) and `tracking` (standard library only). Above them, the stage packages: `data` and `training` depend on `config`, `training` and `deployment` also use `tracking`, and `evaluation` and `feedback` depend only on the engine. `reports` depends on nothing internal, because it takes plain dicts. Above that, `experiments`, whose runner is the composition root: it builds the catalog, engine, index, trainer and models, and calls every stage. The CLI sits on top. There are no cycles. The edge that breaks the layering is the CLI reaching past the façade into `data.interventions` internals and reimplementing the evaluation stage instead of calling a service function. And the engine sits underneath every layer as a concrete dependency, with no abstraction."

Why that answer lands: it's precise about direction and leaves, names the composition root, and identifies the real violation instead of claiming purity.

A2. Four patterns. "Template Method via callback in `Trainer.train`: the engine's `fit` owns the loop, and I inject an `on_step` hook that validates, checkpoints, verifies and prunes. It lets run policy live in this repo while the step mechanics stay in the engine. Transaction Script in `ExperimentRunner.run`: one linear method for the whole lifecycle, readable and mapping directly onto the architecture figure, at the cost of no resume. Specification via Pydantic model validators on `DatasetIntervention`: the methodology rule 'don't presume the weak dimension' becomes a load-time error. And a Null-Object-like explicit 'unavailable' record, in the profile's visual diversity, the rollout log and the environment's missing torch: absence is data, so nothing downstream mistakes it for a zero."

Why that answer lands: each pattern has a file, a mechanism and a payoff specific to this system, which shows the patterns are real and not name-dropped.

A3. Why not a `Protocol` for the engine? "Because there's one engine, and my tests deliberately use the real one, since the risky bugs live at that boundary, so a protocol would have had a single implementation and no test doubles. The cost showed up anyway. With no seam, pure logic like pruning, threshold arithmetic or report rendering can't easily be tested without spinning up a catalog, so a lot of it isn't tested. I'd introduce small protocols where a stage needs only a narrow slice of the engine: a `SplitSource` with `episode_ids(split)` for the integrity check, an `EpisodeCatalog` for intervention resolution. Then unit tests use fakes and the integration tests keep using the real engine. The trigger for a full abstraction would be a second backend."

Why that answer lands: it explains the original reasoning, admits the cost that materialized, and proposes a targeted, incremental fix with a clear trigger for going further.

A4. Two concurrent launches. "Four shared resources. First, the run directory: if it's the same experiment and seed, `make_run_dir` checks for emptiness and then creates, so both can pass the check and write into one directory. Second, the engine catalog: both call `next_version` and then write, which isn't atomic, so both may be allocated the same `training/vN`. Whether the engine rejects the second write I couldn't verify. Third, the JSONL index: appends are unlocked, and model rows can exceed the write buffer, so lines can interleave, and one torn line breaks every later read. Fourth, the GPU: two SmolVLA trainings on an 8 GB card will likely OOM one of them, and that one gets a `crash.json`. No threads are involved; it's all multiprocess. Fixes are an atomic `mkdir(exist_ok=False)`, a lock around version allocation and index appends, and a per-machine GPU lock or a queue."

Why that answer lands: it enumerates every shared resource, gives the precise race for each, separates verified from unverified, and proposes minimal fixes.

A5. "Weakest part of your low-level design?" "The contracts between stages. The edges are typed (YAML goes through Pydantic, the CLI through Typer), but inside the pipeline every handoff degrades to `dict[str, Any]`, and everything on disk is schemaless JSON. That's why a one-word key mismatch made every report print 'n/a' for my own repo's commit, and why an engine change to a stats dict would show up as a `KeyError` after training. The fix is a Pydantic model per persisted artifact (dataset record, model record, summary, report context) with a schema version, used on both write and read. Second on the list is the implicit state machine: I'd add an atomic status file so crashed, killed and completed runs are distinguishable and retries are possible."

Why that answer lands: it picks a genuinely structural weakness, proves it with a real bug, and gives a fix that addresses the class of bugs, not the instance.

Module 14 Summary — the one page to revise

What this module is. The design view of the package: an acyclic four-layer graph with one composition root (the runner), about a dozen real patterns at known locations, SRP and DIP strained at the runner and the engine boundary, typed edges with untyped interiors, an implicit file-existence state machine, a built-in-exception error model, single-process assumptions and non-idempotent writes.

Five claims to make cold

1. No cycles. `config` and `tracking` are leaves, `reports` imports nothing internal, and the runner is the composition root. The CLI is the one layering violation.
2. Patterns: Template Method via callback (Trainer), Transaction Script (runner), Specification (validators), Façade (index and packages), explicit "unavailable" Null Objects.
3. Contracts are typed at the edges and dicts inside; persisted JSON has no schema or version.
4. Run state is implicit in which files exist. Killed equals in-progress, and every state blocks retry.
5. Writes aren't idempotent and nothing dedupes. Concurrency is multi-process only, and unsafe.

Decisions

Chose	Over	Because	Cost
Strict downward layering, `reports` dict-only	shared typed models	decoupling	untyped interiors
Runner as the only composition root	DI container	one wiring point	SRP-heavy runner
Concrete `rde` dependence	project protocols	one engine; real-engine tests	no seam for unit tests or a second backend
Built-in exceptions, no top handler	custom hierarchy	simplicity	tracebacks for user errors; late validation
Errors as data in `fit` and observation	raise everywhere	partial results survive	inconsistent outside `fit`
File-existence state	explicit status	nothing to maintain	ambiguous states; blocked retries

Volunteer unprompted: the untyped contracts and the `n/a` key bug as proof; the implicit state machine; the CLI layering violation; non-idempotent writes; the multi-process races.

Numbers and names: 4 layers; 2 custom exceptions; 6 `raise typer.Exit` sites; 36 raise/except sites outside the CLI; 7 broad or `ImportError` excepts (all in observation or trainer code); 0 threads; 4 shared resources under concurrency (run directory, catalog, index, GPU).

Sentences worth memorizing

- "Typed at the edges, dicts in the middle, schemaless on disk. That's the weakest part of my design."
- "The runner is the composition root and the only seam; that's why it does too much."
- "My run state machine is implicit in which files exist, so a killed run looks like a running one."
- "Reads are idempotent, writes aren't, and nothing dedupes."

Module 15 — High-Level Design & Scaling

Files covered: cross-cutting. Numbers come from `README.md` §1–2, `docs/IMPLEMENTATION_PLAN.md` §3 and §8, and `configs/base.yaml`; every derivation from them is shown. No new source files.

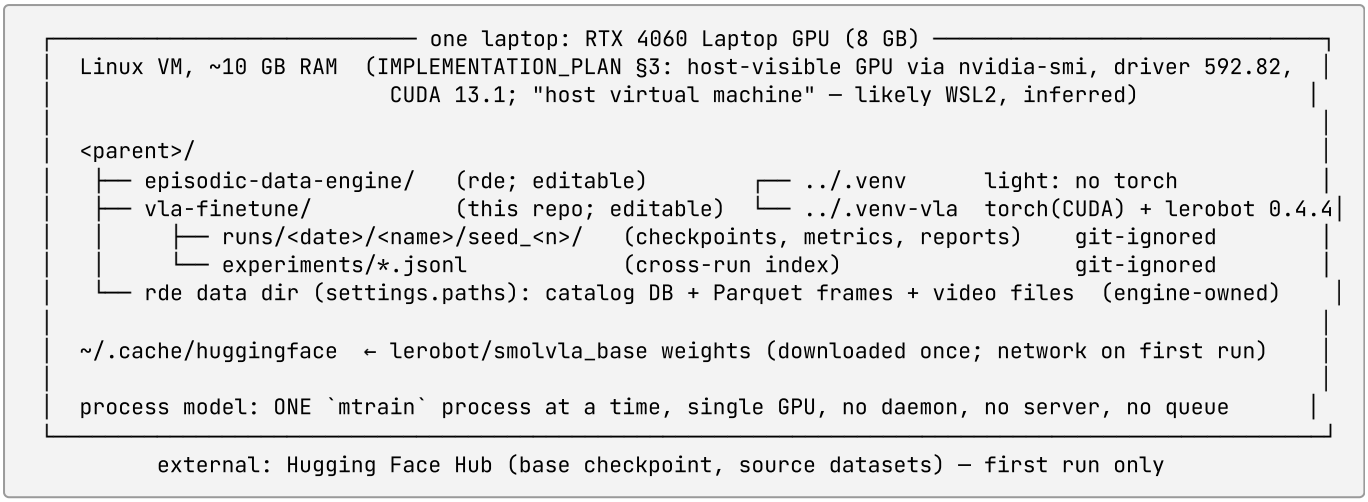
Where we are: Module 14 was the inside view (classes, contracts, states). This module is the outside view: what runs where, how long a run takes and where that time goes, what one machine can sustain, what breaks first as the work grows, and which parts would survive a 1000× scale-up unchanged.

The one-line thesis: This is an **evidence-limited system, not a compute-limited one**. On the README's own numbers, evaluation takes about as long as training, a fine-tuning run sees **well under one epoch** of its data, and the first thing that breaks as you scale is the **statistical validity of comparisons** (shared test sets, real seeds), long before GPU, disk or the index.

15.1 Prerequisites — the absolute basics

- **Deployment topology.** Which processes run on which machines, and what they share (files, databases, GPUs).
- **Latency budget.** A breakdown of where wall-clock time goes in one request, here one experiment run, so you know what to optimize.
- **Throughput / capacity.** Work completed per unit time (steps per second, runs per day), and the ceiling set by the scarcest resource.
- **Epoch.** One full pass over the training data. *Fraction of an epoch* = `steps × batch_size / training_frames`, assuming per-frame sampling. For SmolVLA each sampled frame supervises an *action chunk* of future actions, so per-frame "coverage" understates supervision somewhat. The engine's exact sampling unit is unverified.
- **Bottleneck analysis.** Ordering resources by which one saturates first as load grows. The order tells you what kind of system you have.
- **Scaling ladder.** For each order-of-magnitude increase, the single change without which the system fails.
- **Statistical power.** The probability that an experiment detects a real effect of a given size. It falls with small test sets, few seeds and noisy metrics.
- **Content-addressed cache.** Results stored under a hash of their inputs, so identical requests are served from cache.
- **Multi-tenancy.** Several users or teams sharing one system with isolation (namespacing, quotas, access control).
- **Quality vs infrastructure metrics.** *Quality* metrics say whether the results are right (test-set overlap, confidence intervals, baseline ratios). *Infrastructure* metrics say whether the system is healthy (GPU utilization, run failures, disk).

15.2 Deployment topology, as it actually is



Decision: single-machine, single-process, file-based. It's forced by C1 (one laptop GPU) and C9 (one or two researchers, short runs) from the Module 13 capstone. There's nothing to coordinate, so there's nothing to deploy. *Cost:* everything in Module 14's concurrency table becomes a problem the moment a second process appears.

15.3 The lifecycle of one run, with a latency budget

The README gives **training time** and **per-frame inference time**. I derived evaluation time from those. Lines marked *est.* are estimates I can't measure here.

SO-100 `baseline_full` (1,000 steps, batch 4, 1,908 test frames, about 1,900 val frames, inference about 48 ms/frame at batch 16):

Stage	Time	Derivation
env + git + config (torch import, CUDA init)	~2–5 s	*est.*
intervention: engine build of `training/vN`	seconds–tens of s	*est.*, engine-side, unverified
split check + profile (12 histogram scans: 6 action + 6 state)	seconds	*est.* (DuckDB on inline columns only)
training (incl. 10 × 32-frame validations, 10 checkpoint saves)	**324 s**	README Table 2
of which validation	~15 s	10 × 32 frames × ~48 ms
baselines (NN index on ~15k train frames, score 1,908)	seconds	*est.* (CPU, state-only)
SmolVLA test `overall`	~92 s	1,908 × 48 ms
`task_0` slice (single task ⇒ full duplicate, Module 7)	~92 s	same frames re-inferred
three length slices (partition the test set)	~92 s	same frames re-inferred
temporal re-read, feedback, reports, figures	seconds	*est.*
total	**~10–11 min**	
evaluation share	**~45%** (~275 s of ~610 s)	

ALOHA `aloha_coffee_baseline` (1,000 steps, 4,400 test frames, about 86 ms/frame, `slices: [per_length_bucket]`):

Stage	Time	Derivation
training	**576 s**	README Table 2
test `overall`	~378 s	4,400 × 86 ms
length slice: all 4 test episodes are 1,100 frames, so one bucket = a full duplicate	~378 s	Module 7 tie collapse
total	**~22 min**	
evaluation share	**~57%**, more than training	

What the budget says:

1. **Evaluation, not training, dominates wall-clock.** About 45% on SO-100 and about 57% on ALOHA. **Half of that evaluation time is pure redundancy:** the duplicate `task_0` on single-task SO-100, and the all-in-one-bucket length slice on ALOHA. Predict-once slicing (Module 7) would cut total run time by about 25–30% with *zero* loss of information.
2. **These are lower bounds.** The README's per-frame figure "excludes data loading", and ALOHA's frames come from four AV1 video streams (the config comment says "AV1, 4 cameras"). Decode time is extra and unmeasured.
3. **Validation is cheap (about 5% of training time)** precisely because it's capped at 32 frames. That's the Module 6 trade: cheap, and too noisy to select well.

15.4 Capacity arithmetic: one laptop

Training throughput (SO-100): 1,000 steps / 324 s ≈ **3.1 steps/s**, or about 12 samples/s at batch 4 (an upper bound on step time, since 324 s includes validation and saves).

How much of the data does one run see?

Dataset	Train episodes	≈ Train frames	Samples seen (1,000 steps × 4)	≈ Fraction of an epoch
SO-100	38	38 × ~400 ≈ 15,200 (episodes 326–475 frames)	4,000	**≈ 0.26**
SO-100, extended (+1,000)	38	≈ 15,200	8,000 total	**≈ 0.53**
ALOHA	32	32 × 1,100 ≈ 35,200 (test episodes are 1,100; assumed similar for train)	4,000	**≈ 0.11**

This is the most important number in this module. The headline runs train on **a quarter of an epoch (SO-100) and a ninth of an epoch (ALOHA)**, assuming per-frame sampling and before counting action-chunk supervision. Read that next to the Module 7 result:

- On SO-100, doubling steps (0.26 → 0.53 epoch) cut test MSE by 18%. The model was still learning.
- On ALOHA, at about 0.11 epoch, SmolVLA is **2× worse than nearest-neighbour retrieval**, which by construction has "seen" 100% of the training frames.

So "SmolVLA loses to retrieval on ALOHA" is, plausibly, **"SmolVLA was undertrained relative to a baseline that memorizes everything."** That's a better-supported and more useful interpretation than "retrieval is just strong". It also suggests the cheapest next experiment: ALOHA at 5–10k steps, about 50–100 min on the same laptop by linear extrapolation. The caveat: if the engine samples action-chunk windows rather than frames, the effective coverage is higher, and I can't verify which.

Memory headroom: peak GPU 2.2–3.0 GB of 8 GB (allocator-reported, Module 6), and peak host 3.0–3.3 GB of about 10 GB. **The 8 GB GPU isn't the binding constraint at batch 4.** Batch 8 or 16 probably fits (unverified; activations scale roughly linearly with batch, and the allocator figure excludes the CUDA context). Batch size 4 is **conservative, not forced**, by the measured numbers. That revises the C1 row in the Module 13 capstone. The constraint motivated it, but the measurements show slack.

Runs per day, serially: about 10–11 min (SO-100) or about 22 min (ALOHA) per run → **about 130 SO-100 runs/day or about 65 ALOHA runs/day** at 100% utilization, so realistically a few dozen. **Concurrency:** 2–3 GB peak per training process means two would *fit* in 8 GB, but Module 14 showed two concurrent runs race on the run directory, catalog and index. Capacity is single-process by correctness, not by hardware.

Disk per run (*est.*): SmolVLA is publicly about 450M parameters, so a checkpoint is roughly 0.9 GB in 16-bit or 1.8 GB in 32-bit (unverified which). Kept per run: last 3 periodic, plus best if older, plus `final` (a duplicate of `step_1000`, Module 6), so **4–5 checkpoints ≈ 4–9 GB per run**. At a few dozen runs, that's **hundreds of GB**. Disk fills long before the GPU saturates.

15.5 Bottleneck analysis, in the order things break

Scale the *number of experiments* (the axis a research harness actually grows on):

Order	Bottleneck	Breaks at	Symptom
1	**Statistical validity of comparisons**	the **first** comparison between two interventions	different test sets per intervention (Module 4), fake seeds (Module 11), no intervals (Module 7), feedback tuned on test (Module 8): conclusions are wrong *now*, at 3 runs
2	**Evaluation time**	~10 runs	about half of each run is inference, half of that redundant
3	**Disk**	~20–50 runs	4–9 GB per run in checkpoints, with no retention policy beyond keep-last-3
4	**Single-GPU serial execution**	~50–100 runs per week	a queue of runs; no scheduler
5	**Process-level correctness**	the first time anyone parallelizes	run-directory TOCTOU, catalog version race, torn JSONL lines (Module 14)
6	**Index and catalog as files**	thousands of runs	full-scan queries, no locking, no status
7	**Data path** (video decode, loader)	10–100× dataset size	decode-bound evaluation; the profile's per-dimension DuckDB scans grow

What the ordering says about the system: the *first* failure isn't a resource at all. It's that the harness can't yet produce a trustworthy comparison between two experiments. The *second* is an algorithmic waste (redundant evaluation), and only then come the physical resources. That ordering identifies a **research-methodology system**: its scaling problem is evidence quality per GPU-hour, not GPU-hours. The engineering priority list follows directly: fix the comparison protocol before buying hardware.

15.6 The scaling ladder

Each step names **the one change without which that step fails**, plus what comes along with it.

Today (1×): about 3 published runs, 1 researcher, 1 laptop GPU, 40–48 episodes.

10× (tens of runs, a real intervention study, about 500 episodes).

- **The one thing: a frozen, shared test split plus predict-once evaluation.** Split the full pool once, apply interventions only to its train split (Module 4), score every run on the same episodes, and slice stored predictions instead of re-inferring (Module 7).
- With it: real seed propagation (Module 11), episode-bootstrap intervals from `test_outcomes.jsonl` (Module 7), feedback on validation (Module 8), and a checkpoint retention policy (keep best plus final, drop the `step_N` / `final` duplicate).
- Without it, ten runs produce ten numbers that can't be compared.

100× (hundreds of runs, 2–5 researchers, a few GPUs).

- **The one thing: an explicit run state machine plus a scheduler.** Write `status.json` atomically at every transition (Module 14 §14.7), and have one job queue that owns the GPUs and serializes catalog version allocation.
- With it: the index moves to SQLite or DuckDB (or MLflow as a sink, Module 3), with per-user namespaces for `experiment_name`; checkpoints move to object storage with retention; config gets `extra="forbid"` and fail-fast name validation (Module 11), because a typo now wastes someone else's queue slot; and there's CI with both repos and a pinned engine version (Module 1).
- Without it, concurrent runs corrupt the shared catalog and index.

1000× (thousands of runs, tens of thousands of episodes, multi-GPU training).

- **The one thing: the data path.** Pre-decoded or streaming frame storage and a parallel loader owned by the engine (the `num_workers` and `mixed_precision` fields that exist in the config today and are wired to nothing, Module 2), plus distributed training (DDP or FSDP) inside the engine's `fit`.
- With it: the engine catalog becomes a server database with transactional version allocation; content-addressed `training/vN` (identical builds reused, Module 4); sweeps become DAG jobs with stage caching (Module 11); profiling goes from per-dimension full scans to one pass with all dimensions; and real closed-loop evaluation (simulator farm), because at this scale offline MSE stops being an acceptable proxy for a decision.
- Without it, every run is decode-bound and a single GPU per run can't finish an epoch in useful time.

15.7 Caching strategy

What	Cached today?	Opportunity	Key
Base checkpoint download	yes, by the Hugging Face cache (external)	—	Hub revision
Training dataset version	**no**: every run mints a new `training/vN` (Module 4)	content-addressed build reuse	hash(input version, subset, splits, seed, weights)
Baseline fits (NN index)	**no**: refit every run	cache per `training/vN`	training version plus baseline name
Test predictions	**no**: re-inferred per slice and per re-score	predict once, slice in memory; store vectors	checkpoint hash plus version plus split plus sampling seed
Decoded frames	engine-side, unknown	pre-decoded tensor cache	version plus camera plus resolution
Profile	**no**: recomputed per run on an immutable version	cache per version (immutable, so never stale)	dataset version

Principle: because versions are immutable, **everything keyed by a version is cacheable forever.** That's an unusually strong position, and it's the payoff of Module 4's design. The system doesn't exploit it yet.

15.8 Multi-tenancy

Today: single tenant, none of the mechanisms.

- `experiment_name` is a global namespace in `runs/`, so two users' `baseline_full` collide (`FileExistsError`) or, worse, interleave under TOCTOU.
- The engine catalog is shared, with no access control (engine-side, unverified).
- The index has no `user` field. `environment.json` records `hostname` but not the user.
- `make_run_dir`'s no-overwrite guard is the only isolation.

Minimum for multi-tenancy: a `user` (or `team`) path segment in `runs/` and an index column; catalog writes through a service that serializes allocation; and per-user GPU quotas in the scheduler. **None of this is warranted before the 100× step**, and the C9 constraint (team of one or two) justifies its absence.

15.9 Observability: what to measure, quality first

Today: files only. `metrics/train.jsonl` (loss, LR, grad norm per step), `validation.jsonl`, `checkpoint_errors.jsonl`, `crash.json`, `environment.json`, peak memory and wall clock. There's **no logging** (Module 3), no live view, and no cross-run dashboard.

What I'd add, in priority order. Quality before infrastructure:

#	Metric	Type	Why it's first
1	**test-set identity per run** (hash of test episode IDs) and **"comparable?" flag** in `compare`	quality	the #1 bottleneck; makes the moving test set visible
2	**SmolVLA / nearest-neighbour MSE ratio**, not just vs mean-action	quality	the honest baseline; mean-action ratios (2.6%) flatter
3	**bootstrap 95% CI over test episodes** for MSE	quality	5 test episodes; every difference needs an interval
4	**inference-noise band** (score overall twice, report the spread)	quality	measured once by accident (up to 5%); should be deliberate
5	**val–test gap** for the selected checkpoint	quality	13× today (Module 6); flags a broken selection signal
6	**epochs seen** = steps × batch / train frames	quality	explains under-training (0.11 epoch on ALOHA)
7	per-slice frame and episode counts	quality	stops "long episodes are easier" from 1–2 episodes
8	run status distribution (completed / crashed / killed)	infrastructure	needs the state machine
9	steps/s, eval frames/s, eval share of run time	infrastructure	evaluation dominates
10	GPU utilization (not just allocator peak), disk per run	infrastructure	headroom and retention

Why quality metrics come first: this system's characteristic failure isn't a crash, which is loud and already mostly handled. It's a **plausible, wrong conclusion produced silently**: a comparison across different test sets, a variance from fake replicates, a recommendation that can never say "no action". Infrastructure dashboards would show a perfectly healthy system producing those. The metrics that would have caught every serious finding in this manual are quality metrics.

15.10 Cost model

Formula per run: `cost ≈ GPU_hours × GPU_price + disk_GB × storage_price × retention + researcher_minutes × hourly_rate`.

On the laptop (today): the marginal GPU cost is electricity, which is negligible. The real costs are **researcher time** and **disk**.

Cloud-equivalent order of magnitude (*assumption:* a single entry-level datacenter GPU at roughly $0.5–1 per hour; real prices vary by provider and region, and I haven't looked them up):

- SO-100 run, about 0.18 h → **about $0.10–0.20 per run**
- ALOHA run, about 0.37 h → **about $0.20–0.40 per run**
- A 10-point data-fraction sweep with 3 seeds = 30 runs → **about $5–10**
- Checkpoint storage at 4–9 GB per run quickly exceeds compute cost if kept indefinitely

Where money is actually wasted, in order:

1. **Wrong conclusions** (re-running studies whose comparisons were invalid). That's unbounded, and it's the real cost.
2. **Redundant evaluation**, about 25–30% of every run's GPU time (15.3).
3. **Duplicate and retained checkpoints** (`final` = `step_1000`; keep-3 regardless of value).
4. **Config typos found after training** (Module 11): one full run each.

What survives cost-wise: at these prices, compute is cheap enough that **adding seeds and longer training costs dollars**. The barrier to trustworthy results is protocol, not budget.

15.11 What survives unchanged: the good bones

These would carry through every rung of the ladder without redesign:

1. **Data identity by immutable version, with lineage** (Modules 2 and 4). It scales from a laptop to a catalog server unchanged, and it's what makes caching safe forever.
2. **Interventions expressed as engine transforms with recorded reasons** (Module 4). Only the *order* (split first) needs changing, not the mechanism.
3. **The fully resolved config persisted with every run** (Module 2). It becomes the job specification for a scheduler as-is.
4. **The refusal to fabricate as data** (`"unavailable"`, `verified: false`, `crash.json`, rollout records). That's an honesty contract that only gets more valuable as more people read the results.
5. **Validation-only checkpoint selection, structurally enforced** (Module 6). It's the right discipline; only the *sample* needs fixing.
6. **Per-frame outcomes with full provenance** (`test_outcomes.jsonl`, Module 11). That's the raw material for bootstraps, per-joint analysis and predict-once slicing.
7. **Plain-file artifacts readable without this code** (Module 3). DuckDB over the same JSONL is the 100× index with no format migration.
8. **Git identity of both repositories** (Module 3). Only the path resolution needs fixing.
9. **Dry-run-by-default config resolution from the real catalog** (Module 12). It's the human gate a multi-user system needs.
10. **Lazy, import-light CLI** (Module 12). It stays fast as the dependency stack grows.

The pattern: the **mechanisms** (identity, lineage, honesty, provenance) are the bones, and they scale. The **statistical protocol** (one test set, real seeds, intervals, val-driven curation) is what's missing, and it's cheap to add.

15.12 Honest gaps to own before an interviewer finds them

1. **Evaluation is about 45–57% of run time, half of it redundant** (derived from README numbers). *Consequence:* about 25–30% of GPU time per run is wasted.
2. **Headline runs see about 0.26 (SO-100) and about 0.11 (ALOHA) of an epoch** (per-frame assumption). *Consequence:* the ALOHA loss to retrieval is plausibly under-training, and the README's "retrieval is a strong baseline" framing under-states this.
3. **Batch 4 uses 2.2–3.0 of 8 GB.** *Consequence:* "constrained by the GPU" is softer than it sounds. Larger batches were plausibly available (unverified).
4. **Disk, not GPU, is the first physical limit** (4–9 GB per run, estimated). *Consequence:* there's no retention policy beyond keep-last-3, plus a duplicate `final`.
5. **No caching despite immutable versions.** *Consequence:* every run recomputes builds, baselines and profiles that could be reused forever.
6. **No observability beyond files, and no quality metrics** (test-set identity, NN ratio, intervals, val–test gap, epochs seen). *Consequence:* the system looks healthy while producing incomparable results.
7. **Single tenant by design** (global experiment names, shared catalog, no user field). *Consequence:* fine until a second researcher.
8. **All timing beyond training and per-frame inference is estimated**, since data loading and video decode aren't measured anywhere. *Consequence:* the latency budget's lower bounds are real, and its totals are approximations.

The shape to carry into the Fixes & Roadmap module

```
1 laptop · 8 GB GPU (2–3 GB used) · ~10 GB RAM · 1 process · files + engine catalog
run (SO-100):  train 324 s  ▸  eval ~275 s (½ redundant)  ▸  total ~10–11 min  ▸  ~0.26 epoch
run (ALOHA):   train 576 s  ▸  eval ~756 s (½ redundant)  ▸  total ~22 min     ▸  ~0.11 epoch
breaks in order:  ① comparison validity ② eval time ③ disk ④ serial GPU ⑤ multi-process ra
                  ⑥ file index/catalog ⑦ data path
ladder:  10× frozen test split + predict-once   100× run state machine + scheduler
         1000× data path (decode/loader) + distributed fit
bones that survive: version identity & lineage · resolved config · "unavailable" honesty ·
                    val-only selection · per-frame provenance · plain-file artifacts
```

Drill — answer from memory, files closed

1. Give the latency budget of one SO-100 run and say where you'd cut first.
2. How much of an epoch does a 1,000-step run see on each dataset, and what does that change about how you read the ALOHA result?
3. *Why didn't you just* move this onto a cluster with a job scheduler and MLflow from the start?
4. *What breaks in production:* the team grows from one researcher to five sharing two GPUs. List what fails, in order.

5. *Interviewer framing:* "How would this system scale to a thousand times more data and experiments?"

Appendix A — Model answers to the drill

A1. SO-100 latency budget. "Training is 324 seconds for 1,000 steps, including ten 32-frame validations and ten checkpoint saves. Evaluation is the surprise. Test inference is about 48 milliseconds a frame over 1,908 frames, roughly 90 seconds per pass, and the default slices run three passes: overall, a per-task slice that's a full duplicate because SO-100 has one task, and three length slices that together cover the test set again. So about 275 seconds of evaluation, against 324 of training, around 45% of a ten-to-eleven-minute run. I'd cut there first. Predict once and slice the stored outcomes, and evaluation drops to about 90 seconds with no information lost. That's about a quarter of total run time, recovered with no hardware change. Those evaluation figures are lower bounds, since the per-frame number excludes data loading."

Why that answer lands: it derives the budget from real numbers, finds that evaluation, not training, is the big lever, and quantifies the fix. That's capacity reasoning, not guesswork.

A2. Epochs seen. "At batch 4, 1,000 steps is 4,000 samples. SO-100's 38 training episodes are around 15,000 frames, so that's about a quarter of an epoch, and the extended run about half. ALOHA's 32 training episodes at around 1,100 frames are about 35,000 frames, so 1,000 steps is about a ninth of an epoch. That assumes per-frame sampling. If the engine samples action-chunk windows, coverage is higher, which I couldn't verify. It changes how I read ALOHA: SmolVLA losing two-to-one to nearest-neighbour retrieval is a model that's seen about 11% of the data, against a baseline that has memorized all of it. Plus, doubling training on SO-100 gave 18%. So the defensible reading is 'undertrained', not 'retrieval is just better'. And the cheapest next experiment is ALOHA at five to ten thousand steps, about an hour on the same laptop."

Why that answer lands: a simple calculation reframes the project's most uncomfortable result into a testable hypothesis with a costed next step, and it states the assumption.

A3. Why not a cluster, scheduler and MLflow from day one? "Because none of the first bottlenecks are infrastructure. With one or two researchers and ten-minute runs, a scheduler and a tracking server would have added operational surface while the real problems were methodological: interventions scored on different test sets, seeds that didn't propagate, no intervals. Infrastructure doesn't fix any of those, and it would have produced the same incomparable numbers faster. The file-based design also kept the artifacts readable by DuckDB and pandas, so moving to MLflow or SQLite later is an export, not a migration. The trigger for a scheduler is the second concurrent user, because that's when the run-directory, catalog and index races I know about start firing."

Why that answer lands: it argues from the measured bottleneck order, shows the infrastructure wouldn't have helped the real problem, and names the precise trigger.

A4. Five researchers, two GPUs. "First, results stay incomparable, the same as with one researcher but with more of them. Then name collisions: experiment names are a global namespace in `runs/`, so two people's `baseline_full` either collide on `FileExistsError` or, if launched at the same moment, race into one directory. Then the shared engine catalog: version allocation isn't atomic, so concurrent runs can claim the same training version. Then the JSONL index: unlocked appends of large model rows can interleave, and one torn line breaks every later read of the registry. Then the GPUs: with no scheduler, two runs can land on one card. Each needs 2–3 GB, so they'd fit, but they'd contend. Then disk fills, at four to nine gigabytes of checkpoints per run. The fixes, in that order: frozen test split and real seeds, per-user namespaces, a status file plus a job queue that also serializes catalog writes, a SQLite or MLflow index, and a checkpoint retention policy."

Why that answer lands: it's ordered by when each failure actually occurs, distinguishes correctness races from resource contention, and maps each to a fix.

A5. "How would this scale to 1000×?" "In three steps, each with one non-negotiable change. At 10×, tens of runs and hundreds of episodes, the blocker is comparability, so: freeze one shared test split, apply interventions only to train, propagate real seeds, and predict once for evaluation. At 100×, multiple users and GPUs, it's correctness under concurrency, so: an explicit run state machine and a scheduler that owns GPUs and serializes catalog writes, with the index in SQLite or MLflow. At 1000×, the data path, so: pre-decoded or streaming frames, a parallel loader and distributed training in the engine, plus a transactional catalog service and real closed-loop evaluation, because at that scale offline MSE is too weak to make decisions on. What doesn't change at any step is the core: immutable versioned data with lineage, the resolved config as the job spec, per-frame outcomes with provenance, and the refusal to fabricate. Those are the bones."

Why that answer lands: each rung has exactly one named blocker and its fix, the ordering matches the bottleneck analysis, and it ends by naming what's already right. That shows you can tell the parts that need to change from the parts that don't.

Module 15 Summary — the one page to revise

What this module is. The system seen from outside: one laptop, one process, files plus an engine catalog; about 10–11 minutes per SO-100 run and about 22 minutes per ALOHA run, with evaluation the larger or near-equal share; under one epoch of training per run; and a bottleneck order that begins with statistical validity, not hardware.

Five claims to make cold

1. Evaluation is about 45% (SO-100) and about 57% (ALOHA) of run time, and half of it is redundant slicing. Predict-once saves about 25–30% of each run.
2. 1,000 steps at batch 4 is about 0.26 epoch on SO-100 and about 0.11 on ALOHA. The ALOHA loss to retrieval is plausibly under-training.
3. Peak GPU is 2.2–3.0 GB of 8 GB, so batch 4 is conservative, not forced.
4. Bottleneck order: comparison validity → eval time → disk → serial GPU → process races → file index → data path. It's an evidence-limited system.
5. The ladder: 10× frozen test split plus predict-once; 100× state machine plus scheduler; 1000× data path plus distributed fit.

Decisions

Chose	Over	Because	Cost
Single machine, single process, files	cluster + scheduler + tracking server	C1, C9; nothing to coordinate	every concurrency race latent
Batch 4	larger batch	8 GB caution	under 3 GB used; fewer samples per second
1,000-step budget	epoch-based budget	fast iteration	0.11–0.26 epoch; under-training
No caching	content-addressed caches	simplicity	recompute despite immutable versions
File-only observability	logging plus dashboards	no service to run	no quality metrics; no live view
No multi-tenancy	namespaces and quotas	team of one or two	collisions at user two

Volunteer unprompted: evaluation dominates run time; the under-one-epoch arithmetic and the ALOHA reinterpretation; disk before GPU; immutable versions cached nowhere; quality metrics before infrastructure metrics.

Numbers and names: 324 / 385 / 576 s training; 47–50 ms/frame (SO-100) and 86 ms/frame (ALOHA) at eval batch 16; 1,908 and 4,400 test frames; about 3.1 steps/s; peak GPU 2.2–3.0 GB / 8 GB; host 3.0–3.3 GB / ~10 GB; SO-100 about 15,200 train frames (0.26 epoch), ALOHA about 35,200 (0.11 epoch); checkpoints about 0.9–1.8 GB each (estimated), 4–9 GB per run; cloud-equivalent about $0.10–0.40 per run (assumed $0.5–1/GPU-hour); lerobot 0.4.4, CUDA 13.1, driver 592.82.

Sentences worth memorizing

- "Evaluation, not training, dominates my run time, and half of it was redundant."
- "My ALOHA run saw about a ninth of an epoch; losing to a baseline that memorized everything says 'undertrained' before it says 'retrieval is better'."
- "The first thing that breaks when this scales isn't the GPU. It's whether two of my numbers are comparable."

- "Immutable versions mean everything keyed by a version is cacheable forever; I don't cache anything yet."

Module 16 — Fixes & Roadmap

Files covered: the running gaps register (`docs/study/_gaps.md`, about 200 entries across Modules 1–15), consolidated. Each fix names the file or files to change.

Where we are: Fifteen modules each ended with "honest gaps". This module turns them into a plan. They're deduplicated, so a gap found in three modules appears once, and they're prioritized by **what they do to the project's conclusions**, not by how easy they are. The module ends with a one-day shortlist, a feature roadmap, and an anti-roadmap of tempting changes that would make the system worse.

The one-line thesis: About a dozen fixes matter more than everything else, and almost all of them are about **making two results comparable** (shared test split, real seeds, validation-driven decisions, strict config, non-destructive re-scoring), not about features or infrastructure. Most of them are hours of work, not weeks.

16.1 Prerequisites — the absolute basics

- **Priority levels used here.**
  - **P0:** the gap invalidates a result or a claim the project makes, *now*, at current scale. Fix it before reporting anything new.
  - **P1:** the gap causes wrong output, lost work or misleading artifacts under realistic use, or a doc states something false.
  - **P2:** hygiene, robustness, cost, or something that only bites at larger scale.
- **Effort estimates** assume someone who knows this codebase, including writing a test for the fix: **XS** < 30 min, **S** about 1–2 h, **M** about half a day, **L** 1–3 days. They're estimates, and engine-side work could change them.
- **Regression test.** A test written to fail before the fix and pass after it. Every P0/P1 below names one.
- **Anti-roadmap.** Changes that look like progress but would make the system worse at its current stage, each with the condition under which the answer flips.

16.2 P0: fix before reporting anything new

#	Fix	Gaps consolidated	Files	Effort	Regression test
P0-1	**Freeze one test split; apply interventions only to the train split.** Build `training/v_base` once from the full pool. Each intervention then becomes a transform on `v_base`'s *train* episodes (filter or weight), inheriting val/test unchanged.	M4-1, M11-10, M8 (compare on same slice), M4-9	`data/interventions.py`, `experiments/runner.py`, `experiments/data_fraction.py`; possibly an engine transform that preserves split attributes	L	assert `reduced` and `imbalanced` runs' test episode IDs == baseline's
P0-2	**Make `seed` a real seed.** Copy the config with `training.seed = seed`, re-stamp `experiment_id`, and loop over `config.seeds` from the CLI. Keep the split seed fixed.	S1, M11-1, M12-12	`experiments/runner.py`, `cli.py`	S	two seeds → different `training.seed` in `config.resolved.yaml` and different `experiment_id` / `model_id`
P0-3	**Strict, fail-fast config.** `model_config = ConfigDict(extra="forbid")` on all six models; `Literal` for `training.model`, `selection_metric`, `slices`, `baseline_models` (or a stage-0 lookup against `MODEL_REGISTRY`); bound split fractions to [0, 1] with `val > 0`; require `checkpoint_every % validate_every == 0`.	M2-1, M2-2, M2-8, M2-14, M11-2, M6-5	`config/schema.py`, `experiments/runner.py` (stage 0)	S	`learning_rat:` → error; `model: SmolVLA` → error; `baseline_models: [nearest_neighbour]` → error *before* `make_run_dir`
P0-4	**Close the imbalanced silent no-op.** Raise when `imbalanced` resolves to *every* episode, or when `variable` is set but `variable_value` is `None`. Coerce `task_id` values to `int` in the schema.	M2-4, M12-6, M4-5	`data/interventions.py`, `config/schema.py`, `cli.py`	XS	`imbalanced, variable=episode_length_bucket, value=None` → `ValueError`
P0-5	**Run the feedback loop on validation, not test.** Do one full val evaluation of the selected checkpoint, and pass it to `build_feedback_artifact`. Test stays untouched until the final report.	M8-1	`experiments/runner.py`	S	spy on `build_feedback_artifact`: its `result.split == "val"`
P0-6	**Never overwrite results in `mtrain evaluate` / `test`.** Write to `evaluation/rescore_<UTC-ts>_<split>/`, name files by split (`<split>_overall.json`), and refuse to write `test_overall.json`.	S7, M12-1, M12-2	`cli.py`	S	after `mtrain evaluate --split val`, the original `test_overall.json` is byte-identical
P0-7	**A trustworthy selection signal.** Choose a fixed, seeded, stratified validation subset (frames spread across every val episode and across progress deciles) once per run, score every checkpoint on it, and average 2 stochastic samples per frame.	M6-1	`training/trainer.py`	M	subset covers every val episode; identical subset across checkpoints

Why these seven: each one, unfixed, produces a number that's wrong or a comparison that's invalid *today*. P0-1 is the only one larger than half a day, and it's the one the experiment design depends on.

16.3 P1: wrong output, lost work, or false documentation

Correctness and data integrity

#	Fix	Gaps	Files	Effort
P1-1	**Predict once, slice stored outcomes.** `stratified_evaluate` groups `overall.outcomes` by episode. Measure inference noise deliberately (score overall twice with different sampling seeds).	M7-1, M15-1	`evaluation/offline.py`	S
P1-2	**Bootstrap 95% CI over test episodes** from `test_outcomes.jsonl`, reported in `test_overall.json`, the report and `compare`.	M7-3, M15-6	`evaluation/offline.py`, `reports/*`	S
P1-3	**Explicit run state machine.** Write `status.json` atomically at every transition; `make_run_dir --force` only from `failed`; mark `summary.json` complete.	M3-3, M11-3, M14 §14.7, S13	`tracking/run_dir.py`, `experiments/runner.py`, `cli.py`	M
P1-4	**Atomic writes and tolerant reads.** Write to temp then `os.replace` for JSON; a single `os.write` per JSONL line; `read_all` skips and counts bad lines.	M3-1, M3-2	`tracking/run_dir.py`, `tracking/index.py`	S
P1-5	**Crash coverage.** `except BaseException`, a SIGTERM handler that raises, and `status: running` written before `fit`.	M6-2	`training/trainer.py`, `experiments/runner.py`	S
P1-6	**Selection-fallback honesty.** Add a limitation when `best → final` fires; reject non-finite val metrics (`math.isfinite`).	M6-3, M6-4	`training/trainer.py`, `experiments/runner.py`	XS
P1-7	**Feedback thresholds that can say "no".** An absolute or baseline-relative gate on *whether* to act; concentration tested against the p-percentile base rate (binomial).	M8-2, M8-3	`feedback/artifact.py`	S
P1-8	**`health_check` cross-checks checkpoint ↔ contract** (action dim, camera keys and count, dataset id), and compares preprocessor stats with contract stats.	S8, M9-1, M9-2	`deployment/loader.py`	S
P1-9	**Coverage normalized by `log2(_HIST_BINS)`.** Keep evenness as a separately named metric.	M5-4	`data/profile.py`	XS
P1-10	**Split integrity `ok` = complete, non-empty, disjoint partition with sizes matching config ±1.**	M5-2	`data/split_integrity.py`	XS
P1-11	**One definition of length buckets**, computed once on the full pool with fixed cut points, used by both intervention and evaluation.	M4-4, S11	`data/interventions.py`, `evaluation/offline.py`	S
P1-12	**Engine provenance from `rde.__file__`**, not a guessed path; save `git diff HEAD` for dirty trees.	M3-5, M3-6	`tracking/run_dir.py`, `experiments/runner.py`	XS
P1-13	**The zero-data anchor = evaluate the base checkpoint with no fine-tuning** (its own path); fix or delete the misleading test.	S6, M4-2, M13-3	`data/interventions.py`, `experiments/data_fraction.py`, tests	S
P1-14	**Sweep fault isolation.** A per-fraction `try`, incremental results to `experiments/sweeps/*.jsonl`, nested subsets (prefixes of one permutation), and collision-free names.	M11-11, M11-12, M4-6	`experiments/data_fraction.py`, `data/interventions.py`	S

Artifacts and reporting

#	Fix	Gaps	Files	Effort
P1-15	**Report correctness.** Fix the git key (`vla_finetune`); show SmolVLA MAE; include per-dimension MAE; `n/a` for `None`.	S2, M3-8, M10-1, M10-3, M10-4, M10-15	`reports/experiment_report.py`	XS
P1-16	**The limitations list carries every caveat**: rebalanced ("redistributes attention, cannot close a coverage gap"), selection fallback, epochs seen, test set differs from baseline (until P0-1).	M4-8, M10-12	`experiments/runner.py`	XS
P1-17	`compare` **that can't mislead.** Build the rich table by key (union of columns); add a test-set hash column with a "comparable?" flag; group by experiment and report mean ± sd and n over seeds.	M10-5, M10-6, M10-7, M15-6	`reports/comparison.py`, `cli.py`	S
P1-18	**Model artifact labels.** Store the `selection: {metric, value, split: "val", step}` that justified selection, separately from `reported: test metrics`.	M9-7	`deployment/artifact.py`	XS
P1-19	`export` **includes** `git.json`, `environment.json`, **reports and predictions**; drops the empty `tables/`.	M12-9	`cli.py`	XS
P1-20	`build-intervention --apply` **preserves comments** (ruamel round-trip, or a separate `*.intervention.yaml` merged by the loader); records `kind: imbalanced`.	M12-4, M12-5	`cli.py`, `config/loader.py`	S
P1-21	**Factor runner stage 4 into one function** used by both the runner and the CLI (`run_evaluation_stage(run_dir, config, view, checkpoint)`), reading the run's own resolved config.	M12-3, M14-3	`experiments/runner.py`, `cli.py`	M
P1-22	**Rebuild reports from disk.** `build_report_ctx(run_dir)` plus `mtrain report --run`.	M10-2	`reports/experiment_report.py`, `cli.py`	S

Testing and supply chain

#	Fix	Gaps	Files	Effort
P1-23	**Cross-stage invariant tests**: val-only selection (spy), seed propagation, slices partition overall, artifacts agree, light venv doesn't import torch.	M13-1, M1-7	`tests/`	M
P1-24	**Tests for untested surfaces**: CLI (`typer.testing.CliRunner`), reports, `_prune_checkpoints`, `crash.json`, registry ordering, sweep.	M13-4, M10-16, M12-15, M6-10, M9-11	`tests/`	M
P1-25	**A fixture that exercises real logic**: varied episode lengths and ≥ 10 episodes, so length buckets and length slices actually run.	M13-2	`tests/conftest.py`	XS
P1-26	**CI, a pinned engine and a lockfile.** A two-repo checkout, `uv.lock`, an engine version range; light suite per push, smoke nightly on GPU.	M1-2, M1-3, M1-5, M13-5	`pyproject.toml`, `.github/workflows/`	M

Documentation that states something false (fix the doc, or make it true)

#	Doc claim	Reality	Action
D1	`FAILURE_ANALYSIS.md` §4: "predictions are deterministic given the checkpoint and the frame"	stochastic (README §2.3)	correct; store predicted and target vectors (M8-5)
D2	`FAILURE_ANALYSIS.md` §3: per-dimension per-episode via `episode_metrics`; "sorted by mse"	neither	correct, or implement (M8-6)
D3	`FAILURE_ANALYSIS.md` §6: recommendation "names a concrete next step"	fixed template	correct, or implement attribute analysis (M8-4)
D4	`EXPERIMENTS.md` §4: rebalanced limitation in every report	absent	P1-16 makes it true
D5	`EXPERIMENTS.md` §6: "necessarily" different test splits; `dataset.json` records evaluated IDs	not necessary; it doesn't	P0-1 makes the first moot; correct the second (M4-9)
D6	`EXPERIMENTS.md` §7: `compare` reports mean/std over seeds	not implemented	P1-17 makes it true
D7	`METRICS.md` §2: `evaluate_baselines` computes ratios	it doesn't	correct (M7-7)
D8	`METRICS.md` §6 and profile methodology: entropy "measures how evenly the observed range is used"	evenness among occupied bins	P1-9 makes it true
D9	`API_REFERENCE.md` §4.5/§9.3: use `evaluate --split val`	writes val into `test_overall.json`	P0-6 makes it safe
D10	`API_REFERENCE.md` §5.5: `success_tolerance` defines the proxy	never passed	wire it (one line, M7-12)
D11	`DEPLOYMENT.md` §5: predict invokes `select_action`	`predict_action_chunk`	correct (M9-9)
D12	`DEPLOYMENT.md` Table 2, registry docstring: metrics "justified selection"	test metrics	P1-18
D13	`loader.py` `DeploymentHealthError`: "any checkpoint/contract mismatch"	no comparison	P1-8
D14	`schema.py`: "enforced by `model_training.experiments.selection`"	no such module	point at `Trainer.on_step` (S3)
D15	`schema.py`: identity fields "not user-editable"	editable	enforce with a before-validator (M2-5)
D16	`interventions.py`: length buckets "same method as profile.py"	profile has none	correct (M4-4)
D17	`experiment_report.py`: "assembled entirely from saved run artifacts"	in-memory ctx	P1-22 makes it true
D18	`trainer.py` crash comment: catches SIGTERM	doesn't	P1-5 makes it true
D19	`data_fraction.py` / API Table 8: "isolates the quantity effect"	different test sets per point	P0-1 makes it true
D20	`configs/smolvla/so100_pickplace.yaml`: "point `--base` at it"	no `--base` flag	add the flag or delete the file (M2-9)
D21	`scripts/compare.py`: `--runs a b c`	invalid	fix the usage text (M12-11)
D22	README §2.2: "long SO-100 episodes are easier"	1–2 episodes per slice	downgrade (M7-2)
D23	README §2 point 3: "retrieval is a strong baseline" as the explanation	plausibly under-training (≈0.11 epoch)	add the epoch arithmetic (M15-2)
D24	`plot_results.py`: "reproducible from disk"	selection markers hard-coded	read `model.json` (M10-8)

| D25 | README Table 5: "A fraction of 0.0 is honoured as zero episodes" | the builder raises on zero episodes | correct, or implement P1-13 (M4-2) | | D26 | README Table 5: imbalanced = "under-representation of one regime" | removes the regime entirely | correct the wording (M4-3) | | D27 | README §3.3: "a process killed mid-write cannot leave a corrupt 'best' model" | verification is presence-only and can't run in a killed process | downgrade (M6-9) | | D28 | README §2.4: "scattered … points to a model or coverage limitation" as a finding | the only possible output at p90 with 4–5 episodes | reframe (M8-3) |

Effort: each doc correction is XS. Together they're about 2 hours, and they buy the most credibility per minute in this whole module.

16.4 P2: hygiene, robustness, cost, scale

Grouped, with files. Each is XS–S unless noted.

- **Config and identity:** a config hash separate from the run ID (M2-6); one home for defaults (M2-10); a discriminated-union intervention (M2-12); a single 0.5 default (M2-13); bounded weights (M2-15); dated resume path → a run reference (M2-11). Files: `config/schema.py`, `config/loader.py`, configs.
- **Tracking:** an atomic `mkdir(exist_ok=False)` (M3-4); `environment.json` adds driver, cuDNN, `transformers`, `pip freeze`, and renames compute capability (M3-7); delete `logs/` or tee stdout (M3-9); index rows gain `status` and `schema_version` (M3-10, M14-1); strict JSON types instead of `default=str` (M3-11); route all writes through `RunDir` (M3-12); default dirs not from `parents[3]` (M3-13).
- **Interventions:** validate explicit IDs and weights (M4-7); content-addressed builds to stop version proliferation (M4-10, M15-5, **M**); a clearer unsupported-variable error (M4-12); a stale docstring (M4-11).
- **Profile:** profile the train and test splits separately (M5-6); a real CV (M5-5); report out-of-range values and NULLs (M5-8); drop the duplicate metric (M5-9); a notes fix (M5-10); `to_dict` via `fields()` (M5-11).
- **Trainer:** resume checked against the dataset, verify before load (M6-6); optional optimizer-state save for long runs (M6-7); skip the duplicate `final` when aligned, verify `final`, move it inside `try` (M6-8); a content integrity check (M6-9); prune bookkeeping (M6-10); move the empty-train guard to the top (M6-13); report allocator vs reserved GPU memory (M6-12).
- **Evaluation:** rename per-episode "mse" (M7-5); drop `per_task` from `base.yaml` for single-task data (M7-6); a note when slices are skipped (M7-8); robust `frame_index` (M7-9); frame count in the error distribution (M7-11).
- **Feedback:** consume or drop `retraining_selection` (M8-8); configurable thresholds (M8-10); rename `affected_slice` (M8-9).
- **Deployment:** an observation-to-`Sample` adapter (roadmap); fix the latency breakdown (measured `total_ms`, sync, warm-up, percentiles) (M9-5); registry stages (M9-6, roadmap); identity fields (M9-8).
- **Reports and figures:** object-oriented matplotlib without the global backend (M10-10); guard plotting so it can't abort a completed run (M10-13); wire or delete the unused plot and matrix functions (M10-9); stop the dead `manifest.json` reads (M10-14).
- **Runner and CLI:** a GPU guard (M11-6); NN baseline and total time in `summary.json` (M11-7); relative `run_id` (M11-8); evaluate on `training.device` (M11-14); `crash.json`-aware `validate` and `analyze-failures` (M12-10); dedupe `_target_episodes` (M12-7); resolution-version check (M12-8); `inspect` default (M12-13); remove script wrappers in favour of `python -m` (M12-14); timestamp rollout records (M12-16).
- **Packaging:** a top-level `__init__.py` (M1-6); drop `allow-direct-references`, unused `pytest-cov` and dead `psutil` fallbacks (M1-8); promote the engine's private functions used by tests (M1-4); session-scoped read-only fixtures (M13-6).

- **Scale (defer until the trigger in Module 15):** a scheduler, a SQLite or MLflow index, multi-tenancy namespaces, a checkpoint retention policy and object store (M15-4, M15-7), measuring data loading and decode (M15-8).

16.5 The five-fix shortlist: about one day

Chosen for **maximum change in what the project can truthfully claim, per hour**. Each comes with its regression test.

Order	Fix	Why this one	Time
1	**P0-3 strict, fail-fast config** (`extra="forbid"`, `Literal`s, stage-0 name checks, cadence alignment)	closes the whole class of "typo silently changes the run" and "typo found after training"	1.5 h
2	**P0-2 real seed propagation** plus identity re-stamp plus a seed loop in the CLI	turns the documented multi-seed workflow from fake replicates into real ones; unlocks every variance claim	1.5 h
3	**P0-6 non-destructive re-scoring** (versioned rescore directory, split-named files)	stops published numbers from being overwritten; removes the doc-endorsed val-as-test footgun	1.5 h
4	**P1-1 predict-once slicing**	removes stochastic non-reconciliation *and* about 25–30% of each run's GPU time	2 h
5	**P1-15 + P1-16 + D1–D28 doc and report truthfulness pass** (git key, SmolVLA MAE, per-dimension MAE, limitations, every false doc sentence)	the cheapest credibility in the project; every reader-facing claim becomes true	1.5 h

Total: about 8 hours. Deliberately *not* on the list despite being P0: **P0-1, the frozen test split** (1–3 days, may touch the engine), and **P0-7, the frozen validation subset** (half a day). They're the first two items of the roadmap below, not afternoon work.

After this day you can say: "Config errors fail before the GPU starts, seeds are real, results are never overwritten, slices reconcile with the overall number, and every statement in the docs and reports matches the code."

16.6 Feature roadmap

Phase A — Validity (1–2 weeks). Everything a reviewer needs before believing a comparison.

1. Frozen test split; interventions on train only (P0-1).
2. Frozen stratified validation subset (P0-7); feedback on validation (P0-5).
3. Bootstrap CIs and paired per-episode comparisons on the shared split (P1-2).
4. Three seeds per configuration, as standard.
5. One length-bucket definition (P1-11), and train/test-split coverage in the profile.

Phase B — Experiments the data can answer (1–2 weeks of GPU-evenings).

1. **ALOHA at 5k–10k steps** (≈0.5–1 epoch) to test the under-training hypothesis (Module 15). This is the highest-information experiment available.
2. A batch-size check (8/16) for throughput, given 2–3 GB peak of 8 GB.

3. A nested data-efficiency curve on the frozen split (25/50/75/100%, 3 seeds each) with the 0% point = the base checkpoint.
4. A rebalanced run on SO-100's length bucket, compared *paired* against the full run on the shared test split.
5. A held-out-session or held-out-condition split, if the data supports one, as the first real generalization test (Module 5's content-leakage point).

Phase C — Reliability (about 1 week). The run state machine (P1-3), atomic I/O (P1-4), crash coverage (P1-5), invariant and CLI tests (P1-23 to P1-25), CI plus lockfile plus pinned engine (P1-26), the factored evaluation stage (P1-21), and reports rebuildable from disk (P1-22).

Phase D — Toward closed loop (open-ended).

1. An observation adapter (raw camera frames, joint vector, instruction → model input) so `DeployedModel` takes robot data, not `Sample`s (Module 9).
2. An action-chunk execution loop with control-rate handling.
3. A real latency benchmark: single sample, warmed up, synchronized, p50/p95/p99.
4. A simulator task matching one embodiment, *if one exists*. I haven't checked which LeRobot simulation environments match these datasets. That gives the first non-"unavailable" rollout number.

Phase E — Scale (only on the Module 15 triggers). Scheduler plus GPU lock; SQLite, DuckDB or MLflow index; content-addressed builds; retention policy and object storage; per-user namespaces.

16.7 Anti-roadmap: tempting changes that would make it worse

Tempting change	Why it's worse now	Flips when
Adopt MLflow or W&B	adds a service and a second source of truth while the real problem is comparability; dashboards would show incomparable runs side by side	a second concurrent researcher, or hundreds of runs
Build a FastAPI inference server	`predict` takes dataset `Sample`s; a server would expose a replay harness over HTTP	a robot controller in another process needs to call the model
Replace JSONL with Postgres	migration and operations cost for tens of rows; loses the "readable without our code" property	concurrent writers, or thousands of runs (and even then, DuckDB-over-JSONL first)
Move config to Hydra	the file-per-experiment record *is* the experiment log; Hydra's CLI overrides encourage unrecorded variants	hyperparameter sweeps become routine (and outputs auto-record the resolved config)
Mock the engine so tests run without it	the bugs live at the engine boundary; a mock encodes our assumptions (it would never reveal filter-then-split)	never for boundary tests; *yes* for pure logic (pruning, thresholds, rendering)
Add more slices (per joint, per phase, per object)	5 test episodes; slices are already 1–2 episodes; more slices mean more noise read as findings	the test set reaches tens of episodes, with CIs per slice
Report `success_rate` as task success, or estimate rollout success from offline MSE	breaks the project's central honesty contract	never, without closed-loop evaluation
Close the feedback loop automatically (apply the recommendation and retrain unattended)	the loop runs on test, can't say "no action", and can't produce its defect branch, so automation would overfit the test set at machine speed	feedback runs on val with an absolute gate, *and* a fresh untouched test set exists
Switch to a bigger or different model	the comparison protocol can't yet tell whether *this* model is undertrained	after Phase A and the ALOHA longer-training experiment
Replace the percentile threshold with an absolute one	loses ranking that's robust to normalization	never replace; *add* an absolute gate alongside
Delete the split-integrity check as tautological	it's free, documents the invariant, and guards against a representation change	never delete; strengthen it (P1-10)
A blanket `--force` to overwrite run directories	destroys the write-once guarantee for good runs too	allow `--force` only from `status: failed` (P1-3)
Rebalance by duplicating episodes on disk	heavier, less traceable, and the same effect as weights	never; weights are the honest mechanism
Always save optimizer state	about 3× checkpoint size on a disk-bound laptop	multi-hour or preemptible runs that need exact resume
Add a DAG framework	10–20-minute runs don't need caching or resume infrastructure	multi-hour runs or sweeps over ~10 runs; first try "skip a stage if its artifact is complete"

16.8 Honest gaps to own before an interviewer finds them

1. **The P0 list is long for a project with three published runs.** *Consequence:* the README's results predate the protocol fixes, so present them as "pipeline validation runs", not as findings, until P0-1, P0-2 and P0-7 land.
2. **P0-1 may need engine work.** A transform that preserves split attributes might not exist in the engine. *Consequence:* the most important fix isn't fully within this repository's control, and its estimate is uncertain.

3. **Estimates are untested.** None of these fixes has been attempted, and the engine isn't available here. *Consequence:* treat efforts as planning numbers.
4. **Some README numbers would change after the fixes** (any result scored on a moving test set, any selection from 32 frames). *Consequence:* be ready to say which published numbers are robust (continued-training on the identical split) and which aren't.

The shape to carry into the Interview Kit

```
P0 (claims invalid now):  frozen test split · real seeds · strict/fail-fast config · imbalanced no-op
                          guard · feedback on val · non-destructive rescoring · frozen val subset
P1 (wrong/lost/false):    predict-once · bootstrap CI · run state machine · atomic I/O · crash coverage
                          · health cross-check · coverage metric · compare by key + test-hash + seeds
                          · report truth · 28 doc corrections · invariant/CLI tests · CI + lockfile
P2 (hygiene/scale):       ~60 items, XS–S each, scale items deferred to Module 15 triggers
1-day shortlist:          config ▸ seeds ▸ rescoring ▸ predict-once ▸ truthfulness pass   (~8 h)
Roadmap:                  A validity ▸ B experiments (ALOHA 5–10k steps first) ▸ C reliability
                          ▸ D closed loop ▸ E scale-on-trigger
```

Drill — answer from memory, files closed

1. If you had one day, which five fixes would you make, and why that order?
2. Why is the frozen test split P0 but not on the one-day list?
3. *Why didn't you just* add MLflow? It would have given you experiment comparison for free.
4. *What breaks in production* if you automate the feedback loop today?
5. *Interviewer framing:* "If you had another month on this project, what would you do?"

Appendix A — Model answers to the drill

A1. The one-day list. "First, make the config strict and fail fast: forbid unknown keys, use literal types for the model, metric, slice and baseline names, and check all names before the run directory is created. That closes every 'typo silently changed the run' and 'typo found after training' bug in one change. Second, make `--seed` actually set the training seed and re-stamp the experiment ID, because today replicates share a seed and any variance I report is fake. Third, make re-scoring non-destructive, so `mtrain test` can't overwrite published numbers and `--split val` can't write into a file called test. Fourth, predict once and slice the stored outcomes. That makes slices add up to the overall number and saves about a quarter of each run's GPU time. Fifth, a truthfulness pass: fix the report bugs and correct the nearly thirty doc sentences that don't match the code. The order goes from silent-wrong-results to lost-results to waste to credibility, and each fix comes with a test that fails before it."

Why that answer lands: it's concrete, ordered by consequence with the reason stated, sized to a day, and every item has a test. That's what "prioritize" means.

A2. Why the frozen split isn't in the day. "It's the most important fix. Without it, every intervention is scored on a different test set, so the quantity-versus-coverage comparisons the project exists for aren't like-for-like. But it isn't an afternoon change. It reverses the order of operations from 'filter, then split' to 'split once, then intervene on train', and that may need a transform in the data engine that preserves split assignments, which I can't confirm exists. That's one to three days and partly outside this repo. So it heads the roadmap, and until it lands I'd present the intervention results as unvalidated."

Why that answer lands: it separates importance from effort, explains the dependency honestly, and says how results should be qualified in the meantime.

A3. Why not MLflow? "MLflow would have given me a comparison UI, but the problem wasn't displaying runs side by side. The runs weren't comparable: different test sets per intervention, fake seed replicates, no intervals. MLflow would have shown those incomparable numbers in a nicer table. My artifacts are plain files that DuckDB or pandas can read directly, so adding MLflow later as a sink is cheap. I'd add it when a second researcher starts running experiments concurrently or runs reach the hundreds. That's when a tracking server's locking and UI earn their operational cost."

Why that answer lands: it identifies the real problem the tool wouldn't solve, keeps the option open cheaply, and names the flip condition.

A4. Automating the feedback loop. "It would overfit the test set at machine speed. The feedback artifact is computed on test outcomes, so each automatic iteration would choose the next data intervention by looking at test errors and judge it on the same test errors. It also can't say 'no action', because a 90th-percentile threshold always flags 10% of frames, so it would intervene forever. And its 'data defect' branch is unreachable at these test sizes, so it would always attribute failures to the model. Automation is the right end state, but only after feedback runs on a validation evaluation, there's an absolute or baseline-relative gate on whether to act, and there's a fresh test set the loop has never seen."

Why that answer lands: it combines three independent findings into one decisive argument and states exactly what has to be true before automation is safe.

A5. "Another month?" "Week one: validity. Freeze one test split and apply interventions only to train, propagate real seeds, select checkpoints on a frozen stratified validation subset, run feedback on validation, and add bootstrap intervals. Weeks two and three: the experiments that change conclusions. First, ALOHA at five to ten thousand steps, because the current run saw about a ninth of an epoch, and I think 'SmolVLA loses to retrieval' is really 'SmolVLA is undertrained'. Then a nested data-efficiency curve with three seeds

per point, and a rebalanced run compared paired against the full run on the shared split. Week four: reliability, meaning a run state machine, atomic writes, CI with both repositories and a lockfile, plus an observation adapter so the deployment path takes real robot inputs. I would *not* spend the month on MLflow, a server or a bigger model. None of those helps until two numbers from this system can be compared."

Why that answer lands: it's a sequenced plan where each week's purpose is stated, the headline experiment is justified by a calculation, and it explicitly rejects the tempting distractions. That shows judgement, not just a to-do list.

Module 16 Summary — the one page to revise

What this module is. About 200 gap entries consolidated into 7 P0 fixes (comparability and truthfulness of results), 26 P1 fixes plus 28 doc corrections, and about 60 P2 items. Then a one-day shortlist (about 8 hours), a five-phase roadmap, and a fifteen-row anti-roadmap with flip conditions.

Five claims to make cold

1. The P0s are all about comparability: frozen test split, real seeds, strict config, no imbalanced no-op, val-driven feedback, non-destructive re-scoring, a frozen val subset.
2. One day buys: fail-fast config, real seeds, non-destructive re-scoring, predict-once slicing, a truthfulness pass.
3. The frozen test split is the most important fix and not a one-day fix. It may need engine work.
4. The first experiment after the fixes is ALOHA at 5–10k steps, testing under-training (≈0.11 epoch today).
5. Not now: MLflow, a server, Hydra, mocking the engine, more slices, automated feedback, a bigger model, each with its flip condition.

Decisions

Chose	Over	Because	Cost
Prioritize by effect on conclusions	by ease or by module	wrong results matter most	P0s include a multi-day fix
Fix docs or make them true	leave docs as aspirations	credibility per minute	28 small edits
Validity before experiments before reliability	infrastructure first	the bottleneck order (Module 15)	reliability gaps persist for a few weeks
Add an absolute gate beside the percentile	replace it	keep normalization-robust ranking	two thresholds to explain
Defer scale items to triggers	build now	C9, short runs	rework later, by design

Volunteer unprompted: the README's three runs are pipeline validation until P0-1/2/7 land; P0-1 depends on the engine; continued training on an identical split is the robust result.

Numbers and names: 7 P0 · 26 P1 · 28 doc corrections · ~60 P2; shortlist about 8 h (1.5 + 1.5 + 1.5 + 2 + 1.5); P0-1 effort L (1–3 days); ALOHA target 5–10k steps ≈ 0.5–1 epoch.

Sentences worth memorizing

- "My P0 list is about one thing: making two numbers from this system comparable."
- "In one day I'd make config errors fail before the GPU starts, make seeds real, make re-scoring non-destructive, predict once, and make every doc sentence true."
- "Automating the feedback loop today would overfit the test set at machine speed."
- "Until the test split is frozen, the intervention results are pipeline validation, not findings."

Module 17 — Interview Kit

Files covered: none new. This module packages Modules 1–16 for speaking. Every claim points back to a module (`M4 §4.5`) and its model answer (`M4 A5` = Module 4, Appendix A, answer 5).

Where we are: The manual so far is for understanding. This module is for *delivery*: what to say in 30 seconds, 2 minutes and 5 minutes; how to answer the 45 questions you're most likely to get; how to tell the behavioural stories; which phrases to avoid; what to ask back; and every number and term on one card.

The one-line thesis: The project's strongest story isn't "I fine-tuned a VLA". It's **"I built an evaluation harness honest enough to tell me my VLA tied a nearest-neighbour lookup, and then I audited the harness itself and found where its own comparisons weren't yet valid."**

> **Authorship note before anything else.** `pyproject.toml` lists Kushagra Srivastava as author and a `kushagrathisside/...` repository URL. README line 298 credits the SO-100 fine-tuning runs to `AdityaKishore0698` and the ALOHA run to `chytra3` (M1 §1.4, gap 11). Decide your exact framing *before* the interview, and use it consistently. For example: "a collaborative project. The codebase design is X's; I ran the SO-100 experiments, and I did this audit." Everywhere below, "I" means "the project". Swap in "we" or "the author" wherever that's the truthful version.

17.1 Prerequisites — the absolute basics

- **Pitch.** A self-contained explanation sized to a time box. A 30-second pitch is *what and why*; 2 minutes adds *how and the result*; 5 minutes at a whiteboard adds *architecture and trade-offs*.
- **Answer skeleton.** The three or four beats an answer must hit, in order. Memorize skeletons, not scripts.
- **STAR.** Situation, Task, Action, Result: the standard shape for behavioural answers. Add a fifth beat, *Reflection* ("what I'd do differently").
- **Signal.** What an answer tells the interviewer about you beyond its content: judgement, honesty, depth, ownership.
- **The honest downgrade.** Volunteering the limit of a claim before you're asked. It's the highest-signal move in a technical interview, because it shows you know where the edges are.

17.2 Pitches

30 seconds

> "It's an experiment and evaluation layer for fine-tuning a vision-language-action robot policy, SmolVLA, on real LeRobot datasets. The organizing idea is that the model layer never owns the data. Every data intervention, like halving the dataset or dropping a regime, becomes a lineage-tracked version in a separate data engine, so every checkpoint traces back to raw episodes. On SO-100 pick-and-place, fine-tuning reached 2.6% of the mean-action baseline's error. But it exactly tied a nearest-neighbour lookup over robot state, and on bimanual ALOHA retrieval was twice as good. The harness was built to surface results like that instead of hiding them."

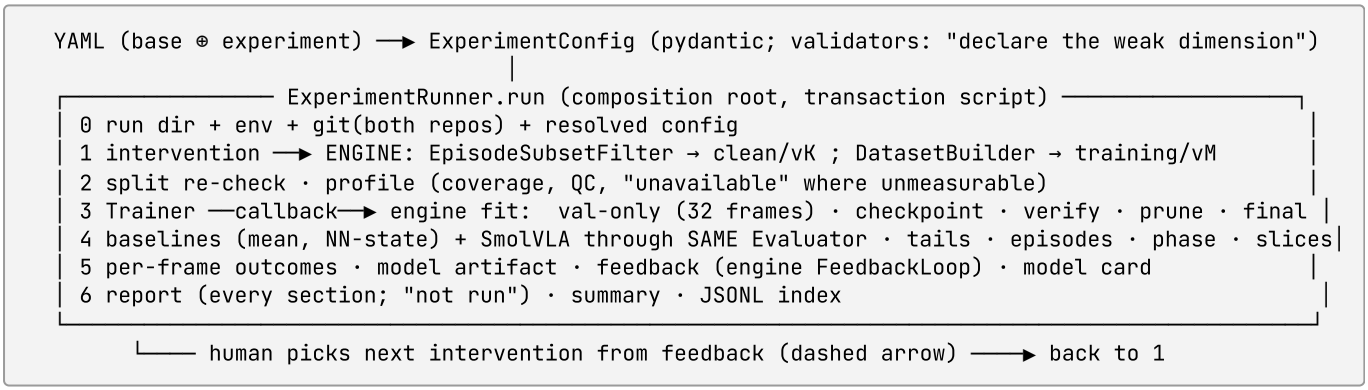
2 minutes

Beats, in order:

1. **The problem (15 s).** VLA fine-tuning results are easy to overclaim: offline error gets read as task success, data changes aren't traceable, and baselines are weak or missing.
2. **The design principle (20 s).** The model layer never owns data. Interventions are engine transforms, data is named by immutable version, and every run directory records the config, both repos' commits and the environment.
3. **The pipeline (30 s).** One command: build the intervention version, re-check the splits, profile, fine-tune with validation-only checkpoint selection, score against mean-action and nearest-neighbour through the same evaluator, decompose the error (tails, per episode, per phase, slices), produce a feedback artifact, then report and model card. Anything it can't measure, like rollout, it records as "unavailable".
4. **The results (25 s).** SO-100: test MSE 0.0290, then 0.0239 after continued training (an 18% improvement on the identical test split, outside the roughly 5% inference noise), equal to nearest-neighbour. ALOHA: 0.216 vs retrieval 0.106.
5. **The honest reading (20 s).** Offline, on single-setup data, robot state almost determines the action. And the ALOHA run saw about a ninth of an epoch, so 'undertrained' is the first hypothesis.
6. **What I'd fix next (10 s).** Interventions currently re-split the data, so they're scored on different test sets. Freezing one test split and making seeds real come first.

5 minutes at a whiteboard

Draw this, top to bottom, talking through each box (M1, M11, M14 §14.2):



Then spend the remaining two minutes on **three trade-offs** (pick based on the interviewer):

- **Lineage vs comparability.** Filter-then-split gives perfect provenance, but a different test set per intervention. The fix is split once, intervene on train only (M4 §4.5).
- **Validation cost vs selection quality.** 32 frames is cheap, but val MSE is about 13× below test. The fix is a frozen stratified subset (M6 §6.4).
- **Honesty markers vs completeness.** Rollout is "unavailable", and so is visual diversity. Nothing is fabricated, so the reports are incomplete by design (M7, M10).

17.3 Question bank

Format: **Q** · skeleton (the beats) · → pointer.

A. Project and motivation

1. **What does this project do, in one sentence?** Experiment, evaluation and feedback layer for SmolVLA on versioned robot data; the model layer never owns the data. → M0, M1 A5
2. **Why build this instead of using LeRobot's training script?** LeRobot trains; this layer adds lineage-tracked interventions, baselines through the same evaluator, decompositions and honesty markers, and records it all per run. Admit that the step itself is the engine's. → M1 §1.3, M1 A3
3. **What did *you* specifically do?** Use your authorship framing; name concrete parts (the SO-100 runs, this audit, whichever modules are yours). → note above, M1 gap 11
4. **What's the headline result?** SO-100 0.0239, equal to NN; ALOHA 0.216 vs NN 0.106; 2.6% of mean-action; 18% from continued training. → M7 §7.2, README
5. **Is the model any good?** Learned the distribution; doesn't beat retrieval offline; probably undertrained on ALOHA; closed loop unmeasured. → M7 A5, M15 A2

B. Data and interventions

6. **Why version data instead of passing episode IDs to training?** Immutable identity, lineage back to raw, recorded reasons; cost: catalog churn and the moving test set. → M4 A2
7. **Walk me through what `reduced` does to the catalog.** Seeded draw over sorted IDs → `EpisodeSubsetFilter` → `clean/vK` → `DatasetBuilder` → `training/vM`; lineage records kept and dropped IDs; tested. → M4 A1
8. **Are your intervention results comparable to the baseline?** No: filter-then-split means a different, smaller test set; the fix is to split once and intervene on train. → M4 A4
9. **Why episode length as the imbalance variable?** SO-100 has one task and the schema has no variation field; length is the only varying attribute; terciles. → M4 §4.3
10. **What does "imbalanced" actually do?** Removes the regime entirely, so it's an out-of-distribution test, not under-representation; its test set lacks that regime. → M4 §4.4
11. **How does rebalancing work, and what can't it do?** Sampling weights on existing episodes, nothing duplicated; can't close a coverage gap. → M4 §4.5
12. **How do you prevent train/test leakage?** Episode-level splits, a single label per episode, re-check, val-only selection; content-level leakage is unchecked, and NN strength is the evidence. → M5 A5
13. **Does your split check actually catch anything?** The overlap branch is near-tautological; it misses unassigned and empty splits; make `ok` mean a complete partition. → M5 A1, A2

C. Training

14. **What do you own in the training loop?** A callback into the engine's `fit`: val-only validation, checkpoint, verify, select, prune, final, crash marker, GPU release. → M6 A1, M1 A3
15. **How do you choose the checkpoint, and how confident are you?** Min val MSE among verified checkpoints; 32 frames of a stochastic policy, val 13× below test; right on SO-100 because the curve was still falling. → M6 A5
16. **Explain the 0.0018 vs 0.0239 gap.** Likely first frames of one episode (easiest phase), winner's curse, 5-episode splits; unverified engine frame order. → M6 A4
17. **What happens if the job is killed?** SIGTERM doesn't raise, so no `crash.json`; periodic checkpoints survive; resume restarts warmup; retry blocked. → M6 A3
18. **Why not save optimizer state?** Disk (Adam moments about 3× size); a fresh warmup on resume; the engine reportedly supports it; flips for preemptible long runs. → M6 §6.3
19. **How much data does a run actually see?** 0.26 epoch SO-100, 0.11 ALOHA (per-frame assumption). → M15 A2

20. **Why batch size 4?** Motivated by 8 GB, but peak was 2.2–3.0 GB, so it's conservative; try 8–16. → M15 §15.4

D. Evaluation and metrics

21. **Why two baselines, and what's nearest-neighbour exactly?** Mean-action is the floor; NN retrieves by robot state, no images or language; same evaluator call. → M7 A1
22. **What does tying NN tell you?** Offline, state nearly determines action on single-setup data; closed loop is where vision should matter. → M7 A5
23. **Why report tails and per-phase error?** Control fails at rare frames; continued training cut the max by 32%; error peaks mid-episode. → M7 §7.3–7.4
24. **How do your slices work, and what's wrong with them?** Each slice re-runs stochastic inference, so they don't reconcile, and it costs about 3× compute; predict once and slice. → M7 A2
25. **Would you defend "long episodes are easier"?** No: 1–2 episodes per slice, 5% inference noise, about 25% per-episode spread. → M7 A3
26. **How do you know a difference between runs is real?** Mostly you don't yet: single seed, 5 test episodes, no intervals; only continued training (same split, 18% vs 5% noise) is defensible. → M10 A5
27. **What's `success_rate`?** Fraction of frames with MAE below 0.1; a proxy, never called success; the tolerance isn't even wired to config. → M7 §7.1, M2 §2.6

E. Feedback loop

28. **Walk me through your data flywheel. Does it work?** Plumbing yes; as a decision procedure, no: always recommends action, defect branch unreachable, runs on test. → M8 A5
29. **Why a percentile threshold?** Robust to model and normalization; can't say "nothing's wrong"; add an absolute gate. → M8 A1
30. **Why is the "data defect" explanation never produced?** Max per-episode failure rate ≈ 0.50 (SO-100) and 0.40 (ALOHA) vs a > 0.5 rule. → M8 A2
31. **What's wrong with running failure analysis on test?** Adaptive overfitting: decisions must use val; test is touched once. → M8 A3

F. Systems and design

32. **Draw the architecture and the dependency graph.** Four layers, acyclic; runner as composition root; CLI the one violation; `rde` under everything. → M14 A1
33. **Name design patterns you used.** Template Method via callback, Transaction Script, Specification validators, Façade, explicit "unavailable". → M14 A2
34. **Weakest part of the low-level design?** Typed at the edges, dicts in the middle, schemaless on disk; the `n/a` git-key bug as proof. → M14 A5
35. **Why JSONL and run directories instead of MLflow?** Serverless, durable, query with DuckDB; durability is on you; flips at the second user. → M3 A2, M16 A3
36. **What happens if two runs start at once?** TOCTOU on the run directory, catalog version race, torn JSONL, GPU contention. → M14 A4
37. **How do you make a run reproducible six months later?** Resolved config plus version lineage plus both commits plus env; gaps: no lockfile, dirty diffs, guessed engine path, git-ignored artifacts, stochastic inference. → M3 A5
38. **What breaks first if this scales?** Comparison validity, then eval time, then disk; an evidence-limited system. → M15 §15.5, A5
39. **How did you deploy it, and what's the latency?** Not to a robot; a local loader that's never executed; 47–50 / 86 ms per frame is evaluator throughput, not control latency. → M9 A5

40. **Is your health check sufficient?** It checks presence and well-formedness, not compatibility; normalization checked on the wrong object. → M9 A1

G. Process, testing, judgement

41. **How is it tested?** 56 tests, 52 torch-free, real engine, no mocks; lineage verified end to end; no cross-stage invariant tests. → M13 A1, A2
42. **Why not mock the engine?** Mocks agree with your assumptions; use real engine for the boundary and fakes for pure logic. → M13 A3
43. **Which decisions came from constraints, and which were arbitrary?** The capstone: GPU, no robot, tiny data, schema, engine; arbitrary thresholds; C3 and C6 not carried through. → M13 A5, M13 §13.6
44. **If you had one day?** Strict config, real seeds, non-destructive re-scoring, predict-once, truthfulness pass. → M16 A1
45. **Another month?** Validity, then ALOHA 5–10k steps, then a nested learning curve, then reliability; not MLflow, a server or a bigger model. → M16 A5
46. **What would you not build?** The anti-roadmap: automated feedback, more slices, a server; each with its flip. → M16 §16.7
47. **What's the most surprising thing you learned?** Evaluation took as long as training; under one epoch; the harness's own comparisons were invalid. → M15 §15.3–15.4

17.4 Behavioural framings (STAR + Reflection)

Hardest bug. *The action-queue bug* (engine-side, documented in the smoke test and API_REFERENCE).

- **S:** evaluation scores looked plausible but slightly off.
- **T:** SmolVLA's `select_action` keeps a stateful queue of predicted action chunks for one continuous rollout.
- **A:** in batched offline evaluation, that queue replayed stale actions across unrelated batches. The fix was switching to `predict_action_chunk` (a fresh forward pass per call), and the regression test used a deliberately non-divisor batch size (3) to force varying batch sizes.
- **R:** evaluation became correct.
- **Reflection:** "stateful APIs designed for control loops are dangerous in batch evaluation; now I check whether any model call carries hidden state."
- *If this wasn't your fix, use the audit instead:* "the hardest bug I *found* was that interventions re-split the data, so every comparison used a different test set." (M4 A5)

What you'd do differently. "Design the comparison protocol before the pipeline. I'd freeze one test split on day one, make the seed flow through to training, and put a bootstrap interval on every number. The mechanisms (lineage, provenance, honesty markers) were right; the statistics weren't carried through." (M13 §13.6, last table)

Proudest. "That the system refuses to fabricate. Rollout is recorded as unavailable with a reason, visual diversity is 'unavailable' instead of a fake score, every report section renders as 'not run' rather than disappearing, and the headline result is a tie with a nearest-neighbour baseline, reported as a tie." (M7, M10)

A trade-off you're unsure about. "Putting the model adapter and its training step inside the data engine. It keeps feature shapes and normalization in lockstep with the engine's training contract, which is why cross-embodiment loading worked. But it inverts the dependency: the data layer owns the model. With one

model family I think it was right. With a second I'd split adapters into their own package, and I'm not sure where the line is." (M1 §1.3)

Conflict or disagreement (if asked). Use the docs-vs-code findings: "the docs said selection was enforced in a module that doesn't exist, and that predictions were deterministic. I treated docs as claims and verified each against the code, and found nearly thirty that needed correcting." (M16 §16.3 D-table)

Failure. "My one-day plan would have been to add MLflow. Auditing showed the runs weren't comparable, so a dashboard would only have displayed incomparable numbers more nicely. I changed the plan to fix the protocol first." (M16 A3)

17.5 Phrases that hurt you, and what to say instead

Don't say	Why it hurts	Say instead
"The model achieves 50–57% success."	`success_rate` is a tolerance proxy, offline	"56.9% of test frames are within 0.1 mean absolute error, an offline proxy, not task success."
"SmolVLA beats the baselines."	it ties NN on SO-100 and loses on ALOHA	"It's at 2.6% of mean-action error and ties a state nearest-neighbour lookup."
"I wrote the training loop."	`fit` lives in the engine	"The engine owns the gradient step; this layer owns validation, selection, checkpointing and crash handling around it."
"An independent check guarantees no leakage."	the overlap branch can't fire; content leakage unchecked	"Splits are episode-level and re-checked for disjointness; content-level leakage between same-session episodes isn't checked."
"Long episodes are easier."	1–2 episodes per slice	"The slice infrastructure works; five test episodes are too few for slice conclusions."
"We ran three seeds." (if using `--seed`)	`--seed` doesn't seed	"Single seed. The seed flag currently only names the directory, which I'd fix first."
"It's deployed."	replay harness, never executed	"There's a local loader with health checks; it hasn't been run against a robot, and it takes dataset samples."
"Inference latency is 48 ms."	evaluator throughput per frame, batch 16	"About 48 ms per frame of evaluator throughput at batch 16, excluding data loading. Not control-loop latency."
"Validation MSE was 0.0018."	32 frames, about 13× below test	"Selected on validation; the honest number is test MSE 0.0239."
"The flywheel automatically improves the data."	human-gated, runs on test, always recommends action	"It produces ranked failures and a recommendation for a human; as a decision rule it needs to move to validation."
"It's fully reproducible."	no lockfile, dirty diffs, stochastic	"Every input is recorded; outputs reproduce within inference noise, a few percent."
"Coverage is 0.9."	occupied-bin evenness	"That metric is evenness among occupied bins. I'd renormalize it by the full bin count."
"Retrieval is just a strong baseline."	ALOHA ≈ 0.11 epoch	"Retrieval is strong on single-setup data, and the ALOHA run was undertrained at about a ninth of an epoch."
"No bugs that I know of."	there are ~200 gaps	"Here are the three I'd fix first, and why."

17.6 Questions to ask them

1. "How do you evaluate policies before closed-loop testing, and how much do you trust offline metrics to rank them?"
2. "When you change a dataset, how do you track which models trained on which version?"

3. "What's your split policy for robot data: by episode, session, scene or operator?"
4. "How many seeds do you typically run before you believe a difference?"
5. "Where does the training loop live relative to the data pipeline in your stack, and who owns the model adapters?"
6. "What's your bottleneck today: GPU hours, data collection, or evaluation throughput?"
7. "How do you handle stochastic policies (diffusion or flow matching) when comparing checkpoints?"
8. "What would a successful first three months look like in this role?"

17.7 The ten sentences that carry the project

1. "The model layer never owns the data. Every intervention is a lineage-tracked dataset version, never a local copy."
2. "Offline, on single-setup data, robot state almost determines the next action. That's why a state nearest-neighbour lookup tied a VLA."
3. "The harness's job was to tell me the truth, and it did: the baseline was that strong."
4. "Test never influences checkpoint selection; the trainer's only validation call hard-codes the val split."
5. "I filter before I split, so each intervention got its own test set. The fix is to split once and intervene on train only."
6. "My ALOHA run saw about a ninth of an epoch; 'undertrained' comes before 'retrieval is better'."
7. "Every number I report is offline, and the code records rollout as unavailable rather than invent one."
8. "Evaluation took about as long as training, and half of it was redundant re-inference."
9. "I honoured the constraints in the mechanisms and didn't carry them through to the statistics."
10. "My P0 list is about one thing: making two numbers from this system comparable."

17.8 Quick-reference card: every number

Datasets

	SO-100 pick-place	ALOHA static coffee
Source	`lerobot/svla_so100_pickplace`	`lerobot/aloha_static_coffee`
Robot / action dims	single arm, 6	bimanual, 14 (state 14 joint + 14 effort = 28)
Cameras / fps	2 / 30	4 / 50 (AV1)
Episodes after QC (train/val/test)	48 (38/5/5); raw 50, dropped [9, 31]	40 (32/4/4)
Test frames	1,908	4,400 (all test episodes 1,100 frames)
Episode length	326–475 frames, ≈ 13 s	—
Tasks	1	1 (per config comment)

Results (README Tables 2–3)

Run	Steps	Test MSE	/ mean-action	NN MSE	within 0.1	Train time	Peak GPU
`baseline_full` (SO-100)	1,000	0.0290	3.2%	0.0239	49.8%	324 s	2.2 GB
`baseline_full_extended`	+1,000	0.0239	2.6%	0.0239	56.9%	385 s	2.2 GB
`aloha_coffee_baseline`	1,000	0.2157	21.4%	0.1062	0.05%	576 s	3.0 GB

Mean-action MSE 0.908 (SO-100), 1.010 (ALOHA). SO-100 2,000-step tails: median 0.089, p90 0.174, p99 0.272, max 0.375 (from 0.553). Per-joint MAE 0.063 (shoulder pan) → 0.157 (gripper). Phase: start 0.051, mid 0.144, end 0.049. Per-episode MAE 0.085–0.111 (SO-100), 0.296–0.405 (ALOHA). Val minimum 0.0018 at step 1,800. Re-scoring noise up to 5% (0.0305 vs 0.0290; 0.0232 vs 0.0239). Inference 47–50 ms/frame (SO-100), 86 ms (ALOHA) at batch 16. Host memory 3.0–3.3 GB. NN confidence–error correlation ≈ −0.79.

Config (`configs/base.yaml`): splits 0.8/0.1/0.1, seed 42, quantile · steps 1,000 · batch 4 · lr 1e-4 · wd 1e-5 · warmup 50 · clip 1.0 · ckpt/val every 100 · `val_batches` 8 (32 frames) · keep 3 · training seed 0 · eval batch 16 · tolerance 0.1 · slices `[per_task, per_length_bucket]` · baselines `[mean_action, nearest_neighbor]` . Extended run: lr 3e-5, warmup 20, `resume_from step_1000` . Schema defaults differ: steps 200, ckpt 50, eval batch 32.

Derived (M15): ≈ 3.1 steps/s; epochs seen ≈ 0.26 (SO-100), 0.53 (extended), 0.11 (ALOHA); eval ≈ 45% (SO-100) and 57% (ALOHA) of run time; run ≈ 10–11 min (SO-100), ≈ 22 min (ALOHA); checkpoints ≈ 4–9 GB per run (estimated).

Thresholds: feedback p90, ≥ 3 failures per episode, concentration > 0.5; terciles 33.3/66.7, ≥ 3 episodes; 10 histogram bins; oversample 3.0; split-sum tolerance 1e-6; ID digest 10 hex characters.

Code and tests: about 2,700 lines in `src` ; 11 CLI commands; 56 tests (49 unit, 3 integration, 4 smoke), 52 light; 2 custom exceptions; 4 index files; 7 run subdirectories; 5 checkpoint files.

Environment: RTX 4060 Laptop 8 GB; ~10 GB VM RAM; lerobot 0.4.4; torch 2.10 (CUDA build); driver 592.82; CUDA 13.1; Python ≥ 3.10 declared, 3.11 used.

17.9 Glossary

Term	Meaning (as used in this project)	Module
Action chunk	sequence of future actions predicted per call by SmolVLA	M1, M9
Adapter (model input)	converts dataset samples to model tensors (`StateAdapter`, `VisionLanguageAdapter`)	M1, M7
Adaptive overfitting	test set biased by decisions made from looking at it	M8
Anti-roadmap	changes that would make the system worse now, with flip conditions	M16
Atomic write	write to temp file, then `os.replace`; never half-written	M3
Banker's rounding	Python `round` rounds .5 to even	M4
Behaviour cloning	supervised regression from observation to demonstrated action	M1
Bootstrap (over episodes)	resampling test episodes to get a confidence interval	M7
Catalog	engine metadata store of versions and episodes	M1
Characterization test	test pinning current (possibly wrong) behaviour	M13
Checkpoint	saved weights plus processors plus metadata at a step (5 files)	M6, M9
Clean version (`clean/vN`)	output of transforms (normalization, episode filtering)	M1, M4
Closed-loop rollout	policy controls a robot or simulator; measures task success	M1
Composition root	the one place collaborators are wired (`ExperimentRunner`)	M14
Content-addressed	identified by a hash of its inputs	M2, M15
Contract (training)	dict describing modalities: plane, shape, normalization	M1
Coverage (profile)	normalized entropy of a 10-bin histogram; actually evenness among occupied bins	M5
Deep merge	nested dicts merge; scalars and lists replace	M2
DoF	degrees of freedom: independently controllable joints	M1
Embodiment	the robot body: kinematics, cameras, action space	M1
Engine (`rde`)	Episodic Data Engine, the sibling repo owning data, `fit`, evaluator, feedback	M1
Episode	one recorded demonstration; the unit of splits and interventions	M1
Epoch	one full pass over the training data	M15
Evenness (Pielou)	entropy normalized by occupied categories	M5
External plane	modality stored outside the frame table (images, video)	M1
Fail fast	validate before expensive work	M11
Feedback artifact	ranked failing episodes, explanations, recommended next action	M8
Flow matching	generative method; makes SmolVLA's predictions stochastic	M1
Frame	one timestep: images, state, action	M1
Inline plane	numeric modality stored with the frame (state)	M1
Intervention	a deliberate change to the training data (full, fraction, reduced, imbalanced, rebalanced, custom)	M2, M4
JSONL	one JSON object per line; append-friendly	M3
Lineage	recorded chain of operations producing a version	M1, M4
LeRobot	Hugging Face robot-learning library and dataset format	M1
Mean-action baseline	always predicts the training mean (MSE ≈ 1 normalized)	M7

Term	Meaning (as used in this project)	Module
Nearest-neighbour baseline	returns the action of the closest training *state*	M7
Normalized units	per-dimension (x − mean)/std	M1
Offline evaluation	predicted vs demonstrated action on recorded data	M1
Orphaned version	catalog version left by a failed run	M4
Percentile threshold	flags the top (100 − p)% of frames by error, always about that fraction	M8
Predict-once slicing	infer once, then group outcomes by slice	M7
Quantile split mode	exact split sizes (inferred)	M2
Rebalanced	upweight episodes by sampling weight; no exclusion	M4
Replicate	a run differing only by random seed	M11
Resolved config	merged config with all defaults, persisted per run	M2
Run directory	`runs/<date>/<name>/seed_<n>/`	M3
Slice	evaluation subset by task or length bucket	M7
SmolVLA	small VLA from LeRobot; base `lerobot/smolvla_base`	M1
`success_rate`	fraction of frames with MAE < 0.1; a proxy	M7
Tautological check	a check whose failure can't be represented by its input	M5
Template Method via callback	engine loop with an injected `on_step` hook	M6, M14
Terciles	33.3/66.7 percentile cut into short/medium/long	M4
Training version (`training/vN`)	built dataset with splits and weights	M1, M4
Transaction Script	one procedure running a whole operation	M11
TOCTOU	race between checking a condition and acting on it	M3
"Unavailable" marker	explicit record that something wasn't measured, with a reason	M5, M10, M12
Validation-only selection	checkpoints compared on val, never test	M6
VLA	vision-language-action model	M1
Winner's curse	the minimum of noisy estimates is biased low	M6

17.10 Honest gaps to own before an interviewer finds them

1. **Authorship ambiguity** (`pyproject.toml` vs README credits). *Consequence:* any inconsistency in "I" vs "we" will be noticed. Settle the framing in advance.
2. **The three published runs predate the protocol fixes.** *Consequence:* present them as pipeline validation; the robust result is continued training on the identical split.
3. **Nothing in this manual was executed.** The engine wasn't available, so engine-side behaviour (Evaluator frame order, sampling unit, version-allocation atomicity) is inferred. *Consequence:* say "I didn't verify X" rather than guessing, if pressed.

The shape to carry into the Data Provenance & Demo module

```
30 s: what + principle + tie result     2 min: problem ▸ principle ▸ pipeline ▸ results ▸ honest read ▸ next fix
5 min: whiteboard runner stages + 3 trade-offs (lineage vs comparability · val cost vs selection · honesty vs completeness)
47 questions → module + appendix pointers      6 behavioural stories (STAR + reflection)
14 phrase swaps · 8 questions to ask · 10 carrying sentences · numbers card · glossary
```

Drill — answer from memory, files closed

1. Give the 30-second pitch, out loud, and time it.
2. *Why didn't you just* report the 56.9% "within tolerance" as the success rate? It's a bigger number.
3. Recite every number on the results table without looking.
4. *What breaks in production* if you say "we ran three seeds" and the interviewer asks to see the configs?
5. *Interviewer framing:* "Tell me about this project."

Appendix A — Model answers to the drill

A1. The 30-second pitch. See §17.2. The test: under 35 seconds, mentions *lineage*, *the tie with nearest-neighbour* and *honesty markers*, and contains no unqualified "success".

Why that answer lands: in 30 seconds it gives a principle, a result and a stance, which is more memorable than a feature list.

A2. Why not call 56.9% the success rate? "Because it isn't one. It's the fraction of test frames whose mean absolute action error is under 0.1, computed offline against recorded demonstrations. A policy can be within tolerance on most frames and still fail the grasp, and in closed loop small errors compound. The whole system is built so that offline accuracy is never presented as task success: the metric is labelled a proxy everywhere and rollout is recorded as unavailable. Calling it success would be the one claim that undoes everything else the project gets right."

Why that answer lands: it explains the technical difference, ties it to the project's design, and shows you won't trade credibility for a bigger number.

A3. Reciting the numbers. SO-100: 0.0290 → 0.0239, NN 0.0239, mean-action 0.908, 3.2% → 2.6%, 49.8% → 56.9% within tolerance, 324 s and 385 s, 2.2 GB. ALOHA: 0.2157, NN 0.1062, mean 1.010, 21.4%, 0.05%, 576 s, 3.0 GB. Splits 38/5/5 and 32/4/4; 1,908 and 4,400 test frames.

Why that answer lands: fluency with your own numbers is the cheapest evidence you did the work.

A4. "We ran three seeds", then the configs. "They'd see that the `--seed` flag only changes the run directory name. The training seed in each resolved config is the same, zero. So the 'three seeds' are three runs with one seed, differing only through GPU nondeterminism and sampling noise, and any spread is an underestimate. That's why I'd never say it. The honest statement is 'single seed', followed by 'and the seed flag doesn't seed yet; that's the second fix on my list.'"

Why that answer lands: it shows you know exactly how the claim would collapse, and that the safe phrasing is already part of your vocabulary.

A5. "Tell me about this project." Deliver the 2-minute pitch (§17.2), then stop and offer a direction: "I can go deeper on the data-versioning design, the evaluation and what the nearest-neighbour tie means, or the flaws I found auditing it. Which is most useful?"

Why that answer lands: it's bounded, it has a result and a self-critique, and it hands control back with three strong branches, each of which you've prepared.

Module 17 Summary — the one page to revise

What this module is. The speaking layer of the manual: three pitches, 47 questions with skeletons and pointers, six behavioural stories, phrase swaps, questions to ask, ten carrying sentences, a numbers card and a glossary.

Five claims to make cold

1. The model layer never owns the data; interventions are lineage-tracked versions.
2. SmolVLA ties state-NN on SO-100 (0.0239) and loses on ALOHA (0.216 vs 0.106).
3. Every number is offline; rollout is recorded as unavailable.
4. Continued training gave 18% on the identical split, outside the 5% noise. It's the robust result.
5. The fixes I'd make first all serve one goal: two numbers from this system being comparable.

Decisions (as you'd narrate them)

Chose	Over	Because	Cost
Lead with the tie	lead with the 2.6% ratio	the honest baseline	less flattering
Volunteer flaws	wait to be asked	the strongest signal	longer answers
"Proxy" every time	"success"	central honesty contract	smaller-sounding numbers
Offer three deep-dive branches	monologue	interviewer chooses, you're prepared	none

Volunteer unprompted: authorship framing; the published runs predate the fixes; the engine-side behaviours you couldn't verify.

Numbers and names: see the §17.8 card. Memorize the results table and the derived epoch fractions.

Sentences worth memorizing: §17.7, all ten.

Module 18 — Data Provenance & the Demo

Files covered: `README.md` §1, §2.4, §4–5 and §10 (as claims), `docs/figures/results.svg`, `docs/diagrams/01-system-architecture.svg`, `configs/base.yaml`, `configs/experiments/aloha_coffee_baseline.yaml` (dataset identity), `tests/conftest.py` (synthetic data). Engine-side ingestion is described only from this repository's call sites and README; I couldn't read the engine.

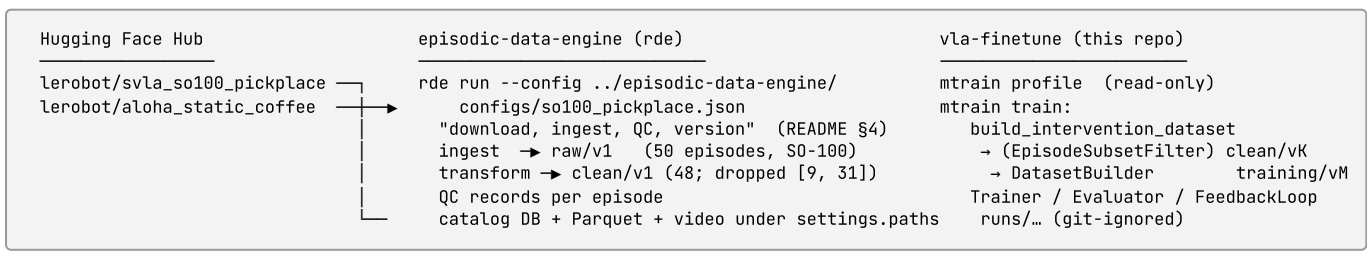
Where we are: Every earlier module assumed data already in the catalog. This module answers where it actually comes from, how it gets in, what this repository owes the sibling engine, and how to *show* the project live without breaking anything, including what to say when something goes wrong in front of an interviewer.

The one-line thesis: The data's path from Hugging Face to a trained checkpoint is fully traceable through engine lineage. But **none of the artifacts behind the README's numbers live in this repository** (runs are git-ignored), so the demo is a walk through *pre-recorded* run directories plus a few safe, read-only commands. Two of the CLI's commands will silently damage those recordings if run during a demo.

18.1 Prerequisites — the absolute basics

- **Data provenance.** The documented origin and processing history of a dataset: where it came from, what was done to it, by what code, and when.
- **Hugging Face Hub dataset.** A versioned dataset repository addressed as `org/name` (here `lerobot/svla_so100_pickplace`, `lerobot/aloha_static_coffee`). Each has its own licence and dataset card. **I haven't checked either licence. Check before redistributing anything.**
- **LeRobot dataset format.** Per-frame tabular data (Parquet) plus camera streams stored as video (the ALOHA config's comment says AV1), with episode boundaries and metadata. The README says the test fixture uses "the real LeRobot v3.0 layout".
- **Ingest → transform → build.** The engine's three stages: ingest produces `raw/vN`, transforms (normalize, QC filtering, episode subsetting) produce `clean/vN`, and build produces `training/vN` with splits and weights. Module 1 has the vocabulary.
- **QC (quality control).** Automated checks on episodes (missing frames, duplicates, discontinuities). Failing episodes are excluded from clean versions or builds.
- **Pre-recorded demo.** Showing completed artifacts instead of running long jobs live. The standard for any ML demo longer than a minute.

18.2 Where the data comes from and how it gets in



Verified from this repository:

- **Dataset identities** are `so100_pickplace` and `aloha_coffee` (`configs/base.yaml`, `aloha_coffee_baseline.yaml`), each with input version `clean/v1`.
- **The SO-100 lineage chain**, as printed in README §1 by `rde version lineage so100_pickplace training/v6`:
  - `ingest [] -> raw/v1 episodes 0->50`
  - `transform ['raw/v1'] -> clean/v1 episodes 50->48 dropped=[9, 31]`
  - `build_training_dataset ['clean/v1'] -> training/v6 episodes 48->48`

  Each step carries a run ID, a fingerprint and the software version. The two dropped episodes are recorded *by ID*.
- **Splits after QC:** 48 episodes → 38/5/5 (SO-100); 40 → 32/4/4 (ALOHA) (README Table 1).
- `training/v6` means at least six training versions of SO-100 were built. That's consistent with Module 4's "every run mints a new version".

Inferred, not verified:

- **What the `clean/v1` transform did beyond dropping episodes 9 and 31.** The test fixture's `clean_version` applies `NormalizeVector("state.joint")`, `NormalizeVector("action.joint")` and then QC, which suggests the real `clean/v1` is normalized. The README's "MSE is in normalized action units" agrees.
- **Why episodes 9 and 31 were dropped** (presumably QC failures, perhaps missing or discontinuous frames). The engine's QC records hold the answer: `rde obs quality`, as `profile.py`'s note suggests.
- **ALOHA's raw episode count before QC**, and which ALOHA episodes (if any) were dropped. Not stated in this repo.
- **The engine ingest config's contents** (`../episodic-data-engine/configs/so100_pickplace.json`).

The synthetic test data (`tests/conftest.py`) is generated by `rde.testing.make_lerobot_dataset`: 8 episodes × 20 frames at 10 fps, two task strings, then ingested, normalized, QC'd and built exactly like real data. No network, and no mocks of the source format (README §9). It's the only data this repository can produce on its own.

18.3 The relationship to the sibling project

Aspect	Engine (`episodic-data-engine`, `rde`)	This repo (`vla-finetune`, `model_training`)
Owns	ingestion, versions, lineage, QC, transforms, `DatasetBuilder`, `DatasetView`, `Evaluator`, `FeedbackLoop`, baselines, `SmolVLAPolicy` **including** `fit`	config, intervention *resolution*, training *procedure*, evaluation *decomposition*, feedback *packaging*, artifacts, reports, CLI
Dependency direction	imports nothing from this repo (README §5)	imports `rde` everywhere (Module 14)
Changes made *to* the engine for this project	three additive changes (README §5, ARCHITECTURE §4): `SmolVLAPolicy.fit()` **with checkpoint save, load and verify**; **an** `episode_ids` **selector on** `Evaluator.evaluate()`; **the** `EpisodeSubsetFilter` **transform**	—
Installed as	editable sibling checkout via `[tool.uv.sources]` (Module 1)	—
Version pin	**none**	—
Git provenance	recorded per run from a guessed path (Module 3)	recorded per run

What to say in an interview: "Three engine changes were made for this project: real gradient fine-tuning with checkpointing, slice-level evaluation by episode ID, and the episode-filter transform interventions are built on. Everything else in the engine predates this layer." (Credit the engine's authorship per the Module 17 note.)

18.4 What's on disk versus in the repo

Artifact	In this repository?	Where it lives
Source code, configs, docs, `results.svg`, architecture SVG	**yes**	git
Datasets (raw, clean, training versions)	no	engine data directory (`settings.paths`)
Run directories behind README Tables 2–4 (`runs/2026-09-03/...` etc.)	**no**, git-ignored (`runs`)	the experimenter's disk
Cross-run index	**no**, git-ignored (`experiments/*.jsonl`)	the experimenter's disk
Checkpoints (selected step 1,800 SO-100 / step 800 ALOHA)	no	inside the run directories
Base model weights	no	Hugging Face cache
Internal audit `docs/CODE_REVIEW.md`	**no**, git-ignored ("private record")	local only

Consequence: every number in the README is backed by files that only exist on the machine that ran them. **Before any demo, copy those run directories somewhere safe** (a tarball and a second disk). Module 12 showed that one wrong command overwrites them, and they're the only copy.

18.5 The rehearsed demo script (about 8 minutes)

Pre-flight checklist (do this the day before, not on the call):

1. `cp -a runs/ runs.backup/` and `cp -a experiments/ experiments.backup/`. Make it a second copy, outside the repo.
2. Activate `../.venv-vla` (or `../.venv` for steps that don't need torch). Run `mtrain --help` once to warm imports.
3. Confirm the engine catalog still has `so100_pickplace@clean/v1` and `training/v6`: `mtrain inspect --dataset so100_pickplace --version training/v6`.
4. Know the three run paths by heart (or put them in shell variables): `BASE=runs/<date>/baseline_full/seed_0`, `EXT=runs/<date>/baseline_full_extended/seed_0`, `ALOHA=runs/<date>/aloha_coffee_baseline/seed_0`.
5. Open `docs/figures/results.svg` and `docs/diagrams/01-system-architecture.svg` in browser tabs.
6. Have a terminal font large enough to share.

The script. Each step lists what to run, what to point at, and what it proves.

#	Do	Point at	What it proves	Time
1	Show `01-system-architecture.svg`	the seven runner stages, and the left column of engine services	the design in one picture: procedure here, mechanism in the engine	45 s
2	`rde version lineage so100_pickplace training/v6`	`dropped=[9, 31]`, three steps, fingerprints	**a trained model traces back to raw episodes, with every exclusion recorded**	45 s
3	`mtrain profile --dataset so100_pickplace --version clean/v1` (scroll to `tasks`, `episode_length`, `coverage.visual_diversity`)	1 task; 326–475 frames; `"status": "unavailable"`	**the imbalance variable came from the data, not a guess; unmeasurable things are marked unavailable**	60 s
4	`mtrain build-intervention --dataset so100_pickplace --version clean/v1 --kind imbalanced --variable episode_length_bucket --variable-value short --target-config /tmp/demo_imbalanced.yaml` (**no `--apply`**)	the printed block of episode IDs and reason	**interventions are resolved from the real catalog, dry-run by default**	45 s
5	`cat $BASE/split_integrity.json`	38/5/5, all overlaps `[]`, `"ok": true`	**episode-level splits, re-checked before training** (say it's a cheap, near-tautological check; Module 5)	30 s
6	Show `results.svg` panel (a), then `head $EXT/metrics/validation.jsonl`	the curve still falling at 1,000; resume point; selected step 1,800	**selection on validation only; continued training kept improving**	60 s
7	`mtrain compare --runs $BASE --runs $EXT --runs $ALOHA`	`mse` vs `nearest_neighbor_mse` columns	**the honest headline: 0.0290 → 0.0239 vs NN 0.0239; ALOHA 0.216 vs 0.106**	60 s
8	`jq '.summary' $EXT/evaluation/test_overall.json` and `jq '.temporal_error.mean_error_by_progress_bucket' $EXT/evaluation/test_overall.json`	mid-episode peak (≈ 0.144) vs ends (≈ 0.05)	**error decomposed by phase: the grasp and transport phase is hard**	45 s
9	`mtrain analyze-failures --run $EXT | head -40`	191/1,908 flagged, max episode 13.8%, "scattered"	**failures as evidence, not a verdict**. Then say, before they ask: at p90 with 5 episodes, "scattered" is the only possible output (Module 8)	60 s
10	`mtrain rollout --run /tmp/demo_rollout_run` (an empty scratch directory, not a real run)	`"status": "unavailable"`	**the system refuses to invent a success rate**	20 s
11	Close with the slide-free summary: "the harness told me a nearest-neighbour lookup was as good as my VLA, and auditing it told me which of its own comparisons weren't valid yet"	—	the project's real value	30 s

Optional live run (no GPU needed, about 1–3 minutes, unmeasured here): write a config with `training: {model: mean_action}` and a *new* `experiment_name` (for example `demo_live_<time>`), and run `mtrain train --config /tmp/demo_live.yaml`. It exercises intervention, split check, profile, baselines, report and index with no torch. That's exactly the path `test_runner_completes_baseline_only_experiment` covers. Show the new run directory tree, then `reports/experiment_report.md` with its "*not run*" and "unavailable" sections. **Point out that the report's model_training git line says `n/a`, and explain the bug** (Modules 3 and 10). Showing a known bug on purpose is a strong move.

Never do these during a demo

Command	Why
`mtrain test --run $EXT` or `mtrain evaluate --run …`	**overwrites** `test_overall.json` and `stratified.json` with new stochastic numbers and loses the length slices (Module 12)
`mtrain evaluate --split val …`	writes validation metrics into `test_overall.json`
`mtrain build-intervention … --apply --target-config configs/experiments/imbalanced.yaml`	erases the template's explanatory comments (Module 12)
`mtrain rollout --run $EXT`	harmless but appends a duplicate record to a published run; use a scratch directory
`mtrain train` with an existing experiment name the same day	`FileExistsError`; looks like a crash
`mtrain train --seed 1` "to show seeds"	it doesn't change the training seed (Module 11)
two `mtrain train` at once	run-directory, catalog and index races (Module 14)

18.6 Recovery table: what to say when things go wrong

What goes wrong	What you see	What to say	What to do
Engine catalog missing or moved	`LookupError: unknown version`	"Data is addressed by catalog version, never by path. The catalog for this machine isn't mounted, which is exactly the failure this design makes loud."	switch to the pre-recorded run directories (steps 5–9 need only files)
`rde` command not found	shell error	"The engine is a sibling package in a separate environment."	activate `../.venv-vla`; skip step 2 and show README §1's printed lineage instead
`import torch` slow or CUDA init hangs	long pause	"This is the heavy environment; the light one doesn't load torch at all, which is why the CLI imports lazily."	Ctrl-C; rerun in `../.venv` for read-only steps
`FileExistsError` on the live run	traceback	"Runs are write-once: it refuses to overwrite an existing run directory. A failed run blocks a same-day retry. That's a known gap, and the fix is a status file plus `--force` from the failed state."	change `experiment_name` and rerun
`JSONDecodeError` on `compare` or `validate`	traceback	"That's a partially written file. My writes aren't atomic yet, and the fix is temp-then-rename."	restore from `runs.backup/`
`compare` columns look misaligned	numbers under the wrong headers	"The terminal table builds headers from the first run and places values by position; the CSV output is keyed correctly."	rerun with `--out /tmp/c.csv` and `column -s, -t /tmp/c.csv`
The interviewer asks to run `mtrain test`	—	"I'd rather not on a published run. Re-scoring currently overwrites results, and SmolVLA is stochastic, so the numbers would move a few percent. That's the third fix on my list."	offer to run it against a copy: `cp -a $EXT /tmp/ext_copy && mtrain test --run /tmp/ext_copy`
A number on screen differs from the README	e.g. stratified slice MSE ≠ overall	"Slices re-run the stochastic model, so they don't reconcile exactly with overall; re-scoring moves MSE up to 5%."	cite README §2.3
The interviewer asks "so does it work on the robot?"	—	"Unknown. There's no robot or simulator here, and the system records rollout as unavailable instead of estimating it. Offline it ties a nearest-neighbour lookup. Closed loop is where I'd expect vision to matter, and that's the experiment I'd most like to run."	—
Run directories were lost	nothing to show	"The artifacts are deliberately not in git because checkpoints are gigabytes; the figure in the README was generated from them by a script."	show `results.svg`, the README tables and the code paths
GPU out-of-memory on any live training	CUDA OOM	"Two models resident at once, or another process on the GPU. The trainer frees its model before evaluation for this reason, and the crash marker records what happened."	show `crash.json` in the run directory

18.7 Honest gaps to own before an interviewer finds them

1. **No artifact behind the README's numbers is in the repository.** *Consequence:* results are reproducible only on the original machine, or by re-running. Back up before demos.
2. **Data licences aren't recorded anywhere in this repository**, and I didn't check them. *Consequence:* don't redistribute exports or datasets without checking each Hugging Face dataset card.

3. **The `clean/v1` transform contents and the reason for dropping episodes 9 and 31 aren't documented here.** They're engine-side. *Consequence:* "why were those dropped?" has to be answered from the engine's QC records, not from memory.
4. **ALOHA's pre-QC episode count and lineage aren't shown anywhere in this repo.** *Consequence:* SO-100 provenance is demonstrable; ALOHA's is asserted.
5. **The architecture diagram presents `DeployedModel` as an after-the-run step.** Module 9 showed it has never been executed. *Consequence:* if you point at it, say so.
6. **Two CLI commands damage the very artifacts a demo shows** (`evaluate` / `test`, `--apply`). *Consequence:* the demo script must avoid them, and you should say why. That turns a hazard into evidence of understanding.
7. **The live baseline-only run's duration is unmeasured here.** *Consequence:* rehearse it once and time it before relying on it.

The shape to carry into Module 0

```
HF Hub (lerobot/svla_so100_pickplace, lerobot/aloha_static_coffee)
  └─► rde run: download · ingest raw/v1 (50) · transform+QC clean/v1 (48; dropped 9, 31) · catalog
        └─► mtrain: intervention → clean/vK? → training/vN (38/5/5) → train → eval → feedback → reports
              └─► runs/ + experiments/  (git-ignored; only copy on the experimenter's disk → BACK UP)
Demo: architecture ▸ lineage ▸ profile ▸ dry-run intervention ▸ split check ▸ val curve ▸ compare
      ▸ phase error ▸ failures (+ honest caveat) ▸ rollout "unavailable" ▸ one-sentence close
Never: mtrain test/evaluate on published runs · --apply on templates · --seed "replicates" · concurrent runs
```

Drill — answer from memory, files closed

1. Trace the SO-100 data from Hugging Face to `training/v6`, naming every version and the episodes dropped.
2. Which three changes were made to the engine for this project?
3. *Why didn't you just* commit the run directories so anyone can verify the README?
4. *What breaks in production:* mid-demo, the interviewer says "run the test command on that run so I can see it live." What do you do?
5. *Interviewer framing:* "Show me something."

Appendix A — Model answers to the drill

A1. The SO-100 data path. "It starts as the Hugging Face dataset `lerobot/svla_so100_pickplace`. The data engine's `rde run` downloads and ingests it as `raw/v1`, 50 episodes. A transform with quality control produces `clean/v1`, 48 episodes, with episodes 9 and 31 dropped and recorded by ID in the lineage. My experiment configs start from `clean/v1`. For the baseline, the intervention is 'full', so there's no extra filter, and the engine's builder produces `training/v6` with 38/5/5 episode-level splits. `rde version lineage` on that version prints all three steps with run IDs and fingerprints, so the model I selected traces back to exact raw episodes. What I can't tell you from memory is why 9 and 31 failed QC. That's in the engine's QC records."

Why that answer lands: it's exact on versions, counts and IDs, shows the verification command, and draws a clean line around what you don't know.

A2. The three engine changes. "Real fine-tuning in `SmolVLAPolicy.fit`, with checkpoint save, load and verification, because the adapter previously had no training step. An `episode_ids` selector on `Evaluator.evaluate`, so slices can be scored by episode set. And the `EpisodeSubsetFilter` transform, which is what makes every data intervention a lineage-tracked engine operation instead of a local copy. They were additive, and the engine still imports nothing from this layer."

Why that answer lands: it's precise about scope and purpose, and it reinforces the one-way dependency.

A3. Why not commit the run directories? "Checkpoints are on the order of a gigabyte each, with several per run, so committing them to git is the wrong tool. The figure script regenerates the README figure from them, and the per-frame outcomes and metrics are small. The real gap is that *nothing* behind the README is published. I'd publish the small artifacts (resolved configs, `dataset.json`, `split_integrity.json`, metrics, evaluation JSON, per-frame outcomes, git and environment files) as a release asset or a dataset repository, and publish the selected checkpoint to the Hub with its model card. Then anyone could verify every table without my disk."

Why that answer lands: it explains why the default is right for checkpoints, admits the actual gap, and proposes a concrete publication split by artifact size.

A4. "Run the test command live." "I'd say: I'd rather not on that directory, because `mtrain test` currently overwrites the run's test results in place, and SmolVLA samples actions stochastically, so the published numbers would change by a few percent with no backup. That's one of the first fixes on my list. Then I'd offer to copy the run to a scratch directory and run it there, so we can watch it re-score and see the number move, which itself demonstrates the inference noise I mention in the README."

Why that answer lands: it turns a trap into a demonstration of deep knowledge of your own tool, and still satisfies the request safely.

A5. "Show me something." "Three minutes, three things. First, lineage: this command shows the model's training data back to raw episodes, including the two QC drops. Second, the honest result: this comparison table shows fine-tuning cut error to 2.6% of the mean-action baseline but exactly tied a nearest-neighbour lookup on robot state. Third, the refusal to fabricate: rollout is recorded as unavailable, and the profile marks visual diversity unavailable instead of inventing a score. Then I'd show you the one bug I always point out myself: the report prints 'n/a' for this repo's git commit because of a key mismatch."

Why that answer lands: it's short, structured, and ends with a self-identified bug, which builds trust faster than any feature.

Module 18 Summary — the one page to revise

What this module is. The data's provenance (Hugging Face → engine `raw/v1` → `clean/v1` → `training/vN`, with recorded drops), the exact boundary with the engine (three additive changes), what's on disk vs in git (no result artifacts in git), a rehearsed 8-minute demo built from read-only commands and pre-recorded runs, and a recovery table.

Five claims to make cold

1. SO-100: `lerobot/svla_so100_pickplace` → `raw/v1` (50) → `clean/v1` (48, dropped 9 and 31) → `training/v6` (38/5/5).
2. Three engine changes: `SmolVLAPolicy.fit` with checkpoints; an `episode_ids` selector on `Evaluator.evaluate`; `EpisodeSubsetFilter`.
3. Every README number is backed by git-ignored files on one machine. Back them up before demos.
4. The demo is read-only: lineage, profile, dry-run intervention, split check, validation curve, compare, phase error, failures, rollout.
5. Never run `mtrain test`/`evaluate` or `--apply` on published artifacts; offer a scratch copy.

Decisions

Chose	Over	Because	Cost
Git-ignore runs	commit artifacts	checkpoints are GBs	README not verifiable from the repo
Pre-recorded demo	live training	runs take 10–22 min	less "live"
Read-only demo commands	full CLI tour	two commands damage artifacts	must explain why
Scratch copy for risky commands	refuse outright	satisfies the request safely	one extra step

Volunteer unprompted: the backup step; why `mtrain test` is excluded; data licences unchecked; ALOHA lineage not shown in this repo; the diagram's `DeployedModel` box.

Numbers and names: 50 → 48 (dropped [9, 31]) → 38/5/5; ALOHA 40 → 32/4/4; `training/v6`; failures 191/1,908 (max episode 13.8%), ALOHA 440/4,400 (22.6%, 12.1%); demo about 8 minutes, 11 steps; fixture 8 × 20 frames at 10 fps, 2 tasks.

Sentences worth memorizing

- "This command traces the model back to raw episodes, including the two QC drops."
- "I won't run `test` on a published run: it overwrites in place, and the model is stochastic. Let me show you on a copy."
- "Every number in the README is backed by files that only live on the machine that ran them; publishing the small ones is on my list."